Adaptive Vision-Based Human Position Tracking and Autonomous Person (Dorsal) Following for Mobile Robots

by

CHRISTOPHER ONYEKACHUKWU OKOH
25006340

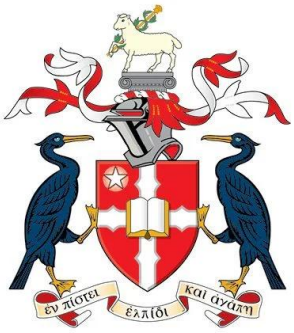
Submitted in partial fulfilment of the requirements for the Degree of

MASTER OF SCIENCE

in

ROBOTICS ENGINEERING

Supervised by Dr. Emanuele Lindo Secco



LIVERPOOL HOPE UNIVERSITY

School of Computer Science and the Environment

September 2026

DECLARATION

I hereby declare that this project is my work and has not been part or entire from any other source except where duly acknowledged. As such, all use of previously published works (books, journals, Internet, etc.) has been acknowledged within the main report to an entry in the References list.

I have also used the generative AI tool during the brainstorming of the project idea, some graphic design and grammar assistance. All AI-generated content was reviewed and approved by the author. The intellectual content, design decisions and analysis within this work remain solely the author's own work.

I further confirm that appropriate ethical approval has been obtained for the research presented in this dissertation.

I agree that an electronic or hard copy of this report may be stored and used for plagiarism prevention and detection. I understand that cheating and plagiarism constitute a breach of the university's regulations and will be dealt with accordingly.

Additionally, I hereby certify that the total word count for this dissertation, including the title page, declaration, abstract, acknowledgements, table of contents, list of figures and references, comply with the University's word count requirements.

I grant permission for Liverpool Hope University to archive this dissertation publicly.

CERTIFICATION PAGE

This is to certify that the project described in this dissertation is the original work of Christopher Onyekachukwu Okoh with registration number 25006340. In partial fulfilment of the requirements for the award of Master of Science (M.Sc.) degree in Robotics Engineering.

…….. ……

DR. EMANUELE LINDO SECCO
SUPERVISOR Date/Sign

…….. ……

DR. JENNIFER CLEAR
(HEAD OF SCHOOL) Date/Sign

…….. ……

EXTERNAL EXAMINER Date/Sign

DEDICATION

This dissertation is dedicated to Almighty God, the provider of all knowledge, for His eternal love and boundless grace during this master's degree program. To my wife, Rindah Joy Okoh, and my children, Jaden and Rinah Okoh, their understanding, spiritual companionship, patience, and encouragement have been a constant source of strength for me throughout this journey.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Dr. Emanuele Lindo Secco, for the guidance, encouragement, and constructive feedback provided throughout this research. I am also grateful to the staff and colleagues, particularly the laboratory technician, Mr. Ian Steel, who provided technical assistance, resources, and support during the development of this research work.

Furthermore, I am very grateful to the Commonwealth Scholarship Commission, UK, and Liverpool Hope University, UK, for their shared funding towards completing this master's program and research project.

ABSTRACT

Autonomous robots which can follow humans in their daily lives and activities are becoming increasingly important. These robots usually integrate a combination of perception, tracking, and motion control techniques to perform safe motion tracking. Multimodal expensive sensors, such as depth sensors (RGB-D cameras), LiDAR, and ultrasonic or infrared sensors, are deployed simultaneously. Owing to the high cost of these sensors, this study considered a low-cost technique for achieving an autonomous robot human (dorsal) following task using a monocular RGB PTZ camera.

This dissertation presents the development and experimental evaluation of a deep learning-based vision system for autonomous person-following using a 4-wheels Robotnik Summit XL mobile robot. A YOLO-based human detector was developed and integrated with the robot's front RGB PTZ camera to identify the human body and obtain the bounding box position, dimensions, and confidence information. The horizontal centre of the bounding box was used to estimate the target position within the image, whereas the bounding-box height was used as a relative human target distance identifier. These parameters were subsequently applied to an adaptive motion-control algorithm to generate robot movement commands according to the observed changes in the visual position and relative distance of the human target. A person-centred tracker algorithm was also developed to maintain the robot 'visually engaged' with the identity of the detected human using a unique ID tracker. This ensures the robustness of the system against visual occlusions, namely, when the perception of the human target has been temporarily lost and then regained, its previous and unique ID is reassigned to the proper human target.

The proposed system was experimentally tested and validated using nine different scenarios covering diverse statuses, such as (a) stationary and (b) dynamic human behaviours, (c) stopping, (d) variable operating distances, (e) lighting variation, and (f) the absence of a target. One trial was conducted for each scenario in this study. The results show detection accuracies from 94.4% to 100% across person-present scenarios, with average confidence scores between 0.85 and 0.90. The system achieved detection rates of approximately 1.3 –5.3 Hz in the recorded experiments, while robot velocity commands between -0.003 m/s to 0.005 m/s were published with a sampling rate between 3.0 – 10.0 Hz. Position-estimation analysis showed variations in the horizontal target position and bounding-box height according to target movement and operating distance, with a walking distance scenario of 2–3 m from the mobile robot producing the lowest horizontal positional standard deviation of 98.04 pixels. The standing scenario produced the lowest bounding box height standard deviation of 34.17 pixels. Finally, in the 'no target' scenario, the system achieved 100% specificity and generated zero linear velocity commands, indicating that the robot remained stationary in the absence of a detected person.

TABLE OF CONTENTS

TITLE PAGE i
DECLARATION ii
CERTIFICATION PAGE iii
DEDICATION iv
ACKNOWLEDGEMENTS v
ABSTRACT vi
TABLE OF CONTENTS vii
LIST OF TABLES xiii
LIST OF FIGURES xiv
LIST OF ABBREVIATIONS xvi
DEFINITION OF TERMS xvii

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study 1
1.2 Motivation of the Study 2
1.3 Statement of the Problem 3
1.4 Research Questions 3
1.5 Aim of the Study 4
1.6 Objectives of the Study 4
1.7 Scope of the Study 4
1.8 Significance of the Study 5
1.9 Justification of the Study 5
1.10 Dissertation Structure 5

CHAPTER TWO: LITERATURE REVIEW

2.1 Theoretical Review 7
2.1.1 Autonomous Mobile Robots (AMRs) 7
2.1.1.1 Evolution of Autonomous Mobile Robots 8
2.1.1.2 Types of Robots 9
2.1.1.3 Applications of Autonomous Mobile Robots 11
2.1.1.4 Navigation Autonomous Mobile Robots 12
2.1.1.5 Challenges of Autonomous Mobile Robots 12
2.1.2 Human-Robot Interaction 13
2.1.2.1 Evolution of Human-Robot Interaction 14
2.1.2.2 Human-Centred Robotics 14
2.1.2.3 Safety Considerations of Human-Robot Interaction 14
2.1.2.4 Person-Following within HRI 15
2.1.3 Autonomous Person-Following Robots 15
2.1.3.1 Historical Development of Autonomous Person-Following Robots 16

2.1.3.2 Components of an Autonomous Person-Following Robots 16
2.1.3.3 General Architecture of Autonomous Person-Following Robots 17
2.1.3.4 Existing Applications of Autonomous Person-Following Robots 18
2.1.4 Computer Vision in Robotics 19
2.1.4.1 Image Acquisition 19
2.1.4.2 Image Processing 20
2.1.4.3 Feature Extraction 20
2.1.4.4 Object Recognition 20
2.1.4.5 Vision-Based Navigation 21
2.1.4.6 Challenges of Computer Vision in Robotics 21
2.1.5 Machine Learning 22
2.1.5.1 Supervised Learning 22
2.1.5.2 Unsupervised Learning 22
2.1.5.3 Reinforcement Learning 23
2.1.5.4 Applications of Machine Learning in Robotics 24
2.1.6 Deep Learning 24
2.1.6.1 Artificial Neural Networks 24
2.1.6.2 Convolutional Neural Networks 25
2.1.6.3 Advantages of Deep Learning over Classical Computer Vision 26
2.1.6.4 Deep Learning in Robotics Perception 26
2.1.7 Object Detection 27
2.1.7.1 Conventional Object Detection Approaches 27
2.1.7.2 Deep Learning-Based Object Detection Approaches 28
2.1.8 YOLO (You Only Look Once) Object Detection 28
2.1.8.1 Development of YOLO 28
2.1.8.2 YOLO Architecture 29
2.1.8.3 Evolution of YOLO (YOLOv1 – YOLOv11) 30
2.1.8.4 Advantages of YOLO 32
2.1.8.5 Limitation of YOLO 33
2.1.8.6 Why YOLO is suitable for Real-time Robotics 33
2.1.9 Object Tracking 33
2.1.9.1 Tracking by Detection 33
2.1.9.2 Single Object Tracking 34
2.1.9.3 Multi-Object Tracking 34
2.1.9.4 Challenges of Object Tracking 35
2.1.10 Robot Operating System (ROS) 35
2.1.10.1 ROS Architecture 35
2.1.10.2 Advantages of ROS 37
2.1.10.3 Limitations of ROS 38
2.1.11 Robot Control Systems 39
2.1.11.1 Motion Control 39
2.1.11.2 Velocity Control 40
2.1.11.3 Proportional Integral Derivative (PID) Control 40
2.1.11.4 Adaptive Control 41

2.1.12 Human Position Estimation 41
2.1.12.1 Boundary Box Estimation 41
2.1.12.2 Monocular Distance Estimation 42
2.1.12.3 Pose Estimation 42
2.1.12.4 Sensor Fusion Approaches 42
2.1.13 Visual Servoing 43
2.1.13.1 Position-Based Visual Servoing (PBVS) 43
2.1.13.2 Image-Based Visual Servoing (IBVS) 44
2.1.13.3 Comparison of Position-Based and Image-Based Visual Servoing 44
2.2 Empirical Review 45
2.2.1 Review of Related Empirical Studies 46
2.2.2 Critical Analysis of Reviewed Studies 48
2.2.3 Review of Classical Vision-Based Person-Following System 48
2.2.3.1 Haar Cascade-Based Person Detection 48
2.2.3.2 Histogram of Oriented Gradient (HOG) 50
2.2.3.2 Optical Flow-Based Tracking 50
2.2.3.4 Colour-Based Tracking 51
2.2.3.5 Background Subtraction 51
2.2.3.6 Critical Analysis of Classical Vision-Based Person-Following Techniques 52
2.2.4 Review of Deep Learning-Based Person-Following Systems 53
2.2.4.1 Faster R-CNN 53
2.2.4.2 Single Shot Multi-Box Detector 54
2.2.4.3 You Only Look Once (YOLO) 55
2.2.4.4 CenterNet 55
2.2.4.5 EfficientDet 56
2.2.4.6 Critical Comparison of Deep Learning-Based Person-Following Systems 56
2.2.5 Review of Tracking Algorithms 58
2.2.5.1 Kalman Filter 58
2.2.5.2 Simple Online Real-time Tracking 58
2.2.5.3 DeepSORT 59
2.2.5.4 ByteTrack 60
2.2.5.5 BoT-SORT 60
2.2.5.6 Critical Comparison of Tracking Algorithms 61
2.2.6 Review of Mobile Robot Platforms 62
2.2.6.1 TurtleBot 62
2.2.6.2 Clearpath Husky 63
2.2.6.3 Robotnik Summit XL 63
2.2.6.4 Fetch Robot 64
2.2.6.5 Boston Dynamics Spot 65
2.2.6.6 Comparative Analysis of Mobile Robot Platforms 66
2.2.7 Review of ROS-Based Person-Following Systems 67

2.2.7.1 ROS Stack Navigation 67
2.2.7.2 ROS Perception Pipeline 68
2.2.7.3 Existing ROS-Based Person-Following Implementations 68
2.2.7.4 Limitations of Existing ROS-Based Person-Following Systems 69
2.2.7.5 Critical Analysis of ROS-Based Person-Following Systems 69
2.2.8 Review of Adaptive Person-Following Controllers 70
2.2.8.1 Proportional Integral Derivative (PID) Control 70
2.2.8.2 Fuzzy Logic Control 70
2.2.8.3 Model Predictive Control 71
2.2.8.4 Reinforcement Learning 72
2.2.8.5 Adaptive Control 72
2.2.8.6 Critical Comparison of Adaptive Person-Following Controllers 73
2.3 Comparative Analysis of Existing Studies on Human-Following Robots 74
2.3.1 Critical Discussion 74
2.4 Research Gap 78

CHAPTER THREE: RESEARCH METHODOLOGY

3.1 Research Design 80
3.1.1 Research Philosophy 80
3.1.2 Research Approach 81
3.2 System Requirements Analysis 82
3.2.1 Functional Requirements 82
3.2.2 Non-Functional Requirements 84
3.3 System Architecture 85
3.3.1 Conceptual Framework 85
3.3.2 Data Flow Architecture 88
3.3.3 System Workflow 89
3.4 Hardware Architecture 90
3.4.1 Robotnik Summit XL Mobile Robot 90
3.4.2 RGB Camera 92
3.4.3 Processing Computer 93
3.4.4 Network Configuration 95
3.5 Software Architecture 96
3.5.1 Ubuntu Linux 96
3.5.2 Robot Operating System (ROS) 97
3.5.3 Python 101
3.5.4 OpenCV 102
3.5.5 YOLOv8 Framework 102
3.5.6 Development Environment 104
3.6 Development of the Vision-Based Perception Module 104
3.6.1 Image Pre-processing 105
3.6.2 Human Detection using YOLOv8 106
3.6.3 Bounding Box Generation 107

3.6.4 Human Position Estimation 109
3.6.5 Detection Message Generation 111
3.7 Development of the Motion Control Module 113
3.7.1 Control Strategy 113
3.7.2 Distance Estimation 114
3.7.3 Motion Decision Logic 115
3.7.4 Velocity Generation 117
3.7.5 Adaptive Motion Controller 119
3.8 System Implementation 122
3.8.1 Software Implementation 122
3.8.2 Integration Process 123
3.9 Experimental Setup 125
3.9.1 Experimental Environment 125
3.9.2 Test Scenarios 125
3.10 Performance Evaluation 127
3.10.1 Evaluation Metrics 127
3.10.2 Data Collection 127
3.10.3 Data Analysis 128
3.11 Safety Considerations 129

CHAPTER FOUR: RESULTS AND DISCUSSION

4.1 Results 130
4.1.1 Detection Performance 130
4.1.2 Motor Control Performance 139
4.1.2.1 Forward Motion 140
4.1.2.2 Backward Motion 141
4.1.2.3 Stop Accuracy 142
4.1.2.4 Response Time 143
4.1.2.5 Motion Smoothness 144
4.1.3 Position Estimation Performance 144
4.1.3.1 Centre Estimation 144
4.1.3.2 Distance Estimation 145
4.1.3.3 Positional Stability 147
4.2 Discussion of Findings 148
4.2.1 Relating Results with Research Questions 148
4.2.2 Comparison with Existing Literature 149

CHAPTER FIVE: SUMMARY AND CONCLUSION

5.1 Summary 151
5.2 Challenges and Limitations of the Study 151
5.3 Implications of the Study 152
5.4 Future Work 153
5.5 Conclusion 154

REFERENCES 156

APPENDIX A 166
APPENDIX B 170

LIST OF TABLES

Table 2.1 – Comparison of Position-Based and Image-Based Visual Servoing 45
Table 2.2 – Comparison of Classical Vision-Based Person-Following techniques 52
Table 2.3 - Comparison of Deep Learning-Based Object Detectors 57
Table 2.4 – Advantages, Disadvantages and Applications of Kalman Filters 58
Table 2.5 – Advantages, Disadvantages and Applications of SORT 59
Table 2.6 – Advantages, Disadvantages and Applications of DeepSORT 59
Table 2.7 – Advantages, Disadvantages and Applications of ByteTrack 60
Table 2.8 – Advantages, Disadvantages and Applications of BoT-SORT 61
Table 2.9 – Comparisons of Tracking Algorithms 61
Table 2.10 – Comparison of Mobile Robot Platform 66
Table 2.11 – Advantages and Limitations of PID Control 70
Table 2.12 - Advantages and Limitations of Fuzzy Logic Control 71
Table 2.13 – Advantages and Limitations of Model Predictive Control (MPC) 71
Table 2.14 - Advantages and Limitations of Reinforcement Learning 72
Table 2.15 – Advantages and Limitations of Adaptive Control 72
Table 2.16 – Comparison of Adaptive Person-Following Controllers 73
Table 2.17 – Comparison Analysis of Existing Studies on Human-Following Robots 75
Table 3.1 – Node Interaction 99
Table 3.2 – ROS Topics Applied in the System's Framework 100
Table 4.1 -Timestamped Message Data for Each Scenario 132
Table 4.2 – Quantitative Analysis for Detection Performance 138
Table 4.3 – Centre Position Estimation Analysis across all Scenarios 145
Table 4.4 – Distance Estimation Analysis 146
Table 4.5 – Positional Stability Analysis 147

LIST OF FIGURES

Figure 2.1 – Evolution of Autonomous Mobile Robots 8
Figure 2.2 – Wheeled Mobile Robot 9
Figure 2.3 – Tracked Mobile Robot 10
Figure 2.4 – Legged Robot 10
Figure 2.5 – Aerial Mobile Robot 10
Figure 2.6 – Underwater Mobile Robot 11
Figure 2.7 – Human-Robot Interaction 13
Figure 2.8 – Person-Following Autonomous Robot 15
Figure 2.9 – Autonomous Robot Person-Following Component Architecture 17
Figure 2.10 – Reinforcement Learning Block Diagram 23
Figure 2.11 – Neural Network Architecture 25
Figure 2.12 – Model Structure of YOLOv8 29
Figure 2.13 – Evolution of YOLO 30
Figure 2.14 – Basic ROS Data Flow 36
Figure 2.15 – Three-Level Hierarchical Motor Control 39
Figure 2.16 – PID Controllers of Mobile Robot System 40
Figure 2.17 – Position-Based Visual Servoing Scheme 43
Figure 2.18 – Image-Based Visual Servoing Scheme 44
Figure 2.19a – Haar Theoretical Face Model 49
Figure 2.19b – Cascade Structure for Haar 49
Figure 2.20 – Histogram of Gradients (HOG) Feature Extraction Process 49
Figure 2.21 – Optical Flow-Based Tracking Process for Estimation and Tracking 50
Figure 2.22 – Colour-Based Tracking Process using Distinctive Colour Information 51
Figure 2.23 – Background Subtraction Human Detection Process 51
Figure 2.24 – Faster R-CNN Person Detection Pipeline 53
Figure 2.25 – Single Shot Multi Box Detector Pipeline 54
Figure 2.26 – YOLO Human Detection Pipeline 55
Figure 2.27 – CenterNet Pipeline 55
Figure 2.28 – Architectural Overview of EfficientDet 56
Figure 2.29 – TurtleBot3 63
Figure 2.30 – Husky A300 63
Figure 2.31 – Robotnik Summit XL 64
Figure 2.32 – Fetch Advanced Mobile Manipulator 65
Figure 2.33 – Spot Robot 65
Figure 3.1 – Conceptual Framework of the Autonomous Vision-Based Human-Following System 86
Figure 3.2 – System Workflow of Autonomous Vision-Based Human-Following System 89
Figure 3.3 – Real World Robotnik Summit XL Robot Back Panel 90
Figure 3.4 - Real World Robotnik Summit XL Robot Hardware Components 91
Figure 3.5 – Axis PTZ Network Camera 92
Figure 3.6 – Schematic of Elements Connected to the Robot's Onboard CPU 93
Figure 3.7 – System Network Structure 95

Figure 3.8a – ROS Communication Architecture 97
Figure 3.8b – ROS Node Graph 98
Figure 3.9 – YOLOv8 Inference Pipeline 103
Figure 3.10 – Bounding Box Parameters Extracted from YOLOv8 Human Detector 108
Figure 3.11–Human Position Estimation using Image Centre & Bounding Box Geometry 109
Figure 3.12 – Relative Distance Estimation Based on Variations in Bounding Box Height 114
Figure 3.13-Motion Decision Logic Implemented in the Adaptive Visual Servo Controller116
Figure 3.14 – Perception to Action Interaction 123
Figure 3.15 – Standing Scenario 124
Figure 3.16 – Walking Forward Scenario 125
Figure 3.17 – Walking Backward Scenario 125
Figure 3.18 – Stopping Scenario 125
Figure 3.19 – Standing >3 m Scenario 125
Figure 3.20 – Walking within 2 m to 3 m Range Scenario 126
Figure 3.21 – Standing <3 m Scenario 126
Figure 3.22 – Lighting Variation Scenario 126
Figure 3.23 – No Target Scenario 126
Figure 4.1 – Active Person Tracker 130
Figure 4.2 – Confusion Matrix to Define the Ground Truth and Detector Output 131
Figure 4.3 – YOLO Detector Message Data 131
Figure 4.4 – Average Confidence Score across all Scenarios 138
Figure 4.5 – Motor Controller Module Message Data 139
Figure 4.6a – Real World Robot Forward Motion 140
Figure 4.6b – Velocity-Time Graph for Forward Motion 140
Figure 4.7a – Real World Robot Backward Motion 141
Figure 4.7b – Velocity-Time Graph for Backward Motion 141
Figure 4.8a – Real World Stop Scenario 142
Figure 4.8b – Velocity-Time Graph for Stop Accuracy 142
Figure 4.9 – Response Time Graph for Forward, Backward and Stop Actions 143
Figure 4.10 – Distance Estimation Graph 146

LIST OF ABBREVIATIONS

YOLO	You Only Look Once
PTZ	Pan-Tilt-Zoom
RGB	Red, Green and Blue
FPS	Frames Per Second
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
Px	Pixel
FOV	Field of View
PID	Proportional Integral Derivative
ROS	Robot Operating System
UKF	Unscented Kalman Filter
SORT	Simple Online and Real-time Tracking

DEFINITION OF TERMS

i. Autonomous Mobile Robot (AMR)

An Autonomous Mobile Robot (AMR) is a robotic system capable of navigating and performing assigned tasks within an environment without continuous human intervention using sensors, perception algorithms, and control systems.

ii. **Person Following**

Person following is the ability of a mobile robot to automatically detect, identify, track, and maintain an appropriate distance from a designated human while adapting its movement to the changing position of the person.

iii. **Human Position Tracking**

Human position tracking involves the continuous estimation and updating of a person's location, direction, and relative distance with respect to the robot using visual or sensor-based information.

iv. Computer Vision

Computer vision is a branch of artificial intelligence that enables computers and robots to acquire, process, analyse, and interpret visual information from images or videos to understand their surrounding environment.

v. Artificial Intelligence (AI)

Artificial Intelligence refers to the capability of computer systems to perform tasks that normally require human Intelligence, including perception, learning, reasoning, decision-making, and problem-solving.

vi. Machine Learning

Machine learning is a subset of artificial intelligence that enables computer systems to learn patterns from data and improve their performance without being explicitly programmed for every possible situation.

vii. Deep Learning

Deep learning is a specialised area of machine learning that employs multilayered artificial neural networks to automatically learn complex features from large datasets for tasks such as object detection, image classification, and recognition.

viii. YOLO (You Only Look Once)

YOLO is a state-of-the-art deep learning-based object detection algorithm that performs object localisation and classification simultaneously in a single forward pass, enabling high-speed real-time detection that is suitable for robotic applications.

ix. Object Detection

Object detection is a computer vision process that locates and identifies objects within an image or video by determining their class labels and spatial locations.

x. Human Detection

Human detection is the process of identifying the presence and location of people within an image or video sequence using computer vision and machine learning techniques.

xi. Object Tracking

Object tracking is the process of continuously estimating the position of a detected object across successive image frames, while maintaining its identity over time.

xii. Bounding Box

A bounding box is a rectangular region that surrounds a detected object within an image. It specifies the location of an object using coordinates and is commonly used to estimate the position, size, and distance of an object.

xiii. Human Pose Estimation

Human pose estimation involves identifying and locating key human body joints, such as the head, shoulders, elbows, and knees, to determine body posture and movement.

xiv. Mobile Robotics

Mobile robotics is a field of robotics that is concerned with the design, development, and control of robots capable of moving autonomously or semi-autonomously within an environment.

xv. Robot Operating System (ROS)

The Robot Operating System (ROS) is an open-source middleware framework that provides software libraries, communication tools, hardware abstraction, and development utilities for designing robotic applications.

xvi. Robotnik Summit XL

Robotnik Summit XL is a four-wheel, differential-drive, autonomous mobile robotic platform designed for research and industrial applications. It served as the experimental platform used to validate the proposed person-following system in this study.

xvii. PID Controller

A proportional–integral–derivative (PID) controller is a feedback control algorithm that continuously computes the error between a desired reference and measured output to produce smooth and stable control actions.

xviii. Adaptive Control

Adaptive control is a control strategy that automatically adjusts the controller parameters in response to changes in system dynamics or environmental conditions to maintain the desired performance.

xix. Real-Time System

A real-time system is a computing system that processes incoming data and produces responses within strict time constraints, ensuring immediate interaction with dynamic environments.

xx. Human–Robot Interaction (HRI)

Human–Robot Interaction is an interdisciplinary study of communication, collaboration, and interaction between humans and robotic systems to achieve safe, intuitive, and efficient cooperation.

xxi. Image Processing

Image processing refers to the manipulation and analysis of digital images to improve the image quality or extract useful information for computer vision applications.

xxii. Visual Servoing

Visual servoing is a robot control technique that uses visual feedback from one or more cameras to guide and adjust the robot's movement toward a desired target.

xxiii. Autonomous Navigation

Autonomous navigation is the capability of a robot to move safely and intelligently within an environment while making independent decisions based on the sensor information.

xxiv. Feedback Control

Feedback control is a control mechanism in which system outputs are continuously measured and compared with the desired reference values to minimise errors and improve system stability.

xxv. Motion Control

Motion control is the process of regulating a robot's linear and angular movements to achieve smooth, accurate, and stable navigation.

xxvi. Differential Drive Robot

A differential drive robot is a mobile robot whose movement is controlled by independently varying the speeds of the left and right drive wheels, thereby enabling forward, backward, and rotational motions.

xxvii. Perception System

A perception system is a collection of sensors and algorithms that enable a robot to perceive, interpret, and understand information about its surrounding environment.

xxviii. Detection Confidence

Detection confidence is a probability score generated by an object detection algorithm that indicates the likelihood that a detected object belongs to a particular class.

xxix. Bounding Box Height

The bounding box height is the vertical size of the bounding box of the detected object in an image. In monocular vision systems, it is commonly used as an approximate indicator of an object's distance from the camera.

xxx. Monocular Vision

Monocular vision is a vision system that relies on a single camera for environmental perception, detection, and distance estimation.

xxxi. Frame Rate

The frame rate refers to the number of image frames processed or captured per second (FPS), which directly influences the responsiveness of a real-time vision system.

xxxii. Person Re-Identification

Person re-identification is the process of recognising and consistently identifying the same individual across multiple image frames or different camera views after a temporary loss of tracking.

xxxiii. Occlusion

Occlusion occurs when a target object is partially or completely blocked by another object, making detection and tracking difficult.

xxxiv. Embedded System

An embedded system is a dedicated computing platform integrated into a robot to efficiently execute specific control, sensing, and processing tasks.

xxxv. Real-Time Person-Following Robot

A real-time person-following robot is an autonomous mobile robot that continuously detects, tracks, and follows a designated individual while processing sensory information and generating motion commands with a minimal delay.

CHAPTER ONE

INTRODUCTION

1.1 BACKGROUND OF STUDY

With the continuous advancement of robotic systems, the benefits of autonomous mobile robots are significant. The major reason for this is that most robot agents that collaborate with humans to perform several tasks in industrial or domestic settings, include mobility in the form of wheeled, tracked, legged, aerial, or underwater mobility. Moreover, the advantages of these robots are that they are cost-effective, flexible, highly efficient, and easy to install. In this regard, Statzon (2023) stated that by 2030, the autonomous mobile robot market size will be worth $26.9 billion, reflecting an annual growth rate of 16.1%, driven by the surging demand for automated supply chains and advanced logistics.

Less discussed is the application of these robots in close proximity to humans which relies on the possibility of socially attuning them, particularly as this can influence the robot's performance at the sensorimotor level (Ciardo *et al*., 2019). On the other hand, personalisation can help improve user engagement in long-term interactions by adapting to the user's personality, preferences, and needs, or by recalling shared memories with the user. Moreover, personalising interactions can facilitate the establishment of rapport and trust between the user and robot. This type of study requires substantial resources, which may not always be technically feasible, especially if robots are deployed in uncertain scenarios, which often do not provide generalisable results owing to the variability of subject needs, thus making it challenging for researchers to publish results (Irfan *et al*., 2019).

Developing intelligence for the interactive features in robots is a relatively growing field that deals with assistive capabilities that can be used to support health workers in life-threatening situations, help elderly people live self-determined and independent lives, and cater to more than one billion people who are currently experiencing different forms of disabilities (Malla & Bhandari, 2023; Graf *et al*., 2019; Ma, 2024). Several initiatives focused on using these technologies to advance healthcare have gradually transformed traditional medical models, thus enhancing the efficiency and quality of medical services (Reegård *et al*., 2024; Xu, 2025). Regardless of these transformations, there are still concerns that some of these service robots have not yet fully achieved human-level expertise, especially in tasks that involve interaction with people and handling large varieties of objects (Odabasi *et al*., 2022).

Learning and adapting to various tasks usually begin with object detection, which provides critical functionalities such as navigation, collision avoidance, object manipulation, equipment condition monitoring, and interaction with dynamic surroundings (Rakhmetova *et al*., 2025). As a platform, computer vision technology provides robots with the capability to detect, recognise, and make decisions. These decisions in mobile robots are integrated with automated steering control mechanisms to perform dynamic navigation (Shafique *et al*., 2021). As robots are increasingly used to assist and work with people, especially in healthcare service delivery, it is crucial that these robotic devices are developed to understand body language for accurate intuition (GP *et al*., 2024).

Robot vision perception and deep learning models, such as convolutional neural networks (CNN), are widely used for target detection, object recognition, and scene understanding. For instance, using deep learning technology, robots can accurately detect the position and category of target objects in complex environments, providing key information for subsequent interactive operations. Deep learning shows excellent performance in semantic segmentation which can help robots understand the environment more accurately, predict and plan their action path in dynamic environments, and realise autonomous navigation and obstacle avoidance for mobile robots, thus improving the accuracy and efficiency of their interaction. Additionally, deep learning provides robots with the ability to learn and imitate human behaviour for improved adaptability in complex environments (Zhang *et al*., 2024).

Considering that these robotic systems are complex hardware devices with many sensors and controllers managed by distributed software, it becomes pertinent to develop their control algorithm on the Robot Operating System (ROS) framework because it allows for the construction of control and navigation programs which are implemented either in a simulated environment, such as Gazebo, or directly on a real robot (Marian *et al*., 2020; Arcos *et al., 2020),* as proposed in this research project. Developing the control terminal based on a ROS platform ensures the execution of core command scheduling and human-computer interaction functions, where the environment's map of the robot's location and navigation path in real time is displayed through a graphical interface, analysing task instructions, and coordinating the software and hardware modules to complete collaborative tasks, such as autonomous navigation and object detection (Zhou & Guowei, 2025).

In addition to autonomous navigation algorithms, person-follower algorithms can be developed within the ROS framework. The primary method applied in the person-follower algorithm is a vision-based approach, where a camera is used to identify and track a person (Dam *et al*., 2020). A person-following robot is an autonomous robotic system designed to track and follow a human companion in various applications and environments. It involves the use of sensors, perception algorithms, planning strategies, and control mechanisms to enable the robot to navigate and maintain the pace of the person it is following. The purpose of person-following robots is to provide companionship or support in scenarios such as healthcare and social interactions in various settings (Montesdeoca *et al*., 2024). As these robots are now required to perform tasks associated with providing healthcare support, significant effort has been invested in developing robust person-follower algorithms (Honig *et al*., 2018).

1.2 MOTIVATION OF THE STUDY

The motivation for this research on a person-following mobile robot emanates from the fact that there is an increasing amount of labour shortages driven by declining birthrates and an aging population which affects the availability of sufficient nursing personnel within the health service system worldwide (Sim *et al*., 2025). Many hospitals, care homes, and rehabilitation centres are experiencing increasing demand for healthcare services owing to ageing populations, rising numbers of patients with chronic illnesses, and limited availability of qualified healthcare professionals. Consequently, few nurses and caregivers often spend a considerable portion of their working hours on non-clinical tasks, such as transporting medical

supplies, delivering medications, carrying equipment, and escorting patients, thereby reducing the time available for direct patient care.

A person-following mobile robot offers a practical solution by autonomously accompanying healthcare workers and carrying essential equipment or supplies during ward rounds while maintaining a safe distance from the human target. This capability can reduce the physical strain associated with repetitive lifting and transportation tasks, minimise staff fatigue, and improve workplace ergonomics. In addition, the robot can enhance operational efficiency by reducing unnecessary trips between wards and storage areas, allowing healthcare professionals to focus on diagnosis, treatment, patient monitoring and compassionate care.

Beyond improving productivity, person-following robots can contribute to better patient outcomes by supporting timely service delivery and reducing delays caused by staff shortage. They also provide a scalable technological solution that complements rather than replaces healthcare workers. Consequently, research in this area has the potential to address pressing workforce challenges while promoting safer, more efficient, and patient-centred healthcare environments.

1.3 STATEMENT OF THE PROBLEM

Autonomous person-following robots have attracted significant research interest due to their potential applications in healthcare, service robotics, and human–robot collaboration. Despite recent advances, existing systems still face significant challenges in reliably tracking human targets in dynamic environments. Many person-following approaches based on monocular cameras face challenges in maintaining continuous target tracking when the person moves outside the camera's field of view or becomes partially occluded (Koide *et al*., 2020). Traditional vision-based approaches can also be sensitive to illumination changes, background complexity, and partial visibility, resulting in unreliable human recognition and tracking (Álvarez-Santos *et al*., 2012). These limitations lead to target loss and affect the stability of the robot-following behaviour. Furthermore, numerous proposed solutions have been validated primarily through simulations, with limited implementation on real robotic platforms, where sensing uncertainties and control constraints are more pronounced. These limitations frequently lead to oscillatory, delayed, or unsafe robot behaviour during indoor operations. Therefore, there is a need to develop an adaptive vision-based person-following system capable of effective human detection, continuous position estimation, and stable real-time navigation on a physical autonomous mobile robot operating in an indoor environment.

1.4 RESEARCH QUESTIONS

To systematically develop and evaluate an adaptive vision-based person-following system, this study will address these questions

i. How can a deep learning-based vision system be developed for robust human detection in autonomous person-following mobile robot?

ii. How can human positional changes be estimated and tracked continuously in real time to support autonomous person following?
iii. How can an adaptive motion control algorithm be designed to achieve stable and responsive person following under dynamic human movement?
iv. To what extent does the proposed adaptive vision-based person-following system improve detection accuracy, following stability, and real-time performance on a Robotnik Summit XL mobile robot?

1.5 AIM OF THE STUDY

The aim of this study was to develop an adaptive vision-based human position tracking and autonomous person-following system capable of robustly following a moving person under dynamic positional and illumination changes using deep learning and ROS.

1.6 OBJECTIVES OF THE STUDY

To achieve this aim, these objectives are considered for this study

- To investigate existing human-following techniques for autonomous mobile robots and design a vision-based human detection system using deep learning.
- To develop a real-time human position estimation algorithm by implementing adaptive forward and backward following control.
- To integrate the perception and control modules within the ROS.
- The proposed system was deployed on a Robotnik Summit XL mobile robot to evaluate the detection, motor control, and position estimation performance under various human-pose scenarios in an indoor environment.

1.7 SCOPE OF THE STUDY

This study focuses on the design, implementation, and evaluation of an adaptive vision-based autonomous person-following system for indoor environments. The proposed system was implemented on a real-world Robotnik Summit XL mobile robot operating within the ROS framework. Human detection and localisation were achieved using the YOLO deep learning object detection algorithm to provide real-time information on the position of a detected person within the camera's field of view (FOV). The control strategy focuses on linear motion in response to changes in the detected distance of the person while maintaining real-time operation.

Finally, an experimental validation is conducted based on the robot's ability to maintain continuous person following under normal indoor operating conditions, including changes in the person's position and movement direction.

1.8 SIGNIFICANCE OF THE STUDY

The advancement of autonomous person-following mobile robots provides an opportunity to make theoretical and practical contributions to robotics, computer vision, and human-robot interaction (HRI). Robotics researchers will benefit from the methodology of integrating adaptive deep learning techniques to ensure perception and control. Computer vision researchers will find it valuable as it demonstrates the practical application of YOLO-based human detection on a real robot, while researchers in the HRI field will find it important through its emphasis on safe and adaptive robot behaviour during human-following tasks. Most importantly, it will critically support the healthcare system, specifically hospitals, where they will be deployed to assist medical doctors during ward rounds to transport medical supplies.

1.9 JUSTIFICATION OF THE STUDY

The rapid advancement of artificial intelligence (AI), computer vision, and autonomous robotics has significantly increased the demand for intelligent mobile robots that can operate safely and effectively in dynamic human environments. The rate of deploying human following robots across industries, especially healthcare, where they are expected to autonomously support human users in performing daily activities, is increasing. As the demand for these applications continues to expand, there is a growing need for robots that can reliably perceive and adapt to continuous changes in human movements.

Despite substantial progress in autonomous navigation, many existing person-following systems remain limited by their dependence on traditional vision algorithms, predefined human trajectories, or simulation-based validation which usually struggle to maintain accurate target tracking under frequent changes in human position. These limitations reduce the reliability and practical applicability of current follow-up robotic systems. This study aims to contribute towards resolving the identified challenges by developing an adaptive vision-based human-following system capable of operating robustly in real-world indoor environments.

1.10 DISSERTATION STRUCTURE

This dissertation is organised into five chapters, each addressing a specific aspect of this research work, from problem formulation to system development, experimental validation, and conclusions.

Chapter One: Introduction

This chapter introduces the research by presenting the background of the study and discussing the increasing importance of autonomous person-following robots in modern service robotics. It identifies the research problem, establishes the motivation for the study, and highlights the existing gaps in current person-following systems research. The chapter further presents the aim and specific objectives of the research, the research questions, the significance

of the study, the scope and limitations, and the justification for the research. Finally, it provides an overview of the organisation of the dissertation.

Chapter Two: Literature Review

This chapter reviews the existing literature relevant to autonomous mobile robotics and intelligent person-following systems. It discusses the theoretical foundations of mobile robots, computer vision, artificial intelligence, deep learning, object detection, object tracking, human–robot interaction, and robot control systems. The chapter also reviews various person-following techniques, including classical computer vision approaches and modern deep learning-based methods, such as YOLO. Existing research is critically analysed to identify their strengths, limitations, and research gaps, thereby establishing the need for the proposed adaptive vision-based person-following framework implemented on a real robotic platform.

Chapter Three: Research Methodology

This chapter describes the methodology adopted for designing, implementing, and evaluating the proposed autonomous person-following system. It presents the overall research design, system architecture, and implementation workflows. This chapter describes the hardware components, including the Robotnik Summit XL mobile robot and onboard camera, as well as the software environment comprising Ubuntu Linux, ROS, Python, OpenCV, and the YOLO deep learning framework. Furthermore, it explains the development of the perception module, detection algorithm, control strategy, ROS communication architecture, and the experimental procedures used to validate the proposed system. The evaluation metrics employed to assess the system performance are also presented.

Chapter Four: Results and Discussion

This chapter presents the results of the implementation of the proposed autonomous person-following robot and discusses the experimental evaluation conducted on the Robotnik Summit XL platform. This chapter analyzes the experimental setup, testing procedures, and scenarios used to evaluate the system under indoor conditions. Quantitative and qualitative results are presented to assess the detection accuracy, tracking performance, response time, following stability, and overall system effectiveness. The obtained results are analysed and discussed with reference to the research objectives and findings reported in related studies.

Chapter Five: Summary and Conclusions

This chapter summarises the entire research by revisiting the objectives, methodology, implementation and key findings. The major conclusions drawn from the experimental results are presented, and the original contributions of this study to the fields of autonomous mobile robotics and vision-based person-following systems are highlighted. This chapter also discusses the limitations encountered during the research and provides recommendations for future work.

CHAPTER TWO

LITERATURE REVIEW

The ability of autonomous person-following robots to perceive, detect, track, and safely follow a human target in real time requires the integration of multiple technologies, including computer vision, deep learning, robot operating system (ROS), and adaptive motion control. As research in these areas continues to evolve, it is essential to examine existing theories, methodologies, algorithms, and experimental findings to establish a comprehensive understanding of the current state of the knowledge. This review aims to provide an understanding of the current state of scientific knowledge, avoid unnecessary duplication of existing work, identify research gaps, and justify the significance of the proposed investigation. This review establishes a theoretical framework that will guide the research methodology, help in choosing appropriate algorithms and technologies, and position the current study within the broader frame of scholarly research.

Through a critical analysis of previous findings, this study aims to identify the limitations of existing person-following systems, including sensitivity to illumination variations, target loss during rapid human movement, unstable motion control, reliance on expensive sensing technologies, and limited validation of many approaches on physical robotic platforms. These identified shortcomings motivate the development of a more adaptive, robust, and experimentally validated vision-based autonomous person-following system.

2.1 THEORETICAL REVIEW

In the context of autonomous person-following robots, the theoretical review encompasses concepts from robotics, artificial intelligence, computer vision, machine learning, autonomous navigation, and control systems. These disciplines collectively provide the knowledge required to design robots capable of perceiving their environment, identifying human targets, making autonomous decisions, and executing safe and reliable motion in dynamic environments.

2.1.1 Autonomous Mobile Robots (AMRs)

Autonomous Mobile Robots (AMRs) are intelligent robotic systems capable of moving through structured or unstructured environments without continuous human intervention. Unlike conventional automated guided vehicles (AGVs), which rely on predefined paths such as magnetic tapes or fixed guide wires, AMRs perceive their surroundings using onboard sensors and make navigation decisions autonomously through artificial intelligence, computer vision, localization, and path-planning algorithms (Siciliano & Khatib, 2016). They integrate mechanical structures, sensing technologies, embedded computing systems, and intelligent control algorithms to perform complex tasks with minimal or no human supervision (Corke, 2017). These robots continuously collect environmental information through cameras, LiDAR, ultrasonic sensors, inertial measurement units (IMUs), and other sensing devices. The acquired data are processed using perception algorithms that enable the robot to recognize objects,

estimate its surroundings, localize itself, plan safe paths, and execute motion commands while adapting to changes in the environment.

2.1.1.1 Evolution of Autonomous Mobile Robots

The evolution of AMRs has been driven by advances in robotics, artificial intelligence, sensing technologies and computational power. The development of AMRs can generally be divided into five major generations.

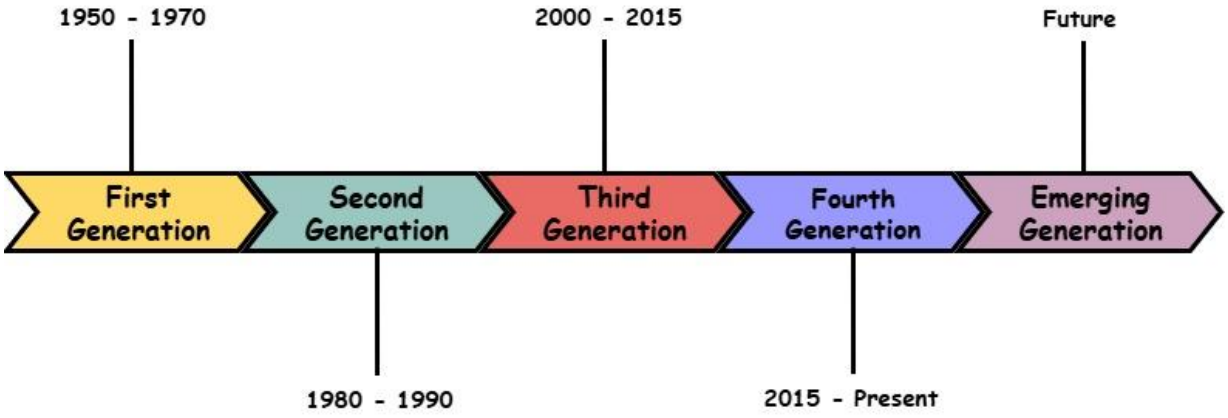


Figure 2.1 - Evolution of Autonomous Mobile Robots

i. First Generation (1950s–1970s): Teleoperated Mobile Robots

The earliest mobile robots were remotely controlled machines that relied entirely on human operators for navigation and task execution. These robots possessed little or no autonomous capability and were primarily developed for hazardous environments, such as nuclear facilities and space exploration. They include

- Shakey Robot (Stanford Research Institute)
- Early remotely operated vehicles (ROVs)

Although pioneering, these robots lacked environmental intelligence and depended heavily on external control.

ii. Second Generation (1980s–1990s): Rule-Based Autonomous Robots

The second generation introduced limited autonomy through rule-based programming and simple sensor integration. These robots could avoid obstacles using ultrasonic and infrared sensors while following predefined behavioural rules. In this period, researchers developed

- Behaviour-based robotics
- Reactive navigation
- Sensor-based obstacle avoidance

Rodney Brooks' subsumption architecture significantly influenced this era by proposing layered behavioural control instead of centralised planning (Brooks, 1986).

iii. Third Generation (2000–2015): Intelligent Navigation

During this era, there were rapid improvements in computing hardware, such as laser scanners, GPS, stereo cameras, and inertial sensors, that enabled robots to perform SLAM, path planning, sensor fusion, and visual navigation. This period also witnessed the emergence of ROS, which standardized robotic software development and accelerated robotics research.

iv. Fourth Generation (2015–Present): AI-Driven Mobile Robots

In this period, robot developments employ deep learning because of its capability to transform robotic perception. Instead of relying on handcrafted image features, robots now utilise algorithms such as convolutional neural networks (CNNs), YOLO, faster R-CNN, SSD, and EfficientDet to achieve human detection, object recognition, semantic segmentation, pose estimation, and scene understanding. This has enabled real-time perception, making autonomous person-following robots significantly more robust than earlier vision systems.

v. Emerging Generation: Human-Centred Intelligent Robots

Modern research focuses on robots capable of collaborating safely with humans in dynamic environments, with specific research directions in human intention prediction, adaptive behaviour learning, social navigation, reinforcement learning, multi-modal perception, and human-aware motion planning. This present study intends to contribute to this emerging generation by proposing an adaptive person-following system capable of responding to continuous changes in human position using deep learning and visual feedback.

2.1.1.2 Types of Robots

Generally, mobile robots are identified according to their locomotion mechanisms. These mechanisms are described as

i. Wheeled Mobile Robots

These robots move using wheels and are the most widely used in indoor environments because of their simplicity, stability, and energy efficiency. Robot products in this category include the Robotnik Summit XL, TurtleBot, and Clearpath Husky. Although, they perform poorly on rough terrains, but they have high speed, low energy consumption, easy control and smooth indoor navigation.



Figure 2.2 - Wheeled Mobile Robot (Wang, 2026)

ii. Tracked Mobile Robots

Tracked robots move using continuous tracks, like military tanks. They are very effective in search and rescue, military operations and mining, this is because they have high terrain adaptability, excellent traction and good stability.



Figure 2.3 - Tracked Mobile Robot (Shandong Guoxing Intelligent Technology Co., Ltd., n.d)

iii. Legged Robots

Legged robots imitate biological locomotion using two, four, or multiple legs. Robotic products such as the Boston Dynamics Spot and ANYmal mimic this type of walking mechanism. They have shown to be efficient in stair climbing and rough terrain mobility even though they have high computational complexity and high energy consumption.



Figure 2.4 - Legged Robot (Humanoid Robotics Technology, 2025)

iv. Aerial Mobile Robots

These robots navigate through the air for tasks such as inspection and surveillance. These robots are quadcopters, UAVs and autonomous drones.



Figure 2.5 - Aerial Mobile Robot (EU Perspectives, 2026)

v. Underwater Mobile Robots

The Autonomous Underwater Vehicles (AUVs) and Remotely Operated Vehicles (ROVs) are specifically designed to operate and perform tasks in water particularly in ocean exploration, pipeline exploration and marine research.



Figure 2.6 - Underwater Mobile Robot (Blueye Robotics, 2026)

2.1.1.3 Applications of Autonomous Mobile Robots

The increasing intelligence of AMRs has expanded their use across numerous and diverse sectors. Some of these sectors are identified with

i. Industrial Automation

AMRs transport materials between production stations, improving efficiency while reducing labour costs. They perform activities in warehouse logistics, factory automation and inventory transportation.

ii. Healthcare

Mobile robots assist healthcare workers through medicine delivery, patient guidance, hospital logistics, and telepresence.

iii. Agriculture

Robots here are applied in crop monitoring, weed detection, precision spraying and autonomous harvesting.

iv. Security and Surveillance

Autonomous robots patrol facilities and detect unauthorized activities using cameras and intelligent vision algorithms.

v. Service Robotics

These are the robots used to perform roles in hotel services, shopping assistants, airport guidance and package deliveries.

vi. Search and Rescue

AMRs assist emergency responders by navigating hazardous environments where human access may be unsafe.

vii. Assistive Robotics

These are mostly identified as person-following robots that assist elderly individuals, hospital patients, people with disabilities, workers carrying heavy loads and doctors during ward rounds.

2.1.1.4 Navigation Methods of Autonomous Mobile Robots

Robot navigation involves determining where the robot is, where it needs to go, and how to move safely to its destination. Common navigation approaches include:

i. Dead Reckoning

Here the robot's wheel encoder measures its last position, direction and time it used to get to its position to estimate its current position. Although simple errors can accumulate over time due to wheel slippage.

ii. Landmark-Based Navigation

The robot identifies known landmarks in the environment to estimate its location.

iii. GPS Navigation

This is suitable for outdoor environments but generally ineffective indoors due to poor satellite signal availability.

iv. SLAM (Simultaneous Localization and Mapping)

Allows robots to build a map while simultaneously estimating their position within it.

v. Vision-Based Navigation

This uses cameras to perceive and interpret the environment. Vision-based navigation is inexpensive, flexible, and particularly effective for person-following applications.

vi. Deep Learning-Based Navigation

This combines computer vision with neural networks to perceive the environment and support intelligent navigation decisions.

2.1.1.5 Challenges of Autonomous Mobile Robots

Despite significant technological advances, AMRs continue to face several challenges in

a. Reliable perception under varying illumination conditions.
b. Partial and complete occlusion of target objects, robust navigation in crowded indoor environments and safe human–robot interaction.
c. Dynamic and unpredictable environments and sensor noise and measurement uncertainty.
d. Real-time processing constraints and limited onboard computational resources.
e. Accurate localization in GPS-denied environments.

f. Energy and battery limitations.

For vision-based person-following robots, additional challenges include maintaining continuous target detection, adapting to rapid human positional changes, preventing oscillatory motion, and ensuring smooth and safe robot behaviour during real-time operation.

2.1.2 Human-Robot Interaction (HRI)

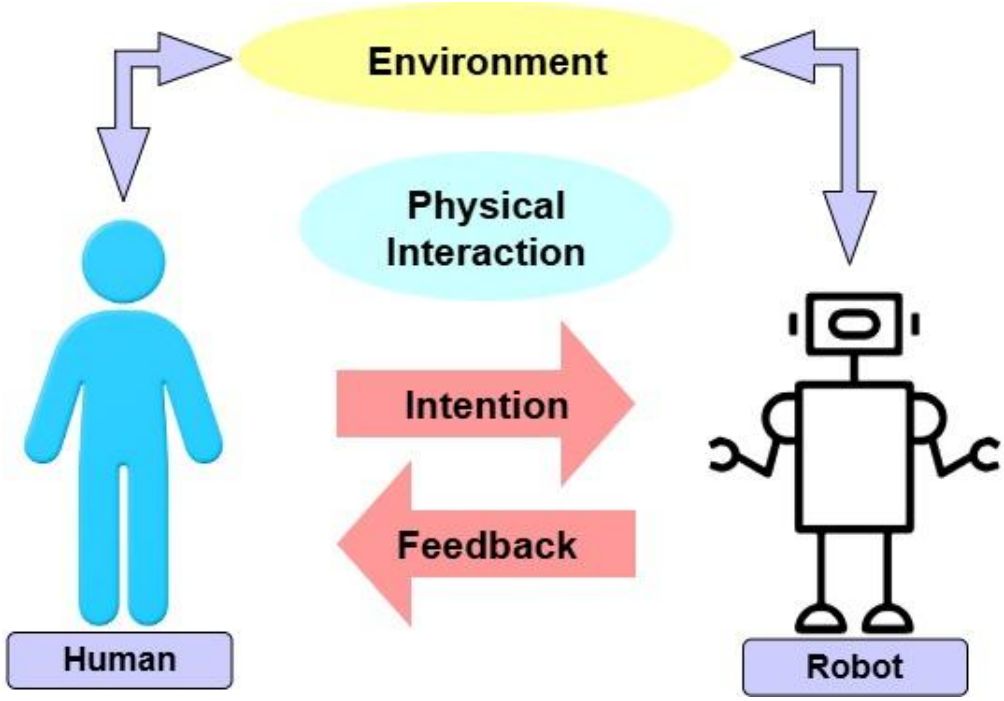


Figure 2.7 - Human-Robot Interaction

Human–Robot Interaction (HRI) is an interdisciplinary field that studies the design, implementation, evaluation, and understanding of interactions between humans and robots. It combines principles from robotics, computer science, artificial intelligence, psychology, cognitive science, human factors engineering, and social sciences to develop robotic systems capable of operating safely, intelligently, and effectively alongside humans. HRI is a concept that enables robots to understand human intentions, interpret human behaviour, and respond appropriately within shared environments. Unlike traditional industrial robots that operate in isolated workspaces, modern autonomous mobile robots increasingly work directly with humans in homes, hospitals, warehouses, offices, and public environments. Consequently, effective interaction has become one of the most important requirements in autonomous robotics.

According to Goodrich & Schultz (2007), HRI encompasses every aspect of communication between humans and robots, including perception, decision-making, navigation, collaboration, safety, and adaptive behaviour. Person-following robots represent one practical application of HRI, where the robot continuously observes a human target, estimates positional changes, and adjusts its movement accordingly to maintain an appropriate following distance.

2.1.2.1 Evolution of Human-Robot Interaction

Human–Robot Interaction has evolved significantly over the past several decades alongside advances in robotics and artificial intelligence. The earliest robots developed during the 1960s and 1970s were primarily industrial manipulators designed for repetitive manufacturing tasks. These robots operated inside protective cages with minimal or no direct interaction with humans because of safety concerns (Siciliano & Khatib, 2016). Human involvement was largely limited to programming and supervising robotic operations.

During the 1980s and 1990s, research shifted towards service robotics, where robots began performing tasks outside structured industrial environments. This period witnessed the development of autonomous navigation algorithms, sensor fusion techniques, and early computer vision systems that allowed robots to perceive and react to their surroundings.

The emergence of machine learning and deep learning in the last decade has dramatically transformed HRI. Modern robots can now recognise human faces, estimate body poses, detect gestures, interpret speech, and follow individuals in real time using advanced vision-based algorithms such as YOLO, Faster R-CNN, SSD, and transformer-based detection networks (Redmon *et al*., 2016; Jocher *et al*., 2023).

2.1.2.2 Human-Centred Robotics

Human-centred robotics is a design philosophy that prioritises human needs, safety, comfort, and usability throughout the development of robotic systems. Rather than requiring humans to adapt to robot behaviour, robots are designed to understand and adapt to human behaviour (Norman, 2013). The key characteristics of human-centred robotic systems are

- Intuitive interaction with minimal user training
- Adaptive behaviour based on human actions
- Safe operation in shared environments
- Reliable perception of human movements
- Predictable and understandable robot behaviour
- Continuous monitoring of user intentions

Vision-based person-following robots are excellent types of human-centred robotic systems because they continuously monitor human movement and dynamically adjust their navigation according to changes in the target's position. Instead of requiring explicit commands, these robots automatically interpret visual information to provide autonomous assistance.

2.1.2.3 Safety Considerations in Human-Robot Interaction

Safety is one of the most critical aspects of HRI especially when autonomous robots operate in human-dense environments. Unlike industrial robots that function within fenced environments, person-following robots must move safely among people while avoiding collisions and maintaining user comfort. Some factors that influence safe HRI, are

- Reliable human detection
- Accurate localisation
- Stable motion control
- Appropriate following distance

- Smooth acceleration and deceleration

Poorly designed control systems may produce oscillatory motion, abrupt turning, excessive acceleration, or unstable navigation, potentially endangering nearby users. Consequently, many mobile robots incorporate safety mechanisms such as speed limits, emergency stop functions, obstacle detection sensors, and velocity smoothing algorithms (ISO 13482, 2014).

2.1.2.4 Person-Following within HRI

Person-following represents one of the most important applications of HRI because it requires continuous perception, decision-making, and adaptive motion in response to dynamic human behavior. Previous person-following systems often relied on vision techniques such as colour segmentation, Haar cascades, Histogram of Oriented Gradients (HOG), optical flow, and template matching. Although computationally efficient, these approaches frequently suffer reduced performance under varying illumination, partial occlusion, background clutter, and rapid human movement.

Recent advances in deep learning have significantly improved person-following performance with Object detection algorithms such as YOLO provide accurate real-time localisation of human targets by predicting bounding boxes directly from image frames. Bounding box information can then be converted into control variables, including horizontal position and apparent target size, which are used to estimate directional movement and following distance.

2.1.3 Autonomous Person-Following Robots

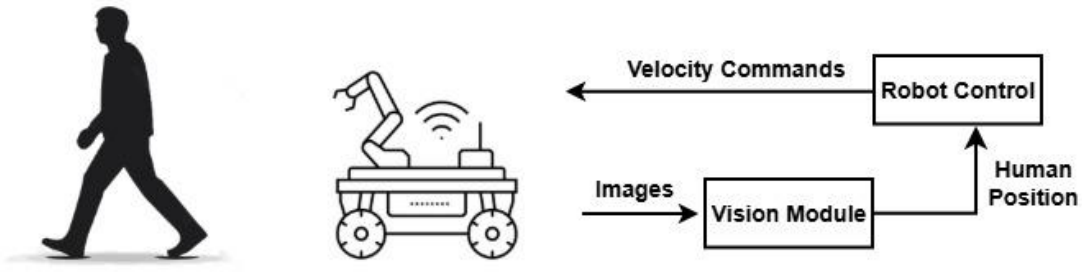


Figure 2.8 - Person-Following Autonomous Robot

Autonomous person-following robots are intelligent mobile robotic systems capable of detecting, identifying, tracking, and following a designated human target without continuous human intervention. These robots integrate sensing, perception, localisation, decision-making, and motion control technologies to maintain a safe and consistent distance from a person while adapting to changes in the person's position and movement within dynamic environments (Chen *et al*., 2020). They continuously perceive their surroundings using onboard sensors such as cameras, LiDAR, ultrasonic sensors, infrared sensors, or depth cameras. Information collected from these sensors is processed by computer vision and artificial intelligence algorithms to estimate the target's position, determine movement direction, and generate

appropriate navigation commands. As a result, these robots can operate with minimal supervision while maintaining continuous interaction with the user.

Person-following robots have become one of the most active research areas within service robotics because they enable natural human–robot collaboration in environments where humans are constantly moving.

2.1.3.1 Historical Development of Autonomous Person-Following Robot

The concept of autonomous person-following robots originated from early research in mobile robotics and autonomous navigation during the 1980s and 1990s. Initial systems relied on simple sensor-based approaches such as infrared beacons, ultrasonic ranging, and radio frequency (RF) transmitters that required the target person to carry dedicated hardware. Although these approaches simplified localisation, they limited the robot's autonomy and flexibility because the target had to wear or carry an external device (Borenstein *et al*., 1996). As computer vision technology advanced during the late 1990s and early 2000s, researchers began exploring vision-based person-following techniques. Early vision systems employed handcrafted image-processing algorithms such as colour segmentation, template matching, Haar cascade classifiers, Histogram of Oriented Gradients (HOG), optical flow, and feature descriptors including SIFT and SURF (Dalal & Triggs, 2005). These methods reduced the need for wearable devices but often struggled with varying illumination, background clutter, occlusions, and complex indoor environments. The emergence of deep learning revolutionised autonomous person-following research as CNNs enabled robots to detect and recognise humans with significantly greater accuracy than traditional computer vision methods. Object detection models such as YOLO, SSD, and Faster R-CNN introduced real-time detection capabilities suitable for mobile robotic applications (Redmon *et al*., 2016; Ren *et al*., 2015; Liu *et al*., 2016).

2.1.3.2 Components of an Autonomous Person-Following Robot

An autonomous person-following robot comprises several interconnected hardware and software components that collectively enable autonomous perception, decision-making, and navigation.

i. Perception System

The perception subsystem acquires information about the surrounding environment and the target person. Cameras are the most used sensors because they provide rich visual information suitable for deep learning-based object detection. Additional sensors such as LiDAR, ultrasonic sensors, depth cameras, inertial measurement units (IMUs), and wheel encoders may complement visual perception to improve localisation accuracy and environmental awareness.

ii. Human Detection Module

The human detection module identifies the presence and location of the target person within captured images. Conventional approaches depended on handcrafted feature extraction techniques, whereas modern systems predominantly utilize deep learning-based object detectors. YOLO has become one of the most widely adopted algorithms because it provides

high detection accuracy while maintaining real-time processing speed, making it particularly suitable for mobile robotic applications (Jocher *et al*., 2023).

iii. Tracking Module

Following detection, the tracking module maintains continuous observation of the selected target across consecutive image frames. Tracking reduces computational complexity by estimating the target's movement rather than performing complete re-detection for every frame. Robust tracking is essential for handling temporary occlusions and rapid human movement.

iv. Decision-making Module

The decision-making module interprets perception results to determine the robot's appropriate response. It estimates the target's relative position, evaluates whether movement is necessary, and generates navigation commands based on predefined behavioral rules or adaptive control algorithms.

v. Motion Control System

The motion controller converts navigation decisions into velocity commands that regulate the robot's linear and angular movement. Controllers may employ proportional (P), proportional–integral–derivative (PID), fuzzy logic, model predictive control (MPC), or reinforcement learning techniques depending on system complexity and performance requirements.

vi. Mobile Robot Platform

The physical robot platform provides locomotion and onboard computational resources. Depending on the application, the robot may utilize differential drive, omnidirectional wheels, tracked mechanisms, or legged locomotion. In this research, the Robotnik Summit XL serves as an experimental platform, offering differential-drive mobility, ROS compatibility, and support for real-time autonomous navigation.

2.1.3.3 General Architecture of Autonomous Person-Following Robots

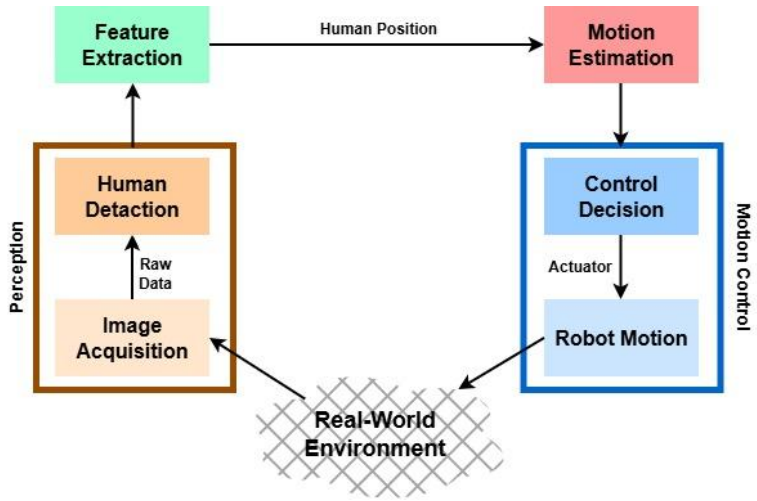


Figure 2.9 - Autonomous Robot Person-Following Component Architecture

Although implementations vary across different research studies, most autonomous person-following robots follow a common layered architecture consisting of perception, processing, planning, control, and actuation components. The process generally follows these stages:

a. **Image Acquisition:** Visual data are captured continuously by the onboard camera.
b. **Human Detection:** Deep learning algorithms identify the person and generate a bounding box around the detected target.
c. **Feature Extraction:** Important positional information, including bounding box centre coordinates and height, is extracted to estimate the person's relative location and distance.
d. **Motion Estimation:** The system determines whether the person is moving forward, backward, left, or right based on positional changes observed over consecutive image frames.
e. **Control Decision:** The navigation controller computes appropriate linear and angular velocities required to maintain the desired following distance.
f. **Robot Motion:** Velocity commands are transmitted through the Robot Operating System (ROS) to the robot's motion controller, enabling continuous autonomous movement.

This modular architecture allows individual subsystems to be modified or upgraded independently while maintaining overall system functionality. Such flexibility is one reason ROS has become the dominant software framework for mobile robotics research.

2.1.3.4 Existing Applications of Autonomous Person-Following Robots

Autonomous person-following robots have been adopted across numerous application domains due to their ability to navigate alongside humans while carrying out assistance tasks.

i. Healthcare and Elderly Assistance

Person-following robots assist elderly individuals, patients, and healthcare workers by transporting medication, carrying personal belongings, and providing continuous mobility support. Their autonomous navigation reduces caregiver workload while improving patient independence.

ii. Warehouse and Logistics

Modern warehouses increasingly utilise autonomous mobile robots to accompany workers during order picking and inventory management. Instead of requiring workers to push heavy carts manually, the robots automatically follow them while transporting goods, improving productivity and reducing physical strain.

iii. Industrial Collaboration

Collaborative mobile robots assist factory workers by delivering tools, transporting materials, and supporting assembly operations. Continuous person-following enables efficient human–robot collaboration while maintaining operational flexibility.

iv. Hospital Service Robots

Hospitals employ autonomous mobile robots for transporting medical supplies, laboratory samples, pharmaceuticals, and equipment between departments. Person-following functionality allows robots to accompany healthcare professionals throughout clinical environments.

v. Security and Surveillance

Security robots equipped with person-following capabilities can escort authorised personnel, monitor suspicious activities, and patrol restricted areas while maintaining continuous visual observation of designated individuals.

vi. Assistive Robotics

Personal assistant robots designed for domestic environments use person-following capabilities to provide everyday assistance, including carrying shopping bags, transporting household items, and supporting individuals with mobility impairments.

vii. Research and Education

Most universities and research laboratories extensively employ person-following robots to investigate computer vision, autonomous navigation, artificial intelligence, reinforcement learning, human–robot interaction, and adaptive control algorithms. Real-world experimental validation contributes significantly to advancing autonomous robotics research.

2.1.4 Computer Vision in Robotics

Computer vision is an aspect of artificial intelligence that enables computers and robotic systems to acquire, process, interpret, and understand visual information obtained from digital images or video sequences. It seeks to emulate the human visual system by enabling machines to recognise objects, estimate distances, interpret scenes, and make intelligent decisions based on visual data (Szeliski, 2022). In robotics, computer vision serves as the robot's main perception mechanism, providing the environmental awareness required for autonomous navigation, object detection, manipulation, and human–robot interaction (Corke, 2017).

The rapid advancement of deep learning has significantly transformed computer vision by enabling robots to perform complex visual tasks such as object detection, semantic segmentation, facial recognition, pose estimation, and autonomous tracking with unprecedented accuracy and speed (Goodfellow *et al*., 2016). Consequently, computer vision has become one of the fundamental technologies underpinning intelligent autonomous mobile robots.

2.1.4.1 Image Acquisition

Image acquisition is the first stage of every computer vision system. It involves capturing visual information from the environment using imaging devices such as digital cameras or vision sensors (Gonzalez & Woods, 2018). The quality of acquired images directly influences the effectiveness of subsequent image processing and object recognition algorithms.

Modern robotic systems utilise various imaging devices including RGB cameras, stereo cameras, depth cameras, thermal cameras, infrared cameras, and event cameras depending on the intended application (Szeliski, 2022). Among these, monocular RGB cameras remain the most widely adopted in autonomous mobile robotics because they are inexpensive, lightweight, and compatible with modern deep learning algorithms (Bradski & Kaehler, 2008).

In autonomous person-following applications, image acquisition involves continuously capturing image frames from the onboard camera while the robot is moving. These image frames provide the visual information required for detecting and locating the target person. Several factors influence image acquisition quality, including camera resolution, frame rate, exposure settings, viewing angle, lens distortion, motion blur, and illumination conditions (Gonzalez & Woods, 2018). Poor image quality can reduce detection accuracy and consequently affect the robot's ability to follow the target reliably.

2.1.4.2 Image Processing

Image processing refers to the computational techniques used to improve, analyse, and transform raw images into forms suitable for computer vision tasks (Gonzalez & Woods, 2018). Before meaningful interpretation can occur, captured images often require preprocessing to remove noise, enhance contrast, improve brightness, or standardise image dimensions.

Common image processing operations include image resizing, colour space conversion, filtering, histogram equalisation, image normalisation, edge detection, thresholding, and morphological transformations (Sonka *et al*., 2015). These operations improve image quality and facilitate subsequent feature extraction and object recognition.

2.1.4.3 Feature Extraction

Feature extraction is the process of identifying distinctive visual characteristics from an image that facilitate object recognition and scene understanding (Szeliski, 2022). Features describe important image properties such as edges, textures, corners, colours, shapes, and spatial relationships. Although, conventional methods are sensitive to illumination changes, viewpoint variations, scale differences, and partial occlusions, deep learning has transformed feature extraction by allowing CNNs to learn hierarchical visual features and complex semantic representations automatically during training (LeCun *et al*., 2015).

2.1.4.4 Object Recognition

Object recognition is the process of identifying and classifying objects present within digital images. In autonomous robotics, object recognition enables robots to interpret their surroundings and make intelligent navigation decisions based on detected objects. As such, object recognition generally involves three interconnected tasks: object classification, object localisation, and object detection. Recent advances in deep learning have significantly improved object recognition accuracy through CNNs (Goodfellow *et al*., 2016).

Several state-of-the-art object detection algorithms have been proposed, including Faster R-CNN (Ren *et al*., 2015), SSD (Liu *et al*., 2016), RetinaNet (Lin *et al*., 2017), EfficientDet (Tan *et al*., 2020), and YOLO (Redmon *et al*., 2016). Among these algorithms, YOLO has become particularly popular in robotics because it simultaneously performs localisation and

classification within a single neural network, thereby achieving high detection accuracy while maintaining real-time processing speeds (Redmon *et al*., 2016).

2.1.4.5 Vision-Based Navigation

Vision-based navigation refers to the use of visual information as the main sensing modality for autonomous robot movement. Instead of relying exclusively on external localisation systems or range sensors, the robot continuously analyses captured images to determine its position relative to surrounding objects and to generate safe navigation commands (Corke, 2017). This generally consists of four major stages: environmental perception, object detection and localisation, motion estimation, and path planning or motion control (Siciliano & Khatib, 2016). These stages collectively enable robots to perceive their environment, estimate target positions, and navigate autonomously. Although, several vision-based navigation techniques have been developed, including visual servoing, visual odometry, SLAM, marker-based navigation, and deep learning-based navigation (Corke, 2017).

2.1.4.6 Challenges of Computer Vision in Robotics

Despite remarkable advances in artificial intelligence, computer vision remains one of the most challenging components of autonomous robotics because robots must interpret complex visual environments while maintaining real-time performance (Szeliski, 2022). One major challenge is illumination variation, where changes in lighting intensity, shadows, reflections, and low-light conditions significantly affect image quality and reduce object detection accuracy (Sonka *et al*., 2015). Similarly, partial and complete occlusions frequently occur when the target person becomes temporarily hidden behind furniture, walls, or other individuals, making continuous tracking difficult as well as rapid human movement which often introduces motion blur and abrupt positional changes that may temporarily disrupt object detection and tracking (Milan *et al*., 2016). Another important challenge is background clutter, where visually similar objects within complex indoor environments increase the probability of false detections (Dalal & Triggs, 2005).

Computer vision systems must also cope with scale variation, where the apparent size of the target changes continuously as the person moves closer to or farther from the camera (Redmon *et al*., 2016). Furthermore, camera motion caused by robot movement introduces vibration and changing viewpoints, which may degrade image quality and tracking stability (Corke, 2017). Another significant challenge is the need for real-time processing. Autonomous robots must perform image acquisition, object detection, motion estimation, decision-making, and motion control within fractions of a second to ensure smooth navigation. This imposes substantial computational demands, particularly on embedded robotic platforms with limited processing resources (Goodfellow *et al*., 2016). Lastly, computer vision algorithms often struggle to generalise across different environments because variations in lighting conditions, furniture arrangements, backgrounds, and human appearance may reduce detection performance outside the training environment (Jocher *et al*., 2023).

2.1.5 Machine Learning

Machine learning is the core subset of artificial intelligence that enables computer systems to learn patterns from data and improve their performance on specific tasks without being explicitly programmed for every possible scenario. Rather than relying solely on manually designed rules, machine learning algorithms identify relationships within data and use these relationships to make predictions, classifications, or decisions. In robotics, machine learning has become a fundamental technology for enabling autonomous perception, adaptive control, navigation, and intelligent decision-making in dynamic environments.

2.1.5.1 Supervised Learning

Supervised learning is the most widely used machine learning paradigm, where a model is trained using labelled datasets consisting of input-output pairs. During training, the algorithm learns the relationship between the input data and the corresponding labels, enabling it to predict the correct output for previously unseen data. Common supervised learning tasks include image classification, object detection, speech recognition, and regression problems. Popular supervised learning algorithms include Support Vector Machines (SVM), Decision Trees, Random Forests, Artificial Neural Networks (ANNs), and Convolutional Neural Networks (CNNs) (Goodfellow *et al*., 2016).

In robotics, supervised learning has significantly improved perception systems by enabling robots to accurately identify people, objects, and environmental features. Modern object detectors such as the YOLO family are examples of supervised deep learning models trained on large annotated image datasets. These models learn visual features directly from labelled data, allowing robots to perform reliable real-time object detection under varying environmental conditions. In autonomous person-following applications, supervised learning enables the robot to distinguish human targets from surrounding objects with high accuracy, thereby improving tracking reliability and navigation performance (Redmon *et al*., 2016; Jocher *et al*., 2023). Despite its effectiveness, supervised learning requires extensive labelled datasets, which are often expensive and time-consuming to produce. The quality of the trained model is also highly dependent on the diversity and representativeness of the training data. Poor-quality datasets may reduce the model's ability to generalize to unfamiliar environments or lighting conditions (Goodfellow *et al*., 2016).

2.1.5.2 Unsupervised Learning

Unsupervised learning involves training algorithms using unlabelled datasets, where the objective is to discover hidden structures, relationships, or patterns within the data without predefined output labels. Unlike supervised learning, the algorithm independently identifies similarities among data samples through clustering, dimensionality reduction, or feature extraction techniques (Bishop, 2006). Common unsupervised learning algorithms include K-Means Clustering, Hierarchical Clustering, Gaussian Mixture Models (GMM), Principal Component Analysis (PCA), and Autoencoders. These methods are widely used for data exploration, anomaly detection, environment mapping, and feature learning.

Also, unsupervised learning supports autonomous exploration by enabling robots to recognise environmental patterns, group similar observations, and build internal representations of unknown environments. For instance, robots may cluster sensor measurements to distinguish different terrain types or identify obstacles without requiring manually labelled training data. Feature extraction techniques such as PCA also reduce computational complexity while preserving important information required for navigation and localisation (Murphy, 2019). Although unsupervised learning reduces the dependence on labelled datasets, the discovered patterns may not always correspond to meaningful real-world categories. Consequently, unsupervised learning is frequently combined with supervised learning to enhance perception systems in intelligent robotic applications.

2.1.5.3 Reinforcement Learning

Reinforcement learning is another machine learning paradigm in which an intelligent agent learns optimal behaviour through continuous interaction with its environment. Instead of learning from labelled examples, the agent performs actions, observes the resulting state changes, receives reward or penalty signals, and gradually improves its decision-making strategy by maximising cumulative rewards over time (Sutton & Barto, 2018). A reinforcement learning system consists of four primary components,

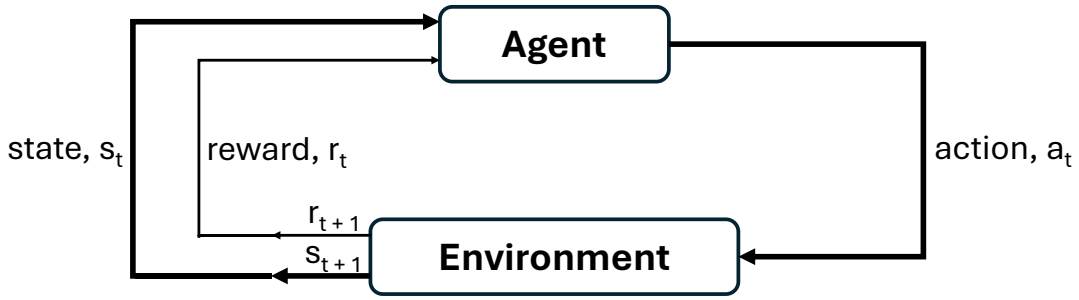


Figure 2.10 - Reinforcement Learning Block Diagram (Sharanya et al., 2023)

i. An *agent*, which performs actions.
ii. An *environment*, within which the agent operates.
iii. A *policy*, defining the agent's behaviour.
iv. A *reward function*, measuring the quality of each action.

Through repeated interactions, the agent develops an optimal policy that maximises long-term performance. Within robotics, reinforcement learning enables robots to acquire complex behaviours without explicitly programmed control rules. Robots can learn obstacle avoidance, manipulation tasks, autonomous navigation, and adaptive motion planning through continuous trial-and-error interactions. Reinforcement learning has demonstrated impressive performance in dynamic environments where deterministic control strategies may struggle to adapt (Kober *et al*., 2013). However, reinforcement learning generally requires large numbers of training iterations and substantial computational resources. Direct training on physical robots can also be impractical because repeated exploration may damage hardware or create unsafe operating conditions. Consequently, reinforcement learning is often trained in simulation before transferring learned policies to physical robots (Sutton & Barto, 2018). Modern reinforcement

learning methods include Q-Learning, Deep Q Networks (DQN), Policy Gradient methods, and Actor–Critic algorithms.

2.1.5.4 Applications of Machine Learning in Robotics

Machine learning has transformed modern robotics by enabling robots to perceive, understand, and interact intelligently with complex environments. Unlike conventional rule-based robotic systems, machine learning allows robots to adapt their behaviour based on sensory information and previous experiences, making them more robust in dynamic real-world scenarios (Murphy, 2019). Key applications of machine learning in robotics include,

- **Object Detection and Recognition:** Deep learning models such as YOLO enable robots to accurately identify people, vehicles, and objects in real time for navigation and interaction (Jocher *et al*., 2023).
- **Human Detection and Tracking:** Machine learning algorithms facilitate reliable person detection and continuous tracking, enabling applications such as autonomous person-following, surveillance, and assistive robotics.
- **Autonomous Navigation:** Machine learning enhances localisation, path planning, and obstacle avoidance by enabling robots to interpret complex sensor data and adapt to changing environments.
- **Robot Manipulation:** Learning-based methods improve robotic grasping, object handling, and manipulation by allowing robots to learn effective strategies from demonstrations or experience.
- **Human–Robot Interaction:** Machine learning enables robots to recognise gestures, body poses, facial expressions, and human activities, thereby improving natural interaction between humans and robots.
- **Predictive Decision Making:** Intelligent robots employ machine learning to anticipate environmental changes and select optimal actions based on learned behavioural patterns.

2.1.6 Deep Learning

Deep learning is a branch of machine learning that enables computers to automatically learn hierarchical representations of data through multiple layers of artificial neural networks. Unlike conventional machine learning methods that rely on manually designed features, deep learning algorithms automatically extract low-level and high-level features directly from raw data. This capability has significantly improved the performance of computer vision, speech recognition, natural language processing, and autonomous robotics (LeCun *et al*., 2015; Goodfellow *et al*., 2016).

2.1.6.1 Artificial Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by the biological structure of the human brain. They consist of interconnected processing units called neurons that receive inputs, perform weighted computations, apply activation functions, and produce outputs. Through iterative training using large datasets, ANNs learn complex relationships

between input and output data by adjusting their connection weights to minimise prediction errors (Haykin, 2009). A typical ANN comprises three primary layers

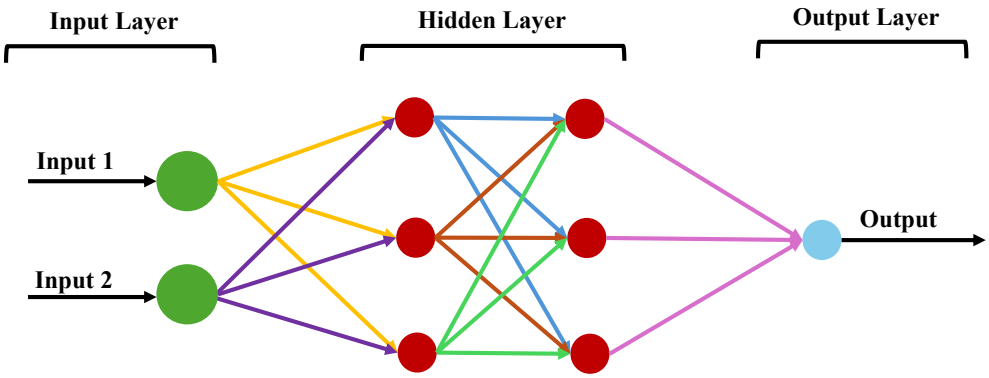


Figure 2.11 - Neural Network Architecture

i. *Input layer*, which receives raw data from sensors or images.
ii. *Hidden layers*, where feature extraction and nonlinear learning occur.
iii. *Output layer*, which produces the final prediction or classification.

The performance of ANNs improves with the availability of larger datasets and increased computational power. However, conventional ANNs are generally less effective for image processing because they treat images as one-dimensional vectors, thereby losing important spatial information. This limitation motivated the development of CNNs, which are specifically designed for visual recognition tasks (Goodfellow *et al*., 2016). In robotic perception, ANNs have been applied to sensor fusion, obstacle classification, motion prediction, autonomous navigation, and decision-making. They provide the learning foundation upon which more advanced deep learning architectures are developed (Haykin, 2009).

2.1.6.2 Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are specialised deep learning architectures developed specifically for image and video analysis. CNNs preserve the spatial relationships within images by employing convolutional filters that automatically learn visual features such as edges, textures, corners, shapes, and semantic object characteristics from training data (LeCun *et al*., 1998). A typical CNN consists of several layers, including,

- Convolutional layers for feature extraction
- Activation layers introducing nonlinear behaviour
- Pooling layers for dimensionality reduction
- Fully connected layers for final classification

The introduction of AlexNet by Krizhevsky *et al*. (2012) demonstrated the superiority of CNNs over conventional computer vision methods by achieving unprecedented performance on the ImageNet Large Scale Visual Recognition Challenge (ILSVRC). Since then, CNN architectures such as VGGNet, GoogLeNet, ResNet, EfficientNet, and the YOLO family have become fundamental components of modern robotic vision systems. In autonomous mobile robotics, CNNs enable robots to perform functions such as human detection, object recognition,

semantic scene understanding, visual localisation, autonomous navigation, person-following, obstacle recognition, and human activity recognition. These capabilities make CNNs the backbone of intelligent robotic perception systems.

2.1.6.3 Advantages of Deep Learning over Classical Computer Vision

Classical computer vision methods rely on manually designed image features such as colour histograms, edge descriptors, Haar features, Histogram of Oriented Gradients (HOG), Scale-Invariant Feature Transform (SIFT), and Speeded-Up Robust Features (SURF). Although these approaches have been successfully applied in various applications, they often experience significant performance degradation when environmental conditions change (Dalal & Triggs, 2005; Lowe, 2004). Deep learning provides several important advantages over classical vision approaches:

i. Automatic feature extraction without manual engineering.
ii. Higher object detection accuracy.
iii. Greater robustness to illumination variations.
iv. Improved performance under partial occlusions.
v. Better handling of scale and viewpoint changes.
vi. Capability to learn complex visual representations from large datasets.
vii. Real-time processing when accelerated by modern GPUs.
viii. Superior generalisation to unseen environments.

These advantages have made deep learning the preferred choice for modern robotic perception, particularly in applications requiring high reliability and adaptability, such as autonomous driving, warehouse robotics, surveillance, and intelligent person-following robots (Redmon *et al*., 2016; Bochkovskiy *et al*., 2020). Nevertheless, deep learning also presents several challenges. Training deep neural networks requires large annotated datasets, significant computational resources, and powerful graphics processing units (GPUs). Furthermore, model performance depends on the quality and diversity of training data, and inference speed may be constrained on low-power embedded robotic platforms (Goodfellow *et al*., 2016).

2.1.6.4 Deep Learning in Robotic Perception

Robotic perception involves enabling robots to sense, interpret, and understand their surrounding environment using information acquired from onboard sensors. Deep learning has substantially advanced robotic perception by enabling robots to identify objects, estimate human positions, recognise activities, understand scenes, and make intelligent navigation decisions in real time (LeCun *et al*., 2015). Modern robotic perception systems typically integrate deep learning with RGB cameras, depth cameras, LiDAR, radar, inertial sensors, and ultrasonic sensors to achieve robust environmental awareness. CNN-based object detectors provide semantic understanding of scenes, while tracking algorithms maintain object identities across successive video frames (Szeliski, 2022).

Within autonomous person-following systems, deep learning enables continuous human detection despite changes in body posture, orientation, scale, illumination, and background clutter. Instead of relying on engineered descriptors, CNN-based detectors learn discriminative human features from extensive datasets, thereby improving detection robustness in real-world

environments (Redmon *et al*., 2016). Recent advances have further improved robotic perception through one-stage object detectors such as the YOLO family, which combines high detection accuracy with real-time inference. The latest YOLO models have become particularly attractive for autonomous robotics because they achieve low computational latency while maintaining excellent detection performance. Consequently, YOLO-based perception systems have been widely adopted in mobile robotics, autonomous vehicles, surveillance systems, and intelligent service robots (Jocher *et al*., 2023).

2.1.7 Object Detection

Object detection is an important task in computer vision that involves identifying the presence of objects within an image and accurately determining their locations using bounding boxes while assigning them to predefined object categories. Unlike image classification, which predicts only the overall class of an image, object detection simultaneously performs localisation and classification, making it one of the most important perception techniques for autonomous robotic systems (Szeliski, 2022).

Object detection serves as the foundation for numerous intelligent robotic applications, including autonomous navigation, human-following robots, surveillance systems, warehouse automation, industrial inspection, autonomous vehicles, and service robotics. For person-following robots, object detection provides continuous information regarding the target's position, size, and movement, allowing the robot to estimate the person's location and adjust its motion accordingly (Siciliano & Khatib, 2016). In the proposed system, the YOLO object detection algorithm is employed to identify a human target and generate bounding box coordinates, which are subsequently used by the motion controller to estimate the relative distance and direction of the person.

2.1.7.1 Conventional Object Detection Approaches

Prior to the emergence of deep learning, object detection primarily relied on engineered image features combined with conventional machine learning classifiers. These approaches required manually designed descriptors capable of capturing object characteristics such as edges, corners, gradients, texture, and shape. The descriptors are typically combined with classifiers such as Support Vector Machines (SVMs), AdaBoost, or Random Forests to distinguish objects from background regions. One of the earliest successful human detection methods was HOG combined with a linear SVM. HOG computes gradient orientation histograms over local image regions, producing descriptors that effectively capture human silhouettes. Furthermore, Dalal and Triggs (2005) demonstrated that HOG descriptors significantly improved pedestrian detection accuracy compared with previous handcrafted methods, making the approach widely adopted in robotics and surveillance applications.

Another influential conventional approach was the Viola–Jones detector, which utilised Haar-like features and an AdaBoost cascade classifier for rapid face detection (Viola & Jones, 2001). Although computationally efficient, its performance deteriorated under varying illumination, scale changes, and occlusions. Similarly, SIFT and SURF offered robustness to image scale and rotation but required considerable computational resources, limiting their suitability for real-time embedded robotic platforms (Lowe, 2004; Bay *et al*., 2008).

Despite their historical importance, these detection methods possess several limitations such as their dependence on manually created features. This restricts their ability to generalise across complex environments, while performance is often degraded by changes in lighting conditions, background clutter, camera viewpoint, and partial object occlusions (Szeliski, 2022). Furthermore, these methods require extensive feature engineering and separate stages for feature extraction and classification, increasing system complexity and reducing processing speed. For autonomous person-following robots operating in dynamic indoor environments, these limitations frequently lead to unstable target detection, intermittent tracking, and reduced robustness during continuous human movement.

2.1.7.2 Deep Learning-Based Object Detection Approaches

Current deep learning has the capability of transforming object detection through end-to-end learning directly from large image datasets. Deep learning detectors automatically learn hierarchical visual features through multiple convolutional layers, eliminating the need for manually designed feature descriptors. This capability has substantially improved detection accuracy, robustness, and generalisation across diverse environments (Goodfellow *et al*., 2016).

Meanwhile, modern object detection algorithms are generally classified into two categories: two-stage detectors and one-stage detectors. Two-stage detectors first generate candidate object regions and subsequently classify each region. Representative examples include Region-based CNN, Faster R-CNN, and Mask R-CNN (Girshick *et al*., 2014; Ren *et al*., 2015; He *et al*., 2017). These approaches achieve high detection accuracy but typically require greater computational resources, making them less suitable for real-time robotic applications. And one-stage detectors perform localisation and classification simultaneously within a single network architecture, resulting in significantly faster inference speeds. Prominent examples are Single Shot Detector (SSD), RetinaNet, and the YOLO family of algorithms (Liu *et al*., 2016; Lin *et al*., 2017; Redmon *et al*., 2016). These detectors are particularly attractive for autonomous robots because they provide high-speed inference while maintaining competitive detection accuracy. Among these methods, YOLO has become one of the most widely adopted object detection algorithms for robotic perception. Unlike region proposal-based detectors, YOLO processes the entire image in a single forward pass through the neural network, simultaneously predicting object classes, confidence scores, and bounding box coordinates (Redmon *et al*., 2016). Subsequent versions, including YOLOv3, YOLOv5, YOLOv7, YOLOv8, and YOLOv11, have progressively improved localisation accuracy, detection speed, robustness to small objects, and computational efficiency (Jocher *et al*., 2023).

2.1.8 YOLO (You Only Look Once) Object Detection

2.1.8.1 Development of YOLO

The rapid advancement of deep learning has significantly enabled YOLO to transform object detection in computer vision as a single regression problem. This unified approach enables the network to simultaneously predict object locations, class labels, and confidence

scores from a single input image, thereby achieving real-time detection speeds while maintaining high detection accuracy YOLO (Redmon *et al*., 2016).

YOLO was first introduced by Joseph Redmon and colleagues in 2016 to address the computational limitations of existing region-based object detectors such as R-CNN, Fast R-CNN, and Faster R-CNN. Earlier detection algorithms required multiple stages of feature extraction, region proposal generation, and classification, making them computationally intensive and unsuitable for real-time robotic applications (Girshick *et al*., 2014; Ren *et al*., 2015). By treating object detection as a single-stage regression problem, YOLO significantly reduced inference time while providing competitive detection performance (Redmon *et al*., 2016).

Since its introduction, YOLO has become one of the most influential object detection frameworks in computer vision, with successive versions improving detection accuracy, robustness, computational efficiency, and deployment on embedded robotic platforms. Continuous improvements have made YOLO particularly suitable for applications requiring rapid perception, including autonomous vehicles, surveillance systems, healthcare, industrial automation, and mobile robotics (Bochkovskiy *et al*., 2020; Jocher *et al*., 2023).

2.1.8.2 YOLO Architecture

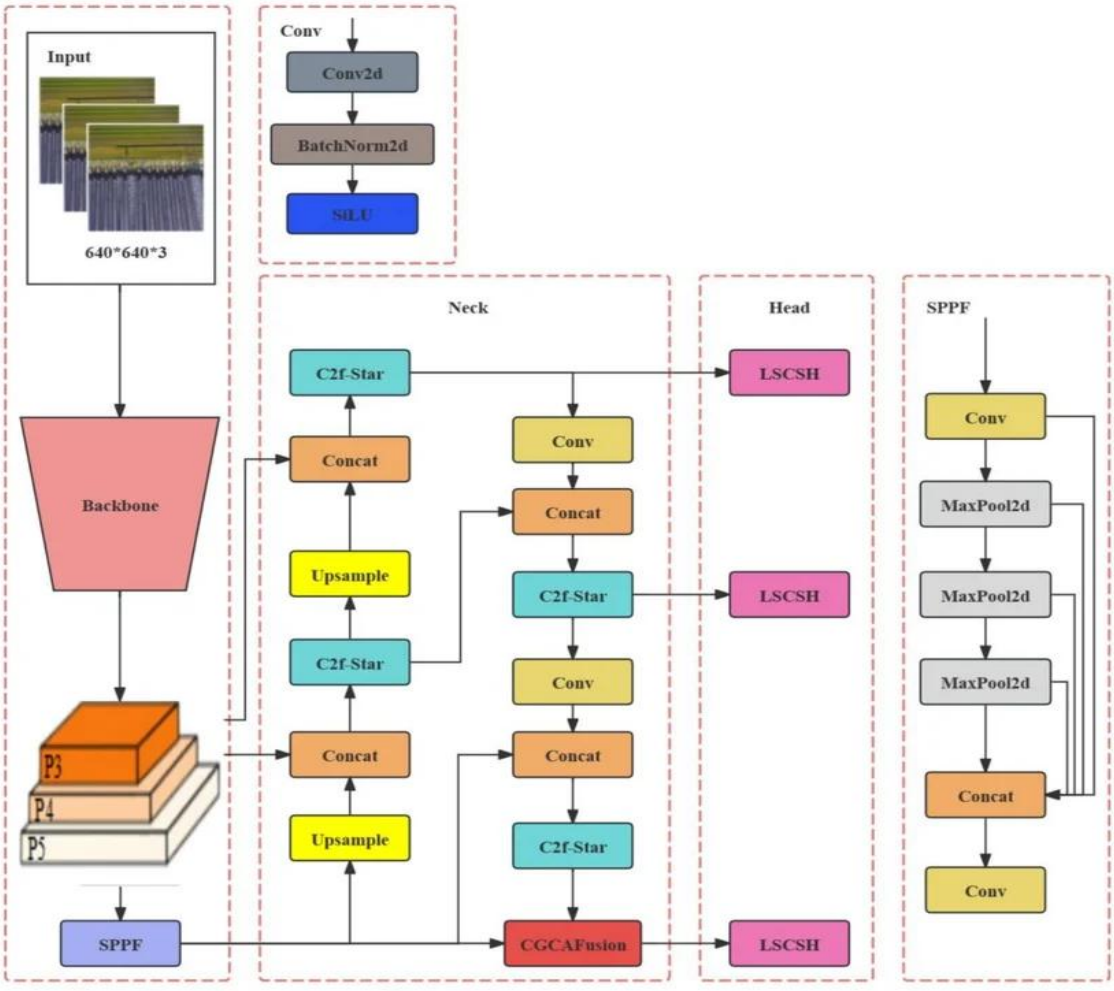


Figure 2.12 - Model Structure of YOLOv8 (Zhang et al., 2024)

The architecture of YOLO is based on a single-stage CNN that processes an entire image in one forward pass. Rather than first generating candidate regions, the network divides the input image into a grid and directly predicts bounding boxes, confidence scores, and object class probabilities for each grid cell. This end-to-end architecture allows the model to learn both object localisation and classification simultaneously (Redmon *et al*., 2016). A typical YOLO network consists of three major components,

i. Backbone

The backbone extracts hierarchical visual features from the input image using multiple convolutional layers. Recent YOLO versions employ advanced feature extraction networks such as CSPDarknet and other optimized convolutional architectures that improve feature representation while reducing computational cost (Bochkovskiy *et al*., 2020).

ii. Neck

The neck combines multi-scale feature maps generated by the backbone using structures such as the Feature Pyramid Network (FPN) and Path Aggregation Network (PAN). This stage enhances the network's ability to detect both small and large objects by fusing semantic information from different feature levels (Lin *et al*., 2017).

iii. Head

The detection head predicts the coordinates of bounding boxes, confidence scores indicating the probability that an object exists, and the corresponding object class. During inference, Non-Maximum Suppression (NMS) removes duplicate detections by retaining only the bounding boxes with the highest confidence values (Redmon *et al*., 2016). The simplicity of this architecture enables YOLO to achieve exceptionally high processing speeds, often exceeding 30 frames per second on modern graphics processing units (GPUs), making it ideal for real-time robotic perception.

2.1.8.3 Evolution of YOLO (YOLOv1-YOLOv11)

Since its first release, the YOLO framework has undergone substantial improvements, with each version addressing limitations of previous models while enhancing speed, accuracy, and deployment flexibility.

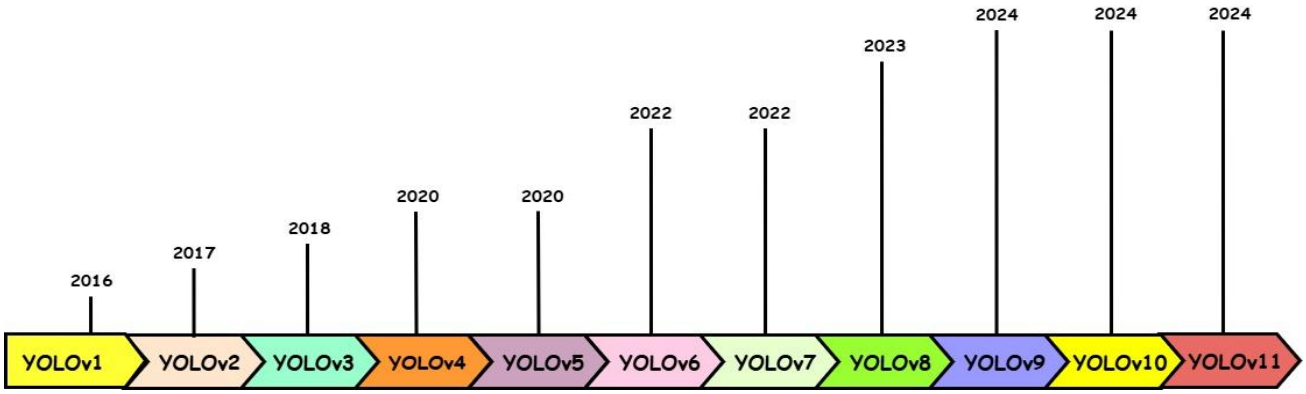


Figure 2.13 - Evolution of YOLO

i. YOLOv1 (2016)

This was where the concept of single-stage object detection was introduced by predicting object locations and class probabilities simultaneously from the entire image. Although revolutionary in speed, YOLOv1 struggled with small objects and localisation accuracy (Redmon *et al*., 2016).

ii. YOLOv2 (YOLO9000) (2017)

This version experienced improved detection accuracy through batch normalization, anchor boxes, higher-resolution training, and multi-scale prediction. YOLO9000 further expanded the detection capability to over 9,000 object categories using joint training (Redmon & Farhadi, 2017).

iii. YOLOv3 (2018)

This version introduced Darknet-53 as a more powerful backbone and incorporated multi-scale feature prediction, significantly improving small object detection while maintaining real-time performance (Redmon & Farhadi, 2018).

iv. YOLOv4 (2020)

This was developed by Bochkovskiy *et al*., (2020), YOLOv4 integrated CSPDarknet53, Mish activation functions, Spatial Pyramid Pooling (SPP), and numerous training optimisations known as the "Bag of Freebies" and "Bag of Specials," leading to substantial improvements in both speed and accuracy.

v. YOLOv5 (2020)

This was developed by Ultralytics, YOLOv5 introduced a PyTorch implementation with simplified deployment, automatic anchor generation, mixed precision training, and multiple model sizes (nano, small, medium, large, and extra-large). Although not an official continuation by the original YOLO authors, it gained widespread adoption due to its ease of use and excellent real-time performance (Jocher *et al*., 2023).

vi. YOLOv6 (2022)

This is an optimised version specifically for industrial applications by improving inference speed, quantisation support, and deployment on edge devices while maintaining competitive detection accuracy.

vii. YOLOv7 (2022)

It focused on trainable optimisation techniques, efficient feature aggregation, and model scaling strategies, becoming one of the fastest and most accurate real-time object detectors available at its release (Wang *et al*., 2022).

viii. YOLOv8 (2023)

An anchor-free detection architecture was introduced here, with improved backbone design, enhanced segmentation capability, pose estimation support, and simplified training through the Ultralytics framework. YOLOv8 demonstrates superior detection performance while reducing computational complexity, making it particularly suitable for robotics and autonomous systems (Jocher *et al*., 2023). Thus, the reason it was used for the development of this research project.

ix. YOLOv9 (2024)

YOLOv9 introduced Programmable Gradient Information (PGI) and the Generalized Efficient Layer Aggregation Network (GELAN), significantly improving feature learning, gradient flow, and computational efficiency, especially for real-time detection tasks.

x. YOLOv10 (2024)

The YOLOv10 further optimised end-to-end object detection by eliminating the need for Non-Maximum Suppression (NMS) during inference, thereby reducing latency while maintaining high detection accuracy. The architecture emphasised holistic efficiency, making it attractive for real-time embedded applications.

xi. YOLOv11 (2024)

The YOLOv11 represents the latest evolution of the YOLO family, introducing improved backbone efficiency, enhanced feature aggregation, greater detection accuracy, and lower computational requirements. It further improves performance on small object detection, robustness under challenging environmental conditions, and deployment on resource-constrained robotic systems. These improvements make YOLOv11 particularly suitable for autonomous mobile robots requiring reliable, low-latency human detection in dynamic indoor environments.

2.1.8.4 Advantages of YOLO

Some advantages that make YOLO one of the most suitable object detection algorithms for autonomous mobile robotics are its single-stage architecture that enables real-time detection, thereby allowing robots to perceive their environment continuously without significant processing delays (Redmon *et al*., 2016). YOLO achieves an excellent balance between detection accuracy and computational efficiency, enabling deployment on embedded computing platforms commonly used in robotic systems (Bochkovskiy *et al*., 2020). Also, the algorithm performs simultaneous localisation and classification, reducing computational overhead while simplifying the overall perception pipeline. Furthermore, it supports multi-object detection, allowing robots to detect multiple people or surrounding objects within a single image frame. Recent versions demonstrate improved robustness against varying viewpoints, illumination changes, scale variations, and partial occlusions, making them suitable for complex real-world environments encountered during mobile robot navigation.

2.1.8.5 Limitations of YOLO

Despite its numerous advantages, YOLO also presents several limitations. One of its limitations is reduced detection accuracy under severe illumination changes or extremely low-light conditions, where image quality deteriorates significantly. The algorithm may also experience degraded performance during heavy occlusion, where a person's body is largely hidden behind surrounding objects. Although later versions have substantially improved small-object detection, detecting very distant objects remains more challenging than detecting large, nearby targets. YOLO additionally requires substantial annotated datasets for effective training, making dataset preparation both time-consuming and computationally demanding. And lastly, achieving high inference speed typically requires GPU acceleration, although lightweight variants can operate on CPUs with reduced performance.

2.1.8.6 Why YOLO is Suitable for Real-Time Robotics

YOLO is particularly well suited for autonomous person-following robots because it satisfies the stringent requirements of real-time robotic perception. Autonomous robots require continuous detection of human targets while simultaneously executing navigation and motion control. Any delay in perception may result in unstable robot behaviour or loss of the target. The ability of YOLO to perform rapid and accurate human detection enables continuous estimation of a person's position, which can subsequently be used to generate motion commands for robot control. Its high frame rate supports smooth forward and backward motion while allowing the robot to respond quickly to changes in human position. Furthermore, the compatibility of modern YOLO implementations with ROS facilitates seamless integration into robotic perception pipelines.

2.1.9 Object Tracking

Object tracking is a fundamental task in computer vision that involves continuously locating and estimating the trajectory of an object across successive image frames after it has been detected. Object tracking maintains the identity and spatial location of the target over time, enabling continuous monitoring of its movement (Smeulders *et al*., 2014). In robotics, object tracking allows autonomous systems to perceive dynamic environments, estimate object motion, and make informed navigation decisions. It is particularly important in autonomous person-following robots, where the robot must continuously estimate the changing position of a human target while maintaining a safe following distance and appropriate orientation (Yilmaz *et al*., 2006). The key objective of object tracking is to determine the position, velocity, and trajectory of moving objects despite variations in illumination, object appearance, background clutter, occlusions, and camera motion. Effective tracking enhances the robot's situational awareness and provides the continuous positional information required for real-time motion planning and control (Szeliski, 2022).

2.1.9.1 Tracking by Detection

Tracking by detection is the dominant paradigm used in modern visual tracking systems. In this approach, an object detector first identifies the objects of interest in every video frame,

after which a tracking algorithm associates detections belonging to the same object across consecutive frames (Bewley *et al*., 2016). Rather than relying solely on appearance models, tracking by detection continuously updates the target location based on new detections, making it more robust to environmental changes and temporary tracking failures. Tracking algorithms such as SORT (Simple Online and Realtime Tracking), DeepSORT, and ByteTrack combine detection outputs with motion prediction techniques, including the Kalman Filter and data association algorithms, to maintain consistent object identities over time (Wojke *et al*., 2017; Zhang *et al*., 2022). Tracking by detection offers several advantages for robotic applications. It is computationally efficient, capable of handling temporary occlusions, and allows the tracker to recover when an object reappears after being briefly lost. Consequently, it has become the preferred tracking framework for autonomous navigation, surveillance, intelligent transportation, and human-following robots.

2.1.9.2 Single Object Tracking

Single object tracking (SOT) focuses on maintaining the location of one specific target throughout a video sequence after its initial detection or manual selection (Yilmaz *et al*., 2006). The tracker continuously estimates the target's position by analysing appearance, motion, or feature similarity between consecutive frames. Because only one object is tracked, computational complexity is relatively low, making single-object tracking suitable for real-time robotic applications. Conventional single object tracking algorithms include correlation filters, optical flow methods, template matching, and feature-based tracking techniques. More recently, deep learning-based trackers such as Siamese Neural Networks have significantly improved tracking robustness under challenging conditions including illumination changes, scale variation, and partial occlusion (Bertinetto *et al*., 2016). Single object tracking is particularly relevant to autonomous person-following robots because the robot is generally required to follow only one designated individual.

2.1.9.3 Multi-Object Tracking

Multi-object tracking (MOT) extends the tracking problem by simultaneously maintaining the identities and trajectories of multiple objects within the scene (Milan *et al*., 2016). Unlike single-object tracking, MOT must solve both object localisation and identity association, ensuring that each detected object retains a consistent identification number even when objects cross paths or become temporarily occluded.

Modern multi-object tracking systems commonly employ the tracking by detection technique, combining deep learning object detectors with motion estimation algorithms such as the Kalman Filter and data association methods including the Hungarian Algorithm. Recent algorithms such as DeepSORT and ByteTrack further incorporate deep appearance features to improve identity preservation under crowded conditions (Wojke *et al*., 2017; Zhang *et al*., 2022). Multi-object tracking has numerous applications in autonomous driving, intelligent surveillance, crowd monitoring, sports analytics, warehouse automation, and collaborative robotics. However, its computational complexity is considerably higher than single-object tracking due to the need for simultaneous identity management and association. Although multi-object tracking represents an active area of research, it falls outside the scope of the

present study. This research is specifically designed for single person following within indoor environments, where only one target is detected and tracked at any given time.

2.1.9.4 Challenges of Object Tracking

Despite significant advancements in computer vision and deep learning, object tracking remains a challenging problem in dynamic environments. One major challenge is partial and complete occlusion, where the target becomes temporarily hidden by surrounding objects or other people. Occlusion can cause the tracker to lose the target or incorrectly associate it with another object (Smeulders *et al*., 2014). Another challenge is illumination variation, where changes in lighting conditions alter the appearance of the tracked object, reducing detection and tracking accuracy. Indoor environments often contain shadows, reflections, and non-uniform lighting that complicate visual perception. Scale variation also affects tracking performance. As the person moves closer to or farther from the robot, the apparent size of the bounding box changes significantly. Robust tracking systems must continuously adapt to these changes while maintaining accurate localisation. Background clutter presents another difficulty when the target shares similar visual characteristics with surrounding objects. In such situations, distinguishing the target from the environment becomes increasingly challenging.

Additionally, rapid human movement can result in motion blur and abrupt changes in target position, increasing the likelihood of temporary tracking failure. Camera motion caused by robot movement may further complicate accurate target localisation. Also, maintaining real-time performance remains a critical challenge. Autonomous robots require continuous perception with minimal latency to ensure smooth navigation and safe interaction with humans. High computational requirements associated with advanced tracking algorithms may limit deployment on embedded robotic platforms.

2.1.10 Robot Operating System (ROS)

The Robot Operating System (ROS) is an open-source robotics middleware that provides a flexible software framework for developing robotic applications. Although referred to as an operating system, ROS is not a conventional operating system; rather, it acts as a middleware layer that facilitates communication between various hardware and software components of a robotic system. It provides standardized tools, libraries, and communication protocols that simplify the design, implementation, and integration of complex robotic applications (Quigley *et al*., 2009). ROS has the capability to modularize robotic systems into independent processes known as nodes, allowing different components such as perception, navigation, localization, mapping, and motion control to operate concurrently while exchanging information through standardized communication mechanisms. This modular architecture promotes software reusability, scalability, and rapid system development. Based on these features, ROS has become the de facto standard software framework for robotics research and development in academia and industry (Quigley *et al*., 2009; Koubaa, 2017).

2.1.10.1 ROS Architecture

ROS employs a distributed architecture in which multiple independent software processes collaborate to perform robotic tasks. Rather than implementing all functionalities within a

single program, ROS decomposes a robotic application into several specialized nodes that communicate through well-defined interfaces. This modular architecture enhances system flexibility, maintainability, and robustness (Quigley *et al*., 2009). At the centre of the ROS communication framework is the ROS Master, which functions as a name server. The ROS Master allows nodes to discover one another, establish communication channels, and exchange data. Once communication has been established, nodes communicate directly through peer-to-peer connections without routing messages through the master, thereby reducing communication overhead. The principal components of the ROS architecture are ROS master, nodes, topics, messages, services, parameter server, packages, and launch files. This distributed architecture allows each subsystem to execute independently while collaborating to accomplish complex robotic tasks.

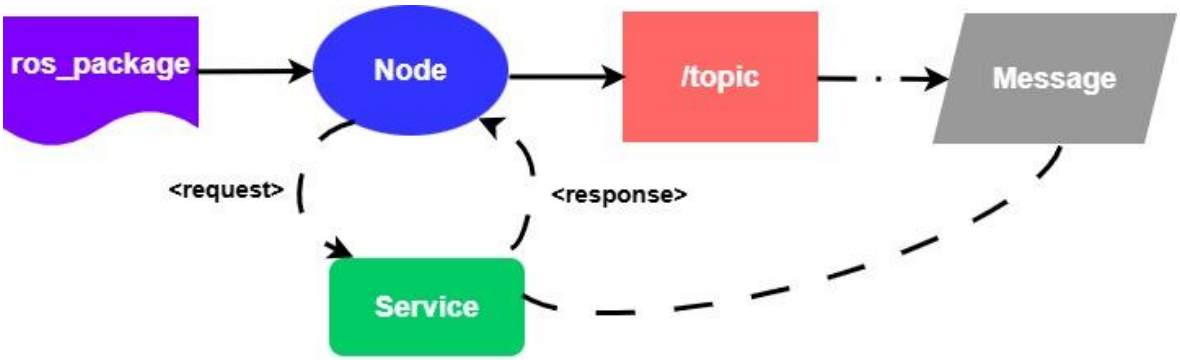


Figure 2.14 - Basic ROS Data Flow (Maralit, 2024)

i. Nodes

Nodes are independent executable processes that perform specific functions within a ROS application. Each node is designed to accomplish a well-defined task, allowing robotic systems to be divided into modular software components (Quigley *et al*., 2009). The ROS nodes are camera drivers, sensor interfaces, object detection algorithms, motion controllers, localization modules, and navigation systems. Nodes communicate with one another using ROS communication mechanisms such as topics and services. Since each node executes independently, failures within one node generally do not affect the operation of the remaining system components.

ii. Topics

Topics provide an asynchronous publish-subscribe communication mechanism within the ROS framework. A node known as the publisher transmits data to a named topic, while one or more subscriber nodes receive the published information (Quigley *et al*., 2009). Topics are particularly suitable for continuously changing information such as camera images, laser scans, robot velocity commands, GPS coordinates, IMU measurements, person detection results. Publishers and subscribers operate independently without direct knowledge of one another, promoting loose coupling between software components.

iii. Messages

ROS messages define the structure and format of data exchanged between nodes. Each topic transmits a specific message type, ensuring that publishers and subscribers interpret data consistently (Quigley *et al*., 2009). ROS provides numerous built-in message types, such as *geometry_msgs/Twist, sensor_msgs/Image*, *sensor_msgs/LaserScan*, *std_msgs/String*, *std_msgs/Float32*, and *std_msgs/Float32MultiArray*. However, custom message types can be defined based on required application-specific data structures.

iv. Services

ROS services provide synchronous request-response communication between nodes. Unlike topics, which continuously stream information, services are intended for operations that require an immediate response (Quigley *et al*., 2009). A service consists of a request and a response with typical applications in resetting hardware, enabling robot controllers, loading configuration parameters, starting or stopping subsystems and querying system status.

2.1.10.2 Advantages of ROS

ROS has become one of the most widely adopted middleware frameworks for robotics research and industrial development due to its modular architecture, flexibility, and extensive ecosystem. It's hardware abstraction, enables application software to be deployed across different robotic platforms with minimal modifications. ROS provides standardized communication interfaces that simplify the integration of sensors, actuators, cameras, LiDARs, manipulators, and mobile robot platforms from different manufacturers. Furthermore, its large collection of open-source packages significantly reduces development time by providing readily available implementations for localisation, SLAM, navigation, motion planning, manipulation, perception, and simulation.

ROS also benefits from a large international research community that continuously develops and maintains software libraries, tutorials, and documentation. This extensive community support has accelerated robotics research worldwide and encouraged the adoption of common software standards. Moreover, ROS integrates seamlessly with widely used artificial intelligence and computer vision libraries, including OpenCV, TensorFlow, PyTorch, and deep learning-based object detection frameworks such as YOLO, thereby facilitating the development of intelligent robotic systems. Another notable strength of ROS is its support for distributed computing, which enables computational tasks to be executed across multiple interconnected computers. This capability is particularly valuable for computationally intensive robotic applications involving deep learning and computer vision.

Building upon ROS 1, ROS 2 introduces several enhancements that make it more suitable for modern robotic systems. Unlike ROS 1, ROS 2 is built on the Data Distribution Service (DDS) communication standard, providing decentralized communication without requiring a central ROS Master. This architecture improves scalability, reliability, and fault tolerance in distributed robotic systems (Macenski *et al*., 2022). ROS 2 also offers native support for Quality of Service (QoS) policies, allowing the configuration of communication reliability, latency, durability, and message delivery according to application requirements. Additionally,

ROS 2 incorporates improved security mechanisms through DDS Security, including authentication, encryption, and access control, making it more appropriate for industrial and collaborative robotic applications. Furthermore, ROS 2 provides significantly improved support for real-time computing and multi-platform compatibility, enabling deployment on Linux, Windows, macOS, and embedded operating systems. Collectively, these features have established ROS and ROS 2 as leading middleware frameworks for developing intelligent autonomous robots across research, industrial automation, healthcare, logistics, agriculture, and service robotics.

2.1.10.3 Limitations of ROS

Despite their numerous advantages, both ROS 1 and ROS 2 possess certain limitations that researchers and developers must consider during system design and implementation. One of the primary limitations is the high technical expertise required when developing ROS-based applications because they require familiarity with distributed communication, package management, message definitions, launch configurations, parameter management, and debugging tools.

A significant limitation of ROS 1 is its lack of native support for deterministic real-time communication. Since ROS 1 was originally designed as a research-oriented middleware rather than a real-time operating framework, message transmission delays and scheduling uncertainties may occur during time-critical robotic operations. Although acceptable for many academic applications, these limitations reduce its suitability for safety-critical autonomous systems requiring strict timing guarantees. ROS 1 also relies on a centralized ROS Master, which represents a potential single point of failure. If the ROS Master becomes unavailable, communication between newly launched nodes cannot be established. In contrast, ROS 2 eliminates this dependency by adopting the decentralized DDS communication architecture, thereby improving system robustness and reliability.

Another limitation is network dependency, as distributed ROS applications require stable network connectivity for reliable communication between nodes executing on different computers. Network congestion, packet loss, or communication interruptions may negatively affect system performance, particularly in wireless robotic applications. Security has historically been another concern for ROS 1 because it lacks built-in authentication, encryption, and access control mechanisms. ROS 2 addresses these shortcomings through DDS Security, providing encrypted communication, authenticated participants, and secure access control. However, configuring these security features introduces additional system complexity and may increase computational overhead.

Both ROS 1 and ROS 2 can also become computationally demanding when integrated with high-resolution sensors, simultaneous localisation and mapping algorithms, and deep learning models. And as such. applications involving CNNs, such as YOLO-based object detection, typically require GPU acceleration to achieve real-time performance. Despite these limitations, ROS remains the dominant middleware for robotics research due to its modularity, scalability, extensive software ecosystem, and widespread community support. Meanwhile, ROS 2 addresses many of the architectural limitations of ROS 1 by providing improved real-time

capabilities, enhanced security, decentralized communication, QoS management, and greater suitability for industrial and production-level robotic systems (Quigley *et al*., 2009).

2.1.11 Robot Control Systems

Robot control systems include the decision-making and actuation mechanisms that govern the movement and behaviour of robotic platforms. They transform sensory information into appropriate motion commands that enable robots to interact safely and intelligently with their environments. In autonomous mobile robots, control systems receive information from perception modules, process environmental and positional data, and generate velocity commands that guide the robot toward accomplishing specific tasks. Effective control systems are fundamental to autonomous navigation, obstacle avoidance, trajectory tracking, and person-following applications because they determine the stability, responsiveness, and safety of robot motion (Siciliano *et al*., 2016). In vision-based person-following robots, the control system continuously processes visual feedback from the perception module to estimate the target person's position relative to the robot. Based on this information, appropriate linear and angular velocity commands are generated to maintain a desired following distance while keeping the target centred within the camera's FOV. The effectiveness of the overall system therefore depends not only on accurate perception but also on robust control algorithms capable of responding rapidly to dynamic human movement.

2.1.11.1 Motion Control

Motion control refers to the regulation of a robot's physical movement to achieve desired positions, orientations, and trajectories. It determines how a robot accelerates, decelerates, turns, and stops while performing assigned tasks. For mobile robots, motion control combines information from sensors, localisation systems, and perception algorithms to produce smooth and stable navigation (Siciliano *et al*., 2016). The motion control system typically consists of three hierarchical levels,

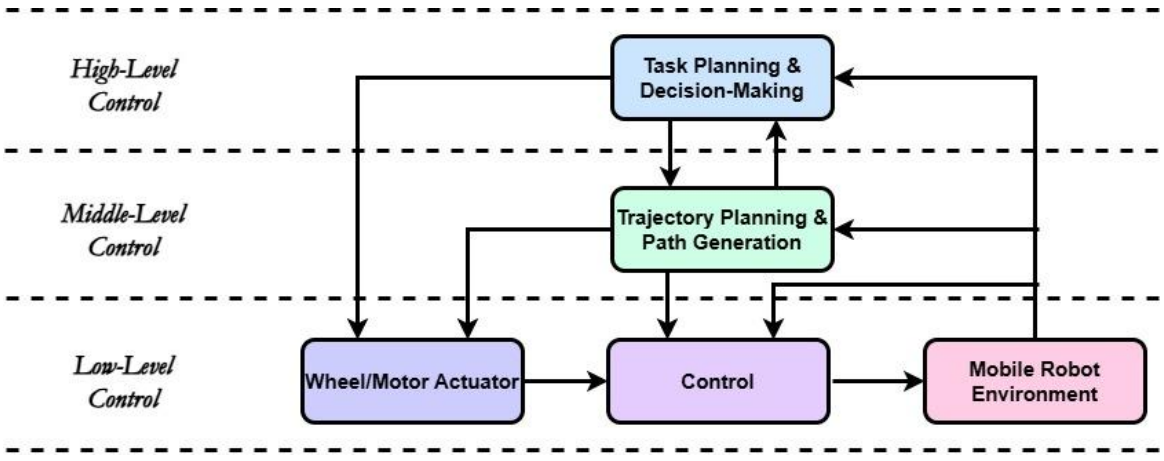


Figure 2.15 - Three-Level Hierarchical Motor Control (Bruyninckx et al., 2001)

i. *High-level control*, responsible for task planning and decision-making.
ii. *Middle-level control*, responsible for trajectory planning and path generation.

iii. *Low-level control*, responsible for controlling wheel motors and actuators.

In autonomous person-following robots, motion control continuously adjusts the robot's trajectory according to the detected position of the target person. As the person's position changes, the controller modifies the robot's translational and rotational motion to maintain continuous tracking while ensuring smooth and stable movement.

2.1.11.2 Velocity Control

Velocity control involves regulating the speed of a robot rather than commanding its absolute position. Mobile robots typically receive velocity commands consisting of linear velocity and angular velocity, which determine forward, backward and rotational motion (Corke, 2017). For differential-drive mobile robots such as the Robotnik Summit XL, velocity control is commonly represented by

- *Linear velocity* (v), controls forward and backward movement.
- *Angular velocity* (ω), controls rotational motion about the robot's vertical axis.

The robot controller continuously adjusts these velocities based on feedback from perception and localisation systems to achieve smooth navigation.

2.1.11.3 Proportional Integral Derivative (PID) Control

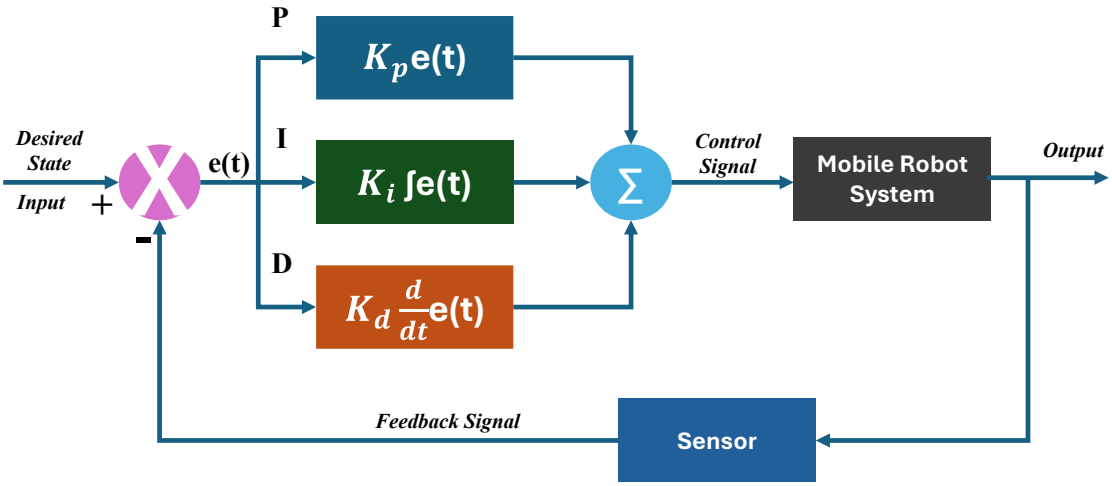


Figure 2.16 - PID Controllers for Mobile Robot System

The Proportional Integral Derivative (PID) controller is one of the most widely used feedback control algorithms in robotics and industrial automation due to its simplicity, robustness, and effectiveness. PID controllers continuously minimise the error between a desired reference value and the measured system output by combining three control actions; proportional, integral, and derivative control (Åström & Murray, 2021). The proportional component generates a control signal proportional to the current error, enabling rapid response to deviations from the desired state. The integral component accumulates past errors to eliminate steady-state error, while the derivative component predicts future system behaviour by considering the rate of change of the error, thereby improving system stability and reducing oscillations. In mobile robotics, PID controllers are widely applied for velocity regulation, steering control, trajectory tracking, heading correction, and path following. Although PID controllers provide excellent performance under relatively stable operating conditions, they

require careful tuning of controller gains. Their performance may degrade when operating in highly dynamic environments where system parameters vary continuously.

2.1.11.4 Adaptive Control

Adaptive control refers to control strategies capable of automatically adjusting controller parameters in response to changes in system dynamics or environmental conditions. Unlike fixed-gain controllers, adaptive controllers continuously modify their behaviour to maintain satisfactory performance despite uncertainties, disturbances, or changing operating conditions (Ioannou & Sun, 2012). Adaptive control is particularly advantageous in autonomous mobile robotics because robots frequently encounter unpredictable environments, varying payloads, sensor uncertainties, and continuously changing human behaviours. Rather than relying on constant controller gains, adaptive controllers dynamically adjust control parameters according to observed system performance. For autonomous person-following robots, adaptive control enables the robot to respond effectively to frequent human positional changes, including variations in walking speed, direction, and following distance. Such adaptability improves motion smoothness, reduces oscillatory behaviour, and enhances overall tracking robustness.

2.1.12 Human Position Estimation

Human position estimation refers to the process of determining the location, orientation, and relative distance of the human with respect to a robotic system using sensory information. It is a fundamental component of autonomous person-following robots because it enables the robot to continuously determine where the target person is located and how the robot should respond to maintain an appropriate following distance. Accurate human position estimation contributes significantly to stable navigation, collision avoidance, and smooth human–robot interaction (Siciliano & Khatib, 2016). Various techniques have been developed for estimating human position in robotics, ranging from traditional computer vision methods to modern deep learning and multi-sensor fusion approaches. These techniques differ in computational complexity, hardware requirements, robustness, and estimation accuracy.

2.1.12.1 Bounding Box Estimation

Bounding box estimation is one of the most widely used techniques for localising objects in digital images. A bounding box is a rectangular region surrounding a detected object and is typically represented by the coordinates of its upper-left and lower-right corners or alternatively by its centre coordinates together with its width and height (Redmon *et al*., 2016). Recent object detection algorithms such as YOLO estimate bounding boxes directly from input images while simultaneously classifying detected objects. For person-following robots, bounding boxes provide several useful measurements, such as horizontal image position, vertical image position, bounding-box width, bounding-box height, and detection confidence. These measurements can be directly utilised for robot control. The horizontal centre of the bounding box indicates whether the person is positioned to the left or right of the camera centre, while the bounding-box height provides an approximate indication of the person's distance from the camera. As the person approaches the robot, the bounding box becomes larger, whereas moving away results in a smaller bounding box. The simplicity and computational

efficiency of bounding-box estimation make it particularly suitable for real-time robotic applications where low processing latency is essential.

2.1.12.2 Monocular Distance Estimation

Monocular distance estimation involves estimating the distance between a camera and an object using images obtained from a single camera. Unlike stereo vision systems or LiDAR sensors, monocular systems do not directly measure depth. Instead, they infer relative distance using visual cues such as object size, perspective, texture, motion, or learned depth representations (Szeliski, 2022). A commonly employed approach in person-following robots is the use of bounding-box dimensions as an indirect distance indicator. Assuming the physical size of a person remains approximately constant, the apparent height of the detected person within the image increases as the person approaches the camera and decreases as the person moves farther away. Mathematically,

$$Distance \propto \frac{1}{Bounding\ Box\ Height} \tag{2.1}$$

Thus, bounding-box height exhibits an inverse relationship with camera to person distance. Although this method does not provide absolute metric distance, it is sufficiently accurate for many indoor autonomous navigation tasks where maintaining a relative following distance is more important than determining exact physical measurements.

2.1.12.3 Pose Estimation

Pose estimation is the process of determining the spatial configuration of the human body by identifying anatomical key-points such as the head, shoulders, elbows, hips, knees, and ankles. Rather than detecting only the person's location, pose estimation estimates the geometric relationship among body joints, thereby enabling robots to interpret human posture, orientation, gestures, and movement intentions (Cao *et al*., 2021). Recent deep learning models, including OpenPose, AlphaPose, MediaPipe Pose, and YOLO-Pose, have significantly improved the accuracy and speed of human pose estimation. These algorithms predict body key-points directly from RGB images without requiring wearable markers or specialised sensing equipment. Pose estimation supports numerous robotic applications like gesture recognition, human activity recognition, human intention prediction, social robotics, healthcare monitoring, and autonomous person following. For person-following robots, pose estimation offers additional information beyond object detection. Human body orientation can be estimated to predict future movement direction, while body posture can indicate whether a person is walking, standing, sitting, or turning. Such information enables more intelligent navigation and smoother interaction. Integrating pose estimation with YOLO-based person detection could improve target tracking robustness, particularly during rapid human movement.

2.1.12.4 Sensor Fusion Approaches

Sensor fusion refers to the integration of information obtained from multiple sensors to produce more accurate, reliable, and robust estimates than can be achieved using any individual sensor alone. Mobile robots frequently combine cameras, LiDAR, ultrasonic sensors, inertial

measurement units (IMUs), wheel encoders, GPS receivers, and depth cameras to improve environmental perception and navigation accuracy (Thrun *et al*., 2005). The main objective of sensor fusion is to compensate for the limitations of individual sensors. Like when, cameras provide rich visual information but are sensitive to lighting variations and occlusions, LiDAR offers accurate distance measurements but lacks colour and texture information and IMUs measure acceleration and orientation but suffer from cumulative drift over time. By combining complementary sensor measurements, robots can achieve improved localisation, tracking, and navigation performance. Common sensor fusion techniques are Kalman Filtering, Extended Kalman Filtering (EKF), Unscented Kalman Filtering (UKF), Particle Filtering, Bayesian Estimation, and Deep Learning-Based Sensor Fusion. In autonomous person-following robots, sensor fusion enhances tracking reliability by combining visual detections with inertial measurements, wheel odometry, or depth sensing. This integration enables the robot to maintain target tracking even when visual information is temporarily degraded due to illumination changes, or rapid motion.

2.1.13 Visual Servoing

Visual servoing integrates three major components which are vision system, control algorithm, and robot actuators. The vision system captures images of the environment and extracts relevant visual features such as object positions, feature points, or bounding boxes. These features are processed to estimate the positional error between the robot and the target object. The controller then computes appropriate motion commands that minimise this error, enabling the robot to track moving targets accurately while adapting to environmental changes.

Visual servoing has become one of the most widely adopted control strategies in autonomous robotics because cameras provide rich environmental information at relatively low cost. Recent advancements in computer vision and deep learning have further enhanced visual servoing by enabling reliable object detection and tracking in complex, dynamic environments.

In autonomous person-following robots, visual servoing enables continuous tracking of a target by using visual measurements to regulate the robot's forward motion and steering. This capability forms the basis of the adaptive vision-based framework developed in this research.

2.1.13.1 Position-Based Visual Servoing (PBVS)

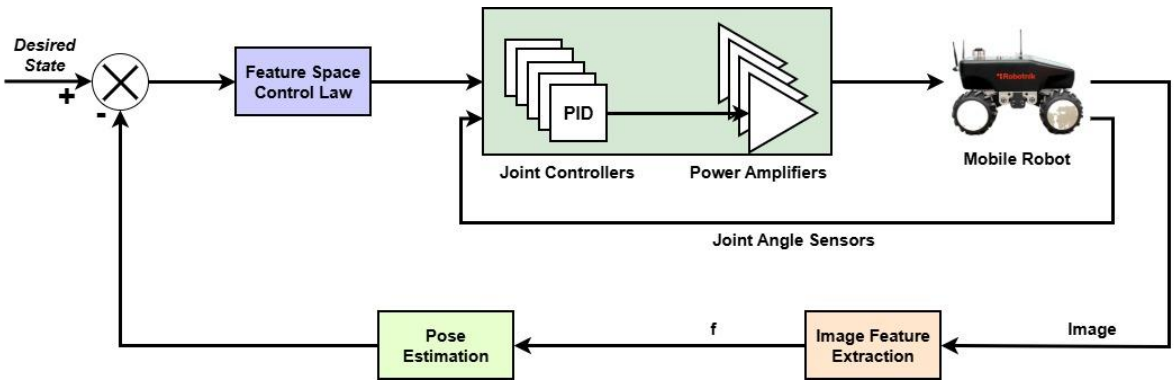


Figure 2.17 - Position-Based Visual Servoing Scheme (Garcia et al., 2009)

Position-Based Visual Servoing (PBVS) is a visual control approach in which the robot first estimates the three-dimensional pose of the target object relative to the camera before computing control commands. The estimated pose generally consists of the target's three-dimensional position and orientation within a defined coordinate system (Chaumette & Hutchinson, 2006). The PBVS process typically involves stages such as image acquisition, detection of visual features, estimation of the object's 3D pose, calculation of positional error, and generation of robot motion commands. Since PBVS relies on explicit three-dimensional geometric information, it usually requires camera calibration and accurate mathematical models of the camera and robot. Stereo cameras, RGB-D cameras, LiDAR sensors, or multiple-view camera systems are commonly employed to obtain reliable depth information.

2.1.13.2 Image-Based Visual Servoing (IBVS)

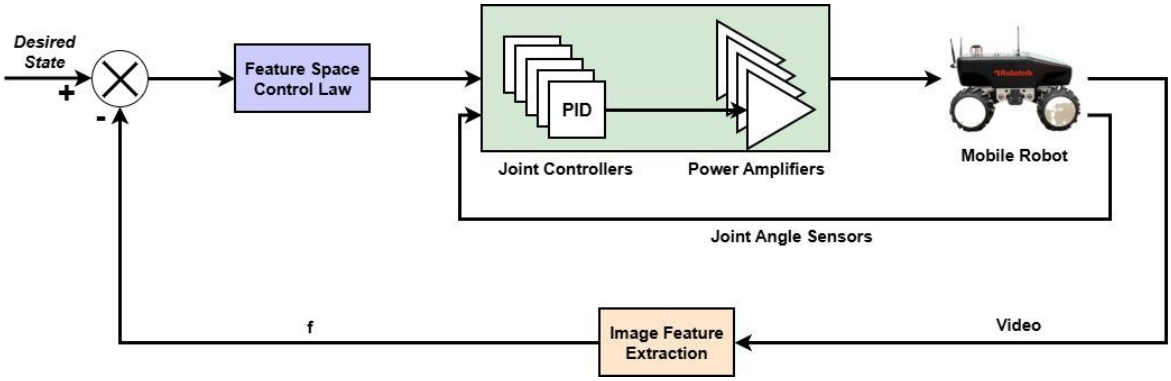


Figure 2.18 - Image-Based Visual Servoing Scheme (Iqbal et al., 2025)

Image-Based Visual Servoing (IBVS) is a visual control technique in which robot motion is controlled directly from image features without explicitly estimating the target's three-dimensional position. Instead of reconstructing the scene geometry, the controller continuously minimises the error between the current image features and their desired positions within the camera image (Chaumette & Hutchinson, 2006). Typical image features are pixel coordinates, feature points, bounding-box centres, bounding-box dimensions, object contours, and image moments. These visual features enable the robot to continuously adjust its movement while maintaining the target within the camera's FOV.

2.1.13.3 Comparison of Position-Based and Image-Based Visual Servoing

Although both PBVS and IBVS utilise camera information to control robotic motion, they differ significantly in their computational requirements, sensing mechanisms, and application domains. PBVS operates within a three-dimensional coordinate framework by estimating the complete spatial pose of the target before computing robot motion. This provides high positioning accuracy and is well suited for precision manipulation tasks. However, it generally requires accurate camera calibration, depth estimation, and greater computational resources.

Conversely, IBVS performs control directly within the image plane by using visual features extracted from camera images. It avoids explicit three-dimensional reconstruction, making it computationally efficient and more robust to certain modelling errors. IBVS is therefore particularly suitable for real-time robotic navigation and object-following applications where

rapid response is more important than absolute positioning accuracy. For autonomous person-following systems operating indoors, IBVS can provide lower computational complexity, reduced hardware requirements, and easier implementation using monocular cameras.

Table 2.1 - Comparison of Position-Based and Image-Based Visual Servoing

S/No.	Feature	Position-Based Visual Servoing (PBVS)	Image-Based Visual Servoing (IBVS)
1.	Control space	Three-dimensional space	Image plane
2.	Depth estimation	Required	Not required
3.	Camera calibration	Essential	Less demanding
4.	Computational complexity	Higher	Lower
5.	Hardware requirements	Often stereo/RGB-D cameras	Single monocular camera sufficient
6.	Robustness to calibration errors	Lower	Higher
7.	Suitability for real-time navigation	Moderate	High
8.	Applications	Robotic manipulation, docking, assembly	Person following, mobile navigation, target tracking

2.2 EMPIRICAL REVIEW

The empirical review examines previous studies related to autonomous mobile robots, vision-based person-following systems, deep learning-based object detection, robot control, and human–robot interaction. Unlike the theoretical review, which discusses concepts and principles, the empirical review critically analyses existing research findings, methodologies, experimental platforms, achievements, and limitations. This analysis provides insight into the current state of knowledge, identifies gaps in the literature, and establishes the need for the proposed adaptive vision-based person-following framework implemented on a real Robotnik Summit XL mobile robot.

Several researchers have investigated autonomous person-following using different sensing modalities, including laser range finders, ultrasonic sensors, RGB-D cameras, stereo vision systems, and monocular cameras. While many of these systems demonstrate satisfactory performance under controlled laboratory conditions, challenges remain in maintaining robust tracking during rapid human movement, illumination variations, partial occlusions, and dynamic indoor environments. Furthermore, although deep learning has significantly improved object detection accuracy, relatively few studies have integrated modern object detectors with adaptive control strategies and validated their performance on full-scale commercial mobile robots operating in real-world environments.

2.2.1 Review of Related Empirical Studies

i. Study 1: YOLO9000: Better, Faster, Stronger

Redmon & Farhadi (2017) introduced YOLO9000, an improved version of the YOLO object detection framework capable of recognising over 9,000 object categories while maintaining real-time detection speed. The study proposed joint optimisation of detection and classification tasks, enabling efficient feature learning from both labelled detection datasets and large-scale image classification datasets. Experimental evaluation on the Pascal VOC and ImageNet benchmarks demonstrated that YOLO9000 significantly improved detection accuracy while maintaining processing speeds suitable for real-time applications. The study established the practicality of deep learning for high-speed object detection and influenced numerous robotic perception systems. However, the work focused primarily on benchmark image datasets and did not investigate robot navigation, person-following behaviour, or adaptive robot control in dynamic environments.

Relevance to this study: The research demonstrates the suitability of YOLO for real-time robotic perception but leaves open the problem of integrating detection outputs with adaptive mobile robot control.

ii. Study 2: YOLOv4: Optimal Speed and Accuracy of Object Detection

Bochkovskiy *et al*., (2020) developed YOLOv4 with the objective of improving detection accuracy while preserving real-time performance. The researchers introduced several innovations, including CSPDarknet53, Mish activation, Cross-Stage Partial Networks, Mosaic data augmentation, and Self-Adversarial Training. Experimental results demonstrated state-of-the-art detection performance on the MS COCO dataset with improved inference speed compared to previous YOLO versions. Although the proposed model significantly advanced real-time perception, the study concentrated solely on object detection performance and did not investigate robotic navigation or autonomous human-following applications.

Relevance to this study: The work validates the effectiveness of modern YOLO architectures for robotic vision but does not address adaptive control for autonomous following.

iii. Study 3: YOLOv8-Based Object Detection for Intelligent Robotic Applications

Jocher *et al*., (2023) introduced YOLOv8 as part of the Ultralytics object detection framework. YOLOv8 incorporates anchor-free detection, improved feature extraction, enhanced segmentation capabilities, pose estimation, and simplified deployment. The framework demonstrates excellent inference speed while maintaining high detection accuracy across multiple benchmark datasets. Its Python interface and compatibility with OpenCV, PyTorch, and ROS make it particularly attractive for robotic perception systems. Despite these improvements, the most published work focused mainly on computer vision performance rather than autonomous mobile robot control or human-following strategies.

Relevance to this study: The present research adopts YOLOv8 as the primary perception algorithm and extends its application to adaptive autonomous person-following on a physical mobile robot.

iv. Study 4: A Survey of Vision-Based Human Following Robots

Islam *et al*., (2019) conducted a comprehensive survey of autonomous human-following robots employing vision-based sensing technologies. The authors reviewed traditional computer vision techniques, machine learning approaches, and deep learning methods for detecting and tracking humans in indoor and outdoor environments. The survey identified illumination variation, target occlusion, cluttered backgrounds, and computational complexity as major challenges affecting tracking performance.

The review concluded that deep learning-based perception substantially improves detection robustness but emphasised that effective robot control remains an open research problem requiring further investigation.

Relevance to this study: The survey identifies many of the challenges directly addressed by the proposed adaptive vision-based person-following framework.

v. Study 5: Robust Person Following for Mobile Robots in Dynamic Environments

Montesdeoca *et al*., (2022) proposed a robust person-following framework combining visual perception with probabilistic tracking techniques. The system employed RGB cameras and tracking algorithms to maintain continuous target localisation while accounting for temporary occlusions and abrupt human movement. Experimental evaluation demonstrated improved robustness compared with purely appearance-based trackers. However, the approach relied heavily on handcrafted tracking algorithms rather than modern deep learning object detectors and was computationally demanding in crowded environments.

Relevance to this study: The research highlights the importance of robust tracking but predates the widespread adoption of modern YOLO-based perception systems.

vi. Study 6: Deep Learning for Vision-Based Mobile Robot Navigation

Tai *et al*., (2017) investigated the application of deep learning to autonomous mobile robot navigation using end-to-end learning techniques. CNNs were employed to map visual information directly to robot control commands. Experimental results demonstrated that learning-based approaches could successfully navigate cluttered environments while reducing dependence on handcrafted feature engineering. Despite promising results, the approach lacked interpretability and required extensive training data. Moreover, the system focused on autonomous navigation rather than person following.

Relevance to this study: The study demonstrates the growing importance of deep learning for robotic autonomy while supporting the integration of intelligent perception with robot motion control.

vii. Study 7: ROS-Based Autonomous Mobile Robot Development

Quigley *et al*. (2009) introduced the Robot Operating System (ROS) as an open-source middleware for robotic software development. Through several experimental robotic platforms, the study demonstrated that ROS enables modular software design, distributed computation, hardware abstraction, and rapid integration of perception, planning, and control

algorithms. Although ROS itself does not provide person-following algorithms, it has become the standard software framework supporting modern autonomous robotic research.

Relevance to this study: The proposed adaptive framework is implemented entirely within the ROS ecosystem, allowing modular integration of YOLO perception, robot control, and communication.

2.2.2 Critical Analysis of Reviewed Studies

The studies when reviewed demonstrated significant progress in autonomous robotics, deep learning, and computer vision. Recent YOLO architectures have substantially improved the speed and accuracy of object detection, making real-time robotic perception increasingly practical. Likewise, ROS has become the dominant middleware for integrating sensing, perception, planning, and control modules within autonomous robotic systems. Nevertheless, several important limitations remain evident. Most deep learning studies concentrate mainly on improving detection accuracy using benchmark datasets without addressing adaptive robot control. Inversely, many person-following studies employ classical tracking techniques that are less robust than contemporary deep learning detectors under varying illumination, partial occlusions, and dynamic indoor environments. Furthermore, most investigations validate their approaches exclusively through simulation or laboratory-scale prototypes, with relatively few demonstrating deployment on commercial autonomous mobile robots operating in realistic environments. Another notable limitation is that many existing person-following systems assume smooth and predictable human motion, resulting in oscillatory behavior, unstable control, or target loss when individuals rapidly change direction or speed. Additionally, comparatively little research has combined real-time YOLO-based perception with adaptive velocity control specifically designed to respond to continuous human positional changes during autonomous following.

2.2.3 Review of Classical Vision-Based Person-Following Systems

Early autonomous person-following robots predominantly relied on classical computer vision techniques for human detection and tracking. These methods were attractive because of their relatively low computational requirements and suitability for real-time implementation of hardware with limited processing capabilities. Before the emergence of deep learning, techniques such as Haar Cascade classifiers, HOG, optical flow, colour-based tracking, and background subtraction formed the foundation of many vision-based robotic perception systems. Although these approaches enabled significant advances in autonomous human following, their performance was often constrained by variations in illumination, viewpoint, occlusion, and complex environmental conditions. Consequently, many of these limitations motivated the development of modern deep learning-based perception systems.

2.2.3.1 Haar Cascade-Based Person Detection

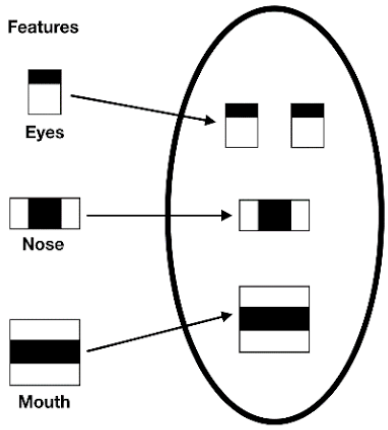


Figure 2.19a - Haar Theoretical Face Model (Lasri et al., 2019)

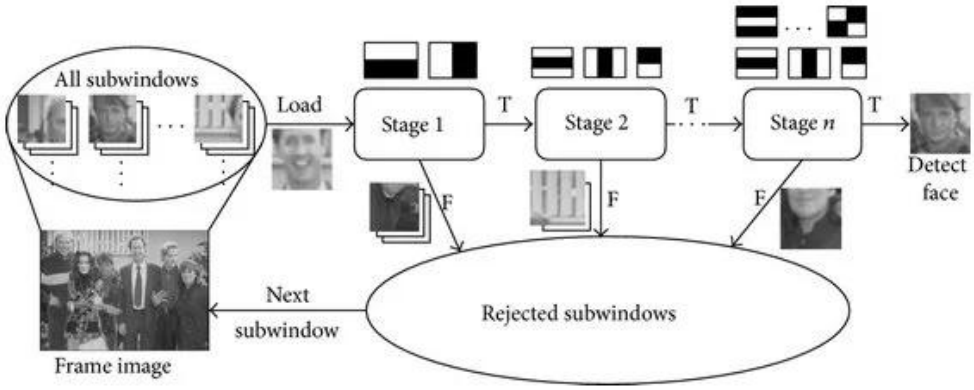


Figure 2.19b - Cascade Structure for Haar Classifiers (Kim et al., 2015)

The Haar Cascade classifier, introduced by Viola & Jones (2001), represented one of the earliest real-time object detection algorithms widely adopted in robotics and computer vision. The method employs Haar-like rectangular features combined with the AdaBoost learning algorithm and a cascade of increasingly complex classifiers to rapidly reject non-object regions while focusing computational resources on promising image areas. Haar Cascade classifiers gained popularity due to their computational efficiency and ability to operate in real time on processors with limited computational resources. Consequently, early mobile robots frequently utilised Haar classifiers for detecting human faces or upper bodies during person-following tasks (Bradski & Kaehler, 2008). However, their effectiveness depends heavily on the similarity between training data and operational environments. A major limitation of Haar Cascade detectors is their sensitivity to illumination changes, viewpoint variations, background clutter, and partial occlusions. Performance deteriorates considerably when individuals rotate, bend, or become partially hidden. Furthermore, Haar features capture only simple intensity differences rather than high-level semantic information, limiting their robustness in complex indoor environments.

2.2.3.2 Histogram of Oriented Gradients (HOG)



Figure 2.20 - Histogram of Gradients (HOG) Feature Extraction Process

The Histogram of Oriented Gradients (HOG) descriptor proposed by Dalal & Triggs (2005), significantly improved pedestrian detection by representing local gradient orientations rather than raw pixel intensities. HOG descriptors are commonly combined with Support Vector Machine (SVM) classifiers to distinguish pedestrians from background objects. Unlike Haar Cascade classifiers, HOG is more robust to moderate illumination changes because gradient orientations remain relatively stable under varying lighting conditions. Consequently, HOG became one of the most widely adopted pedestrian detection techniques for mobile robots before deep learning became dominant. Despite its improved robustness, HOG suffers from several drawbacks. Dense sliding-window searches over multiple image scales are computationally intensive, making real-time performance difficult on embedded robotic platforms. Detection accuracy also decreases under severe occlusion, crowded environments, and significant pose variations. Consequently, HOG-based systems often struggle to maintain reliable human tracking in dynamic real-world environments.

2.2.3.3 Optical Flow-Based Tracking

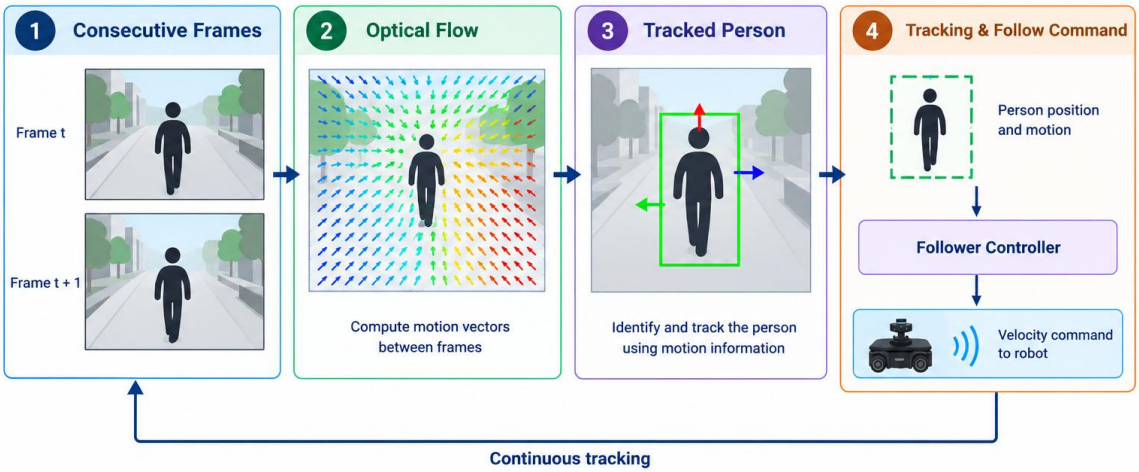


Figure 2.21 - Optical Flow-Based Tracking Process for Estimating and Tracking

Optical flow estimates apparent motion between consecutive image frames by analysing pixel displacement over time. Classical methods such as the Lucas–Kanade algorithm (Lucas & Kanade, 1981) and the Horn–Schunck method (Horn & Schunck, 1981) became widely used for estimating human movement in robotic applications. Optical flow offers several advantages

for person-following because it directly measures motion without requiring explicit object recognition. Robots can therefore estimate walking direction and target velocity using only image sequences. Optical flow has also been integrated with Kalman filters and particle filters to improve temporal tracking performance. However, optical flow estimates become unreliable under rapid camera movement, motion blur, texture-less surfaces, illumination changes, and dynamic backgrounds. Since optical flow estimates motion rather than object identity, robots may inadvertently follow moving background objects or lose the intended target during temporary occlusions. Consequently, optical flow is often combined with complementary detection methods rather than being used independently.

2.2.3.4 Colour-Based Tracking

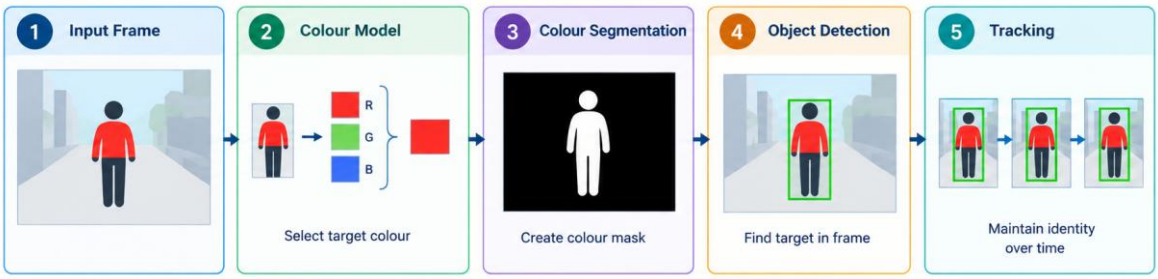


Figure 2.22 - Colour-Based Tracking Process using Distinctive Colour Information

Colour-based tracking represents one of the simplest person-following approaches. It segments image regions according to colour distributions in colour spaces such as RGB, HSV, or YCbCr. Histogram matching and algorithms such as Mean Shift and CAMShift enable continuous tracking of coloured targets (Comaniciu *et al*., 2003). The primary advantage of colour tracking lies in its computational efficiency, enabling real-time operation on low-cost processors. Colour tracking also provides smooth tracking when targets wear distinctive clothing against relatively uncluttered backgrounds. Nevertheless, colour-based approaches are highly sensitive to illumination variations, shadows, reflections, camera white balance, and background objects with similar colours. Their inability to distinguish between individuals wearing similar clothing significantly limits their applicability in crowded or uncontrolled environments. Consequently, colour tracking is generally unsuitable as a standalone solution for robust autonomous person following.

2.2.3.5 Background Subtraction

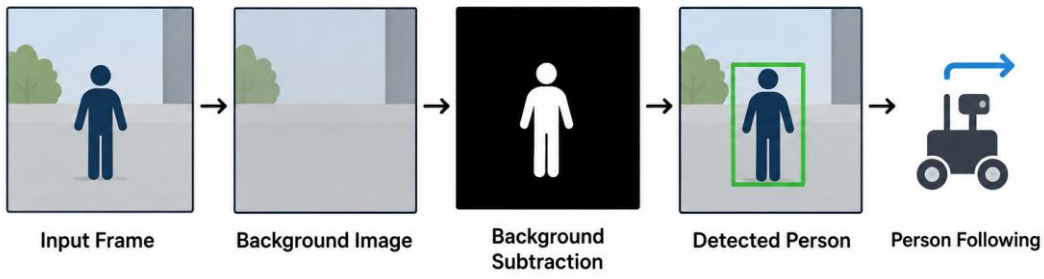


Figure 2.23 - Background Subtraction Human Detection Process

Background subtraction detects moving objects by comparing current image frames against a reference background model. Pixels exhibiting significant deviations are classified as

foreground, enabling moving individuals to be segmented from static environments (Stauffer & Grimson, 1999). It is computationally efficient and performs well in controlled indoor environments with stationary cameras. It has therefore been employed in surveillance systems and some early robotic tracking applications. However, mobile robots introduce continuous camera motion, violating the stationary-camera assumption fundamental to background subtraction. Furthermore, changing illumination, moving shadows, environmental dynamics, and multiple moving objects often produce false detections. As a result, background subtraction is rarely employed as the primary perception technique for modern autonomous mobile robots.

2.2.3.6 Critical Analysis of Classical Vision-Based Person-Following Techniques

Although classical vision algorithms laid the foundation for autonomous person-following research, they exhibit several common limitations. Most methods rely on manually engineered features rather than automatically learned semantic representations, restricting their ability to generalise across diverse environments. Consequently, their performance degrades significantly under varying illumination, pose changes, partial occlusions, cluttered backgrounds, and complex human activities. Among the reviewed approaches, HOG generally achieves higher pedestrian detection accuracy than Haar Cascade due to its use of gradient-based descriptors. Optical flow provides valuable motion information but lacks object-level semantic understanding, making it unsuitable for robust long-term tracking when used alone. Colour tracking offers high computational efficiency but is extremely sensitive to illumination changes and colour ambiguities. Background subtraction performs well only in static-camera scenarios and is generally incompatible with mobile robotic platforms. These limitations have driven the widespread adoption of deep learning-based object detectors such as YOLO, SSD, and Faster R-CNN, which learn hierarchical visual representations directly from large datasets. Such models demonstrate substantially improved robustness to illumination variation, occlusion, scale changes, and complex backgrounds while maintaining real-time performance, making them considerably more suitable for autonomous person-following in real-world environments.

Table 2.2 – Comparison of Classical Vision-Based Person-Following Techniques

S/No.	Technique	Detection Accuracy	Processing Speed	Illumination Robustness	Occlusion Handling	Major Limitation
1.	Haar Cascade	Moderate	Very High	Poor	Poor	Sensitive to pose and lighting changes
2.	HOG + SVM	High	Moderate	Moderate	Moderate	Computationally intensive for sliding-window detection
3.	Optical Flow	Moderate	High	Poor	Poor	Tracks motion rather than object identity

4.	Colour Tracking	Moderate	Very High	Very Poor	Poor	Highly dependent on colour consistency
5.	Background Subtraction	Moderate	High	Poor	Poor	Unsuitable for moving cameras

2.2.4 Review of Deep Learning-Based Person Following Systems

The limitations of classical computer vision algorithms have led to the widespread adoption of deep learning-based object detection techniques for autonomous person-following robots. Unlike traditional approaches that rely on handcrafted features, deep learning models automatically learn hierarchical feature representations from large annotated datasets, enabling superior performance in complex environments characterised by illumination variation, viewpoint changes, occlusion, scale variation, and background clutter (LeCun *et al*., 2015). The remarkable improvements in detection accuracy and robustness have made deep learning the dominant paradigm for robotic perception and autonomous navigation.

Deep learning-based object detectors are generally categorised into two-stage and one-stage detectors. Two-stage detectors first generate candidate object regions before classification, whereas one-stage detectors perform localisation and classification simultaneously. Two-stage methods generally achieve higher localisation accuracy but incur greater computational cost, while one-stage methods provide significantly faster inference suitable for real-time robotic applications (Zou *et al*., 2019).

2.2.4.1 Faster R-CNN

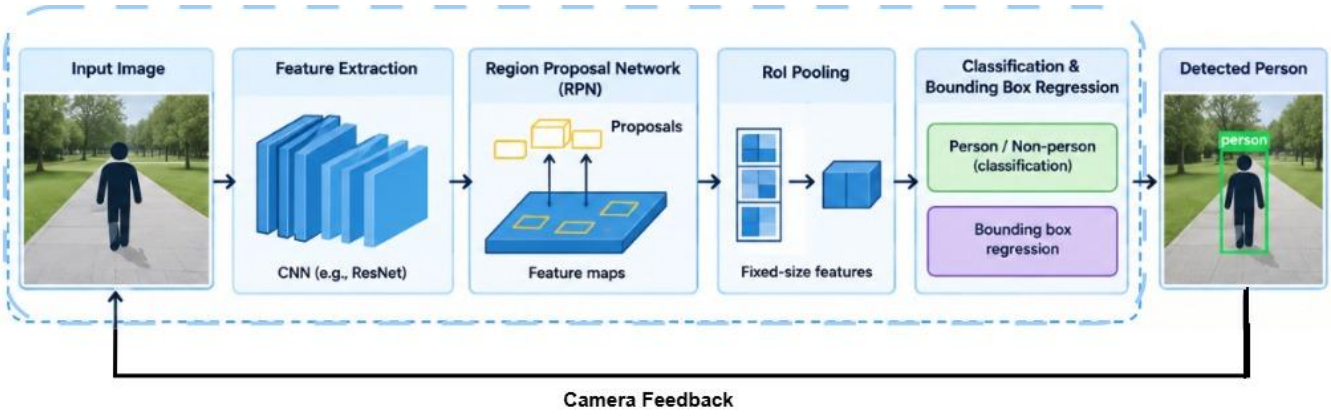


Figure 2.24 - Faster R-CNN Person Detection Pipeline

Faster Region-Based Convolutional Neural Network (Faster R-CNN), introduced by Ren *et al*., (2015), represents one of the most influential two-stage object detection frameworks. Unlike its predecessors, Fast R-CNN and R-CNN, Faster R-CNN integrates a Region Proposal Network (RPN) that generates candidate object regions directly within the neural network, significantly improving detection efficiency. The architecture consists of three major components: a deep convolutional backbone for feature extraction, the Region Proposal

Network for generating candidate object locations, and a classification and regression network for identifying object categories and refining bounding boxes. This two-stage detection process enables highly accurate localisation and classification, making Faster R-CNN one of the most accurate object detectors available. Several robotic perception studies have employed Faster R-CNN for pedestrian detection because of its high localisation accuracy and robustness under partial occlusion. However, its relatively slow inference speed limits its applicability to real-time autonomous person-following, particularly on embedded robotic platforms with constrained computational resources. Real-time deployment typically requires powerful Graphics Processing Units (GPUs), making Faster R-CNN more suitable for offline analysis or high-performance robotic systems than lightweight mobile robots.

2.2.4.2 Single Shot Multi-Box Detector (SSD)

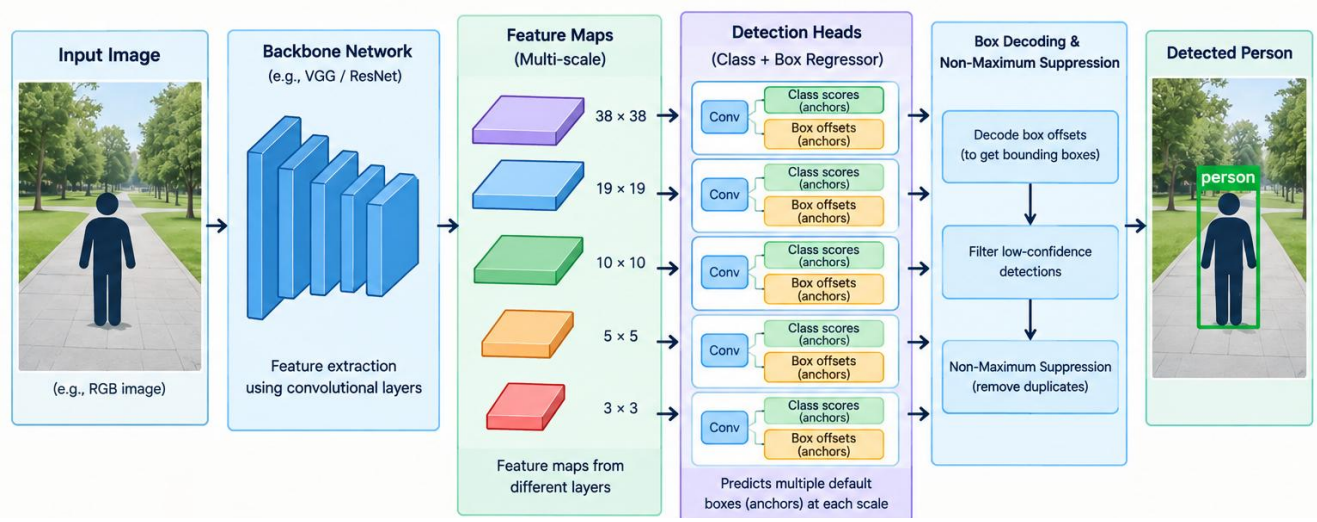


Figure 2.25 - Single Shot Multi-Box Detector Pipeline

The Single Shot Multi-Box Detector (SSD), proposed by Liu *et al*., (2016), was developed to address the speed limitations of two-stage detectors. SSD performs object localisation and classification simultaneously using multiple feature maps at different resolutions, allowing efficient detection of objects across various scales. Unlike Faster R-CNN, SSD eliminates the region proposal stage, significantly reducing computational complexity while maintaining competitive detection accuracy. Multi-scale feature maps enable SSD to detect both large and moderately small objects, making it attractive for robotic perception tasks. SSD offers considerably higher frame rates than Faster R-CNN and has therefore been adopted in numerous autonomous mobile robots requiring near real-time operation. Nevertheless, SSD exhibits reduced detection performance for very small or heavily occluded objects because shallow feature maps contain limited semantic information. Although SSD represented a major advancement in real-time object detection, subsequent detectors such as YOLO and EfficientDet generally outperform SSD in both speed and detection accuracy.

2.2.4.3 You Only Look Once (YOLO)

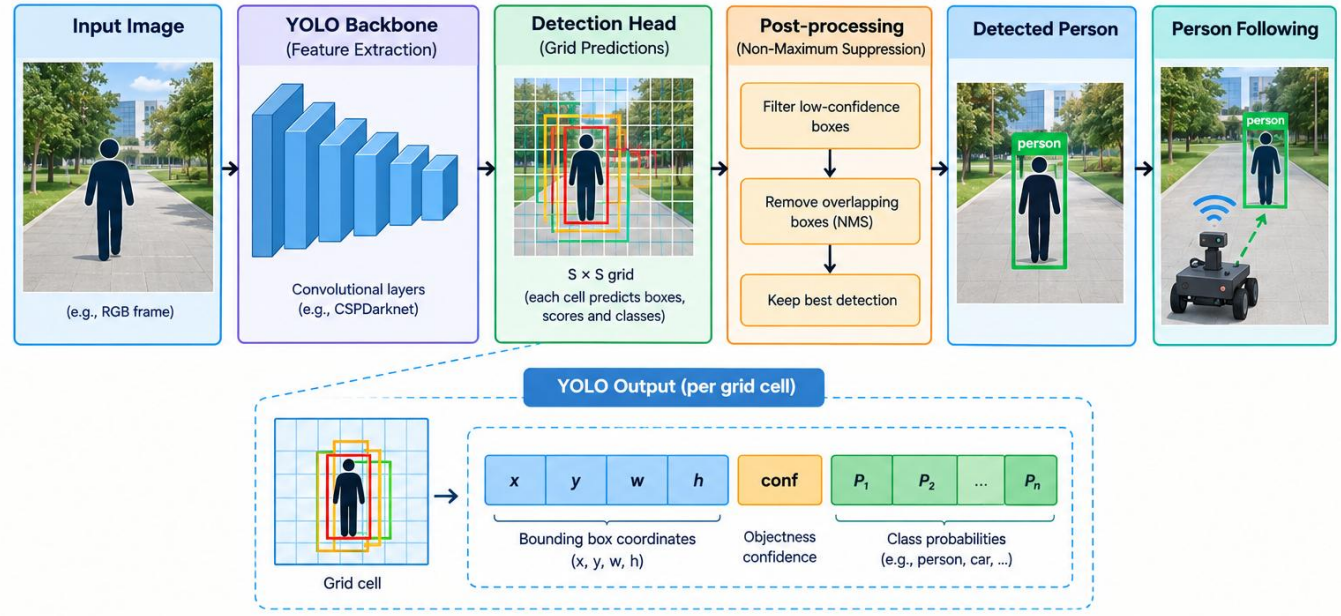


Figure 2.26 - YOLO Human Detection Pipeline

The You Only Look Once (YOLO) family of detectors has revolutionised real-time object detection by treating detection as a single regression problem. Introduced by Redmon *et al.*, (2016), YOLO predicts object locations, class probabilities, and confidence scores directly from an input image using a single forward pass through a convolutional neural network. Unlike region proposal methods, YOLO processes the entire image globally, enabling extremely high inference speed while preserving contextual information. Lightweight variants such as YOLO-Nano and YOLOv8n provide inference speeds exceeding 100 frames per second on modern GPUs while maintaining high detection accuracy. Consequently, YOLO is particularly well suited for autonomous person-following robots that require continuous perception and low-latency decision-making.

2.2.4.4 CenterNet

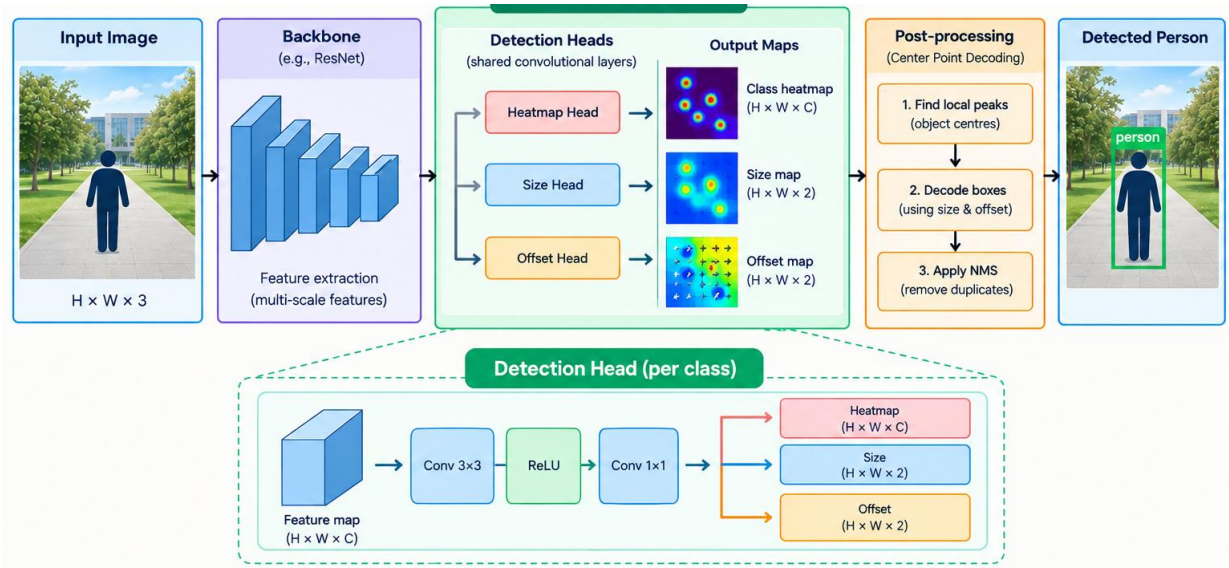


Figure 2.27 - CenterNet Pipeline

CenterNet, proposed by Zhou *et al*., (2019), introduced an anchor-free object detection paradigm. Rather than predicting predefined anchor boxes, CenterNet detects objects by estimating their centre points and subsequently regressing object dimensions. The anchor-free design simplifies network architecture, reduces hyperparameter tuning, and improves computational efficiency. Furthermore, CenterNet demonstrates superior localisation accuracy compared with many anchor-based detectors while reducing the number of false positive detections. Despite these advantages, CenterNet generally achieves lower inference speeds than lightweight YOLO models and requires careful optimisation for deployment on embedded robotic hardware. Nevertheless, its accurate localisation capabilities make it attractive for applications requiring precise human position estimation.

2.2.4.5 EfficientDet

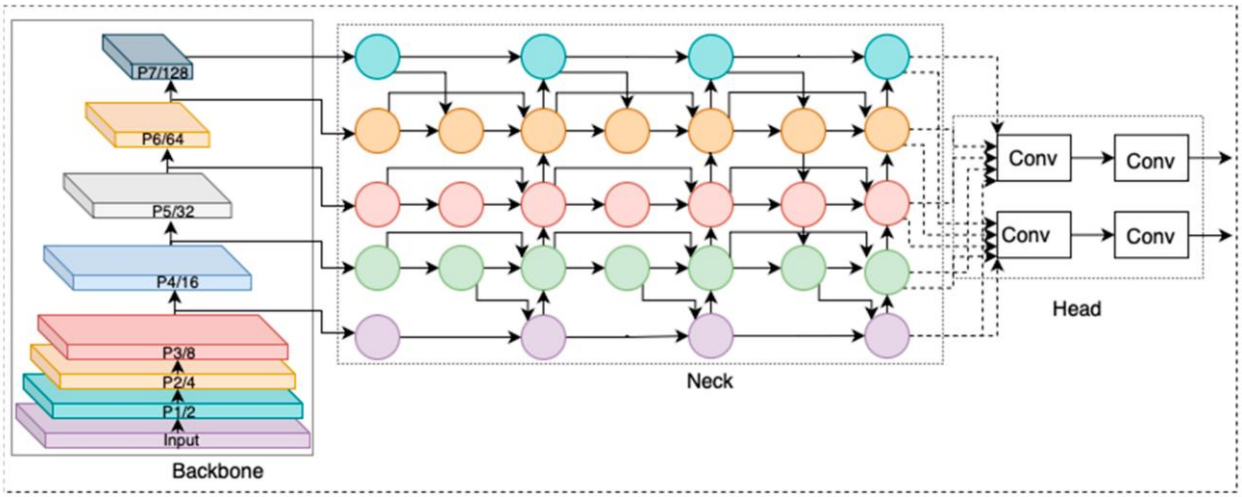


Figure 2.28 - Architectural Overview of EfficientDet (Ali & Zhang, 2024)

EfficientDet, introduced by Tan *et al*., (2020), was designed to maximise detection accuracy while minimising computational cost through compound model scaling. The architecture combines EfficientNet backbones with a Bidirectional Feature Pyramid Network (BiFPN), enabling efficient multi-scale feature fusion. One of EfficientDet's principal contributions is its balanced scaling strategy, which simultaneously scales network depth, width, and image resolution according to available computational resources. Multiple model variants (EfficientDet-D0 to EfficientDet-D7) allow deployment across diverse hardware platforms ranging from embedded processors to high-performance GPUs. EfficientDet achieves state-of-the-art accuracy with fewer parameters than many competing detectors. However, its inference speed remains generally lower than lightweight YOLO variants, making YOLO a preferred choice for applications demanding very high frame rates, including autonomous person-following robots.

2.2.4.6 Critical Comparison of Deep Learning-Based Person-Following Systems

Deep learning has significantly improved robotic perception by providing robust feature learning, superior object localisation, and improved generalisation across diverse operating environments. However, detector selection involves balancing detection accuracy, computational efficiency, inference speed, and hardware constraints. Among these reviewed

detectors, Faster R-CNN consistently achieves the highest localisation accuracy but exhibits relatively slow inference speeds because of its two-stage architecture. SSD substantially improves processing speed while maintaining acceptable detection performance, although its accuracy decreases for small or partially occluded objects. CenterNet simplifies object detection through anchor-free localisation and achieves competitive accuracy with reduced architectural complexity. EfficientDet provides an excellent balance between accuracy and computational efficiency through compound scaling but remains computationally demanding for many embedded robotic systems.

The YOLO family represents the most balanced solution for autonomous person-following robots. Its single-stage architecture provides high detection accuracy while maintaining extremely fast inference speeds suitable for real-time navigation. Continuous architectural improvements have further enhanced robustness, efficiency, and deployment flexibility across embedded platforms. Consequently, YOLO has become the preferred object detector for numerous autonomous robotic applications, including mobile service robots, autonomous navigation, surveillance robots, warehouse automation, and human-following systems.

Table 2.3 – Comparison of Deep Learning-Based Object Detectors

S/No.	Detector	Detection Type	Typical FPS*	Detection Accuracy	Computational Requirements	Suitability for Embedded Robotics
1.	Faster R-CNN	Two-stage	5–10	Very High	Very High (GPU recommended)	Low
2.	SSD	One-stage	20–45	High	Moderate	Moderate
3.	YOLO (latest lightweight variants)	One-stage	50–150+	Very High	Moderate	Excellent
4.	CenterNet	One-stage (Anchor-free)	25–45	High	Moderate–High	Good
5.	EfficientDet	One-stage	20–40	Very High	Moderate–High	Good

**Actual FPS depends on the model variant (e.g., nano vs. large), input resolution, processor, GPU, and optimization framework.*

The comparative analysis demonstrates that while Faster R-CNN and EfficientDet excel in localisation accuracy, their computational demands limit deployment on resource-constrained robotic platforms. SSD and CenterNet provide useful compromises between speed and accuracy but still exhibit limitations under challenging real-world conditions. Overall, the YOLO family offers the most favorable trade-off between inference speed, detection accuracy, computational efficiency, and embedded deployment, explaining its widespread adoption in autonomous mobile robotics and its selection as the perception framework for this research.

2.2.5 Review of Tracking Algorithms

Tracking algorithms have evolved from probabilistic estimation methods such as the Kalman Filter to modern tracking by detection frameworks that combine deep learning with motion prediction and appearance modelling.

2.2.5.1 Kalman Filter

The Kalman Filter is one of the earliest and most influential recursive estimation algorithms used for object tracking. It estimates the hidden state of a moving object by combining predictions from a mathematical motion model with noisy sensor observations (Kalman, 1960). In robotic person-following applications, the Kalman Filter predicts the future position and velocity of a detected individual while correcting prediction errors using incoming visual measurements. The algorithm consists of two sequential stages: prediction and measurement update. During prediction, the current state estimate is propagated forward according to a motion model. During the update stage, the prediction is corrected using new observations obtained from sensors such as cameras or LiDAR. The Kalman Filter is computationally efficient, making it highly suitable for real-time robotic applications. It performs well when target motion is smooth and approximately linear. However, its performance deteriorates under highly nonlinear human movement, sudden direction changes, long-term occlusions, or inaccurate motion models. Resulting to Kalman Filters often integrated with more sophisticated tracking algorithms rather than being used independently.

Table 2.4 - Advantages, Disadvantages and Applications of Kalman Filters

Advantages	Disadvantages	Applications
• Computationally efficient • Low memory requirements • Real-time performance • Simple mathematical formulation • Excellent for smooth target motion	• Assumes linear system dynamics • Sensitive to model inaccuracies • Performance degrades during long occlusions • Cannot recognise object identity	• Mobile robotics • Autonomous navigation • Radar tracking • GPS localisation • Sensor fusion

2.2.5.2 Simple Online Realtime Tracking

SORT, proposed by Bewley *et al.* (2016), represented a key breakthrough in real-time multiple object tracking. Instead of learning object appearance, SORT combines Kalman Filter motion prediction with the Hungarian assignment algorithm to associate detections across consecutive frames. SORT assumes that modern object detectors provide sufficiently accurate detections, allowing relatively simple motion estimation to achieve competitive tracking performance. The tracker predicts object locations using the Kalman Filter before matching predicted positions with new detections based on Intersection-over-Union (IoU). The main

strength of SORT lies in its extremely high processing speed, frequently exceeding several hundred frames per second on modern hardware. Consequently, SORT became widely adopted in autonomous driving, intelligent surveillance, and robotics. Nevertheless, SORT relies solely on motion information and therefore struggles during prolonged occlusions, crowded scenes, and interactions between multiple similar targets because it lacks appearance modelling.

Table 2.5 - Advantages, Disadvantages and Applications of SORT

Advantages	Disadvantages	Applications
• Extremely fast • Simple implementation • Low computational cost • Excellent real-time performance	• Cannot recover identities after occlusion • Sensitive to missed detections • Identity switching occurs frequently • No appearance modelling	• Autonomous vehicles • Mobile robots • Intelligent surveillance • Real-time pedestrian tracking

2.2.5.3 DeepSORT

DeepSORT extends SORT by incorporating deep appearance descriptors into the data association process (Wojke *et al*., 2017). In addition to Kalman Filter motion prediction, DeepSORT extracts discriminative visual features from each detected object using a convolutional neural network. These appearance embeddings enable the tracker to distinguish visually similar individuals even when temporary occlusions occur. Matching therefore considers both motion consistency and appearance similarity, substantially reducing identity switching. DeepSORT has become one of the most widely adopted tracking algorithms in robotics because of its balance between tracking accuracy and computational efficiency. It is particularly suitable for person-following robots operating in crowded environments where maintaining consistent target identity is essential. However, extracting appearance features increases computational complexity compared with SORT, requiring greater GPU resources for real-time operation.

Table 2.6 - Advantages, Disadvantages and Applications of DeepSORT

Advantages	Disadvantages	Applications
• Excellent identity preservation • Robust under partial occlusion • Reduced identity switching • High tracking accuracy	• Higher computational cost • Requires GPU acceleration for optimal performance • Slightly lower frame rate than SORT	• Person-following robots • Human–robot interaction • Smart surveillance • Multi-camera tracking • Crowd monitoring

2.2.5.4 ByteTrack

ByteTrack, introduced by Zhang *et al.* (2022), proposed a novel tracking strategy that associates both high-confidence and low-confidence detections rather than discarding low-confidence predictions. Conventional tracking algorithms ignore low-confidence detections because they are assumed to represent false positives. ByteTrack instead demonstrates that many low-confidence detections correspond to partially occluded or distant objects. By incorporating these detections into the association process, ByteTrack significantly improves tracking continuity while maintaining high processing speed. ByteTrack has achieved state-of-the-art performance on several Multiple Object Tracking (MOT) benchmarks and is increasingly adopted in robotic perception systems requiring robust tracking under crowded conditions.

Table 2.7 - Advantages, Disadvantages and Applications of ByteTrack

Advantages	Disadvantages	Applications
• Excellent tracking accuracy • Handles occlusion effectively • Preserves identities well • Very high processing speed • Simple integration with modern detectors	• Performance depends strongly on detector quality • Increased association complexity • Limited appearance modelling	• Autonomous robots • Intelligent transportation • Video surveillance • Crowd analytics • Human tracking

2.2.5.5 BoT-SORT

BoT-SORT (Bag-of-Tricks SORT), proposed by Aharon *et al.* (2022), further improves DeepSORT by integrating robust motion compensation, appearance modelling, and advanced data association techniques. Unlike previous trackers, BoT-SORT incorporates camera motion compensation to improve tracking when the observing camera itself is moving—a particularly important capability for autonomous mobile robots. It also combines high-quality appearance embeddings with enhanced Kalman filtering and refined association strategies. Experimental evaluations demonstrate that BoT-SORT achieves state-of-the-art performance on multiple tracking benchmarks, outperforming DeepSORT and ByteTrack in identity preservation and overall tracking accuracy. Although computationally more demanding, BoT-SORT provides one of the most reliable tracking frameworks currently available for autonomous person-following applications.

Table 2.8 - Advantages, Disadvantages and Applications of BoT-SORT

Advantages	Disadvantages	Applications
• Outstanding tracking accuracy • Excellent identity preservation • Robust under camera motion • Effective during long occlusions • State-of-the-art MOT performance	• Highest computational complexity • Greater GPU requirements • More complex implementation	• Autonomous mobile robots • Human-following robots • Intelligent surveillance • Autonomous driving • Smart city monitoring

2.2.5.6 Critical Comparison of Tracking Algorithms

Tracking algorithms have evolved considerably from purely motion-based estimators to sophisticated deep learning-assisted frameworks capable of maintaining target identity under challenging environmental conditions. The Kalman Filter provides efficient state estimation but cannot independently resolve object identity or handle complex nonlinear motion. SORT significantly improves real-time tracking performance through motion prediction and efficient data association but remains susceptible to identity switching during occlusion. DeepSORT addresses these shortcomings by incorporating deep appearance descriptors, substantially improving identity preservation in crowded environments. ByteTrack further advances tracking robustness by intelligently associating low-confidence detections, thereby improving tracking continuity under occlusion without sacrificing speed. BoT-SORT combines motion compensation, appearance modelling, and refined association techniques to achieve state-of-the-art tracking accuracy, making it particularly suitable for autonomous mobile robots operating in dynamic environments. A high-performance detector such as YOLO combined with DeepSORT, ByteTrack, or BoT-SORT enables reliable target detection, continuous tracking, and stable human-following behaviour under realistic operating conditions. Considering the need for real-time operation, robust identity preservation, and adaptability to mobile platforms, ByteTrack and BoT-SORT currently represent some of the most promising tracking solutions for intelligent autonomous robotics.

Table 2.9 - Comparisons of Tracking Algorithms

S/No.	Algorithms	Motion Prediction	Appearance Model	Speed	Occlusion Handling	Computational Requirement	Typical Applications
1.	Kalman Filter	✓	✗	Very High	Poor	Very Low	State estimation, sensor fusion
2.	SORT	✓	✗	Very High	Moderate	Low	Real-time object tracking

3.	DeepSORT	✓	✓	High	Good	Moderate	Person-following, surveillance
4.	ByteTrack	✓	Limited	Very High	Excellent	Moderate	Real-time multi-object tracking
5.	BoT-SORT	✓	✓	High	Excellent	High	Robotics, autonomous driving, surveillance

2.2.6 Review of Mobile Robot Platforms

The capabilities of a mobile robot platform significantly influence the performance of person-following systems, particularly in terms of sensing, mobility, computational capacity, payload, and environmental adaptability. Consequently, selecting an appropriate robotic platform is a critical design consideration in autonomous robotics research (Siegwart *et al*., 2011).

Modern research has employed numerous robotic platforms for autonomous navigation and person-following applications, including TurtleBot, Clearpath Husky, Robotnik Summit XL, Fetch Robot, and Boston Dynamics Spot. These platforms differ considerably in their hardware configurations, sensing capabilities, locomotion mechanisms, and intended research applications. The following sections review these platforms and compare their suitability for autonomous person-following research.

2.2.6.1 TurtleBot

TurtleBot is one of the most widely adopted open-source mobile robot platforms for education and robotics research. Developed through collaboration between Willow Garage, Open Robotics, and ROBOTIS, TurtleBot was designed to provide an affordable and ROS-compatible platform for research in autonomous navigation, computer vision, simultaneous localisation and mapping (SLAM), and human–robot interaction (Quigley *et al.*, 2009). The current TurtleBot3 series integrates sensors such as an RGB camera, LiDAR, inertial measurement unit (IMU), wheel encoders, and onboard embedded computers. These sensing capabilities enable implementation of localisation, obstacle avoidance, path planning, and object detection algorithms. Owing to its low cost, extensive documentation, and strong ROS integration, TurtleBot has become one of the most widely used research platforms in universities and robotics laboratories worldwide (ROBOTIS, 2024). Despite these advantages, TurtleBot possesses a relatively low payload capacity and limited processing power compared with industrial mobile robots. Its differential-drive locomotion also restricts operation to relatively smooth indoor environments, making it less suitable for outdoor navigation and computationally intensive deep learning applications.

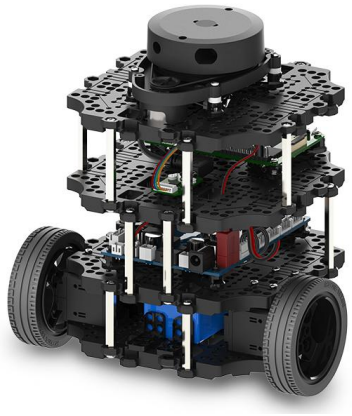


Figure 2.29 - TurtleBot3 (ROBOTIS, 2026)

2.2.6.2 Clearpath Husky

The Clearpath Husky is a rugged four-wheel unmanned ground vehicle specifically designed for outdoor autonomous robotics research. Unlike lightweight educational platforms, Husky employs a skid-steering drive mechanism that enables traversal of rough terrain including grass, gravel, mud, and uneven outdoor environments (Clearpath Robotics, 2024). The platform supports integration of multiple sensing modalities including LiDAR, stereo cameras, RGB-D cameras, GPS receivers, IMUs, robotic manipulators, and environmental sensors. Its high payload capacity enables researchers to integrate numerous sensors and powerful onboard computers for autonomous perception and navigation. Clearpath Husky has been widely employed in autonomous driving research, agricultural robotics, mining automation, environmental monitoring, and outdoor human-following systems because of its robustness and excellent mobility. However, its relatively large dimensions and higher acquisition cost make it less practical for indoor service robotics and laboratory-based human–robot interaction research.



Figure 2.30 - Husky A300 (Clearpath Robotics, 2026)

2.2.6.3 Robotnik Summit XL

The Robotnik Summit XL is an industrial-grade autonomous mobile robot developed for research and industrial automation. The platform employs a four-wheel omnidirectional drive system that enables highly manoeuvrable navigation within both indoor and outdoor environments. Its omnidirectional mobility provides superior manoeuvrability compared with conventional differential-drive robots, particularly in confined environments where precise positioning is required (Robotnik Automation, 2024). Summit XL supports numerous sensing

devices including LiDAR, RGB cameras, RGB-D cameras, stereo vision systems, GPS, IMUs, ultrasonic sensors, and laser scanners. Furthermore, its modular architecture enables straightforward integration of additional computational hardware such as NVIDIA GPUs for deep learning inference and high-performance embedded computers. A major advantage of Summit XL is its seamless integration with ROS, allowing researchers to implement autonomous navigation, localisation, perception, manipulation, and human-following algorithms within a unified software framework. Its relatively large payload capacity also enables simultaneous deployment of multiple sensing systems without compromising mobility.



Figure 2.31 - Robotnik Summit XL (Robotnik Automation, 2024)

2.2.6.4 Fetch Robot

The Fetch Robot is a mobile manipulation platform designed primarily for service robotics and human–robot interaction research. Unlike conventional mobile robots, Fetch integrates a differential-drive mobile base with a seven-degree-of-freedom robotic manipulator, enabling simultaneous navigation and object manipulation (Fetch Robotics, 2021). The robot incorporates LiDAR, RGB-D cameras, force sensors, IMUs, and depth perception systems that facilitate autonomous navigation, object recognition, grasp planning, and collaborative human–robot interaction. These capabilities have made Fetch a popular research platform in healthcare robotics, domestic assistance, warehouse automation, and service robotics. Although Fetch provides exceptional flexibility for manipulation research, the inclusion of an articulated robotic arm substantially increases system complexity, computational requirements, and acquisition cost. Consequently, it is generally employed for manipulation and service robotics rather than dedicated autonomous person-following research.



Figure 2.32 - Fetch Advanced Mobile Manipulator (Fetch Robotics, 2021)

2.2.6.5 Boston Dynamics Spot

Boston Dynamics Spot represents one of the most advanced quadrupedal autonomous mobile robots currently available. Unlike wheeled mobile robots, Spot utilises four articulated legs that enable locomotion across stairs, rocky terrain, industrial facilities, and hazardous environments where wheeled platforms experience significant limitations (Boston Dynamics, 2024). Spot integrates multiple stereo cameras, depth sensors, IMUs, optional LiDAR sensors, and sophisticated perception algorithms that provide comprehensive environmental awareness. Advanced dynamic control algorithms enable the robot to maintain balance while traversing highly irregular terrain. The platform has demonstrated remarkable performance in industrial inspection, mining, construction monitoring, disaster response, infrastructure inspection, and security applications. However, Spot's acquisition cost, maintenance requirements, and computational complexity remain considerably higher than those of wheeled research robots. Although Spot provides unparalleled terrain adaptability, wheeled mobile robots generally remain more energy efficient, mechanically simpler, and easier to control for indoor autonomous person-following applications.



Figure 2.33 - Spot Robot (Boston Dynamics 2024)

2.2.6.6 Comparative Analysis of Mobile Robot Platforms

The identified platforms demonstrate that different robotic systems have been developed to address different research and industrial needs. TurtleBot remains one of the most suitable educational platforms owing to its affordability, extensive ROS ecosystem, and ease of software development. Nevertheless, its limited payload capacity and indoor-only operation restrict more advanced robotics research. Clearpath Husky provides outstanding outdoor mobility and supports numerous sensing configurations, making it highly appropriate for autonomous field robotics. Conversely, its relatively large size reduces manoeuvrability within indoor environments. Fetch Robot extends conventional mobile robotics through integrated manipulation capabilities, making it particularly suitable for service robotics and human–robot collaboration. Boston Dynamics Spot represents the current state of the art in legged locomotion and demonstrates exceptional adaptability to challenging terrain, albeit at substantially greater financial and computational cost. Among the reviewed platforms, the Robotnik Summit XL provides the most balanced combination of industrial robustness, omnidirectional mobility, high payload capacity, sensor flexibility, and ROS compatibility. These characteristics make it particularly suitable for implementing computationally intensive deep learning algorithms while enabling experimental validation under realistic operating conditions. Consequently, Summit XL provides an ideal platform for developing and evaluating adaptive vision-based autonomous person-following systems.

Table 2.10 - Comparison of Mobile Robot Platforms

S/No.	Platform	Sensors	Mobility	Payload Capacity	Research Suitability
1.	TurtleBot	LiDAR, RGB camera, IMU, wheel encoders	Indoor differential drive	Low	Education, SLAM, autonomous navigation
2.	Clearpath Husky	LiDAR, GPS, RGB-D camera, IMU, stereo cameras	Four-wheel skid steering	High	Outdoor robotics, agriculture, autonomous navigation
3.	Robotnik Summit XL	LiDAR, RGB/RGB-D cameras, stereo vision, GPS, IMU, ultrasonic sensors	Four-wheel omnidirectional	High	Deep learning, autonomous navigation, person-following
4.	Fetch Robot	LiDAR, RGB-D camera, force sensors, IMU	Differential drive with manipulator	Moderate	Service robotics, mobile

					manipulation, HRI
5.	Boston Dynamics Spot	Stereo cameras, depth sensors, IMU, optional LiDAR	Quadrupedal locomotion	Moderate	Inspection, hazardous environments, industrial autonomy

2.2.7 Review of ROS-Based Person-Following Systems

The Robot Operating System (ROS) has become the actual middleware for developing autonomous robotic systems due to its modular architecture, hardware abstraction, distributed communication framework, and extensive ecosystem of reusable software packages (Quigley *et al.*, 2009). Rather than functioning as the conventional operating system, ROS provides libraries, communication protocols, development tools, and package management systems that simplify the integration of perception, localisation, navigation, planning, and control algorithms within a unified robotic framework (Cousins, 2010).

In person-following robotics, ROS provides an ideal software architecture because it allows independent modules for image acquisition, object detection, tracking, localisation, robot control, and navigation to communicate through standardised messages. This modularity enables researchers to replace or upgrade individual components without redesigning the entire system. Consequently, ROS has become the dominant software framework for implementing autonomous person-following robots in both academic research and industrial applications (Mohamed *et al*., 2008; Macenski *et al.*, 2022).

2.2.7.1 ROS Stack Navigation

The ROS Navigation Stack is one of the most widely adopted software frameworks for autonomous mobile robot navigation. It enables robots to autonomously move from one location to another while avoiding obstacles and following planned trajectories. The navigation stack integrates several core packages including localisation, mapping, global path planning, local path planning, cost map generation, and velocity control (Marder-Eppstein *et al.*, 2010). The principal components of the ROS Navigation Stack are

- Adaptive Monte Carlo Localisation (AMCL) - used for probabilistic robot localisation.
- *move_base* - coordinates global and local planning.
- Global planners - computes collision-free paths.
- Local planners - generates feasible velocity commands.
- Costmaps - represents static and dynamic obstacles.
- Recovery behaviours - enables robots to recover from navigation failures

Within person-following applications, the navigation stack can be integrated with human detection algorithms to continuously update the robot's navigation goal based on the estimated position of the target. Instead of navigating toward predefined map coordinates, the robot dynamically follows the moving human while avoiding surrounding obstacles.

The modular design of the navigation stack has made it one of the most widely adopted frameworks for autonomous mobile robotics. Nevertheless, the conventional ROS Navigation

Stack was originally designed for waypoint navigation rather than dynamic human following. Consequently, additional perception and tracking modules are required to continuously update navigation goals in response to human movement.

2.2.7.2 ROS Perception Pipeline

The ROS perception pipeline provides the software infrastructure required for acquiring, processing, and interpreting sensory information. It enables seamless communication between cameras, LiDAR sensors, inertial measurement units, depth cameras, and perception algorithms through ROS topics, services, and action interfaces. A typical ROS-based perception pipeline for autonomous person-following consists of the following stages

i. Sensor acquisition - Where image data are obtained from RGB or RGB-D cameras.
ii. Image preprocessing - Performs image conversion, resizing, filtering, and normalisation.
iii. Object detection - Where deep learning models such as YOLO identify human targets.
iv. Object tracking - Uses algorithms such as DeepSORT or ByteTrack to maintain target identity.
v. Position estimation - Where bounding box information is converted into relative target position.
vi. Motion control - Where navigation commands are generated and published to the robot base.

ROS provides numerous packages that simplify implementation of these stages, including, *image_transport, cv_bridge*, *image_pipeline*, OpenCV integration, PCL (Point Cloud Library), and tf and tf2 transformation libraries.

The perception pipeline's modularity enables researchers to replace one detector with another—for example, replacing HOG with YOLO—without modifying other components of the robotic system. This flexibility significantly accelerates robotics research and software maintenance. Furthermore, ROS supports distributed computing, allowing computationally intensive perception tasks to execute on dedicated external computers while lightweight motion controllers operate onboard the robot. This capability is particularly advantageous for deep learning applications requiring GPU acceleration.

2.2.7.3 Existing ROS-Based Person-Following Implementations

Numerous researchers have employed ROS as the software framework for implementing autonomous person-following robots. Early ROS-based systems relied primarily on classical computer vision algorithms such as Haar Cascade classifiers, HOG, colour tracking, and optical flow. These approaches generally combined OpenCV with ROS image-processing packages to detect and track pedestrians while publishing velocity commands to the mobile robot. With the rapid development of deep learning, more recent ROS implementations have integrated convolutional neural networks including Faster R-CNN, SSD, and especially the YOLO family of object detectors. These systems typically employ the following ROS architecture:

- Camera node for image acquisition.
- Detection node executing the deep learning model.

- Tracking node maintaining target identity.
- Controller node generating velocity commands.
- Navigation node handling robot motion.
- Visualisation node using RViz for monitoring system performance.

Several recent studies have demonstrated that combining ROS with YOLO-based object detection significantly improves human detection accuracy while maintaining real-time operation (Redmon *et al.*, 2016; Redmon & Farhadi, 2018; Bochkovskiy *et al*., 2020).

2.2.7.4 Limitations of Existing ROS-Based Person-Following Systems

Despite the widespread adoption of ROS, existing ROS-based person-following systems exhibit several limitations. Many implementations rely on classical computer vision algorithms that perform poorly under illumination changes, background clutter, and partial occlusions. These systems frequently lose the target during rapid human movement or when multiple individuals appear within the camera's field of view. Most existing systems are validated only within simulation environments such as Gazebo. Although simulation provides a convenient development platform, it cannot accurately reproduce sensor noise, communication delays, lighting variation, wheel slippage, and environmental uncertainty encountered during real-world deployment. Consequently, algorithms that perform well in simulation often exhibit reduced robustness on physical robots.

Furthermore, many existing controllers generate oscillatory robot motion because velocity commands are computed directly from noisy detection measurements without adequate filtering or adaptive control. This behaviour reduces navigation stability and may compromise safety in indoor environments shared with humans. ROS 1 also presents inherent architectural limitations. Its communication framework lacks native real-time guarantees, built-in cybersecurity mechanisms, and deterministic Quality of Service (QoS), which are increasingly important for safety-critical robotic applications. These limitations motivated the development of ROS 2, which introduces Data Distribution Service (DDS)-based communication, improved real-time performance, enhanced security, and configurable QoS policies (Macenski *et al.*, 2022).

2.2.7.5 Critical Analysis of ROS-Based Person-Following Systems

ROS has transformed robotics research by providing a flexible, modular, and scalable software framework that enables rapid integration of perception, navigation, localisation, and control algorithms. Its extensive package ecosystem, hardware abstraction, and distributed communication model have significantly accelerated the development of autonomous mobile robots. However, existing ROS-based person-following implementations continue to face challenges related to computational efficiency, target loss during occlusion, dependence on accurate object detection, limited real-world validation, and the absence of adaptive control strategies. These limitations indicate that achieving robust autonomous person-following requires not only advanced deep learning algorithms but also efficient system integration, reliable communication, and stable robot control.

2.2.8 Review of Adaptive Person-Following Controllers

Over the past decades, numerous control strategies have been proposed for autonomous person-following robots, ranging from classical Proportional–Integral–Derivative (PID) controllers to intelligent approaches based on fuzzy logic, Model Predictive Control (MPC), reinforcement learning, and adaptive control. Each control technique exhibits distinct characteristics regarding computational complexity, stability, robustness, and suitability for real-time robotic applications. The following sections review these major control strategies and discuss their applicability to autonomous person-following systems.

2.2.8.1 Proportional Integral Derivative (PID) Control

The Proportional Integral Derivative (PID) controller remains one of the most widely used control algorithms in robotics and industrial automation because of its simplicity, robustness, and ease of implementation (Åström & Hägglund, 2006). A PID controller generates a control signal based on the current error (proportional term), accumulated past errors (integral term), and predicted future error derived from the error rate of change (derivative term). In autonomous person-following robots, PID controllers are commonly used to regulate linear and angular velocities according to the difference between the desired and measured human position. The horizontal displacement of the detected person from the image centre is used to control robot orientation, while the estimated distance to the person controls forward or backward movement. PID controllers provide smooth and predictable behaviour when system dynamics remain relatively constant. However, their performance depends heavily on accurate parameter tuning. Fixed PID gains often become suboptimal when human motion changes rapidly, environmental conditions vary, or sensor measurements become noisy.

Table 2.11 - Advantages and Limitations of PID Control

Advantages	Limitation
• Simple implementation • Low computational cost • Excellent real-time performance • Well-understood mathematically • Reliable for linear systems	• Requires careful gain tuning • Performance degrades under nonlinear dynamics • Sensitive to measurement noise • Limited adaptability

2.2.8.2 Fuzzy Logic Control

Fuzzy logic control extends classical control by representing human reasoning through linguistic variables rather than precise mathematical models (Zadeh, 1965). Instead of relying on exact equations, fuzzy controllers employ expert-defined rules such as:

- IF the person is *far*, THEN move *fast*.
- IF the person is *slightly left*, THEN turn *slowly left*.

This rule-based approach enables controllers to operate effectively under uncertainty and incomplete information. Fuzzy controllers have therefore been widely applied in mobile robotics where perception measurements are often noisy and imprecise.

For person-following applications, fuzzy logic controllers can simultaneously process distance, relative orientation, target velocity, and obstacle information to generate smooth motion commands. Unlike PID controllers, fuzzy systems naturally accommodate nonlinear robot behaviour without requiring an explicit mathematical model. However, designing effective fuzzy rule bases requires considerable expert knowledge. As system complexity increases, the number of rules grows rapidly, making controller design and optimisation increasingly difficult.

Table 2.12 - Advantages and Limitations of Fuzzy Logic Control

Advantages	Limitation
• Handles nonlinear behaviour • Robust to noisy measurements • Human-readable decision rules • Smooth control responses • No precise mathematical model required	• Rule design requires expertise • Scalability issues • Difficult optimisation • No guaranteed optimality

2.2.8.3 Model Predictive Control

Model Predictive Control (MPC) is an advanced optimisation-based control strategy that predicts future system behaviour over a finite time horizon and computes optimal control actions while satisfying system constraints (Camacho & Bordons, 2007). Unlike reactive controllers, MPC continuously predicts the future trajectory of both the robot and the target person. At each control cycle, an optimisation problem is solved to determine the velocity commands that minimise tracking error while respecting constraints such as maximum speed, acceleration limits, obstacle avoidance, and collision prevention. MPC explicitly considers future behaviour, which makes it generally produce smoother trajectories and greater stability than conventional PID controllers. These characteristics have made MPC increasingly attractive for autonomous vehicles, industrial robots, and mobile robot navigation.

Despite these advantages, MPC requires repeated online optimisation, resulting in considerably higher computational requirements. Consequently, its real-time implementation often requires powerful processors or GPU acceleration.

Table 2.13 - Advantages and Limitations of Model Predictive Control (MPC)

Advantages	**Limitation**
• Excellent stability • Predictive behaviour • Handles system constraints • Smooth trajectories • High tracking accuracy	• High computational cost • Requires accurate system models • Complex implementation • Challenging real-time optimisation

2.2.8.4 Reinforcement Learning

Reinforcement Learning (RL) enables autonomous agents to learn optimal control policies through repeated interaction with their environment (Sutton & Barto, 2018). Rather than relying on manually designed control laws, RL learns behaviours by maximising cumulative rewards based on trial-and-error experience. In person-following robotics, reinforcement learning algorithms can learn navigation strategies that simultaneously optimise following distance, orientation, obstacle avoidance, and energy efficiency. Recent advances in Deep Reinforcement Learning (DRL) combine neural networks with reinforcement learning to solve highly complex robotic control problems directly from visual observations. Reinforcement learning-based controllers possess remarkable adaptability and can discover control strategies beyond human intuition. Nevertheless, they generally require enormous training datasets, extensive computational resources, and carefully designed reward functions. Furthermore, ensuring safety during online learning remains a significant challenge for real-world robotics.

Table 2.14 - Advantages and Limitations of Reinforcement Learning

Advantages	Limitation
• Learns optimal behaviour • Highly adaptive • Handles complex nonlinear systems • No explicit mathematical model required • Suitable for dynamic environments	• Long training time • High computational demand • Difficult reward design • Safety concerns during learning • Limited explainability

2.2.8.5 Adaptive Control

Adaptive control represents a family of controllers capable of automatically modifying their control parameters in response to changing system dynamics (Ioannou & Sun, 2012). Unlike fixed-gain controllers, adaptive controllers continuously estimate system behaviour and update controller gains during operation. Walking speed, direction, acceleration, and stopping behaviour change continuously, making fixed controller parameters unsuitable for all operating conditions. Adaptive controllers can adjust robot velocity according to variations in target distance, image position, environmental conditions, and sensor uncertainty. This is achieved as the controller developed in this research follows an adaptive philosophy by continuously updating robot motion based on real-time visual feedback obtained from YOLO-based human detection. Rather than relying on predefined trajectories, robot velocity is dynamically adjusted according to the continuously changing position of the target person.

Table 2.15 - Advantages and Limitations of Adaptive Control

Advantages	Limitation
• Excellent robustness • Automatically adjusts parameters • Handles changing environments • Improved tracking stability	• More complex design • Stability analysis is challenging • Increased computational requirements

• Suitable for uncertain systems	• Requires reliable parameter estimation

2.2.8.6 Critical Comparison of Adaptive Person-Following Controllers

The reviewed controllers demonstrate varying trade-offs between computational efficiency, robustness, and adaptability. PID controllers remain attractive because of their simplicity and low computational requirements but exhibit limited adaptability in highly dynamic environments. Fuzzy logic controllers improve robustness to uncertainty by incorporating expert knowledge but become increasingly difficult to design as system complexity increases.

Model Predictive Control provides excellent trajectory optimisation and stability through predictive optimisation; however, its computational burden limits deployment on resource-constrained robotic platforms. Reinforcement learning offers the greatest adaptability and has demonstrated promising results for autonomous navigation, yet its substantial training requirements and limited interpretability currently restrict widespread deployment in safety-critical systems. Furthermoe, adaptive control combines many of the desirable characteristics of classical and intelligent control methods by continuously adjusting controller behaviour according to changing operating conditions. Its ability to accommodate unpredictable human motion while maintaining stable robot behaviour makes it particularly suitable for autonomous person-following applications.

Table 2.16 - Comparison of Adaptive Person-Following Controllers

S/No.	Controller	Stability	Computational Complexity	Computational Cost	Adaptability	Suitability for Real-Time Person Following
1.	PID	High (well-tuned)	Low	Low	Low	Good
2.	Fuzzy Logic	High	Moderate	Moderate	Moderate	Good
3.	Model Predictive Control (MPC)	Very High	High	High	Moderate	Very Good (high-end hardware)
4.	Reinforcement Learning	Variable –High	Very High	Very High	Very High	Moderate (after training)
5.	Adaptive Control	High	Moderate	Moderate	High	Excellent

This comparison indicates that although each controller has specific strengths, adaptive control provides the most appropriate balance between stability, computational efficiency, and responsiveness for vision-based autonomous person-following. Unlike fixed-gain controllers, adaptive control continuously adjusts robot behavior in response to changing human motion, making it particularly suitable for real-time deployment in dynamic indoor environments.

2.3 COMPARATIVE ANALYSIS OF EXISTING STUDIES ON HUMAN-FOLLOWING ROBOTS

A careful comparative analysis of representative studies related to autonomous human-following robots have been identified. The reviewed studies are compared based on the object detection algorithm, tracking approach, robotic platform, experimental environment, major strengths, and identified limitations. This comparison highlights the evolution of person-following technologies from traditional computer vision methods to modern deep learning-based approaches while revealing the research gaps addressed by this present study.

2.3.1 Critical Discussion

The comparative analysis reveals a clear progression in autonomous person-following research. Early studies primarily relied on classical computer vision techniques such as HOG, SVMs, colour-based tracking, and Kalman filtering for pedestrian detection and target tracking (Dalal & Triggs, 2005; Comaniciu *et al*., 2003; Kalal *et al*., 2012). Although these approaches were computationally efficient and suitable for real-time implementation on resource-constrained robotic platforms, they were highly sensitive to illumination variations, background clutter, viewpoint changes, and partial occlusions, often resulting in target loss in dynamic environments (Bradski, 1998; Yilmaz *et al*., 2006).

The emergence of deep learning significantly transformed person-following research by substantially improving object detection performance. Deep CNN-based detectors, including Faster R-CNN, Single Shot Multi-Box Detector (SSD), and successive versions of the YOLO family, achieved considerably higher detection accuracy while maintaining real-time inference capabilities (Ren *et al.*, 2015; Liu *et al.*, 2016; Redmon *et al.*, 2016; Redmon & Farhadi, 2018; Bochkovskiy *et al*., 2020). Among these methods, the YOLO family has become the preferred choice for autonomous mobile robotics because it offers an excellent balance between detection accuracy, computational efficiency, and inference speed, making it suitable for real-time robotic perception (Jocher *et al.*, 2023). Recent studies have further enhanced tracking robustness by integrating advanced multi-object tracking algorithms such as DeepSORT, ByteTrack, and BoT-SORT, which significantly improve target identity preservation during temporary occlusions and crowded scenes (Wojke *et al*., 2017; Zhang *et al.*, 2022; Aharon *et al.*, 2022).

Despite these technological advances, several important limitations remain. Many existing human following systems have been evaluated primarily within simulation environments or on educational robotic platforms such as TurtleBot, with relatively few studies demonstrating deployment on industrial mobile robots operating under realistic environmental conditions (Quigley *et al.*, 2009; Macenski *et al.*, 2022). Furthermore, much of the existing literature, identified, focuses predominantly on perception and tracking while providing comparatively limited attention to adaptive motion control capable of responding smoothly to continuous changes in human position. Robust operation under illumination variation, dynamic indoor environments, temporary occlusions, and prolonged autonomous deployment therefore remains an active research challenge (Islam *et al*., 2018; Chen *et al.*, 2024).

Table 2.17 - Comparative Analysis of Existing Studies on Human-Following Robots

S/No.	Author(s)	Detection Method	Tracking Method	Robot Platform	Environment	Strengths	Limitations
1.	Treptow *et al*., (2006)	Thermal-vision-based human detection using an elliptical contour model and an integral-image feature model	Particle-filter-based tracking for maintaining person tracks	ActivMedia PeopleBot equipped with a thermal camera	Indoor office/laboratory environment, including corridors and rooms	Thermal vision reduces dependence on visible illumination; capable of detecting and tracking people in real time; evaluated on a physical mobile robot	Older feature-based detection approach; thermal appearance can vary with environmental and human factors
2.	Satake *et al*., (2013)	Laser Scanner	Kalman Filter	Mobile robot	Indoor	Robust localisation	No visual perception
3.	Munaro *et al*., (2015)	RGB-D Camera	Skeleton Tracking	Service robot	Indoor	Accurate distance estimation	Requires depth camera
4.	Chen *et al*., (2017)	RGB Vision	Particle Filter	Mobile robot	Indoor	Robust target recovery	Limited under heavy occlusion
5.	Islam *et al*., (2018)	Survey	Various	Ground/Aerial/ Underwater	Various	Comprehensive taxonomy	No experimental validation
6.	Redmon & Farhadi (2018)	YOLOv3	None	Generic	Indoor/Outdoor	Real-time detection	No temporal tracking
7.	Bochkovskiy *et al*., (2020)	YOLOv4	None	Generic	Various	Improved accuracy	Higher computational demand
8.	Algabri *et al*., (2020)	MobileNet-SSD	HSV colour-feature histogram	Mobile robot with RGB-D camera	Indoor	Robust human following under partial occlusion	Relied on RGB-D and LiDAR sensors,

						and moderate illumination changes	increasing sensor dependence
9.	Koide *et al*., (2020)	OpenPose monocular human-pose detection using ground-plane information and estimated human height	Unscented Kalman Filter (UKF) with target identification using Convolutional Channel Features and online boosting	Mobile robot with an NVIDIA Jetson TX2 onboard computing platform	Indoor and outdoor environments	Monocular-camera solution; estimates target position in robot coordinates; incorporates target identification; demonstrated person-following in different environments	More complex than simple bounding-box-based following; requires ground-plane and human-height estimation
10.	Tsai & Yao (2021)	HOG features + SVM human detector, with foreground extraction	Block Matching Algorithm (BMA) + colour-histogram matching + Kalman-filter prediction	Wheeled mobile robot	Indoor and outdoor	Real-time specific-person tracking; reduced detection computation using block matching; maintained target tracking among multiple people; achieved >85% precision and recall	Required a stereo/depth camera for target distance; relied on hand-crafted HOG/colour features
11.	Zhang *et al*., (2022)	YOLOv5	ByteTrack	Mobile robot	Indoor	Improved identity preservation	High GPU requirement
12.	Aharon *et al*., (2022)	YOLOv5	BoT-SORT	Generic robot	Indoor	Excellent tracking robustness	Computationally intensive

13.	Macenski *et al*., (2022)	ROS Navigation	ROS Navigation Stack	Various	Indoor/Outdoor	Modular ROS2 architecture	Focus on navigation rather than person following
14.	Shi *et al*., (2022)	Monocular-vision pedestrian detection and height estimation	Focuses on estimating pedestrian height and distance from monocular images	Focuses on monocular-vision measurement	Pedestrian/road-scene visual environments	Demonstrates that geometric information about a pedestrian can be extracted from a single camera	Does not constitute a complete robot person-following system and does not evaluate robot tracking or motion control
15.	Alkandary *et al*., (2025)	YOLOv3–YOLOv10	DeepSORT/StrongSORT	Experimental platform	Indoor	Comprehensive detector-tracker comparison	Did not implement person-following control.
16.	Ye *et al*., (2025)	Visual person detection/localisation	Field-based target-search and recovery (RPF-Search)	Mobile robot	Unknown dynamic environments	Robust recovery from target loss caused by occlusion	Focuses primarily on target-search/recovery rather than developing a dedicated YOLO-based detector or adaptive visual-servoing controller
17.	Song *et al*., (2026)	Enhanced YOLOv8	Bounding-box feedback	Mobile Robot	Indoor	Improved detection, path planning and following	Limited evaluation to specific scenarios.
18.	**Present Study**	**YOLOv8**	**Continuous visual target estimation**	**Robotnik Summit XL**	**Real indoor environment**	**Real-time detection, adaptive forward/backward following, ROS implementation on industrial robot**	**Single target, monocular camera, no obstacle avoidance (current implementation)**

2.4 RESEARCH GAP

The preceding literature review demonstrates that considerable progress has been achieved in the development of autonomous human-following robots through advances in computer vision, deep learning, mobile robotics, and intelligent control. Early research predominantly employed classical computer vision techniques, including Haar Cascade classifiers, HOG, colour tracking, optical flow, and background subtraction, owing to their low computational requirements and ease of implementation. Although these methods enabled real-time operation on resource-constrained robotic platforms, they exhibited limited robustness under changing illumination, background clutter, viewpoint variations, and partial occlusions, frequently resulting in unstable tracking and target loss (Yilmaz *et al*., 2006).

The introduction of deep learning-based object detectors such as Faster R-CNN, SSD, and the YOLO family significantly improved human detection accuracy and robustness (Ren *et al.*, 2015; Liu *et al.*, 2016; Redmon *et al.*, 2016; Bochkovskiy *et al*., 2020). More recent studies have further enhanced tracking performance through the integration of algorithms such as DeepSORT, ByteTrack, and BoT-SORT, enabling improved target identity preservation in crowded environments and during temporary occlusions (Wojke *et al*., 2017; Zhang *et al.*, 2022; Aharon *et al*., 2022). Despite these advances, several important research challenges remain unresolved.

A major limitation of many existing human-following systems is the assumption that human motion is relatively smooth and predictable. In practical indoor environments, human movement is often characterised by abrupt directional changes, variable walking speeds, sudden stops, and backward motion. These continuous positional changes frequently lead to unstable robot behaviour, oscillatory motion, delayed responses, or complete loss of the target because many controllers employ fixed control parameters that cannot adapt to rapidly changing conditions (Åström & Hägglund, 2006; Sutton & Barto, 2018).

Another significant limitation concerns sensing requirements. Numerous published systems depend on expensive perception hardware such as LiDAR sensors, RGB-D cameras, stereo vision systems, or multi-sensor fusion architectures to estimate human position and distance (Munaro *et al.*, 2015; Siegwart *et al*., 2011). While these sensing modalities generally improve localisation accuracy, they substantially increase system cost, computational complexity, power consumption, and deployment difficulty. Such requirements limit the practical adoption of autonomous person-following robots in cost-sensitive service robotics applications.

Furthermore, although deep learning has dramatically improved object detection performance, many existing systems still exhibit reduced robustness under varying illumination conditions, motion blur, cluttered backgrounds, and partial or prolonged occlusions. Detection failures under these conditions often propagate to the robot controller, producing erratic navigation behaviour and reducing overall system reliability (Chen *et al.*, 2024; Bochkovskiy *et al*., 2020).

A further observation from the reviewed literature is that relatively limited attention has been devoted to adaptive motion control for indoor person following. Many existing approaches concentrate primarily on perception and tracking accuracy while employing relatively simple proportional or fixed-gain controllers for robot motion. Consequently, robot navigation often exhibits oscillations, delayed responses, overshoot, or unstable following

behaviour when human position changes rapidly. More sophisticated control methods, such as Model Predictive Control and reinforcement learning, have demonstrated improved performance but generally require significantly greater computational resources and complex optimisation procedures that may not be suitable for real-time deployment on mobile robotic platforms (Camacho & Bordons, 2007; Sutton & Barto, 2018).

Another important research gap concerns the integration of high-performance deep learning with lightweight real-time control architectures. Many published systems employ computationally intensive perception algorithms without sufficiently considering efficient controller design and distributed robotic implementation. Consequently, achieving reliable real-time performance remains challenging, particularly when deploying deep neural networks on robots with limited onboard computational resources (Macenski *et al.*, 2022).

Lastly, a substantial proportion of reported person-following systems are validated primarily within simulation environments such as Gazebo or using small educational robotic platforms, including TurtleBot. Although simulation provides an effective platform for algorithm development, it cannot fully reproduce the sensor noise, communication latency, wheel slip, illumination variability, and environmental uncertainties encountered during real-world deployment. Consequently, relatively few studies have demonstrated complete implementation and experimental validation on industrial autonomous mobile robots operating under realistic indoor conditions (Quigley *et al.*, 2009; Macenski *et al.*, 2022).

In addressing some of these identified gaps, this research proposes an adaptive vision-based autonomous person-following framework implemented on a Robotnik Summit XL industrial mobile robot. The proposed system employs YOLOv8 for robust real-time human detection using a single RGB camera, thereby eliminating dependence on expensive sensing hardware such as LiDAR and RGB-D cameras. A lightweight adaptive visual control strategy is developed to regulate forward and backward robot motion in response to continuous changes in human position while maintaining stable real-time performance. Unlike many previous studies that rely primarily on simulation, the proposed framework is implemented and experimentally validated on a physical industrial mobile robot operating in real indoor environments using the Robot Operating System (ROS). Through the integration of modern deep learning-based perception, adaptive control, and real-world robotic experimentation, this research contributes toward the development of practical, robust, and computationally efficient autonomous person-following systems while providing a foundation for future research in obstacle avoidance, multi-person tracking, and vision-language robotics.

CHAPTER THREE

RESEARCH METHODOLOGY

This chapter provides a systematic description of the procedures, techniques, hardware platforms, software tools, and experimental processes employed to achieve the research objectives. It explains how the proposed framework was conceived, developed, integrated, and validated through real-world experimentation, thereby ensuring that the research is transparent, reproducible, and scientifically rigorous.

3.1 RESEARCH DESIGN

The system's framework was designed in a systematic, logical, and scientifically rigorous manner. The system setup combined principles from engineering design, computer vision deep learning-based human detection using YOLOv8 with monocular human position estimation, an adaptive visual motion controller and an experimental evaluation to achieve real-time autonomous person-following. The experimental outcome was evaluated under various indoor operating conditions to assess its detection accuracy, tracking performance, response time, and overall performance.

3.1.1 Research Philosophy

The philosophical foundation of this research is rooted in engineering research, applied research, and experimental research. These complementary philosophies provide the theoretical and methodological basis for designing, implementing, and validating the proposed autonomous person-following system.

a. Engineering Research

Engineering research focuses on the systematic design, development, optimisation, and evaluation of technological solutions to practical engineering problems (Blessing & Chakrabarti, 2009). It provides innovative systems, algorithms, and technologies that address identified challenges through scientific principles and rigorous experimentation to address the limitations of existing autonomous mobile robots in adapting to continuous human positional changes. The emphasis is therefore placed on system development, performance optimisation, and experimental validation using a real industrial robotic platform.

b. Applied Research

Applied research aims to generate practical knowledge that can be directly applied to solving real-world problems rather than developing purely theoretical concepts (Kothari, 2004). It focuses on translating scientific theories and technological advances into usable products, processes, or systems capable of addressing specific societal or industrial needs. In this case, an autonomous mobile robot to robustly follow a human target in dynamic indoor environments

for real deployments in healthcare assistance, warehouse automation, and intelligent service robotics.

c. Experimental Research

Experimental research is characterised by the systematic manipulation and observation of variables to evaluate the effectiveness of a proposed system under controlled conditions (Creswell & Creswell, 2018). Multiple experimental scenarios were conducted to objectively assess system performance. Also, quantitative and qualitative data collection was done for carefully designed experiments.

3.1.2 Research Approach

The research approach adopted in this study is a design and implementation approach supported by a systematic system development methodology and rigorous experimental validation. The reason for this approach was because of the need to not only investigate the challenges associated with autonomous person-following robots but also to develop, implement, and experimentally evaluate a practical solution for in real-world environments (Blessing & Chakrabarti, 2009; Creswell & Creswell, 2018).

a. Design and Implementation Approach

In this approach, the system design was conceived, developed, integrated, and evaluated to address the identified research problem. The approach commenced with a comprehensive review of existing literature on autonomous mobile robots, computer vision, deep learning, person-following systems, and ROS-based robotic applications. The knowledge obtained from the literature review informed the design of the framework that included a perception module, a human position estimation module, an adaptive motion control module, and a ROS-based communication system.

b. System Development Methodology

The development of the system followed an iterative engineering methodology consisting of requirement analysis, system design, software development, module integration, testing, evaluation, and refinement. The methodology allowed successive improvements to be incorporated following observations obtained during implementation and experimental testing. It began with identifying the functional and non-functional requirements of the proposed system based on the research objectives and limitations identified in existing literature. The functional requirements included real-time human detection, position estimation, autonomous forward and backward movement, adaptive speed control, and reliable communication between software modules while the non-functional requirements focused on computational efficiency, robustness, safety, modularity, scalability, and real-time performance.

c. Experimental Validation

The system's efficiency and effectiveness were evaluated through experimental validation. This provided objective evidence of system performance under realistic operating

environments while demonstrating the practical feasibility of the proposed approach beyond simulation. Some of the experimental scenarios executed are static human detection, person-following task, stopping behaviour and varying human–robot separation distances were designed to evaluate different aspects of system performance.

3.2 SYSTEM REQUIREMENTS ANALYSIS

3.2.1 Functional Requirements

Functional requirements are the specific operations and services that the system was required to perform to achieve the research objectives. They are the expected behaviour of the developed system and provided measurable criteria for evaluating whether the implemented solution successfully satisfied the operational requirements (Pressman & Maxim, 2020).

a. Human Detection

The developed system automatically detected the presence of a human target within the camera's field of view (FOV) using the YOLOv8 deep learning object detection model. The perception module processed incoming RGB images in real time and identified the human class together with its corresponding confidence score. Only detections identified within the predefined confidence threshold were considered valid for subsequent processing while accurate human detection formed the foundation of the entire person-following framework because all subsequent localisation and motion control operations depended upon reliable detection.

b. Human Localisation

After successful detection, the system determined the position of the detected human within each captured image. Then the human localisation was achieved by extracting the centre coordinates of the bounding box generated by the YOLOv8 detector. The horizontal centre coordinate provided information regarding the person's relative position within the image, enabling the robot to determine whether the target was centred or displaced from the camera's optical axis. This localisation information served as the primary visual feedback for the autonomous motion controller.

c. Distance Estimation

The developed system estimated the relative distance between the robot and the detected human using the monocular vision. Since only a single RGB camera was employed, direct depth measurement was not available. Consequently, the height of the detected bounding box was utilised as an indirect estimate of human distance. As the target moved closer to the robot, the bounding-box height increased, whereas it decreased as the target moved farther away. This image-based distance estimation enabled the robot to maintain an appropriate following distance without requiring additional depth sensors such as LiDAR or RGB-D cameras (Hartley & Zisserman, 2004).

d. Forward Movement

The autonomous controller generated forward velocity commands whenever the detected person moved beyond the predefined desired following distance. The adaptive motion controller continuously compared the estimated human distance with the reference distance and commanded the mobile robot to move forward whenever the target was farther than the desired separation distance. The generated velocity was proportional to the distance error while remaining within predefined safety limits to ensure smooth and stable motion.

e. Backward Movement

The system also supported autonomous backward movement whenever the detected person moved excessively close to the robot by continuously monitoring the estimated human distance using the bounding-box height, the adaptive controller generated backward velocity commands until the desired separation distance was restored. This functionality enabled the robot to adapt naturally to continuous changes in human position while maintaining a safe following distance throughout indoor operation.

f. Robot Stopping

The developed system automatically stopped the robot whenever the detected person remained within the predefined acceptable following distance or when no valid human target was detected. Similarly, if the target became temporarily lost because of occlusion or moved outside the camera's FOV, the controller immediately published zero linear and angular velocity commands to ensure safe robot operation. Automatic stopping served as an important safety mechanism that prevented unintended robot movement and reduced the possibility of collisions during autonomous operation (ISO 13482, 2014).

g. Continuous Tracking

The developed framework continuously monitored the detected human throughout the robot’s operation. Thus, each incoming camera frame was processed independently, enabling the robot to update the target's position and estimated distance in real time. Continuous visual feedback allowed the robot to respond dynamically to changes in human movement, thereby maintaining robust person-following behaviour under varying indoor conditions.

h. Real-Time Processing

The entire autonomous person-following system operated in real time to ensure responsive interaction between the robot and the human target. The perception module, human position estimation algorithm, adaptive motion controller, and ROS communication framework processed incoming data with minimal latency so that velocity commands were generated promptly. Real-time processing was particularly important in autonomous mobile robotics because delays in perception or control could result in unstable robot behaviour, inaccurate following, or safety hazards (Quigley *et al*., 2009; Macenski *et al*., 2022).

3.2.2 Non-functional Requirements

Non-functional requirements are the quality attributes and operational characteristics that the developed autonomous person-following system exhibited during implementation and deployment. Unlike functional requirements, which described what the system was required to accomplish, non-functional requirements specified how well the system performed its intended functions. These requirements addressed important quality characteristics such as performance, reliability, accuracy, robustness, scalability, computational efficiency, and operational safety. The non-functional requirements are described below.

a. Real-Time Response

The system was developed to operate in real time so that the robot could respond promptly to changes in the target person's position. Image acquisition, human detection, position estimation, adaptive motion control, and velocity command generation were performed with minimal processing delay which ensured smooth robot movement. Real-time operation was essential because delays in perception or control could reduce tracking accuracy and negatively affect the stability of autonomous person-following behaviour (Siciliano & Khatib, 2016). The use of the YOLOv8 object detector and ROS-based distributed communication contributed significantly to achieving real-time system performance (Jocher *et al*., 2023; Macenski *et al*., 2022).

b. Reliability

The developed framework was required to operate reliably throughout extended periods of autonomous operation without unexpected failures or communication interruptions. Reliable communication between ROS nodes ensured that image data, detection results, and robot velocity commands were transmitted correctly between software modules. The modular ROS architecture enhanced system reliability by allowing individual components to operate independently while communicating through well-defined topics. Extensive testing and parameter tuning further improved the stability and reliability of the implemented system.

c. Accuracy

Human detection accuracy directly influenced the quality of position estimation and the effectiveness of the adaptive motion controller. The adoption of the YOLOv8 deep learning model provided high detection precision while maintaining real-time processing performance. Accurate estimation of the bounding-box centre coordinates and height enabled the robot to maintain an appropriate following distance and generate stable motion commands.

d. Robustness

The system was required to maintain acceptable performance under varying indoor operating conditions. Robustness referred to the ability of the perception and control modules to continue functioning despite moderate illumination variations, temporary target occlusions, background clutter, and continuous human positional changes. The robustness of the framework was enhanced with deep learning-based object detection and adaptive motion

control, both of which demonstrated improved resilience compared with conventional computer vision techniques.

e. Scalability

The software architecture was designed to support future expansion without requiring substantial modification of the existing implementation. The modular ROS-based architecture enabled additional functionalities, such as obstacle avoidance, multi-person tracking, gesture recognition, simultaneous localisation and mapping (SLAM), or vision-language reasoning, to be incorporated as independent ROS nodes. This scalability represented an important design consideration because autonomous robotic systems frequently evolve as new sensing technologies and intelligent algorithms become available.

f. Computational Efficiency

The entire framework was designed to utilise computational resources efficiently while maintaining real-time operation. Computational efficiency was particularly important because deep learning algorithms are generally computationally demanding. Efficient image processing, lightweight ROS communication, and GPU-accelerated YOLOv8 inference enabled the perception module to process incoming images rapidly without introducing excessive computational overhead.

g. Safety

Operational safety constituted one of the most critical non-functional requirements of the developed autonomous person-following system. The controller was designed to generate conservative velocity commands, maintain a predefined safe following distance, and immediately stop the robot whenever the target was lost or remained within the acceptable distance threshold. Maximum forward and backward speeds were deliberately limited to minimise the risk of unintended collisions during indoor experiments. These safety measures were implemented in accordance with established principles for personal service robots and autonomous mobile robotic systems (ISO 13482, 2014).

3.3 SYSTEM ARCHITECTURE

3.3.1 Conceptual Framework

The proposed system was designed as a modular vision-based autonomous person-following framework that integrated computer vision, deep learning, adaptive motion control, and robot actuation within the ROS module. The modular architecture facilitated independent development, testing, and future expansion of each subsystem while ensuring efficient communication through ROS topics and message passing.

The architecture consisted of five major subsystems: the perception subsystem, the human position estimation subsystem, the adaptive motion control subsystem, the robot control subsystem, and the continuous visual feedback mechanism. These components operated

collaboratively to enable the mobile robot to detect, localise, estimate the relative position of, and autonomously follow a human target in real time.

The perception subsystem formed the first stage of the framework. An RGB camera mounted on the mobile robot continuously captured images of the surrounding environment. Following a successful human detection, the human position estimation subsystem extracted the horizontal centre coordinate and the height of the detected bounding box. The adaptive motion control subsystem continuously received the detected human position information and compared the estimated distance with the desired following distance. Motion smoothing and safety constraints were incorporated into the controller to minimise oscillations and ensure stable robot behaviour during continuous human movement. The robot control subsystem interpreted the incoming velocity commands and converted them into motor control signals that produced the corresponding translational motion of the robot. Throughout the system's operation, the onboard camera continuously acquired new images, thereby creating a closed-loop visual feedback mechanism. This continuous perception–action cycle enabled the robot to adapt dynamically to changes in the target person's position while maintaining autonomous person-following behaviour.

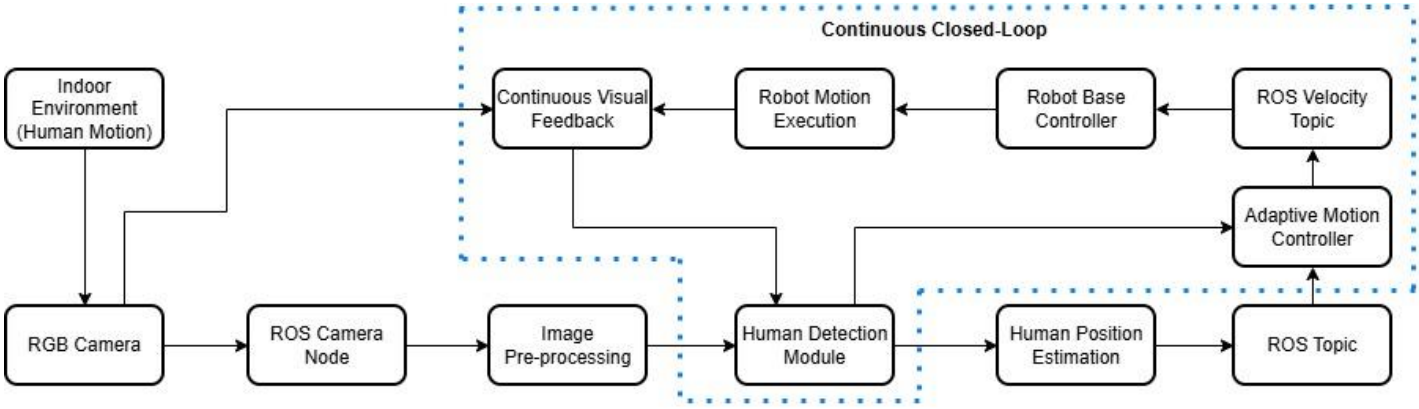


Figure 3.1 – Conceptual Framework of the Autonomous Vision-Based Human-Following System

Stage 1: Human Target Movement

The framework begins with the movement of the target person within an indoor environment. The person's motion constitutes the primary input to the autonomous system. Since human movement is inherently dynamic and unpredictable, the robot must continuously estimate positional changes in real time.

Stage 2: Image Acquisition

Visual information is acquired using the front RGB camera mounted on the Robotnik Summit XL. The camera continuously captures image frames of the surrounding environment and publishes them through a ROS image topic. This stage represents the perception interface between the robot and its environment.

Stage 3: Image Pre-processing

Captured images undergo pre-processing before object detection. Operations include image decoding, resizing, colour conversion where necessary, and formatting into a representation suitable for inference by the YOLOv8 model. Pre-processing improves computational efficiency while preserving relevant visual information.

Stage 4: Human Detection

The pre-processed image is analysed using the YOLOv8 deep learning object detector. The model identifies objects belonging to the "person" class and produces bounding-box coordinates, confidence score and object class. Among all detected objects, only the human class is selected for further processing.

Stage 5: Human Position Estimation

The detected bounding box is used to estimate the relative position of the target person. The two important parameters extracted are the

- *Bounding box centre (x-coordinate)* represents the person's horizontal position within the camera image.
- *Bounding box height* provides a monocular approximation of the person's distance from the robot.

These measurements are published through the custom ROS topic. The detection message serves as the communication interface between the perception subsystem and the motion controller.

Stage 6: Adaptive Motion Controller

The adaptive controller receives continuous visual feedback from the detection module and computes the appropriate robot motion. Unlike conventional fixed-position controllers, the proposed controller continuously updates robot velocity according to changes in the person's estimated position. The controller determines whether the robot should move forward, move backward, stop or maintain safe following distance. The controller also applies dead-band thresholds and motion smoothing to minimise oscillatory behaviour and improve stability during indoor operation.

Stage 7: Velocity Command Generation

Following motion computation, the controller publishes linear velocity commands through the ROS topic. The commands include the interface between the high-level perception system and the low-level robot controller.

Stage 8: Robot Motion Execution

The Robotnik Summit XL receives the velocity commands and executes the corresponding motion through its differential-drive base controller. The robot continuously adjusts its position to maintain an appropriate distance from the target person while adapting to changes in human movement.

3.3.2 Data Flow Architecture

This is the movement of information throughout the autonomous person-following system, from image acquisition to robot motion execution. It describes how visual information captured by the onboard camera was transformed into meaningful motion commands through successive processing stages. The structure adopted a distributed communication model based on the ROS framework, where software modules communicated through publishers, subscribers, and ROS topics.

a. Camera Image Acquisition

The data flow process commenced with the RGB camera which continuously captured images of the surrounding indoor environment containing the target person. These images were published by the ROS camera driver through the */image/compressed* topic. Image compression reduced communication overhead while maintaining sufficient visual quality for real-time human detection. The captured images subsequently served as the input to the perception subsystem for further processing (Bradski & Kaehler, 2008).

b. Human Detection Module

The compressed image stream was received by the perception module, where image decoding and pre-processing operations were performed before executing the YOLOv8 deep learning object detector. The detector analysed each image frame and identified the presence of a human target by generating a bounding box together with the corresponding confidence score. From the detected bounding box, the horizontal centre coordinate and bounding-box height were extracted to represent the target's relative position within the image.

c. ROS Topic Communication

After human detection and position estimation, the extracted detection information was encapsulated into a ROS message and published through the */person_detection* topic. The published message contained the human detection status, horizontal centre coordinate, vertical centre coordinate, bounding-box width, bounding-box height, and confidence score. The use of ROS topics enabled asynchronous communication between the perception module and the adaptive motion controller without requiring direct coupling between software components. This publisher–subscriber communication model simplified module integration and enhanced system modularity (Quigley *et al*., 2009).

d. Adaptive Motion Controller

The adaptive motion controller subscribed to the */person_detection* topic and continuously received the estimated human position parameters. Upon receiving a valid detection message, the controller compared the estimated human distance with the predefined desired following distance. Once the appropriate control action had been determined, the controller generated linear velocity commands and published them through the */summit_xl/cmd_vel* topic for execution by the mobile robot.

e. Robot Base Controller

The Robotnik Summit XL base controller subscribed to the */summit_xl/cmd_vel* topic and interpreted the received velocity commands as translational motion instructions. As the robot moved, the onboard camera continuously captured new images, which re-entered the perception subsystem and completed the closed-loop data flow cycle. This continuous feedback mechanism enabled the robot to adapt dynamically to changes in human position and maintain stable autonomous person-following behaviour.

3.3.3 System Workflow

The system workflow is the sequence of operations performed by the developed autonomous person-following framework from system initialization to continuous robot operation. The workflow followed a closed-loop perception–decision–action paradigm, which was adopted in autonomous robotics because it enables continuous interaction between environmental perception and robot control (Siciliano & Khatib, 2016).

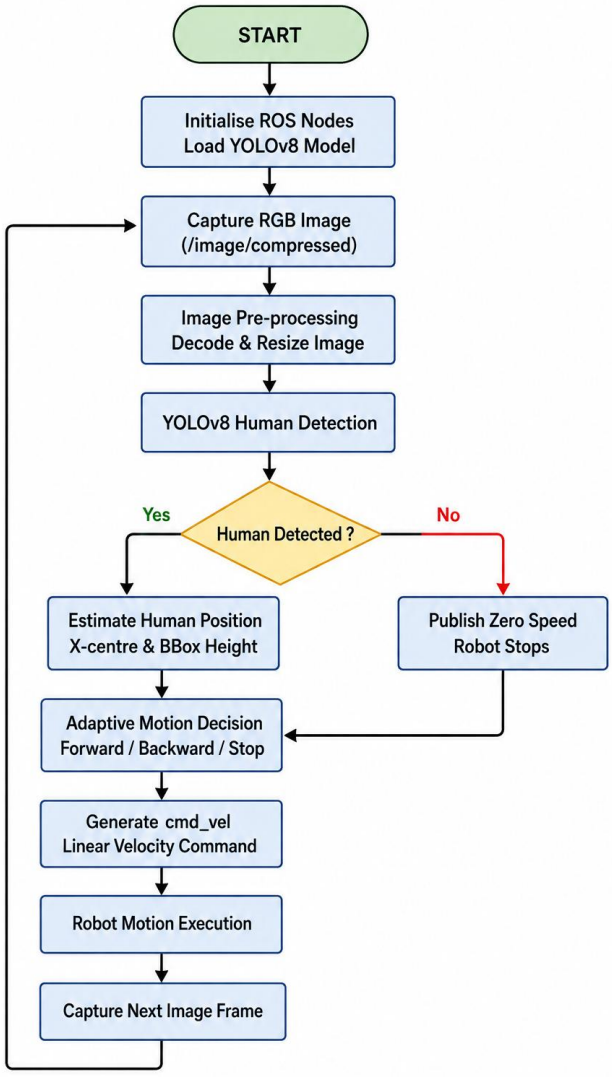


Figure 3.2 - System Workflow of Autonomous Vision-Based Human-Following System

The workflow commenced when the autonomous person-following application was launched within the ROS environment. During initialization, the required ROS nodes, communication topics, and software modules were activated. The acquired image frames from the RGB camera underwent pre-processing before being analysed by the YOLOv8 deep learning object detector. The detector examined every image frame to identify the presence of a human target and generated the corresponding bounding box together with its confidence score. When a valid human detection was obtained, the horizontal centre coordinate and the height of the bounding box were extracted to estimate the target's relative position within the camera frame. The horizontal centre coordinate represented the person's lateral position, while the bounding-box height was used as an indirect estimate of the distance between the robot and the target person. The estimated position information was subsequently transmitted through the */person_detection* ROS topic to the adaptive motion controller. The controller continuously compared the estimated distance with the predefined desired following distance and evaluated the positional relationship between the robot and the detected person. Based on this comparison, the controller determined the most appropriate control action, forward movement, backward movement, or stopping. Motion smoothing and predefined safety constraints were simultaneously applied to minimise oscillatory behaviour and ensure stable autonomous operation.

3.4 HARDWARE ARCHITECTURE

3.4.1 Robotnik Summit XL Mobile Robot

The Robotnik Summit XL autonomous mobile robot was used as the robotic platform in this study. The Summit XL is an industrial-grade unmanned ground vehicle (UGV) designed for indoor and outdoor research applications, autonomous navigation, logistics, surveillance, inspection, and human–robot interactions. Its modular hardware architecture, robust mobility, and seamless integration with the ROS module made it an appropriate platform for implementing and experimentally validating the proposed autonomous vision-based person-following framework (Robotnik Automation, 2019; Koubaa, 2017).

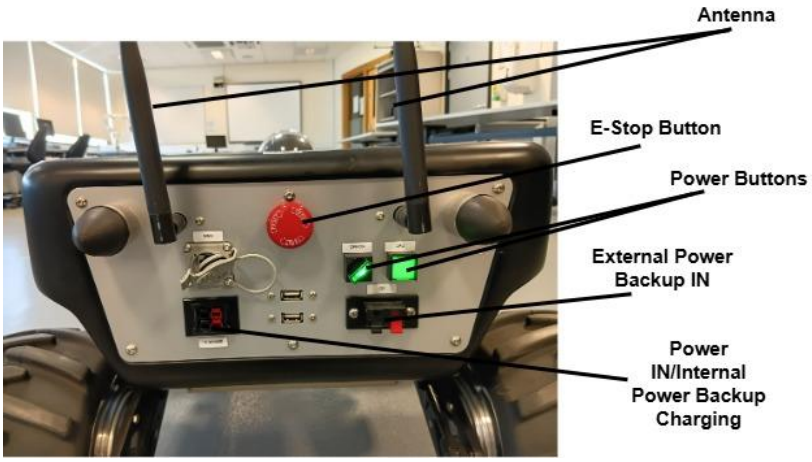


Figure 3.3 - Real-World Robotnik Summit XL Robot Back Panel

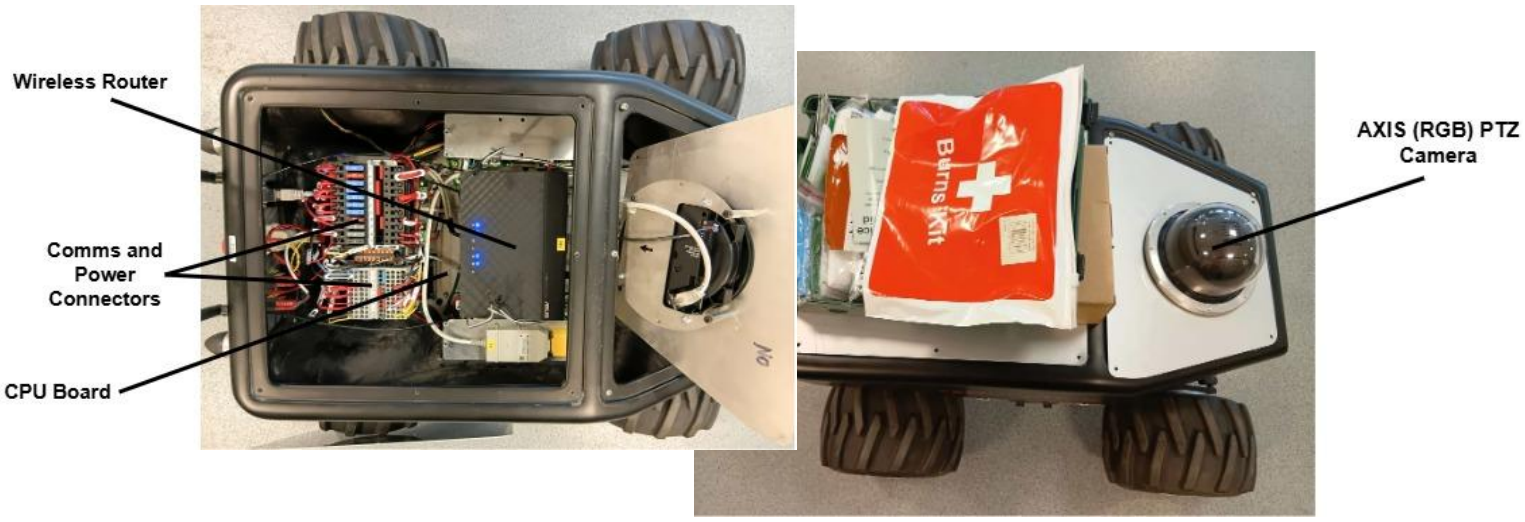


Figure 3.4 - Real World Robotnik Summit XL Robot Hardware Components

a. Robot Specifications

Robotnik Summit XL has a compact yet rugged chassis that enables it to carry sensing and computing equipment while maintaining excellent manoeuvrability and operational stability. Also, it has 4 rubber wheels arranged in a skid-steering configuration, brushless motors, an Inertial Measurement Unit (IMU), a Wi-Fi router, a Linux-based embedded computer, and ROS software architecture (Arregi & Secco, 2023).

b. Sensors

Multiple onboard sensors supported the autonomous navigation and robotic operation. These included wheel encoders for odometry estimation, an Inertial Measurement Unit (IMU) for estimating robot orientation and acceleration, and safety sensors integrated into the robot's navigation system. In this project, the primary sensing device was the onboard RGB camera mounted at the front of the robot, demonstrating that reliable autonomous person-following could be achieved using low-cost vision sensors alone (Corke, 2017; Forsyth & Ponce, 2012).

c. Drive System

Its drive system is a four-wheel skid-steering drive mechanism in which each wheel was independently powered by an electric motor. Steering was achieved by varying the rotational speeds of the wheels on the left and right sides of the robot rather than through mechanical steering components. This drive configuration provided excellent manoeuvrability and enabled the robot to execute forward, backward, rotational, and zero-radius turning movements.

d. Computing Hardware

With its onboard industrial computer responsible for executing the robot operating system, hardware drivers, navigation software, and low-level motion control algorithms, it executed the robot middleware that managed communication between the robot hardware and the external perception system. Since the deep learning inference using YOLOv8 is

computationally intensive, the perception module was executed on a remote workstation equipped with GPU acceleration. The workstation processed incoming camera images, performed real-time human detection, estimated the target position, and transmitted the resulting detection information back to the robot through the ROS communication network.

e. Communication Interface

The communication interfaces employed during this research included the */image/compressed* topic for transmitting RGB camera images, the */person_detection* topic for publishing human localisation information, and the */summit_xl/cmd_vel* topic for transmitting velocity commands to the robot base controller. This publisher–subscriber communication architecture enabled loose coupling between software modules, thereby improving system modularity, scalability, and maintainability (Koubaa, 2017; Martinez & Fernández, 2015).

3.4.2 RGB Camera

Installed on the robot is a 12 VDC powered AXIS P5514 PTZ Network Camera mounted on the front of the Robotnik Summit XL mobile robot as its only sensing device and connected to the onboard component. The camera can move both in the X-axis and in the Y-axis, and it offers a 360° pan with Auto-flip, 180° tilt, and 12x zoom capability for an autonomous person-following process (Arregi & Secco, 2023).



Figure 3.5 - Axis PTZ Network Camera

a. Camera Resolution

With its high video resolution of 720p the camera captures high-quality colour images that provided sufficient spatial information for real-time human. The applied resolution for the captured image sufficiently balances between detection accuracy and computational efficiency. During implementation, the incoming image frames were resized before inference to satisfy the input requirements of the YOLOv8 detector while maintaining real-time performance.

b. Frame Rate

The camera continuously acquired image frames at a rate suitable for real-time robotic perception. A sufficiently high frame rate ensured that successive images accurately represented the continuous movement of the target person, thereby enabling timely updates of

the robot control commands. Higher frame rates reduce motion blur and improve temporal tracking performance, allowing autonomous robots to respond more rapidly to changes in the surrounding environment (Sonka *et al*., 2015). The selected frame rate proved adequate for supporting smooth human detection and continuous robot motion during indoor experiments.

c. Camera Position

The RGB camera was mounted on the front of the Robotnik Summit XL mobile robot. This mounting position provided an unobstructed forward-facing FOV, allowing the perception subsystem to continuously observe the target person during robot operation. During initialization, the camera's tilt is set from its default home position to a normalized float of -0.00725 for a full human view. The front-mounted configuration also aligned naturally with the robot's direction of travel, simplifying the transformation of visual information into corresponding motion commands (Corke, 2017).

d. Image Acquisition

Image acquisition was performed continuously throughout robot operation using the ROS camera driver. The camera's captured images were published through the */summit_xl/front_ptz_camera/image_color/compressed* ROS topic. The use of compressed image transport reduced network bandwidth while maintaining image quality sufficient for deep learning-based human detection. The perception module subscribed to this image topic, decoded the compressed image frames, performed image pre-processing operations, and subsequently executed the YOLOv8 object detector.

3.4.3 Processing Computer

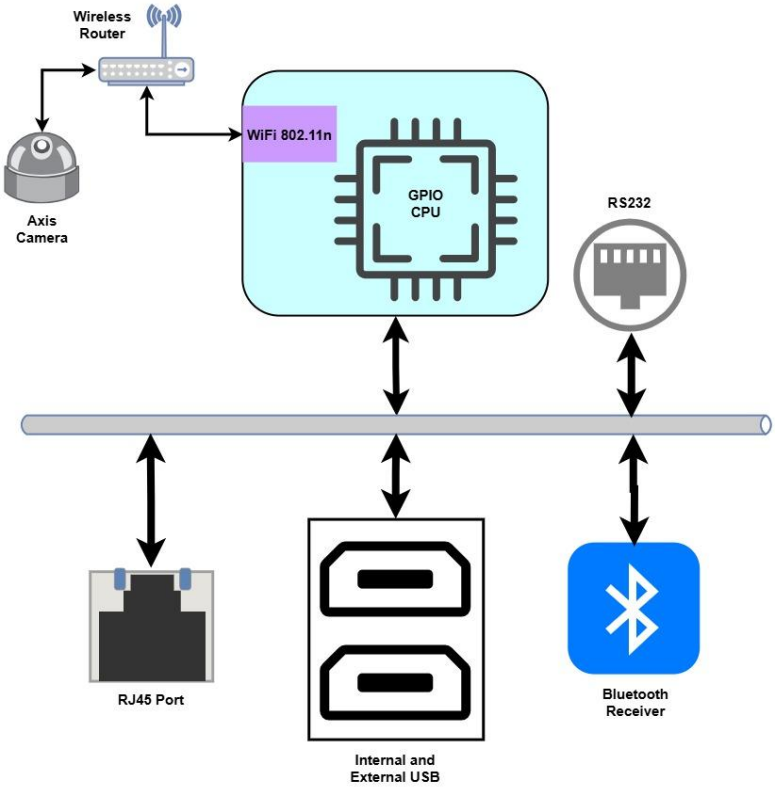


Figure 3.6 - Schematic of Elements Connected to the Robot's Onboard CPU

The robotic system employed a distributed computing architecture comprising its onboard computer integrated the remote workstation. The onboard computer was responsible for robot hardware management, ROS communication, and motion execution, whereas the remote workstation executed the deep learning-based perception module and adaptive control algorithms.

a. Central Processing Unit (CPU)

The remote computer's multi-core Central Processing Unit (CPU) is responsible for executing the ROS framework, Python application programs, image pre-processing routines, and communication between the perception and control modules. The CPU coordinated the execution of software processes, managed memory resources, and handled data transmission between ROS nodes.

b. Graphics Processing Unit (GPU)

The perception subsystem utilised a Graphics Processing Unit (GPU) to accelerate deep learning inference. The YOLOv8 object detection model performed numerous parallel mathematical operations during feature extraction and object classification, making GPU acceleration essential for achieving real-time processing speeds.

c. Random Access Memory (RAM)

The system's Random Access Memory (RAM) supports simultaneous execution of multiple software applications, including Ubuntu Linux, ROS, Python, OpenCV, the YOLOv8 inference engine, and associated system processes. Adequate memory capacity ensured efficient storage of image data, neural network parameters, ROS messages, and temporary computational variables during system execution. Sufficient RAM allocation reduced memory bottlenecks, improved application responsiveness, and contributed to stable long-duration operation of the autonomous person-following framework (Silberschatz *et al*., 2018).

d. Operating System

Ubuntu Linux was the software environment employed as the operating system because of its stability, extensive software support, and compatibility with robotic middleware. The onboard computer integrated within the Robotnik Summit XL operated on Ubuntu 16.04 LTS together with ROS Kinetic Kame, as built by the robot manufacturer. The remote perception workstation operated on Ubuntu 20.04 LTS, with ROS Noetic Ninjemys providing compatibility with modern deep learning libraries, including PyTorch, OpenCV, CUDA, and the Ultralytics YOLOv8 framework. The use of Ubuntu on both computing platforms facilitated seamless communication through the ROS network while allowing the perception subsystem to utilise more recent software libraries than those supported by the robot's onboard computer (Robotnik Automation, 2019; Canonical, 2020).

3.4.4 Network Configuration

The robotic system used a distributed network architecture to facilitate communication between the mobile robot and the remote processing workstation. The network enabled the exchange of sensor data, human detection results, and robot control commands through ROS. This distributed configuration allowed computationally intensive perception tasks to be executed on a high-performance workstation while the robot's onboard computer managed hardware interfacing and motion execution. Such distributed robotic architectures improve computational efficiency, software modularity, and system scalability (Koubaa, 2017).

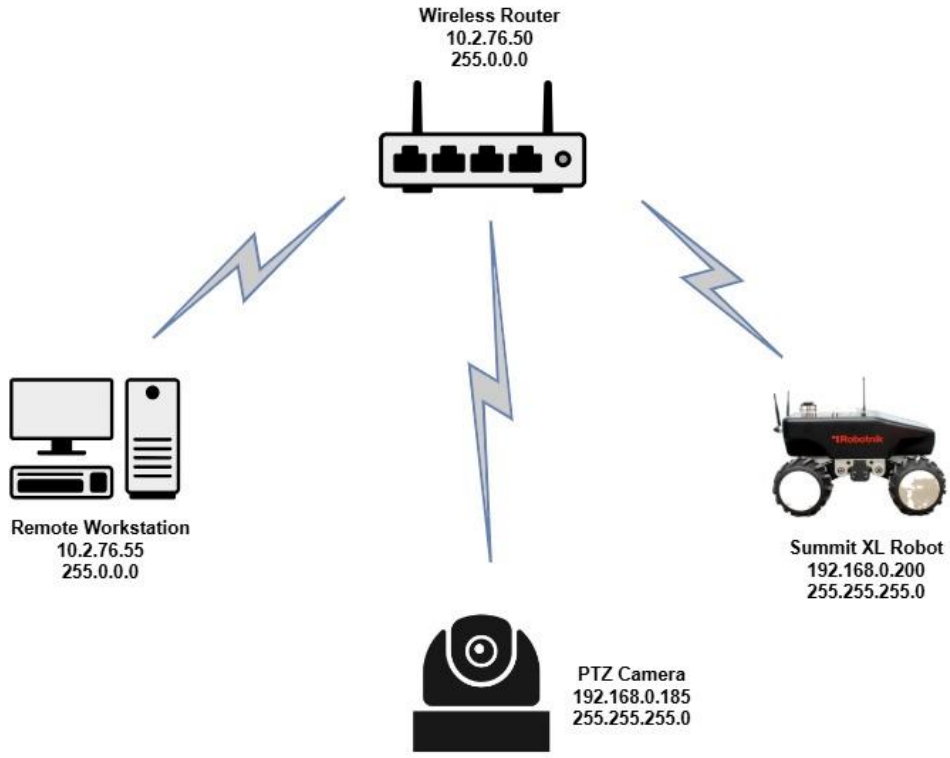


Figure 3.7 – System Network Structure

a. ROS Master Configuration

The Robotnik Summit XL onboard computer hosted the ROS Master, which acted as the central coordination service for all ROS nodes participating in the distributed robotic system. The ROS Master maintained information about active publishers, subscribers, services, and available topics, thereby enabling software modules running on different computers to discover one another and establish communication. For distributed operation, the onboard computer was configured as the ROS Master using the appropriate ROS_MASTER_URI (http://ROBOT_IP:11315) environment variable, while both the robot and the external workstation were assigned valid network addresses through the ROS_IP (WORKSTATION_10.2.76.55) environment variables. This configuration ensured that all ROS nodes communicated with the same master and exchanged information seamlessly across the network (Koubaa, 2017; Martinez & Fernández, 2015).

b. Robot Network

Communication between the Robotnik Summit XL and the remote workstation was established through a local Ethernet network. The high frequency wireless connection within a close range provided a stable, high-bandwidth, and low-latency communication channel suitable for transmitting continuous image streams and real-time robot control commands. The onboard RGB camera published compressed image frames to the ROS image topic, which were received by the remote workstation for processing. The use of a local area network (LAN) minimised communication delays and contributed to reliable real-time operation throughout the experimental evaluation (Robotnik Automation, 2019).

c. Communication Architecture

Ethernet communication was utilised to establish network connectivity between the robot and the remote workstation executing the perception module. The communication architecture followed the publisher–subscriber model implemented by ROS. Independent software modules exchanged information through predefined ROS topics without requiring direct interaction between processes. This loose coupling simplified software development and allowed each module to operate independently while maintaining synchronised communication across the system.

3.5 SOFTWARE ARCHITECTURE

3.5.1 Ubuntu Linux

Ubuntu Linux served as the operating system for the development and implementation of this system's framework. Ubuntu is an open-source Linux distribution developed by Canonical Ltd. and is widely adopted in robotics research because of its stability, security, extensive software repositories, and strong compatibility with ROS. Its long-term support (LTS) releases provide continuous security updates and software maintenance, making Ubuntu particularly suitable for long-duration robotic applications (Canonical, 2020; Koubaa, 2017). The experimental platform employed two versions of Ubuntu Linux, Ubuntu 16.04 LTS, which was required for compatibility with ROS Kinetic Kame and the manufacturer's software drivers and Ubuntu 20.04 LTS, providing compatibility with ROS Noetic Ninjemys and modern deep learning frameworks on the remote workstation. The Ubuntu Linux hosted the ROS middleware, Python runtime environment, OpenCV computer vision library, deep learning framework, and the custom-developed perception and control algorithms. The operating system also managed process scheduling, memory allocation, file systems, network communication, and hardware resource allocation required for continuous robot operation.

A major advantage of Ubuntu Linux is its native compatibility with open-source robotics software. The operating system provides package management through the Advanced Package Tool (APT), enabling straightforward installation and maintenance of ROS distributions, computer vision libraries, and machine learning frameworks. Ubuntu also offers excellent support for software development tools, including Python, C++, Git, CUDA, and integrated

development environments (IDEs), thereby simplifying the implementation and debugging of robotic applications (Canonical, 2020). Ubuntu further facilitated distributed robotic computing by supporting standard TCP/IP networking, Secure Shell (SSH), and Ethernet communication. These capabilities enabled reliable communication between the Robotnik Summit XL and the external processing workstation through the ROS network.

3.5.2 Robot Operating System (ROS)

The Robot Operating System (ROS), as the middleware, integrated all hardware and software components of the autonomous person-following system. ROS provided a distributed software framework that enabled independent software modules to communicate efficiently through standardised interfaces. This modular architecture simplified system integration by allowing perception, control, communication, and robot hardware components to operate independently while exchanging information in real time (Koubaa, 2017).

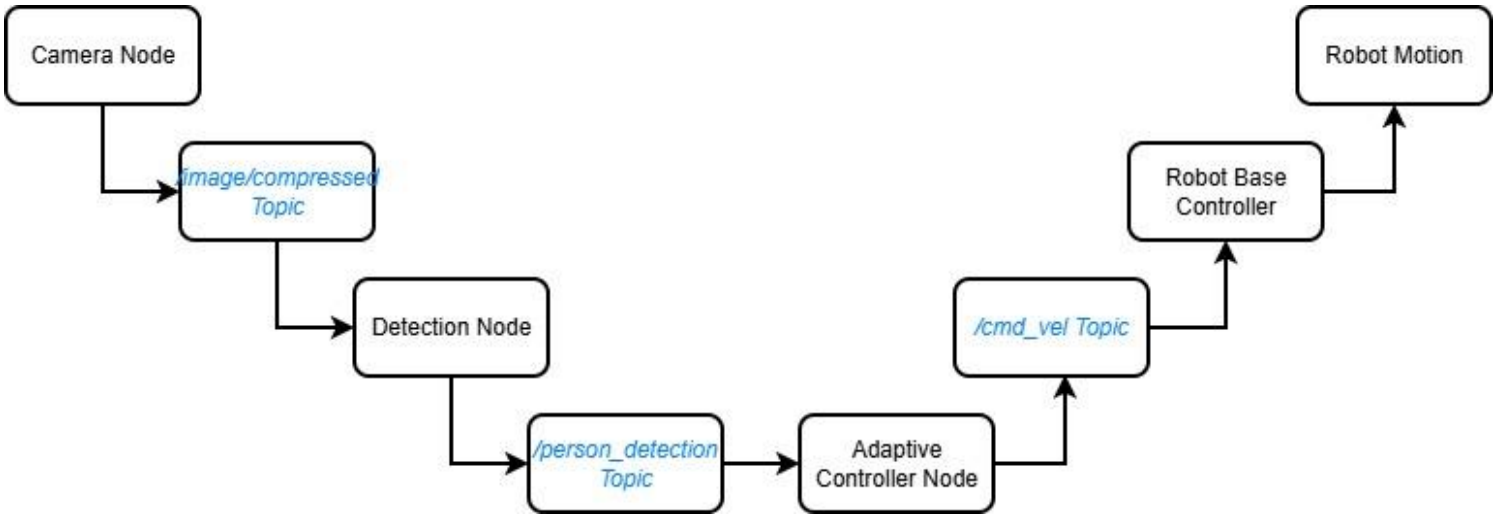


Figure 3.8a - ROS Communication Architecture

a. ROS Nodes

The system was organised into multiple ROS nodes, each responsible for performing a specific function within the autonomous person-following framework. The camera driver node continuously acquired RGB images from the onboard PTZ camera and published them to the ROS image topic. The perception node subscribed to the image stream, performed image pre-processing, executed the YOLOv8 object detector, estimated the target person's position, and published the resulting detection information.

An adaptive motion controller node subscribed to the human detection topic, computed the required robot motion based on the estimated human position, and generated velocity commands for the robot. This modular approach reduced software coupling and simplified development, debugging, testing, and subsequent modification of individual components.
Four principal functional nodes were considered in the person-following architecture: the camera node, YOLO detector node, follower controller node, and robot base-control node.

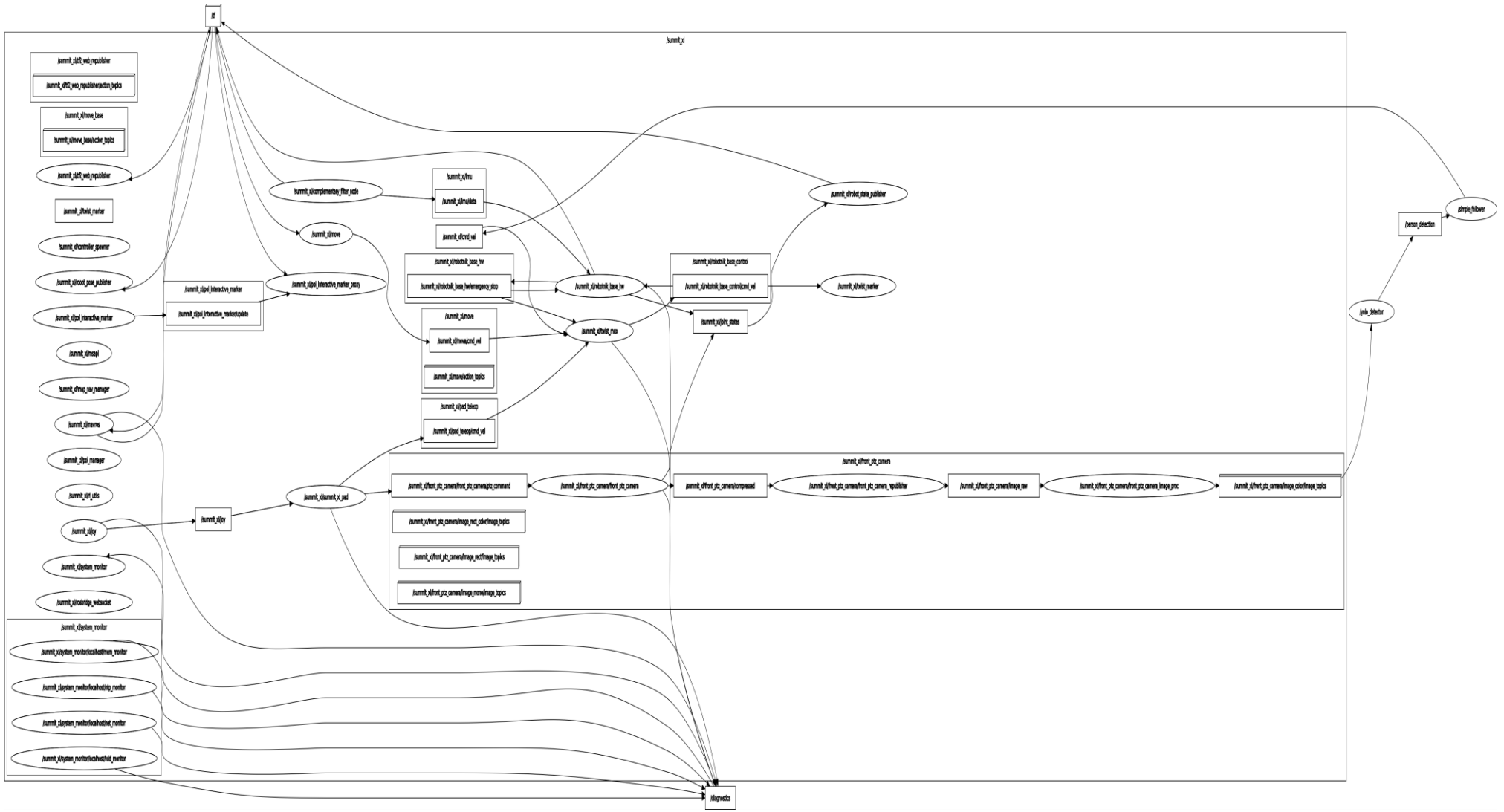


Figure 3.8b - ROS Node Graph

i. **Camera Node -** The camera node provided the visual input required by the perception subsystem through ROS topics. The camera node acquired successive RGB frames and published them through the camera image interface. In the implemented system, the compressed image stream was accessed through the */image/compressed* topic.

ii. **YOLO Detector Node -** The perception component is subscribed to the camera image stream to process the incoming frames using the YOLOv8 object-detection model. The node performed image decoding, preprocessing, neural-network inference, person-class filtering, confidence evaluation, and bounding-box extraction.

iii. **Follower Controller Node -** The follower controller node implemented the decision-making and motion-control logic of the proposed autonomous person-following system. It subscribed to the */person_detection* topic and continuously processed the visual information generated by the YOLO detector. The node extracted the horizontal bounding-box centre and bounding-box height from each received detection message. The horizontal position determined the person's alignment relative to the camera centre, while bounding-box height was used as a relative indicator of target distance.

iv. **Robot Base-Control Node -** The robot base-control node is the hardware interface responsible for receiving the generated velocity commands and converting them into physical robot motion. It formed the lowest-level component of the person-following control pipeline.

Table 3.1 - Node Interaction

S/No.	ROS Node	Primary Function	Input	Output
1.	Camera node	Image acquisition	RGB camera	/image/compressed
2.	YOLO detector	Person detection and localisation	/image/compressed	/person_detection
3.	Follower controller	Motion decision and velocity generation	/person_detection	/summit_xl/cmd_vel
4.	Robot base	Robot motion execution	/summit_xl/cmd_vel	Wheel/motor actuation

b. ROS Topics

ROS topics formed the main communication mode between the various ROS nodes using the publisher–subscriber communication model. Topics provided asynchronous communication channels through which data could be exchanged continuously without requiring direct interaction between software modules.

Table 3.2 - ROS Topics Applied in the System's Framework

S/No.	ROS Topic	Message Type	Function
1.	/image/compressed	sensor_msgs/CompressedImage	Transmitted compressed RGB camera images from the camera node to the YOLOv8 detector.
2.	/person_detection	std_msgs/Float32MultiArray	Transmitted detected-person information, including detection status, bounding-box centre coordinates, width, height, and confidence score, from the YOLO detector to the follower controller.
3.	/summit_xl/cmd_vel	geometry_msgs/Twist	Transmitted linear velocity commands from the follower controller to the Summit XL base-control system.
4.	/e_stop	std_msgs/Bool	Provided emergency-stop state information used to enable or disable robot motion during system operation and testing.

c. ROS Messages

ROS messages served as the standard data structures used to exchange information between nodes. Each topic transmitted messages with predefined data types that ensured consistent communication throughout the distributed robotic system.

The camera node transmitted compressed image messages containing RGB image data, while the perception module published human detection information using a customised message structure consisting of the detection status, horizontal centre coordinate, vertical centre coordinate, bounding-box width, bounding-box height, and detection confidence. The adaptive controller interpreted this information to estimate the target's relative position before generating robot velocity commands using the standard *geometry_msgs/Twist* message type.

d. ROS Services

Although the primary communication mechanism employed in this research relied on ROS topics, ROS services were also available within the ROS to support synchronous communication whenever immediate responses were required. Unlike topics, which continuously stream information, ROS services operate according to a request–response

communication model and are typically employed for configuration, parameter retrieval, hardware initialisation, and system diagnostics. During system implementation, the person-following algorithm did not require frequent service calls because perception and control were executed continuously through ROS topics. Nevertheless, ROS services remained available for robot configuration, node management, hardware initialisation, and system maintenance provided by the Robotnik software framework.

3.5.3 Python

The purpose of Python as the programming language for implementing this project is due to its simplicity, extensive scientific computing ecosystem, and excellent compatibility with robotics and artificial intelligence frameworks. These characteristics contributes to reduced software development time as it facilitates the integration of multiple software components within a unified development environment (Lutz, 2013; Matthes, 2019). Also, it has a seamless integration with the ROS using *rospy* client library that enables developers to create ROS nodes, publish and subscribe to topics, exchange messages, and interact with robotic hardware using relatively simple programming constructs. Furthermore, Python provides extensive support for computer vision through the OpenCV library. The ease of integration between Python and OpenCV facilitated efficient implementation of the vision pipeline while maintaining real-time image processing performance (Bradski & Kaehler, 2008).

Python is compatible with YOLOv8 object detector and was implemented using the Ultralytics Python package, which is built upon the PyTorch deep learning framework. Python therefore provided direct access to pretrained deep neural network models, GPU acceleration through CUDA, and efficient tensor-based computations required for real-time human detection. The availability of these artificial intelligence libraries considerably simplified the implementation of the perception subsystem (Paszke *et al*., 2019; Jocher *et al*., 2023).

Python further supported rapid software prototyping and iterative algorithm development. Throughout the implementation phase, perception algorithms, adaptive control strategies, and ROS communication modules were developed, tested, modified, and validated with minimal programming overhead. This flexibility proved particularly valuable during system integration and experimental evaluation, where multiple refinements were required to improve robot-following performance under varying indoor conditions.

3.5.4 OpenCV

OpenCV (Open-Source Computer Vision Library) version 4.6 was employed as the computer vision library for image acquisition, decoding, pre-processing, and manipulation within the system development. Its comprehensive Python interface and compatibility with deep learning frameworks made it suitable for integrating image processing with the YOLOv8 object detection model used in this research (Bradski & Kaehler, 2008; Laganière, 2017).

Within the developed framework, OpenCV acted as an intermediary between the ROS image acquisition subsystem and the deep learning perception module. It received compressed image streams published by the onboard RGB camera, decoded them into image matrices, performed the required pre-processing operations, and supplied the processed images to the YOLOv8 detection algorithm.

a. Image Decoding

The onboard RGB camera continuously published compressed image frames through the ROS image topic */summit_xl/front_ptz_camera/image_color/compressed*. OpenCV was utilised to decode the incoming compressed image stream into an uncompressed colour image represented as a multidimensional matrix. This conversion transformed the received image into a format suitable for subsequent image processing and deep learning inference. Efficient image decoding ensured that image acquisition introduced minimal processing delay and enabled the perception subsystem to operate continuously throughout the experimental evaluation.

b. Image Processing

OpenCV performed several image pre-processing operations before passing each image frame to the YOLOv8 object detector. These operations included image resizing, colour space management, image copying, and format conversion to satisfy the input requirements of the deep learning model. Image resizing ensured that all input images possessed uniform dimensions compatible with the neural network architecture, thereby reducing computational complexity and improving inference efficiency.

OpenCV also managed image visualisation during system testing by displaying the detected human target together with the corresponding bounding box, confidence score, and target identifier. This visual feedback facilitated debugging, performance evaluation, and verification of the perception subsystem during system development. Furthermore, OpenCV provided efficient matrix manipulation and image handling routines that supported continuous processing of successive camera frames without introducing significant computational overhead. The library's highly optimised implementation enabled the perception subsystem to maintain real-time performance while processing continuous video streams generated by the onboard camera (Kaehler & Bradski, 2016).

3.5.5 YOLOv8 Framework

The YOLOv8 framework was the core perception component of the autonomous person-following system. It was responsible for detecting the target person within the RGB camera images and estimating the corresponding bounding-box location required for robot motion control. YOLOv8 was selected because it combines high detection accuracy with low inference latency, making it particularly suitable for real-time robotic perception applications. YOLOv8 employs a single-stage detection architecture that predicts object classes and bounding boxes simultaneously, thereby achieving high processing speed while maintaining competitive detection performance (Jocher *et al*., 2023; Terven & Cordova-Esparza, 2023).

The framework was implemented using the Ultralytics Python package and executed on the remote workstation. The detected human position was subsequently transmitted to the mobile robot through the ROS communication network, enabling the adaptive motion controller to generate appropriate robot velocity commands.

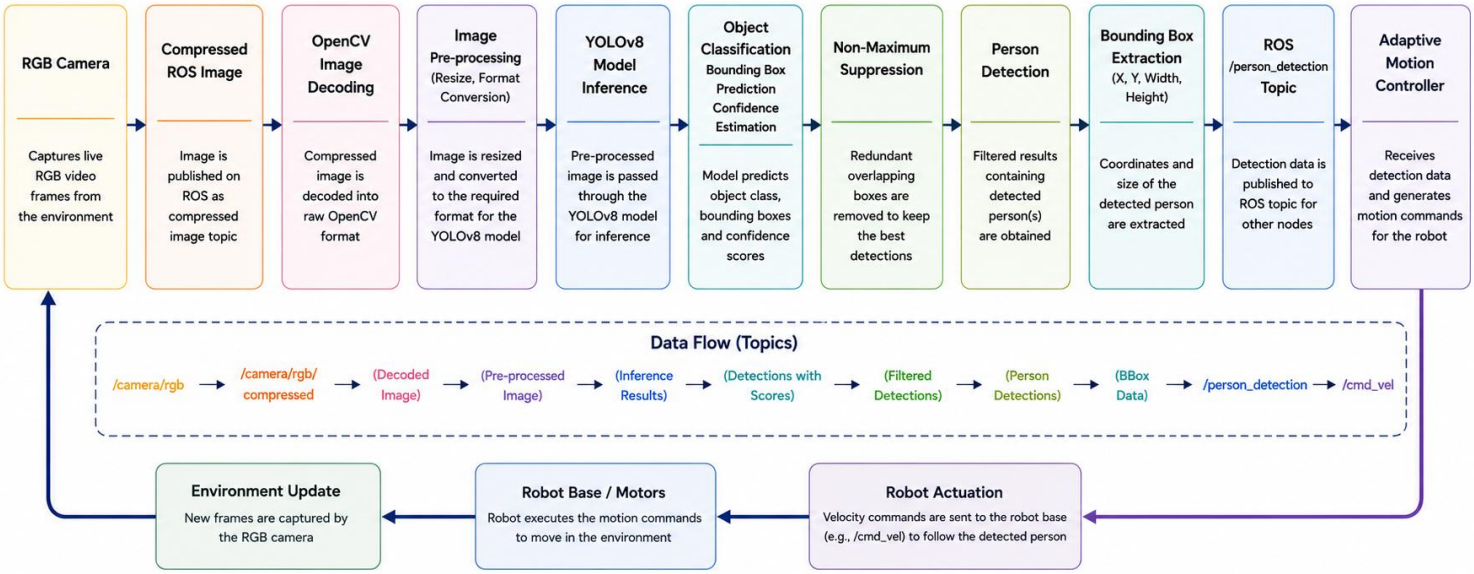


Figure 3.9 - YOLOv8 Inference Pipeline

a. Ultralytics Framework

The Ultralytics framework provides an open-source implementation of the YOLO object detection family with a simplified programming interface for training, validation, inference, and model deployment. It integrates seamlessly with the PyTorch deep learning framework and provides built-in support for GPU acceleration, model optimisation, image preprocessing, confidence thresholding, and non-maximum suppression. In this project, the framework provided a reliable software environment for deploying the person detection model while maintaining real-time performance during continuous robot operation.

b. Model Loading

The perception module loaded the pretrained YOLOv8 model during system initialisation before image processing commenced. The model parameters were stored as pretrained network weights and imported into memory using the Ultralytics Python interface. Once loaded, the neural network remained resident in memory throughout system operation, eliminating repeated loading overhead and improving computational efficiency. The pretrained model had been trained on the Microsoft Common Objects in Context (MS COCO) dataset, which contains 80 object categories (Lin *et al.*, 2014), including the *person* class used in this project. During implementation, the perception module filtered the detection results so that only objects belonging to the *person* category were considered. This reduced unnecessary computational processing and ensured that robot motion decisions were based exclusively on the detected human target.

c. Inference Pipeline

The inference pipeline transformed each incoming RGB image into human detection information suitable for robot control. The process began when compressed camera images published through the ROS image topic were decoded and pre-processed using OpenCV. Each image was resized to satisfy the input dimensions required by the YOLOv8 model before being forwarded to the neural network for inference.

The neural network extracted hierarchical visual features from the image and predicted bounding boxes, object classes, and confidence scores for all detected objects. Non-maximum suppression was subsequently applied to remove duplicate detections and retain only the highest-confidence prediction for each object. The perception module then selected detections corresponding to the *person* class and extracted the horizontal centre coordinate, vertical centre coordinate, bounding-box width, bounding-box height, and confidence score.

These detection parameters were published through the */person_detection* ROS topic, where they were received by the adaptive motion controller. The controller utilised the horizontal centre coordinate to estimate the person's lateral position within the camera view, while the bounding-box height served as a monocular indicator of the person's relative distance from the robot. The resulting information enabled continuous estimation of the target's position and supported smooth real-time robot motion throughout the experimental evaluation.

3.5.6 Development Environment

The software components of this project were developed within an integrated software development environment comprising Visual Studio Code (VS Code), Git version control, and ROS package management framework. These development tools collectively facilitated software implementation, debugging, version management, modular code organisation, and system integration throughout the research. The use of a structured development environment improved software maintainability, reproducibility, and scalability, which are essential characteristics of modern robotic software engineering (Wilson *et al*., 2014; Koubaa, 2017).

a. Visual Studio Code (VS Code)

Visual Studio Code (VS Code) was the Integrated Development Environment (IDE) used to implement the perception, control, and communication modules of the autonomous person-following framework. It was selected because it provides a lightweight yet powerful programming environment supporting Python development, ROS applications, debugging, syntax highlighting, intelligent code completion, and integrated terminal functionality.

The integrated debugging tools enabled efficient identification and correction of programming errors during software development. In addition, extensions supporting Python, ROS, and Git improved developer productivity by providing automatic code formatting, error detection, project navigation, and source code management within a unified interface. These capabilities considerably simplified the iterative development and testing process required during implementation of the proposed robotic system (Microsoft, 2024).

b. Git Version Control

Git was used as the distributed version control system throughout software development. It enabled systematic tracking of source code modifications, preservation of development history, and recovery of previous software versions whenever required. The use of version control proved particularly beneficial during iterative refinement of the perception algorithms and adaptive motion controller, where multiple software revisions were evaluated before selecting the final implementation. It facilitated modular software development by allowing independent modification of individual program files while maintaining consistency across the overall project. Furthermore, version control supports collaborative software engineering by enabling multiple developers to contribute to the same codebase while preserving complete revision histories and minimising conflicts during software integration (Chacon & Straub, 2014).

c. ROS Packages

The software development was organised into ROS packages following the standard ROS workspace structure. Each ROS package contained related source code, configuration files, launch files, dependency information, and executable programs associated with a specific subsystem of the proposed autonomous person-following framework. This modular organisation simplified software maintenance and improved code reusability throughout the development process.

The ROS package management system automatically resolved software dependencies, compiled required components, and facilitated execution of the developed nodes using ROS launch files. This modular software architecture improved scalability by allowing individual packages to be modified, replaced, or extended independently without affecting the operation of the remaining components. Consequently, future enhancements such as obstacle avoidance, multi-person tracking, simultaneous localisation and mapping (SLAM), or vision-language robotic modules can be integrated with minimal modification to the existing software framework (Koubaa, 2017; Martinez & Fernández, 2015).

3.6 DEVELOPMENT OF THE VISION-BASED PERCEPTION MODULE

3.6.1 Image Pre-processing

This forms the second stage of the perception pipeline and was performed immediately after image acquisition. Its objective was to transform the raw camera images into a standardised format suitable for deep learning inference. Appropriate pre-processing improves image consistency, reduces computational overhead, and ensures compatibility between the acquired images and the input requirements of the YOLOv8 object detection model.

a. Colour Conversion

In image decoding, the colour format conversion was done to ensure compatibility between OpenCV and the YOLOv8 framework. OpenCV internally stores colour images using the Blue-Green-Red (BGR) colour ordering, whereas the deep learning framework expects images in

the Red-Green-Blue (RGB) format. Consequently, each decoded image was converted from the OpenCV BGR representation to the RGB colour space before inference. This colour conversion ensured accurate representation of image features and prevented degradation in object detection performance caused by wrong colour channel ordering.

b. Image Resizing

The decoded RGB images were subsequently resized to the input dimensions required by the YOLOv8 neural network. Standardising image dimensions ensured that every input frame possessed identical spatial characteristics, thereby simplifying batch processing and improving computational efficiency during inference. Image resizing also reduced the computational workload associated with deep neural network processing while preserving sufficient visual information for reliable human detection. Selecting a fixed input resolution allowed the perception module to achieve consistent inference speed throughout continuous robot operation. Furthermore, the uniform image dimensions ensured compatibility with the pretrained YOLOv8 architecture, which expects images of predetermined input sizes during object detection (Jocher *et al*., 2023).

3.6.2 Human Detection Using YOLOv8

The human detection stage accurately identifies the target person within each captured image and estimate the corresponding spatial information required for robot motion control. This was achieved using the YOLOv8 object detection framework, which performed real-time localisation and classification of human targets from continuous RGB image streams. The resulting bounding-box information formed the basis for estimating the person's relative position and distance from the robot (Jocher *et al*., 2023; Terven & Cordova-Esparza, 2023).

a. Model Architecture

The system design used the Ultralytics for the YOLOv8 object detection framework. YOLOv8 is a single-stage convolutional neural network that simultaneously performs feature extraction, object localisation, and object classification within a unified deep learning architecture. Unlike earlier YOLO versions that relied on anchor-based prediction, YOLOv8 adopts an anchor-free detection strategy, reducing computational complexity while improving localisation accuracy and simplifying the prediction process (Jocher *et al*., 2023).

The architecture consists of three principal components: the *backbone*, *neck*, and *detection head*. The backbone extracts hierarchical visual features from the input image using successive convolutional layers. These features are then aggregated by the neck through multi-scale feature fusion, enabling robust detection of objects appearing at different sizes. The detection head predicts object locations, class labels, and confidence scores directly from the fused feature maps. Rather than detecting all available object categories, the perception module filtered the model output to retain only detections belonging to the *person* class. This reduced unnecessary computational processing during subsequent stages and ensured that robot motion decisions were generated exclusively from human detections.

b. Detection Process

The detection process commenced immediately after completion of image pre-processing. Each resized RGB image was supplied as input to the pretrained YOLOv8 model, which performed forward propagation through the neural network to extract image features and identify objects present within the scene. The network generated a collection of candidate detections consisting of bounding-box coordinates, predicted object classes, and associated confidence scores. Non-Maximum Suppression (NMS) was subsequently applied to eliminate duplicate detections corresponding to the same physical object by retaining only the prediction with the highest confidence score.

Following non-maximum suppression, detections belonging to the *person* class were selected for further processing. For each valid detection, the perception module extracted the bounding-box centre coordinates, width, height, and confidence score. The horizontal centre coordinate represented the person's lateral position relative to the robot's field of view, while the bounding-box height served as a monocular estimate of the person's relative distance from the camera. These parameters were published through the */person_detection* ROS topic and supplied to the adaptive motion controller for robot navigation. The detection process was executed continuously for every incoming camera frame, thereby enabling the robot to update the target's position in real time throughout the person-following operation.

c. Confidence Threshold

A confidence threshold was applied during the inference stage to improve detection reliability and minimise false positive detections. The confidence score generated by YOLOv8 represented the probability that a predicted bounding box corresponds to a valid object of the specified class. Only detections whose confidence values exceeded the predefined threshold were accepted as valid human detections. Applying an appropriate confidence threshold reduced erroneous detections caused by background objects, illumination variations, and image noise. The confidence threshold was selected empirically through preliminary experimental testing to achieve an appropriate balance between detection precision and recall under the indoor operating conditions considered in this project.

3.6.3 Bounding Box Generation

Bounding box generation constituted the stage in which the spatial location of the detected human target was represented numerically for subsequent position estimation and robot motion control. After a successful human detection by the YOLOv8 model, each detected person was enclosed within a rectangular bounding box that defined the object's position and size within the image plane. The bounding box parameters were extracted directly from the detector and subsequently utilised by the adaptive controller to estimate the target's horizontal position and relative distance from the robot. This approach eliminated the need for specialised depth sensors by exploiting geometric information contained within the two-dimensional image (Szeliski, 2022; Hartley & Zisserman, 2004). During implementation, the perception module extracted four primary bounding box parameters, namely the horizontal coordinate (*x-coordinate*), vertical coordinate (*y-coordinate*), bounding box *width*, and bounding box *height*.

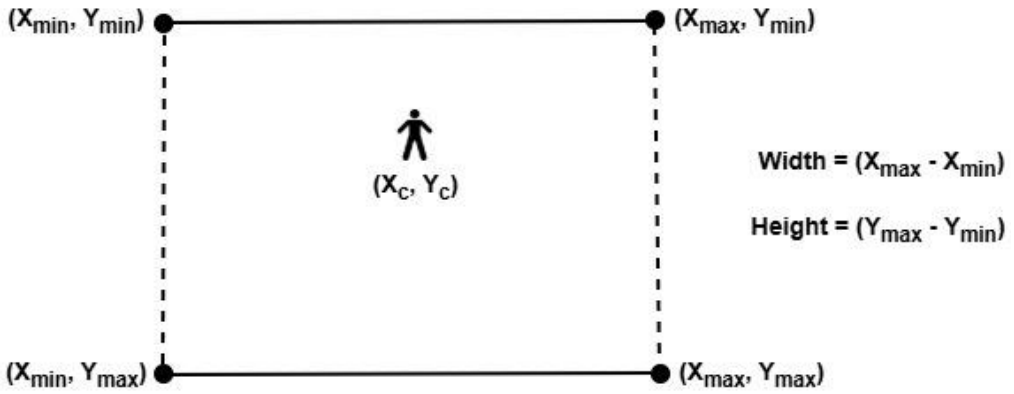


Figure 3.10 - Bounding Box Parameters Extracted from YOLOv8 Human Detector

a. X-Coordinate

The X-coordinate represented the horizontal position of the detected person's bounding box within the image. Rather than using the left boundary of the bounding box directly, the horizontal centre of the bounding box was computed because it provided a more reliable representation of the person's position relative to the camera's optical axis. The centre coordinate was calculated as

$$X_c = \frac{X_{min} + X_{max}}{2} \tag{3.1}$$

where:

- X_{min} is the left boundary of the bounding box,
- X_{max} is the right boundary of the bounding box,
- X_c represents the horizontal centre coordinate.

The horizontal centre coordinate was compared with the image centre to determine the person's lateral displacement.

b. Y-Coordinate

The Y-coordinate represented the vertical position of the detected bounding box within the image plane. Just like the horizontal position, the vertical centre coordinate was calculated using the upper and lower boundaries of the bounding box,

$$Y_c = \frac{Y_{min} + Y_{max}}{2} \tag{3.2}$$

where:

- Y_{min} is the upper boundary,
- Y_{max} is the lower boundary,
- Y_c represents the vertical centre coordinate.

Although the vertical position was not directly employed for motion control, it provided useful localisation information that could support future extensions involving pose estimation, activity recognition, and three-dimensional human localisation.

c. Bounding Box Width

The bounding box width represented the horizontal dimension of the detected human within the image and was calculated as

$$W = X_{max} - X_{min} \tag{3.3}$$

where W is the width of the detected object.

The bounding box width reflected the apparent horizontal size of the detected person and contributed to describing the overall dimensions of the detected target.

d. Bounding Box Height

The bounding box height represented the vertical size of the detected person within the image and was calculated as

$$H = Y_{max} - Y_{min} \tag{3.4}$$

where H is the bounding box height.

Among all extracted detection parameters, the bounding box height played the most significant role in the proposed human following framework. Because a monocular RGB camera was employed, direct depth measurement was unavailable. Instead, the apparent height of the detected person was used as a relative indicator of distance based on the principles of perspective projection.

3.6.4 Human Position Estimation

Human position estimation is where the geometric information obtained from the YOLOv8 detector was converted into positional information for robot navigation by analysing the relationship between the image centre and the detected bounding-box centre, while the bounding-box height was utilised as a monocular indicator of the person's relative distance. These parameters collectively enabled the adaptive motion controller to determine the appropriate steering and translational velocity required for autonomous human following.

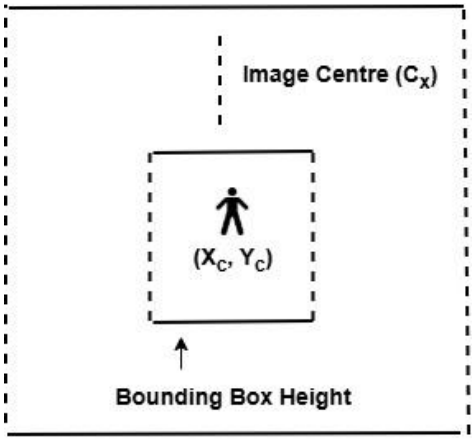


Figure 3.11 - Human Position Estimation using Image Centre and Bounding Box Geometry

a. Image Centre

The image centre represented the geometric reference point of the captured image and served as the desired alignment position for the detected human target. It was computed from the image dimensions as

$$C_x = \frac{W_I}{2} \tag{3.5}$$

$$C_y = \frac{H_I}{2} \tag{3.6}$$

where:

- W_I is the image width,
- H_I is the image height,
- C_x represents the horizontal image centre,
- C_y represents the vertical image centre.

The horizontal image centre acted as the reference against which the detected human position was continuously compared. When the person's bounding-box centre coincided with the image centre, the robot was considered correctly aligned with the target. Any deviation from this reference indicated that steering correction was required to restore alignment.

b. Bounding-Box Centre

The centre of the detected bounding box represented the estimated position of the target person within the camera image. The horizontal and vertical centre coordinates were computed,

$$X_{cv} = \mathrm{X}_{\min} + \frac{W}{2} \tag{3.7}$$

$$Y_{ch} = \mathrm{Y}_{\min} + \frac{H}{2} \tag{3.8}$$

where:

- X_{min} is the right boundary
- Y_{min} denotes the lower boundary
- W and H are bounding box width and height respectively
- X_{cv} and Y_{ch} represent the vertical and horizontal centre coordinates of the detected person.

The lateral position error used for robot steering was then calculated as

$$e_x = C_x - X_c \tag{3.9}$$

where e_x is the horizontal position error.

A positive error indicated that the detected person appeared on one side of the image, while a negative error indicated displacement towards the opposite side.

c. Distance Estimation

The framework estimated the relative distance between the robot and the detected person using the height of the detected bounding box. This approach was based on the principle of perspective projection, whereby the apparent size of an object increases as it approaches the camera and decreases as it moves farther away (Hartley & Zisserman, 2004).

Rather than estimating the human's absolute distance in metres through camera calibration, this project employed the bounding-box height as a relative distance indicator. A larger bounding-box height implied that the person was closer to the robot, whereas a smaller height indicated that the person had moved farther away. The adaptive motion controller continuously compared the measured bounding-box height with a predefined reference value representing the desired following distance. The three motion conditions established during implementation were,

- *Bounding-box height greater than the reference value,* the person was considered too close, and the robot reduced its forward motion or moved backward when necessary.
- *Bounding-box height approximately equal to the reference value***,** the desired following distance had been achieved, and the robot maintained its position.
- *Bounding-box height smaller than the reference value***,** the person was considered farther away, and the robot generated forward motion to reduce the separation distance.

This monocular distance estimation strategy enabled continuous adjustment of the robot's motion without requiring additional depth sensors.

3.6.5 Detection Message Generation

This the final stage of the perception subsystem. After human detection and position estimation, the extracted detection parameters were encapsulated into a ROS message and transmitted to the adaptive motion controller through the ROS communication network.

a. ROS Topic: /person_detection

The */person_detection* topic served as the primary communication channel between the YOLOv8 perception module and the adaptive motion controller. After each image frame was processed, the perception node published the detection results to this topic. The adaptive controller continuously monitored the topic and processed each incoming message to determine the robot's subsequent motion. Also, a *Float32MultiArray* ROS message type was used to transmit the detection parameters because it provided a lightweight and computationally efficient mechanism for exchanging multiple numerical values within a single message. This reduced communication overhead while maintaining real-time performance throughout the person-following process (ROS, 2024). The published message contained six parameters arranged in the following sequence: D, X_c, Y_c, W, H, C, where:

- **Detected** = Person detection status
- $\boldsymbol{X_c}$ = Horizontal centre coordinate
- $\boldsymbol{Y_c}$ = Vertical centre coordinate
- **W** = Bounding-box width
- **H** = Bounding-box height

- **C** = Detection confidence score

This standardised message structure enabled the adaptive controller to interpret every detection consistently regardless of image content or target position.

b. Published Parameters

The perception node published the following parameters for every valid human detection.

- **Person Detected**

The first parameter indicated whether a valid human target had been detected within the current image frame. A value of *1.0* represented successful human detection, whereas *0.0* indicated that no valid person had been identified. This parameter enabled the adaptive controller to distinguish between valid detections and situations where the target was temporarily absent due to occlusion or movement outside the camera's field of view. When no valid detection was available, the controller suspended robot motion or initiated the appropriate search behaviour.

- **X-Coordinate (X_c)**

This is the horizontal centre coordinate of the detected bounding box. This value indicated the person's lateral position within the image and served as the principal input for steering control.

- **Y-Coordinate (Y_c)**

This is the vertical centre coordinate of the detected bounding box.

- **Bounding-Box Width (W)**

This is the horizontal size of the detected bounding box. This value described the apparent width of the detected person within the image and was retained primarily for completeness of the detection information.

- **Bounding-Box Height (H)**

This is the vertical size of the detected bounding box and constituted the principal measurement used for estimating the person's relative distance from the robot. The adaptive controller continuously monitored this parameter to regulate forward motion, backward motion, and stopping behaviour.

- **Detection Confidence (C)**

This is the confidence score generated by the YOLOv8 detector. This score expressed the estimated probability that the detected object corresponded to a valid human target. The confidence values ranged between 0 and 1, with larger values indicating greater detection certainty. The adaptive controller utilised this information to reject unreliable detections and

ensure that robot motion was generated only from sufficiently confident observations, thereby improving overall system stability.

3.7 DEVELOPMENT OF THE MOTION CONTROL MODULE

The motion control module was developed to transform the visual information generated by the perception subsystem into appropriate motion commands for autonomous human following.

3.7.1 Control Strategy

During the development of this system an adaptive image-based visual servoing (IBVS) strategy was adopted to control the motion of the Robotnik Summit XL mobile robot. Visual servoing refers to the use of visual information obtained from one or more cameras to control the motion of a robotic system. In visual servoing the robot's motion continuously adjusts based on real-time observations of the target within the camera's FOV. This enables the robot to respond dynamically to changes in the person's position while maintaining continuous visual contact (Chaumette & Hutchinson, 2006; Corke, 2017).

In the development of the project, the adaptive visual servoing strategy utilised the positional information generated by the YOLOv8 perception module to regulate the linear velocity of the robot. The controller continuously received the human detection message published through the */person_detection* ROS topic and computed appropriate motion commands that enabled the robot to maintain alignment with the detected person while preserving a desired following distance.

a. Adaptive Image-Based Visual Servoing

The image-based visual servoing (IBVS) approach was used because all control decisions were derived directly from image features rather than reconstructed three-dimensional coordinates. Specifically, the controller utilised two image measurements:

- the *horizontal centre coordinate* of the detected bounding box; and
- the *bounding-box height*, which served as a monocular estimate of the person's relative distance.

The horizontal centre coordinate was compared with the image centre to determine the lateral alignment error.

Forward and backward motion were regulated using the detected bounding-box height. Since the apparent height of the detected person increases as the distance to the camera decreases, the controller compared the measured bounding-box height with a predefined reference height corresponding to the desired following distance.

When the measured height was smaller than the reference value, the controller interpreted the target as being farther from the robot and generated forward linear motion. Conversely, when the measured height exceeded the reference value, the controller generated backward motion to restore the desired separation distance. If the measured height remained within a

predefined tolerance of the reference value, the controller commanded the robot to remain stationary while maintaining visual alignment.

The adaptive behaviour of the controller was achieved by continuously updating translational commands based on successive image frames. As the human's position changed within the image, the controller automatically adjusted the robot's linear and angular velocities to compensate for lateral displacement and distance variation. This continuous feedback mechanism enabled smooth and responsive person-following without requiring explicit environmental mapping or external localisation systems. Furthermore, the controller incorporated motion smoothing and dead-band thresholds to minimise oscillatory behaviour caused by small detection fluctuations. Minor variations in the detected bounding-box position that did not represent meaningful human movement were therefore ignored, resulting in smoother robot trajectories and improved following stability during experimental evaluation.

3.7.2 Distance Estimation

This is a fundamental component of the motion control module because it enabled the robot to regulate its separation from the target person during autonomous following. The framework estimated the person's relative distance using the height of the bounding box generated by the YOLOv8 human detector. The principle of perspective projection applied here can be described as the pinhole camera model, where the apparent size of an object captured by a camera varies inversely with its distance from the camera.

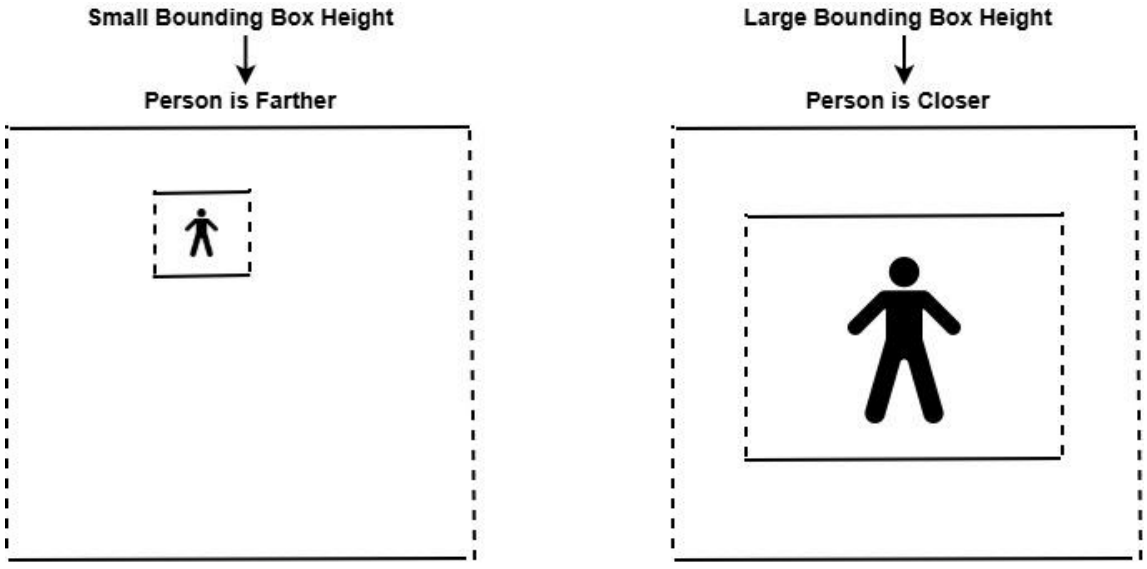


Figure 3.12 - Relative Distance Estimation Based on Variations in Bounding Box Height

The bounding-box height was calculated as in equation 3.4.

During preliminary experimental calibration, a reference bounding-box height corresponding to the desired following distance was determined empirically. This reference value served as the target operating point for the motion controller throughout the experimental evaluation. The distance error was computed as

$$e_d = H_r - H_B \tag{3.10}$$

where:

- H_r is the reference bounding-box height,
- H_B represents the measured bounding-box height,
- e_d is the relative distance error.

The computed distance error was continuously monitored by the adaptive motion controller to determine the robot's translational motion. Three operating conditions were established:

- When the measured bounding-box height was *smaller* than the reference height, the person was considered farther from the robot than the desired following distance. Consequently, the controller generated a positive linear velocity to move the robot forward.
- When the measured bounding-box height was *greater* than the reference height, the person was considered too close to the robot. The controller therefore reduced the forward velocity or generated backward motion to increase the separation distance.
- When the measured bounding-box height remained *within a predefined tolerance* around the reference value, the desired following distance was considered to have been achieved. Under this condition, the controller maintained zero linear velocity while continuing to monitor the person's position.

Because bounding-box measurements may fluctuate slightly owing to image noise, temporary occlusions, or minor body movements, the controller incorporated tolerance limits and temporal filtering before generating motion commands.

3.7.3 Motion Decision Logic

The motion decision logic formed the decision-making component of the motion control module. It was used to determine the appropriate robot motion based on the relative position of the detected person obtained from the perception subsystem. The controller continuously analysed the person's horizontal position and estimated relative distance and subsequently generated one of three translational motion commands: forward movement, backward movement, or stop. In addition, a dead-band mechanism was incorporated to suppress unnecessary robot movements caused by minor fluctuations in object detection, thereby improving motion smoothness and overall system stability (Siciliano *et al*., 2010; Corke, 2017).

Unlike conventional threshold-based controllers that utilize fixed velocity commands, this designed decision logic continuously updates the robot's motion according to successive visual observations. This adaptive behaviour enabled the robot to respond naturally to changes in the person's movement while maintaining a comfortable and safe following distance.

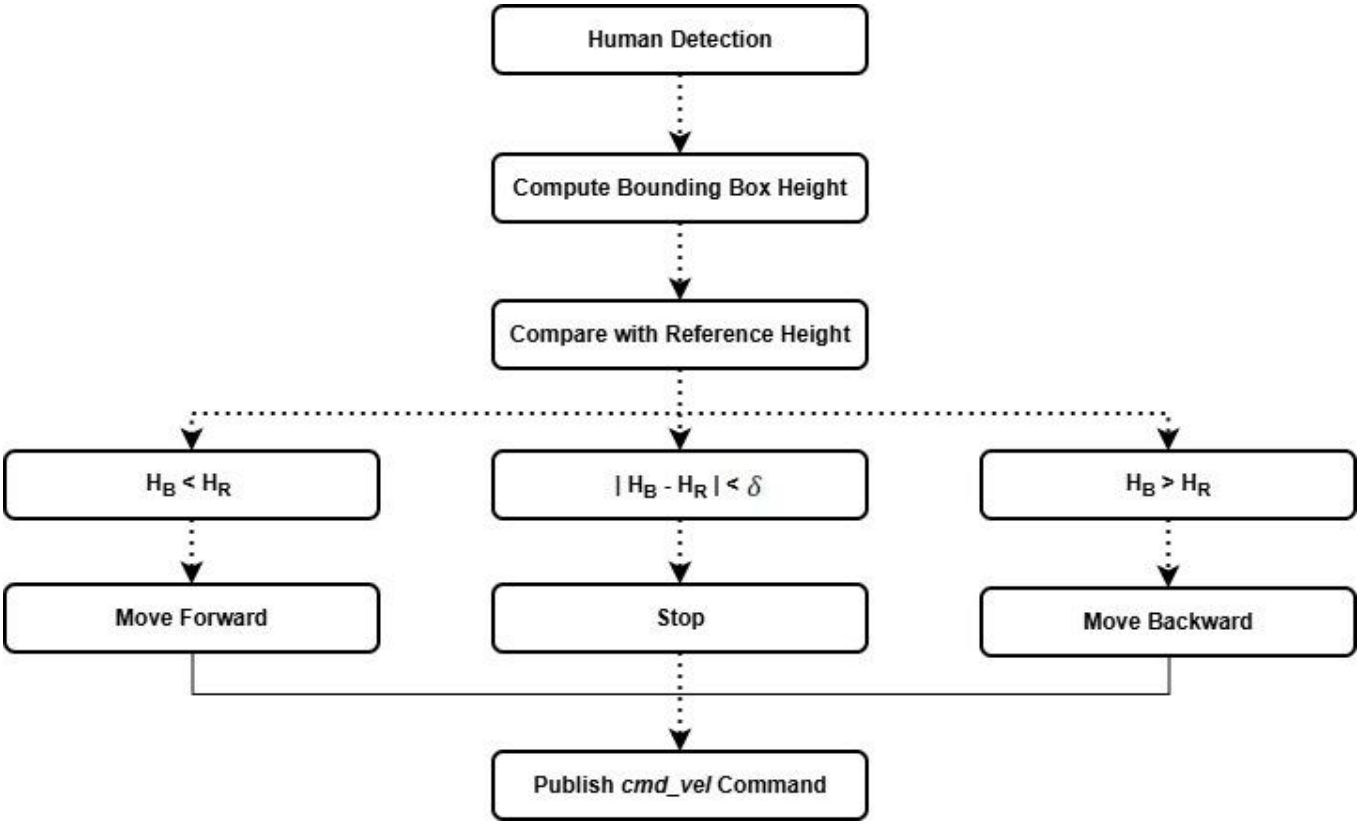


Figure 3.13 - Motion Decision Logic Implemented in the Adaptive Visual Servo Controller

a. Forward Motion Decision

Forward motion was initiated whenever the detected person moved farther away than the desired following distance. This condition was identified by comparing the measured bounding-box height with the predefined reference height established during system calibration. When

$$H_B < H_r \tag{3.11}$$

where:

- H_B is the measured bounding-box height
- H_r represents the reference bounding-box height

The controller interpreted the detected person as moving away from the robot. Consequently, a positive linear velocity command was generated and published through the */summit_xl/cmd_vel* topic, enabling the robot to move forward and reduce the separation distance to 2m to 3m. The forward velocity was generated gradually rather than instantaneously to produce smooth robot motion and minimise abrupt acceleration. This adaptive behaviour improved both tracking stability and user comfort during human following.

b. Backward Motion Decision

Backward motion was generated whenever the detected person approached the robot beyond the desired following distance. Under this condition, the bounding-box height exceeded the reference value, indicating that the person occupied a larger proportion of the image. Thus,

$$H_B > H_r \tag{3.12}$$

indicated that the person was closer than the desired operating distance. The controller therefore generated a negative linear velocity command, causing the robot to move backward until the set separation distance of (2m to 3 m) was restored. This behaviour ensured that the robot maintained a comfortable following distance throughout operation.

c. Stop Decision

The stop condition was activated whenever the measured bounding-box height remained sufficiently close to the reference height. Under this situation, the desired following distance had been achieved, and no translational movement was required.

$$|H_r - H_B| < \delta \tag{3.13}$$

where:

- δ denotes the allowable distance tolerance.

When this condition was satisfied, the controller assigned v=0, where (v) represents the robot's linear velocity. Although translational motion ceased under this condition, the controller continued to monitor the person's horizontal position within the image.

d. Dead-Band Mechanism

Object detection obtained from successive image frames may exhibit small fluctuations due to image noise, illumination changes, camera vibration, or slight body movements. If every minor variation were interpreted as genuine human motion, the robot would continuously accelerate, decelerate, and oscillate around the desired position. These undesirable behaviours were minimised by adding a dead-band mechanism into the controller. The dead-band is a tolerance region around both the desired horizontal alignment and the desired following distance within which no corrective translational motion was generated. When the detected person remained within the predefined tolerance limits, the controller maintained the current motion state instead of issuing new velocity commands. Consequently, insignificant changes in the measured bounding-box position were ignored, thereby reducing unnecessary robot movements.

3.7.4 Velocity Generation

This is when the adaptive visual servoing controller was converted into a quantitative linear velocity command. The is where the robot's forward and backward movement was regulated according to the estimated distance between the robot and the detected person. Rather than

operating the robot using a fixed speed, the proposed controller adjusted the magnitude of the linear velocity according to the observed distance error. This enabled the robot to approach a person smoothly, reduce its speed when the desired following distance was approached, and reverse when the person became too close. The distance error is expressed as

$$e_d = H_r - H_B \tag{3.14}$$

where H_r represents the reference bounding-box height and H_B represents the current measured bounding-box height.

The relationship between bounding-box height and linear velocity was selected such that a smaller bounding-box height, indicating that the person was farther away, resulted in positive linear velocity, whereas a larger bounding-box height, indicating that the person was closer, resulted in negative linear velocity. A proportional control relationship was therefore adopted,

$$v_x = K_d\, e_d \tag{3.15}$$

where:

- v_x is the commanded linear velocity;
- K_d is the distance-control gain; and
- e_d is the distance error.

This relationship allowed the magnitude of the robot's velocity to vary according to the severity of the distance error. When the person was substantially farther from the desired following distance, a larger forward velocity was generated. As the person approached the desired distance, the velocity progressively decreased. Similarly, when the person became too close, the error changed sign and the controller generated a negative linear velocity to move the robot backwards. Meanwhile, excessive robot speeds were prevented by constraining the generated velocity to be within predefined operational limits.

$$v_{min} \leq v_x \leq v_{max} \tag{3.16}$$

This velocity saturation mechanism ensured that the robot remained within safe operating limits even when a large detection error occurred. Sudden changes in the detected person's position or temporary inaccuracies in bounding-box estimation could otherwise result in unnecessarily large velocity commands.

A dead band was also applied around the desired following distance. When the distance error remained within a predefined tolerance,

$$e_d < \delta_d \tag{3.17}$$

the linear velocity was set to zero, $v_x = 0$, where δ_d represents the distance dead-band. This prevented the robot from continuously moving forward and backward in response to small variations in the detected bounding-box height. Such variations could arise from natural human body movement, camera noise, changes in posture, or small fluctuations in YOLOv8 detection.

In addition to velocity limiting and dead-band control, the generated velocity was smoothed before being transmitted to the robot base. Instead of allowing the commanded velocity to change abruptly between successive camera frames, the controller gradually adjusted the output. A simple temporal smoothing relationship can be represented as

$$v_x^t = \alpha\, v_x^{t-1} + (1 - \alpha\, v_{cmd}^t) \tag{3.18}$$

where v_{cmd}^t represents the newly calculated velocity, v_x^{t-1} represents the previous velocity command, and α represents the smoothing coefficient. The use of temporal smoothing reduced abrupt changes in robot speed and contributed to more stable following behaviour.

The resulting linear velocity was incorporated into the ROS velocity command and transmitted to the Robotnik Summit XL through the */summit_xl/cmd_vel* interface. A positive value of v_x commanded forward movement, a negative value commanded backward movement, and a value of approximately zero resulted in no translational movement.

3.7.5 Adaptive Motion Controller

This formed the control component responsible for converting continuously changing visual observations into stable and appropriate robot motion. The controller operated on the detection information generated by the YOLOv8 perception module and continuously adjusted the Robotnik Summit XL's motion according to the estimated horizontal position and relative distance of the target person. The design incorporated three principal mechanisms, motion smoothing, safe distance maintenance, and adaptive gain selection. These mechanisms were introduced to reduce unnecessary oscillations, maintain an appropriate separation between the robot and the person, and provide an appropriate control response under different target-position conditions. The controller was implemented as a feedback system in which the robot's motion was continuously influenced by new visual observations. As a result, the control output was not determined from a single detection but from a sequence of measurements obtained from successive camera frames.

a. Motion Smoothing

This was introduced to reduce abrupt changes in the velocity commands generated from successive human detections. Although YOLOv8 provided accurate real-time detection, the estimated bounding-box centre and height could vary slightly between consecutive frames because of detection uncertainty, camera motion, changes in human posture, and image noise. Directly transmitting these instantaneous measurements to the robot controller could therefore result in rapid fluctuations between velocity commands and produce undesirable oscillatory motion. This was resolved by applying temporal smoothing to the calculated linear velocity commands. The smoothed linear velocity was represented in equation 3.18

$$v_x^t = \alpha\, v_x^{t-1} + (1 - \alpha\, v_{cmd}^t)$$

A similar procedure was applied to the angular velocity command. By incorporating information from the previous control cycle, the controller prevented sudden changes in robot velocity and produced more continuous motion. The smoothing mechanism was particularly important because the perception and control processes operated continuously and therefore generated new commands at a relatively high frequency.

The smoothing coefficient was selected experimentally to provide a compromise between responsiveness and stability. Excessive smoothing could cause the robot to respond too slowly when the person changed direction, whereas insufficient smoothing could result in jerky motion and oscillation. The selected value was therefore determined during experimental calibration based on the observed response of the physical robot.

b. Safe Distance Maintenance

Maintaining an appropriate distance from the target person was an important safety and behavioural requirement of the proposed system. The controller used bounding-box height as a relative indicator of the distance between the person and the robot. A reference bounding-box height was established during calibration to represent the desired following distance.
The distance error was calculated as

$$e_d = H_r - H_B \tag{3.20}$$

where H_r represents the reference bounding-box height and H_B represents the measured bounding-box height. The controller interpreted changes in this error to determine the appropriate translational response. When H_B was substantially smaller than H_r, the detected person was considered farther away and the robot generated forward motion. Conversely, when H_B became substantially larger than H_r, the person was considered too close and the controller reduced the forward velocity or generated backward motion.

A tolerance region was introduced around the reference value to prevent unnecessary motion caused by small variations in bounding-box height. Thus, when

$$|H_r - H_B| < \delta \tag{3.21}$$

the linear velocity was reduced to zero. The parameter δ represented the acceptable distance tolerance. Velocity saturation was also applied to ensure that the robot did not exceed its predefined operating limits,

$$v_{min} \leq v_x \leq v_{max} \tag{3.22}$$

This constraint was particularly important when large visual errors occurred, since an unusually small or large bounding box could otherwise result in an excessive velocity command. Combining the distance dead-band with velocity saturation therefore provided a practical mechanism for maintaining a stable following distance while limiting potentially unsafe motion.

It is important to note that the proposed method provided relative distance regulation rather than metric distance measurement. The bounding-box height was used as a visual proxy for target distance, and the desired following distance was established through system calibration.

c. Adaptive Gain Selection

Adaptive gain selection was employed to ensure that the controller responded appropriately to variations in the magnitude of the visual tracking error. In a conventional proportional controller, a fixed gain may provide satisfactory behaviour under one operating condition but may result in excessive responsiveness when the error becomes large or insufficient responsiveness when the error is small. This consideration was particularly relevant to the proposed system because the apparent size and position of the detected person changed continuously as the person moved relative to the robot.

The basic relationship between the distance error and linear velocity was expressed as

$$v_x = K_d \, e_d \tag{3.23}$$

where K_d represented the distance-control gain.

The controller adjusted its effective response according to the magnitude of the visual error while maintaining predefined limits on the resulting velocity. Larger errors produced stronger corrective action, while smaller errors resulted in progressively smaller corrections. This approach reduced the likelihood of excessive movement when the robot was already close to the desired position. The gain values were determined through an iterative calibration process using the physical Robotnik Summit XL. Initial gain values were selected conservatively and subsequently adjusted by observing the robot's response to different target positions and distances. The calibration considered several factors, including response speed, overshoot, oscillation, stopping behaviour, and the ability of the robot to maintain visual alignment with a moving person.

The adaptive gain mechanism was therefore designed as a practical engineering solution rather than a computationally intensive model-based adaptive controller. This was appropriate for the research because the primary objective was to obtain responsive and stable real-time person-following using monocular vision and lightweight control computation. The approach also allowed the perception module and motion controller to remain computationally independent, which was beneficial given that YOLOv8 inference represented the major computational demand of the system.

d. Integration of the Adaptive Controller

The three mechanisms were integrated into a single feedback-control process. The YOLOv8 detector first provided the bounding-box centre and height of the detected person. The controller then calculated the horizontal and distance errors and used these values to determine the required angular and linear velocities. The generated velocities were subjected to gain adjustment, safety constraints, dead-band filtering, and temporal smoothing before being transmitted to the Robotnik Summit XL through the */summit_xl/cmd_vel* interface.

3.8 SYSTEM IMPLEMENTATION

The implementation translated the system architecture, perception methodology, and adaptive motion-control strategy described in the preceding sections into executable ROS components. The system was implemented as a modular software framework consisting primarily of a YOLOv8-based human detection node and a follower control node. These components communicated through ROS topics and operated as part of a continuous perception–decision–actuation loop.

3.8.1 Software Implementation

The software implementation consisted of two developed Python scripts, *yolo_detector.py* and *follower.py*. The first implemented the visual perception and human-detection functions, while the second implemented the decision-making and adaptive motion-control functions. The separation of the two scripts ensured that each node had a clearly defined responsibility. The YOLO detector was responsible for converting camera images into structured human-detection information, whereas the follower node was responsible for converting this information into appropriate robot motion commands.

a. Python Scripts

Python was selected as the language for the implementation of this project because of its extensive support for computer vision, deep learning, ROS development, and rapid prototyping. It provided access to OpenCV for image processing, the Ultralytics framework for YOLOv8 inference, and the ROS Python client library for communication between nodes. Each Python script was implemented as an independent ROS node. The nodes were initialised using the ROS Python client library and configured with the required publishers, subscribers, processing functions, and control parameters.

b. YOLO Detector Implementation

The *yolo_detector.py* node implemented the visual perception component of the proposed system. Its main responsibility was to receive camera images, process them using YOLOv8, identify human targets, and publish the resulting detection information to the follower node.

The implementation began by loading the YOLOv8 model through the Ultralytics framework. The model used in the research was the lightweight YOLOv8n model, which provided a practical compromise between detection performance and inference speed for real-time robotic operation. The detector subscribed to the compressed camera image topic, */image/compressed* while the incoming *sensor_msgs/CompressedImage* messages were decoded using OpenCV and the resulting image was passed to the YOLOv8 inference pipeline. For each incoming frame, the detector examined the returned detections and retained detections corresponding to the *person* class. Also, the confidence threshold applied, reduced the probability of treating weak or uncertain detections as valid targets. For an accepted detection, the bounding-box coordinates were represented as $(X_{min}, Y_{min}, X_{max}, Y_{max})$, where (X_{min}, Y_{min}) is the upper-left corner and (X_{max}, Y_{max}) is the lower-right corner. Thus, the

bounding-box dimensions were then calculated with equation 3.3 and 3.4 while the bounding-box centre was calculated with equation 3.7 and 3.8.

These parameters provided the information required by the follower controller for horizontal alignment and relative distance estimation. The detector subsequently published the detection information through */person_detection* using a *std_msgs/Float32MultiArray* message. The detector also supported visual debugging during development by providing processed image information showing the detected bounding box and associated detection information. This facilitated verification of the detector's behaviour before integration with the physical robot.

c. Follower Node Implementation

The *follower.py* node implemented the decision-making and motion-control component of the proposed system. The follower node first validated the received detection information. When a valid person detection was available, the node extracted the bounding-box centre and height. These values were used as the primary visual features for control. A zero-velocity command was generated when the target was lost or when the robot reached the desired following condition. This ensured that the robot did not continue executing an outdated motion command after visual information had become unavailable.

3.8.2 Integration Process

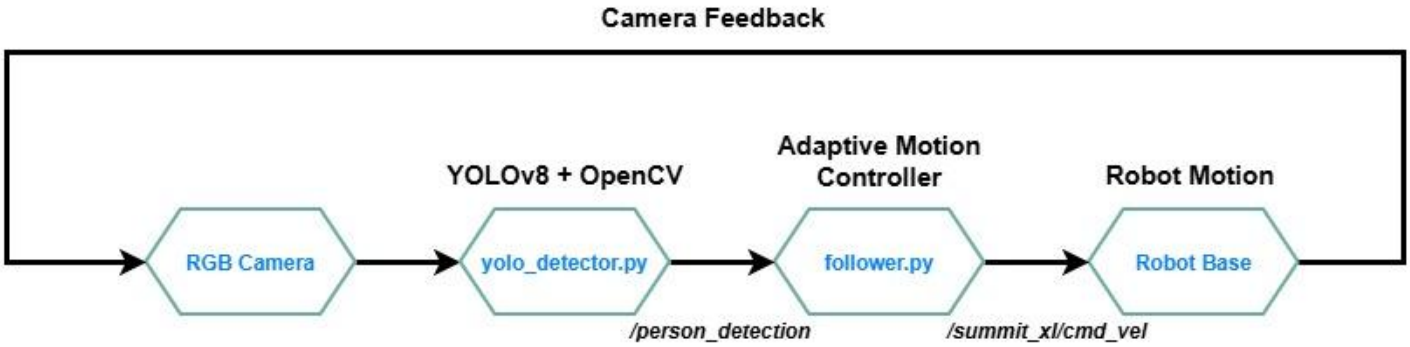


Figure 3.14 - Perception to Action Interaction

The integration process brought together the camera, human-detection, ROS communication, motion-control, and robot-actuation components into a unified autonomous person-following system. The process began as the RGB camera continuously acquired images and transmitted them as compressed image stream through the *ROS /image/compressed* topic. The YOLOv8 detection node subscribed to this image stream, decoded and pre-processed the incoming frames using OpenCV, and performed real-time inference to identify a human in the camera FOV. For each valid detection, the system extracted the bounding-box centre coordinates, width, height, and confidence score, which were packaged into a *std_msgs/Float32MultiArray* message and published through the */person_detection* topic. The follower controller subscribed to this topic and used the detected person's horizontal position to calculate the visual alignment error relative to the image centre, while the bounding-box height was used as a relative indicator of the person's distance from the robot. Based on these visual measurements, the adaptive motion-control module determined whether the robot should move forward, move backward, or stop, while dead-band filtering, velocity limiting, and

motion smoothing were applied to reduce unnecessary or abrupt movements. The resulting linear and angular velocity commands were encapsulated within a *geometry_msgs/Twist message* and published through the */summit_xl/cmd_vel* topic. The Summit XL base-control system received these commands and converted them into appropriate wheel and motor actions, resulting in physical robot movement towards, away from, or around the target person. The robot's subsequent movement produced a new visual observation, which was again captured by the camera and processed by the perception subsystem, thereby establishing a continuous closed-loop perception–control–actuation cycle.

3.9 EXPERIMENTAL SETUP

3.9.1 Experimental Environment

The experimental development of the autonomous person-following system was conducted in an indoor laboratory environment to provide a controlled and repeatable setting for assessing the interaction between the vision-based perception module and the Robotnik Summit XL mobile robot. The laboratory provided sufficient operating space for the robot to perform forward and backward movements while allowing the human participant to move naturally within the camera's FOV. The experimental environment contained typical indoor features, including walls, furniture, laboratory equipment, and other background elements, thereby providing a realistic environment for evaluating the robustness of the detection and following system beyond an artificially simplified setting. Lighting conditions were maintained within normal indoor operating conditions, with the experiments conducted under available laboratory illumination rather than relying exclusively on controlled studio lighting. This allowed the system's response to natural variations in brightness and shadows within the laboratory to be observed. The floor surface was predominantly level and sufficiently smooth to support stable movement of the mobile robot. The indoor environment was selected because it provided a safe and manageable setting for repeated experiments while still presenting practical visual and operational conditions relevant to autonomous person-following applications. The same general experimental area and camera configuration were maintained across the trials to improve consistency and enable meaningful comparison of the system's performance under the different experimental scenarios.

3.9.2 Test Scenarios

The autonomous person-following system was evaluated under nine experimental scenarios designed for easy evaluation and to assess the system’s ability to detect, track, and respond appropriately to different human movements and environmental conditions.

- **Scenario 1 (Standing)**

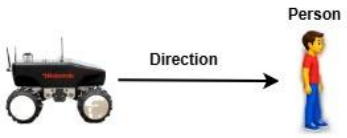


Figure 3.15 - Standing Scenario

Evaluated the system's ability to maintain a stationary state when the person remained still within the predefined following-distance range, thereby assessing the effectiveness of the stopping condition and avoidance of unnecessary robot movement.

- **Scenario 2 (Walking Forward)**

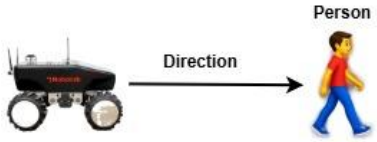


Figure 3.16 - Walking Forward Scenario

Evaluated the robot's response when the person moved progressively away from the robot, with the system expected to detect the reduction in bounding-box height and generate appropriate forward velocity commands to maintain the desired following distance.

- **Scenario 3 (Walking Backward)**

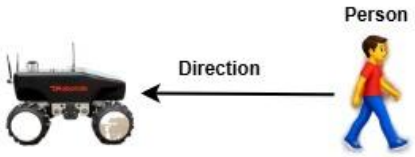


Figure 3.17 - Walking Backward Scenario

Assessed the opposite response, where the person moved towards the robot and the increase in bounding-box height was expected to cause the controller to reduce forward motion or generate backward movement when the person became too close.

- **Scenario 4 (Stopping)**

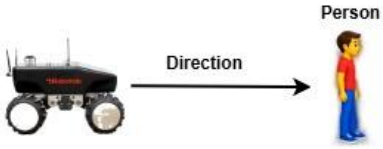


Figure 3.18 - Stopping Scenario

Evaluated the robot's ability to respond when a previously moving person suddenly stopped, with the controller expected to detect the stabilisation of the target's position and distance and subsequently reduce the velocity to zero.

- **Scenario 5 (Standing at a Distance >3m)**

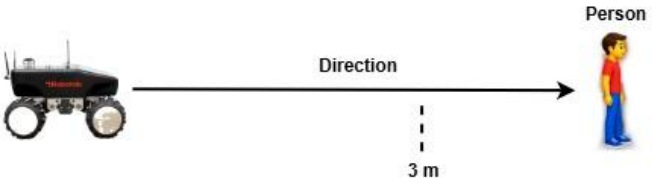


Figure 3.19 - Standing >3m Scenario

Evaluated the distance-control behaviour of the system by positioning the person at a distance greater than 3 meters with the controller expected to detect the human target with a smaller bounding box and then begins to move forward.

- **Scenario 6 (Walking Forward and Backward within 2m to 3m range)**

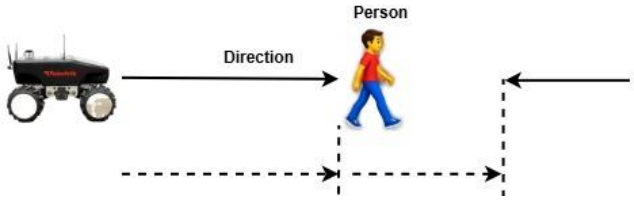


Figure 3.20 - Walking within 2m to 3m Range Scenario

Evaluated the distance-control behaviour of the system by making forward and backward movements within its predefined stop distance.

- **Scenario 7 (Standing at a Distance <3m)**

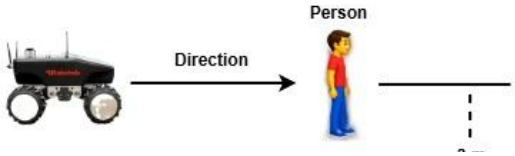


Figure 3.21 - Standing <3m Scenario

Evaluated the distance-control behaviour of the system by positioning the person at a distance lesser than 3 meters with the controller expected to detect the human target with a larger bounding and then begins to move backwards.

- **Scenario 8 (Lighting Variation)**

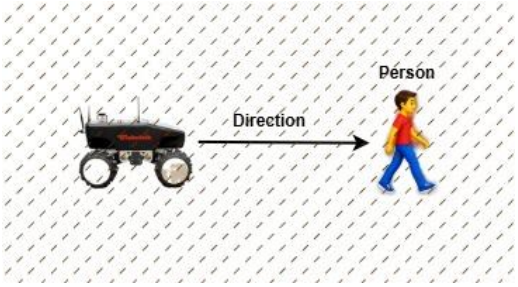


Figure 3.22 - Lighting Variation Scenario

Examined the robustness of the YOLOv8 perception and following system under different indoor illumination conditions, including relatively bright, normal, and reduced-light conditions, to determine whether changes in image quality affected detection reliability and robot-following performance.

- **Scenario 9 (No Target)**



Figure 3.23 - No Target Scenario

Evaluated the robot's behaviour when there is no human target in the camera's FOV.

In all, these scenarios provided a structured basis for evaluating detection reliability, distance regulation, detection responsiveness, stopping behaviour, illumination robustness, and overall stability of the autonomous person-following system under representative indoor operating conditions.

3.10 PERFORMANCE EVALUATION

3.10.1 Evaluation Metrics

The performance of the system was evaluated using a combination of perception, computational, tracking, and motion-control metrics to provide a comprehensive assessment of its effectiveness under the defined experimental scenarios.

- **Detection accuracy** was used to determine the proportion of relevant human instances correctly detected by the YOLOv8 model.
- **Precision** measured the proportion of reported person detections that were correct, thereby indicating the extent of false-positive detections.
- **Recall** measured the proportion of actual person instances successfully detected by the system and was particularly relevant for assessing target-loss behaviour.
- **Frames per second (FPS)** was used to quantify the processing rate of the perception pipeline and assess whether the system operated sufficiently close to real-time requirements.
- **Specificity** measures how well your person detector correctly identifies situations where there is no target person**.**
- **F1 Score** combines precision and recall into one measure. It is particularly useful for a single measure of the detector's overall ability to correctly detect a person while avoiding incorrect detections.
- **Success rate** represented the proportion of experimental trials in which the robot successfully detected, followed, and maintained the target person according to the predefined experimental criteria.
- **Robot response time** measured the time required for the robot to respond to a change in the person's position or movement state, such as starting, stopping, approaching, or moving away from the robot.

3.10.2 Data Collection

Data were collected during the experimental trials using three complementary sources, ROS bag recordings, system log files, and experimental video recordings. ROS bags were used as the primary source of time-stamped robotic and perception data, allowing relevant ROS topics such as */image/compressed, /person_detection*, and */summit_xl/cmd_vel* to be recorded and subsequently replayed for detailed analysis of the relationship between human detection, target position, controller decisions, and robot velocity commands. System log files generated during

execution were used to capture operational information from the YOLOv8 detector and follower controller, including detection events, confidence values, bounding-box measurements, processing information, controller states, velocity commands, detection losses, and system warnings. These logs provided additional information for calculating performance measures such as detection reliability, response time, processing behaviour, and tracking continuity. In addition, video recordings of the experimental trials were collected to provide an independent visual record of the robot–person interaction and to support qualitative assessment of following behaviour, target acquisition, stopping response, directional changes, motion smoothness, and system failures that might not have been fully represented by numerical ROS data. The three data sources were subsequently compared and synchronised, where appropriate, to improve the reliability of the evaluation and to support quantitative and qualitative analysis of the proposed person-following system across the defined experimental scenarios. This approach also ensured that the experimental results could be independently examined after each trial rather than relying solely on observations made during real-time system operation.

3.10.3 Data Analysis

The experimental data were analysed using a combination of statistical analysis, graphical representation, tabular presentation, and comparative evaluation to determine the performance and reliability of the system. The recorded ROS bag data, system log files, and experimental video observations were first organised according to the defined test scenarios and performance metrics.

- **Statistical analysis:** This was used to calculate descriptive measures such as mean, minimum, maximum, standard deviation, and, where appropriate, percentage-based performance measures for detection accuracy, precision, recall, FPS, latency, following-distance error, tracking stability, response time, success rate, and average robot speed. These measures provided an objective assessment of the consistency and variability of system performance across repeated experimental trials.
- **Graphs:** They were used to illustrate temporal relationships and system responses, including changes in bounding-box height, estimated following distance, linear velocity, angular velocity, detection confidence, processing rate, and response latency over time.
- **Tables:** They were used to summarise the numerical results obtained from individual scenarios and to provide concise comparisons between experimental conditions.
- **Performance comparison:** It was conducted across the different test scenarios to determine how variations in human movement, target distance, horizontal position, and illumination affected the perception and control performance of the system. Where appropriate, the obtained results were also compared with findings reported in related person-following studies to establish the relative performance and practical significance of the implemented approach.

3.11 SAFETY CONSIDERATIONS

Throughout the design, implementation, and experimental evaluation of the system safety considerations were made to minimise potential risks to human participants. All experimental trials were conducted within a controlled indoor laboratory environment, where the operating area was inspected and kept sufficiently clear of unnecessary obstacles and personnel before robot operation. The robot's movement was constrained to predefined safe operating conditions, including appropriate limits on linear velocity, to reduce the possibility of unintended collisions or excessive acceleration during following experiments. An emergency-stop mechanism was maintained as an essential safety measure throughout the trials, allowing robot motion to be immediately interrupted whenever an unsafe condition, unexpected movement, communication failure, or loss of control was observed. The human participant's safety was prioritised by ensuring that the participant was informed of the experimental procedure, remained within the designated test area, and could stop an experiment whenever necessary. The system was also configured to stop the robot when a valid person detection was lost for a predefined period rather than allowing previously generated velocity commands to continue indefinitely. Appropriate precautions were taken to protect the participant from unnecessary physical interaction with the robot, particularly during experiments involving changes in walking direction, distance, and stopping behaviour.

CHAPTER FOUR

RESULTS AND DISCUSSION

This chapter presents the experimental evaluation, and discussion of the autonomous vision-based person-following system developed on the Robotnik Summit XL mobile robot. It focuses on the evaluation process defined by the experimental scenarios and performance metrics, including detection accuracy, precision, recall, following-distance error, tracking stability, success rate, and robot response time. Both quantitative data obtained from ROS bag recordings and system logs and qualitative observations from experimental video recordings were analysed to determine the reliability, responsiveness, and practical effectiveness of the system.

4.1 RESULTS

The results collected for evaluation in this study was done using the *rosbag* tool across nine scenarios to test the entire system design on the regular human dorsal poses in indoor environments. The results evaluated includes the detection performance, motion control performance and position estimation performance.

4.1.1 Detection Performance

During detection, the person tracking module, *person_tracker.py* maintains the identity of the detected human target across consecutive frames using a unique tracker ID as shown in figure 4.1 below. The ID allows the system to distinguish the selected person from other detected individuals and maintain target continuity during movement. The tracker updates the target's bounding-box position and ID as new detections are received, providing consistent target information to the follower controller. When it loses and regains perception of a human target, it reassigns the previous unique ID before it loses the human target.

```
robotics_lhu@roboticsLab4:~$ rosrun follow_me_yolo person_tracker.py
[INFO] [1784296336.704000]: Person Tracker Started
[INFO] [1784296341.055025]: Target locked ID: 1
[WARN] [1784296344.480454]: Different person detected. Keeping ID 1
[WARN] [1784296347.718328]: Different person detected. Keeping ID 1
[WARN] [1784296361.220685]: Different person detected. Keeping ID 1
```

Figure 4.1 - Active Person Tracker

In analysing the detection performance, some fundamental parameters will be defined to provide a detailed detection result. These parameters are ground truth (GT), and detector output (true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN)).

i. **Ground Truth (GT)** – This defined as the condition where a person is present or not present.

ii. **Detector Output** – This is defined as the conditions where the robot's detector detects a person or no person.

iii. **True Positive (TP)** – This is when a person is present in the camera image i.e. ground truth = human present.

iv. **True Negative (TN)** – This occurs when there is no person present and no person is detected.

v. **False Negative (FN)** – This occurs when a person is present, but the detector fails to detect the person i.e. ground truth is not equal to detector's output.

vi. **False Positive (FP)** – This occurs when the detector reports a person detection but there is no person corresponding to the detector's output.

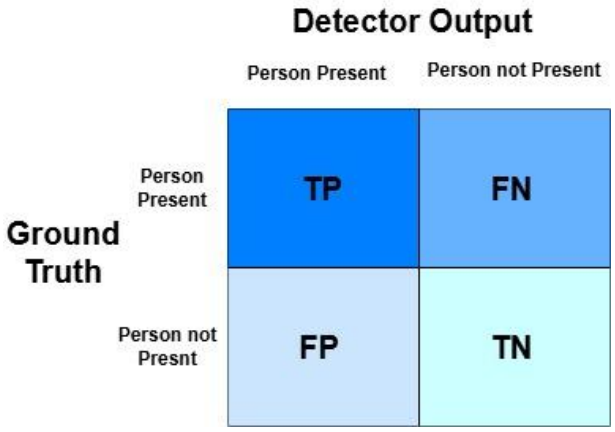


Figure 4.2 - Confusion Matrix to define the Ground Truth and Detector Output

The detection output values were extracted from the recorded timestamped message data gotten after applying the *rosbag* tool. For each of the scenarios performed, the message data was gotten within an average of 60 seconds to 80 seconds. A message data includes the following features;

[*detected, x_center, y_center, width, height, confidence*]

```
(yolo_env) robotics_lhu@roboticsLab4:~/catkin_ws$ rosrun follow_me_yolo yolo_detector.py
[INFO] [1784298086.104235]: Starting YOLO Person Detector
[INFO] [1784298087.187623]: BBox x1=384.5 x2=725.1 center=554.8 image_width=1920
[INFO] [1784298087.222601]: YOLO>FPS: 2.05 Inference Time: 487.6 ms
[INFO] [1784298087.307555]: BBox x1=395.8 x2=751.1 center=573.5 image_width=1920
[INFO] [1784298087.316559]: YOLO>FPS: 13.62 Inference Time: 73.4 ms
[INFO] [1784298087.375237]: BBox x1=387.1 x2=784.8 center=586.0 image_width=1920
[INFO] [1784298087.419939]: YOLO>FPS: 13.01 Inference Time: 76.9 ms
[INFO] [1784298087.556948]: BBox x1=387.7 x2=802.0 center=594.8 image_width=1920
[INFO] [1784298087.591899]: YOLO>FPS: 9.17 Inference Time: 109.0 ms
[INFO] [1784298087.715288]: BBox x1=407.2 x2=816.7 center=612.0 image_width=1920
[INFO] [1784298087.744941]: YOLO>FPS: 9.93 Inference Time: 100.7 ms
[INFO] [1784298087.810363]: BBox x1=536.4 x2=867.5 center=702.0 image_width=1920
```

Figure 4.3 - YOLO Detector Message Data

Where the detected value of 1 or 0 is combined with the confidence score which ranges from 0 to 1 to determine if a person is present or not present.

Table 4.1 - Timestamped Message Data for each Scenario

S/No	Scenarios	Camera Frames	Detection Messages	Performance Duration (Seconds)	Detection Rate (Hz)
1.	Standing	1501	299	68.737	4.338
2.	Forward	1360	271	63.359	4.269
3.	Backward	1627	324	68.352	4.737
4.	Stopping	1075	214	82.867	2.585
5.	Standing >3m	1371	273	60.659	4.5
6.	Standing at 2m to 3m	1853	369	81.732	4.509
7.	Standing <3m	1506	300	60.583	4.943
8.	Light Variation	1647	328	80.961	4.040
9.	No Target	2013	401	74.930	5.349

i. Scenario One (Standing)

With 299 message data received, it was observed that detection value for all the message data was equal to 1. Therefore, if 299 message data were evaluated with the human correctly detected in all conditions, TP = 299, FP = 0, FN = 0 and TN = 0.

- Detection Accuracy (%) = $\frac{TP+TN}{TP+TN+FP+FN}$ x 100% = $\frac{299+0}{299+0+0+0}$ x 100% = 100%
- Precision (%) = $\frac{TP}{TP+FP}$ x 100% = $\frac{299}{299+0}$ x 100% = 100%
- Recall (%) = $\frac{TP}{TP+FN}$ x 100% = $\frac{299}{299+0}$ x 100% = 100%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1501}{68.737}$ = 22 FPS
- Specificity = $\frac{TN}{TN+FP}$ = 0%
- F1 Score = $\frac{2TP}{2TP+FP+FN}$ x 100% = $\frac{2 \text{ x } 299}{2(299)+0+0}$ x 100% = 100%
- Success rate – Does the robot remain stationary while continuously detecting the stationary person, without unintended forward movement?

  Success rate = $\frac{Detection\ Accuracy}{Expected\ detections}$ x 100% = $\frac{100}{100}$ x 100% = 100%

In this scenario where the ability of the detection system to maintain recognition of a stationary target is evaluated, it achieved a detection accuracy of 100%, with both precision and recall reaching 100%. The resulting F1 score was also 100%, indicating that the detector successfully maintained target recognition without recorded false-positive or false-negative detections under this experimental condition. The average confidence score was approximately 0.90, demonstrating a high level of confidence in the detected person. The system operated at approximately 22 FPS, providing sufficiently frequent visual observations for the follow-me controller. The results demonstrate that the detection system was highly reliable when the target remained stationary under the tested conditions. The perfect recall indicates that the target was consistently detected throughout the evaluation period.

ii. Scenario Two (Forward)

With 271 message data received, it was observed that detection value for 265 message data was equal to 1 while 6 was equal to 0. Therefore, if 271 message data were evaluated with the human correctly detected in the identified conditions, TP = 265, FP = 0, FN = 6 and TN = 0.

- Detection Accuracy (%) = $\frac{\text{TP+TN}}{\text{TP +TN+ FP+FN}}$ x 100% = $\frac{265 + 0}{265 + 0 + 0 + 6}$ x 100% = 97.8%
- Precision (%) = $\frac{\text{TP}}{\text{TP + FP}}$ x 100% = $\frac{265}{265 + 0}$ x 100% = 100%
- Recall (%) = $\frac{\text{TP}}{\text{TP + FN}}$ x 100% = $\frac{265}{265 + 6}$ x 100% = 97.8%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1360}{63.359}$ = 22 FPS
- Specificity = $\frac{\text{TN}}{\text{TN + FP}}$ x 100% = $\frac{0}{0 + 0}$ x 100% = 0%
- F1 Score = $\frac{2\text{TP}}{2\text{TP + FP+FN}}$ x 100% = $\frac{2 \text{ x } 265}{2(265)+ 0 + 6}$ x 100% = 98.9%
- Success rate – Does the robot move forward in response to the person moving forward and maintains following behavior without losing the target?
  Success rate = $\frac{Successful\ following}{Expected\ following}$ x 100% = $\frac{97.8}{100}$ x 100% = 97.8%

This forward walking scenario evaluated detection performance while the human target moved away from the robot. The system achieved a 97.8% in both detection accuracy and recall, while precision remained at 100%. The resulting F1 score of 98.9% demonstrates that the system maintained a high level of detection performance during forward target movement. The average confidence score of 0.85, which is slightly lower than that observed in the stationary scenarios is a reduction that is consistent with the increased difficulty associated with a moving target and changing target-camera distance. The system operated at 22 FPS, providing relatively frequent observations during target movement. The results demonstrate that forward target motion had only a limited effect on detection reliability.

iii. Scenario Three (Backward)

With 324 message data received, it was observed that detection value for 319 message data was equal to 1 while 5 was equal to 0 and the confidence score ranged between 0.8 and 0.9. Therefore, if 324 message data were evaluated with the human correctly detected in the identified conditions, TP = 319, FP = 0, FN = 5 and TN = 0.

- Detection Accuracy (%) = $\frac{\text{TP+TN}}{\text{TP +TN+ FP+FN}}$ x 100% = $\frac{319 + 0}{319 + 0 + 0 + 5}$ x 100% = 98.5%
- Precision (%) = $\frac{\text{TP}}{\text{TP + FP}}$ x 100% = $\frac{319}{319 + 0}$ x 100% = 100%
- Recall (%) = $\frac{\text{TP}}{\text{TP + FN}}$ x 100% = $\frac{319}{319 + 5}$ x 100% = 98.5%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1627}{68.352}$ = 24 FPS
- Specificity = $\frac{\text{TN}}{\text{TN + FP}}$ x 100% = $\frac{0}{0 + 0}$ x 100% = 0%
- F1 Score = $\frac{2\text{TP}}{2\text{TP + FP+FN}}$ x 100% = $\frac{2 \text{ x } 319}{2(319)+ 0 + 5}$ x 100% = 99.2%

- Success rate – Does the robot respond appropriately to the person moving backward and does not continue inappropriate forward motion?
  Success rate = $\frac{Successful\ backward\ response}{Expected\ backward\ response}$ x 100% = $\frac{98.5}{100}$ x 100% = 98.5%

The backward walking scenario assessed the ability of the detector to maintain recognition while the human target moved away from the robot. The system achieved a 98.5% in both detection accuracy and recall, with 100% precision. The resulting F1 score was 99.2%, indicating highly reliable detection during backward movement. The average confidence score was approximately 0.85, while the measured processing rate was observed to be higher than that of the forward walking scenario at 24 FPS. The high recall indicates that only a small proportion of person-present observations were missed. These results suggest that backward target movement did not significantly compromise the ability of the YOLO detector to maintain recognition of the target.

iv. Scenario Four (Stopping)

With 214 message data received, it was observed that detection value for 202 message data was equal to 1 while 12 was equal to 0 and the confidence score ranged between 0.8 and 0.9. Therefore, if 214 message data were evaluated with the human correctly detected in the identified conditions, TP = 202, FP = 0, FN = 12 and TN = 0.

- Detection Accuracy (%) = $\frac{TP+TN}{TP+TN+FP+FN}$ x 100% = $\frac{202+0}{202+0+0+12}$ x 100% = 94.4%
- Precision (%) = $\frac{TP}{TP+FP}$ x 100% = $\frac{202}{202+0}$ x 100% = 100%
- Recall (%) = $\frac{TP}{TP+FN}$ x 100% = $\frac{202}{202+12}$ x 100% = 94.4%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1075}{82.867}$ = 13 FPS
- Specificity = $\frac{TN}{TN+FP}$ x 100% = $\frac{0}{0+0}$ x 100% = 0%
- F1 Score = $\frac{2TP}{2TP+FP+FN}$ x 100% = $\frac{2 \text{ x } 202}{2(202)+0+12}$ x 100% = 97.1%
- Success rate – Does the robot effectively reduces or stops its movement when the person stops?
  Success rate = $\frac{Successful\ stops}{Expected\ stops}$ x 100% = $\frac{94.4}{100}$ x 100% = 94.4%

The stopping scenario so far produced the lowest detection performance among the person-present scenarios. The system achieved 94.4% in both detection accuracy and recall, although precision remained at 100% while F1 score was 97.1%. The average confidence score was approximately 0.85. However, the measured FPS decreased to 13 FPS, substantially lower than the rates recorded in other person-present scenarios. The reduction in recall and FPS compared to other scenarios indicates that the stopping condition faces a slight detection delay between the human stop action and the velocity command response of the robot. Nevertheless, the 100% precision indicates that when the system reported a person, the detection was considered valid according to the experimental ground truth.

v. Scenario Five (Standing >3m)

With 273 message data received, it was observed that detection value for all message data was equal to 1 and the confidence score was an average of 0.9. Therefore, if 273 message data were evaluated with the human correctly detected in all conditions, TP = 273, FP = 0, FN = 0 and TN = 0.

- Detection Accuracy (%) = $\frac{TP+TN}{TP+TN+FP+FN}$ x 100% = $\frac{273+0}{273+0+0+0}$ x 100% = 100%
- Precision (%) = $\frac{TP}{TP+FP}$ x 100% = $\frac{273}{273+0}$ x 100% = 100%
- Recall (%) = $\frac{TP}{TP+FN}$ x 100% = $\frac{273}{273+0}$ x 100% = 100%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1371}{60.659}$ = 23 FPS
- Specificity = $\frac{TN}{TN+FP}$ x 100% = $\frac{0}{0+0}$ x 100% = 0%
- F1 Score = $\frac{2TP}{2TP+FP+FN}$ x 100% = $\frac{2 \text{ x } 273}{2(273)+0+0}$ x 100% = 100%
- Success rate – Does the robot respond appropriately without losing the target when the person is detected at the larger distance?
  Success rate = $\frac{Successful\ detection}{Expected\ detection}$ x 100% = $\frac{100}{100}$ x 100% = 100%

When evaluated, the >3 m scenario showed the ability of the detector to recognise a stationary person at a distance farther than the pre-defined robot operating distance. The system achieved 100% in detection accuracy, precision, and recall, resulting in an F1 score of 100%. The average confidence score was approximately 0.90, while the system operated at 23 FPS. The results indicate that the increased target distance did not prevent the detector from successfully recognising the person under the tested conditions. In particular, the perfect recall demonstrates that the target remained detectable despite the increased separation between the robot and the person. This provides evidence that the system can maintain a reliable person detection beyond the 3 m operating distance.

vi. Scenario Six (Standing at 2m to 3m)

With 369 message data received, it was observed that detection value for 365 message data was equal to 1 while 4 was equal to 0 and the confidence score ranged between 0.8 and 0.9. Therefore, if 369 message data were evaluated with the human correctly detected in the identified conditions, TP = 365, FP = 0, FN = 4 and TN = 0.

- Detection Accuracy (%) = $\frac{TP+TN}{TP+TN+FP+FN}$ x 100% = $\frac{365+0}{365+0+0+4}$ x 100% = 98.9%
- Precision (%) = $\frac{TP}{TP+FP}$ x 100% = $\frac{365}{365+0}$ x 100% = 100%
- Recall (%) = $\frac{TP}{TP+FN}$ x 100% = $\frac{365}{365+4}$ x 100% = 98.9%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1853}{81.732}$ = 23 FPS
- Specificity = $\frac{TN}{TN+FP}$ x 100% = $\frac{0}{0+0}$ x 100% = 0%
- F1 Score = $\frac{2TP}{2TP+FP+FN}$ x 100% = $\frac{2 \text{ x } 365}{2(365)+0+4}$ x 100% = 99.5%

- Success rate – Does the robot respond appropriately to both approaching and receding target movement within the 2–3 m range?
  Success rate = $\frac{Successful\ response}{Expected\ response}$ x 100% = $\frac{98.9}{100}$ x 100% = 98.9%

In this scenario, movement in both the forward and backward directions within the 2 m and 3 m range from the robot were evaluated. The system achieved 98.9% in both the detection accuracy and recall, with 100% precision. The resulting F1 score of 99.5% represents one of the highest overall detection scores recorded across the movement scenarios. The average confidence score was approximately 0.85, and the measured operating rate was 23 FPS. The high recall and F1 score indicate that the detector was highly effective at maintaining target recognition while the target changed position within the 2–3 m range. This is particularly important for the proposed follow-me application because this distance represents a practical operating region.

vii. Scenario Seven (Standing <3m)

With 300 message data received, it was observed that detection value for all message data was equal to 1 and the confidence score was an average of 0.9. Therefore, if 300 message data were evaluated with the human correctly detected in all conditions, TP = 300, FP = 0, FN = 0 and TN = 0.

- Detection Accuracy (%) = $\frac{TP+TN}{TP+TN+FP+FN}$ x 100% = $\frac{300+0}{300+0+0+0}$ x 100% = 100%
- Precision (%) = $\frac{TP}{TP+FP}$ x 100% = $\frac{300}{300+0}$ x 100% = 100%
- Recall (%) = $\frac{TP}{TP+FN}$ x 100% = $\frac{300}{300+0}$ x 100% = 100%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1506}{60.583}$ = 25 FPS
- Specificity = $\frac{TN}{TN+FP}$ x 100% = $\frac{0}{0+0}$ x 100% = 0%
- F1 Score = $\frac{2TP}{2TP+FP+FN}$ x 100% = $\frac{2 \text{ x } 300}{2(300)+0+0}$ x 100% = 100%
- Success rate – Does the robot maintain an appropriate stationary response when the person is within 3 m?
  Success rate = $\frac{Successful\ response}{Expected\ response}$ x 100% = $\frac{100}{100}$ x 100% = 100%

For this <3 m scenario, the detection performance was evaluated as the stationary human target was relatively close to the robot. The system achieved 100% detection accuracy, precision, and recall, and thus producing an F1 score of 100%. The average confidence score was 0.90, while the detector operated at 25 FPS. These results indicate that close-range operation did not adversely affect person detection under the conditions tested. The perfect recall demonstrates that the target remained consistently identifiable, while the perfect precision indicates that all reported detections corresponded to the intended target. The relatively high FPS also indicates that the system maintained a responsive detection rate at the shorter target distance.

viii. Scenario Eight (Light Variation)

With 328 message data received, it was observed that detection value for 323 message data was equal to 1 while 5 was equal to 0 and the confidence score ranged between 0.8 and 0.9. Therefore, if 328 message data were evaluated with the human correctly detected in the identified conditions, TP = 323, FP = 0, FN = 5 and TN = 0.

- Detection Accuracy (%) = $\frac{\text{TP+TN}}{\text{TP +TN+ FP+FN}}$ x 100% = $\frac{323 + 0}{323 + 0 + 0 + 5}$ x 100% = 98.5%
- Precision (%) = $\frac{\text{TP}}{\text{TP + FP}}$ x 100% = $\frac{323}{323 + 0}$ x 100% = 100%
- Recall (%) = $\frac{\text{TP}}{\text{TP + FN}}$ x 100% = $\frac{323}{323 + 5}$ x 100% = 98.5%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{1647}{80.961}$ = 20 FPS
- Specificity = $\frac{\text{TN}}{\text{TN + FP}}$ x 100% = $\frac{0}{0 + 0}$ x 100% = 0%
- F1 Score = $\frac{2\text{TP}}{2\text{TP + FP+FN}}$ x 100% = $\frac{2 \text{ x } 323}{2(323) + 0 + 5}$ x 100% = 99.2%
- Success rate – Does the robot behavior remain acceptable under changing illumination and person remain detectable?

  Success rate = $\frac{Successful\ outcome}{Expected\ outcome}$ x 100% = $\frac{98.5}{100}$ x 100% = 98.5%

The lighting variation performance evaluated the robustness of the person detector for movement and standing under a reduced illumination condition for an indoor environment. The detector achieved 98.5% in both detection accuracy and recall, with 100% precision. The resulting F1 score was 99.2%. The average confidence score was 0.85, and the system operated at 20 FPS. The high recall demonstrates that the detector was able to maintain reliable target recognition despite variations in lighting. Although the performance was marginally lower than the perfect scores observed under some stationary and walking conditions, the resulting F1 score of 99.2% indicates that the reduction was relatively small. These results suggest that the detector demonstrated good robustness to the lighting changes introduced during the experiment.

ix. Scenario Nine (No Target)

With 401 message data received, it was observed that detection value for 401 message data was equal to 0 and the confidence score in all message data revealed 0.0.

Therefore, if 401 message data were evaluated with no human detected in the identified conditions, TP = 0, FP = 0, FN = 0 and TN = 401.

- Detection Accuracy (%) = $\frac{\text{TP+TN}}{\text{TP +TN+ FP+FN}}$ x 100% = $\frac{0 + 401}{0 + 401 + 0 + 0}$ x 100% = 100%
- Precision (%) = $\frac{\text{TP}}{\text{TP + FP}}$ x 100% = $\frac{0}{0 + 0}$ x 100% = 0%
- Recall (%) = $\frac{\text{TP}}{\text{TP + FN}}$ x 100% = $\frac{0}{0 + 0}$ x 100% = 0%
- FPS = $\frac{\text{Number of Camera frames}}{\text{Performance Duration}}$ = $\frac{2013}{74.930}$ = 27 FPS
- Specificity = $\frac{\text{TN}}{\text{TN + FP}}$ x 100% = $\frac{401}{401 + 0}$ x 100% = 100%

- F1 Score = $\frac{2TP}{2TP + FP+FN}$ x 100% = $\frac{2 \times 0}{2(0)+ 0 + 0}$ x 100% = 0%
- Success rate – Does the robot remain stationary and produces no unintended movement?

  Success rate = $\frac{Successful\ outcome}{Expected\ outcome}$ x 100% = $\frac{100}{100}$ x 100% = 100%

In this experiment that has to do with no-target, the system achieved 100% detection accuracy and 100% specificity, with a corresponding false-positive rate of 0%. This indicates that the detector did not identify a person. The precision and recall values in this scenario were both 0%, with an F1 score of 0%. These can be interpreted as a no positive person-present observation. Also, the average confidence score was 0.0, was consistent with the absence of a detected person. The system operated at 27 FPS, which was the highest FPS recorded among the nine scenarios. And the average confidence score drawn across the scenarios have been depicted on the graph below.

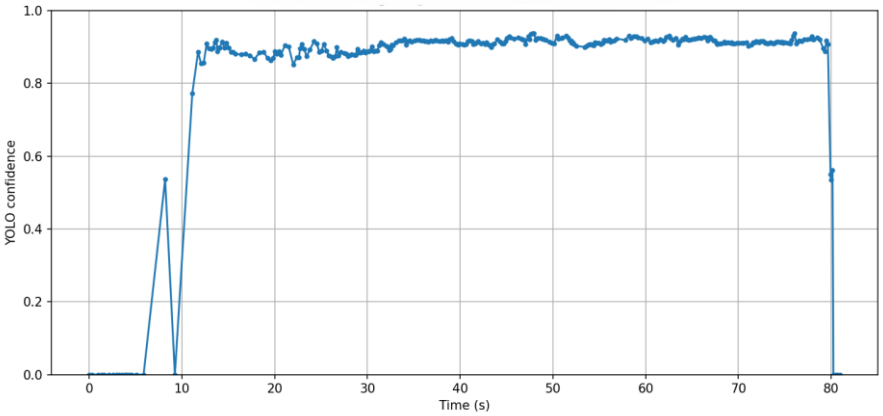


Figure 4.4 - Average Confidence Score across all Scenarios

These evaluations showed the level of the system's detector performance and reliability across nine scenarios depicting regular human dorsal poses within an indoor environment. It considered human walking motion, human distance from the robot, robot stopping behaviour, indoor lighting conditions, and the absence of a human target. The evaluated results have been tabulated below.

Table 4.2 – Quantitative Analysis for Detection Performance

S/No	Scenarios	Detection Accuracy (%)	Precision (%)	Recall (%)	Confidence Score (%)	FPS	Specificity (%)	F1 Score (%)	Success rate (%)
1.	Standing	100	100	100	0.9	22	0	100	100
2.	Forward	97.8	100	97.8	0.85	22	0	98.9	97.8
3.	Backward	98.5	100	98.5	0.85	24	0	99.2	98.5
4.	Stopping	94.4	100	94.4	0.85	13	0	97.1	94.4
5.	Standing >3m	100	100	100	0.9	23	0	100	100

6.	Standing at 2m to 3m	98.9	100	98.9	0.85	23	0	99.5	98.9
7.	Standing <3m	100	100	100	0.9	25	0	100	100
8.	Light Variation	98.5	100	98.5	0.85	20	0	99.2	98.5
9.	No Target	100	0	0	0.0	27	100	0	100

From this tabulated quantitative analysis, it is indicative that the YOLO-based person detection system utilized in this experiment achieved high detection performance across all tested person-present conditions. Detection accuracy was at least 94.4%, while precision remained at 100% across all eight target-present scenarios. Recall was also high, ranging from 94.4% to 100%, and F1 scores ranged from 97.1% to 100%. The results further demonstrate that the detector was robust to changes in varying distances and illumination. In particular, the perfect performance at both less than 3 m and greater than 3 m indicates that the tested distance variations did not substantially degrade target detection. The stopping condition represented the most challenging person-present scenario, producing the lowest recall and F1 score. This suggests that target stopping may introduce greater detection variability than continuous target movement. Also, the no-target experiment achieved 100% specificity and a 0% false-positive rate. This is an important result for the safety and reliability of an autonomous follow-me application, since unintended movement in response to a false detection could compromise system behaviour.

4.1.2 Motor Control Performance

The motion control performance was evaluated to determine how effectively the robot translated the detected target behaviour into linear motion commands. The evaluation considered forward motion, backward motion, stopping accuracy, response time, and motion smoothness. The analysis was based on the recorded */summit_xl/cmd_vel* messages contained in the nine experimental *rosbag* files.

```
^C(yolo_env) robotics_lhu@roboticsLab4:~/catkin_ws$ rosrun follow_me_yolo follow
er.py
[INFO] [1786630449.011914]: ---------------------------------
[INFO] [1786630449.035450]:  Summit XL Simple Follow Controller
[INFO] [1786630449.042452]:  Proof Of Concept V6
[INFO] [1786630449.049392]: ---------------------------------
[INFO] [1786630449.082313]: Listening: /person_detection
[INFO] [1786630449.088779]: Publishing: /summit_xl/cmd_vel
[INFO] [1786630468.782581]: height=931.5 target=720.0 error=-211.5 linear=-0.003
[INFO] [1786630468.882766]: height=931.5 target=720.0 error=-211.5 linear=-0.003
[INFO] [1786630468.982656]: height=931.9 target=720.0 error=-211.9 linear=-0.003
[INFO] [1786630469.082631]: height=932.3 target=720.0 error=-212.3 linear=-0.003
[INFO] [1786630469.183091]: height=932.3 target=720.0 error=-212.3 linear=-0.003
[INFO] [1786630469.282540]: height=931.7 target=720.0 error=-211.7 linear=-0.003
[INFO] [1786630469.382843]: height=932.0 target=720.0 error=-212.0 linear=-0.003
[INFO] [1786630469.482507]: height=931.5 target=720.0 error=-211.5 linear=-0.003
```

Figure 4.5 - Motor Controller Module Message Data

4.1.2.1 Forward Motion

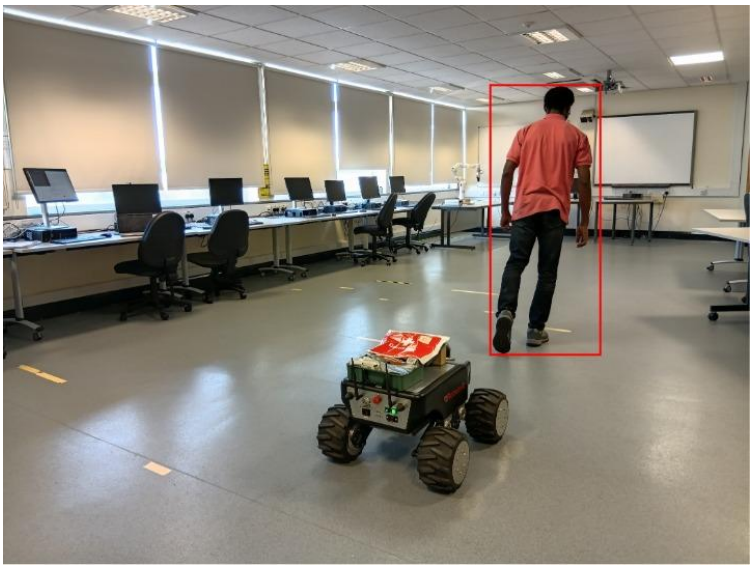


Figure 4.6a - Real-World Robot Forward Motion

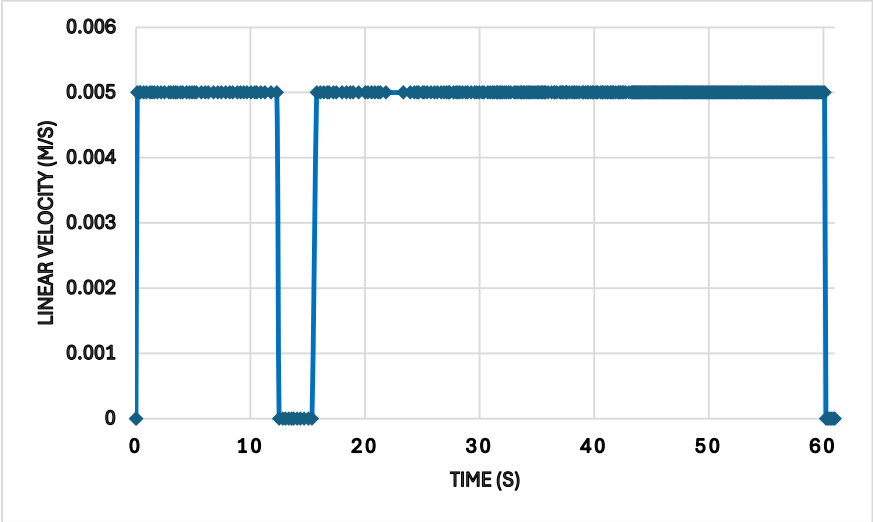


Figure 4.6b - Velocity-Time Graph for Forward Motion

The forward motion was evaluated using the average of walking forward, stopping, 2–3 m forward/backward, and lighting variation scenarios. The controller generated positive linear velocity commands when the detected change in target bounding-box height indicated that the person was moving away from the robot. The recorded command data showed a maximum positive linear velocity of 0.005 m/s. This velocity value was consistent unless in situations where the human target performed stop actions, thus resulting in the velocity drops observed across the defined period indicated on the graph above.

When the average of all the scenarios for this walking forward experiment was done, 331 of 351 recorded velocity commands were non-zero, corresponding to 94.3% of the recorded command messages. This indicates that the robot generated forward movement commands for a greater portion of the experiment while the human target was walking forward. Thereby

indicating that the robot controller can generate forward motion in response to a human target movement. Moreover, the velocity drop observed is a demonstration that the controller was configured for cautious movement rather than high-speed following.

4.1.2.2 Backward Motion

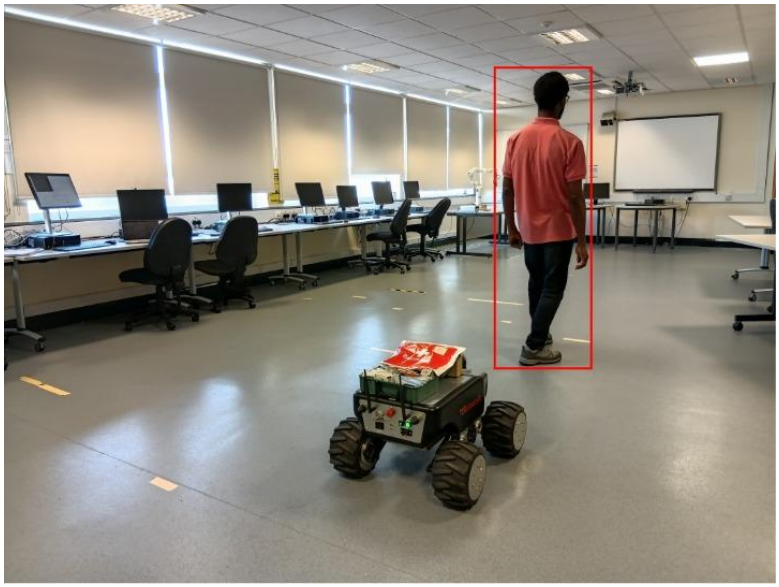


Figure 4.7a - Real-World Backward Motion

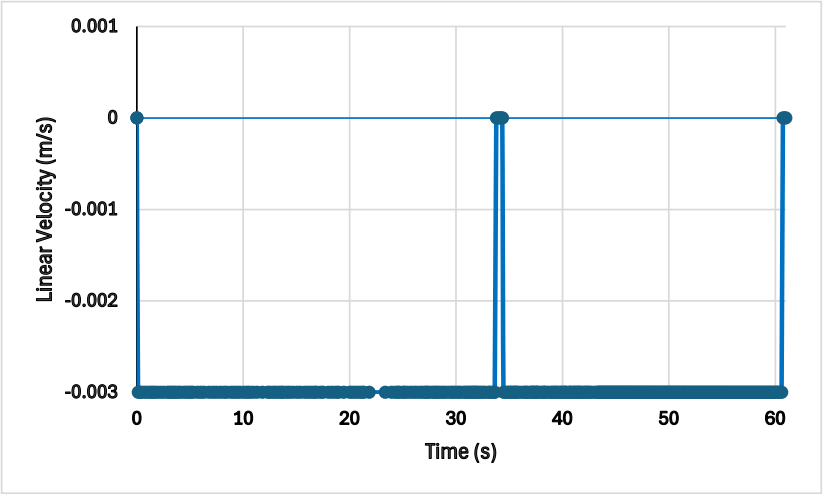


Figure 4.7b - Velocity-Time Graph for Backward Motion

This was evaluated using the average of walking backward and 2-3 m Forward/Backward scenarios. In this case, negative linear velocity commands were generated when the system observes an increase in target bounding-box height thereby interpreted as the person moving toward the robot. The recorded minimum linear velocity was −0.003 m/s, demonstrating that the controller generated reverse motion commands.

The walking backward experiment contained 355 */summit_xl/cmd_vel* messages from the average of the scenarios used and 346 were non-zero, giving a non-zero command ratio of 97.47%. The high proportion of active commands indicates that the controller responded

consistently during the backward walking experiment. Therefore, the result demonstrates that the follower controller could produce an effective backward linear motion with a low velocity command of -0.003 m/s indicating a safety cautious design.

4.1.2.3 Stop Accuracy

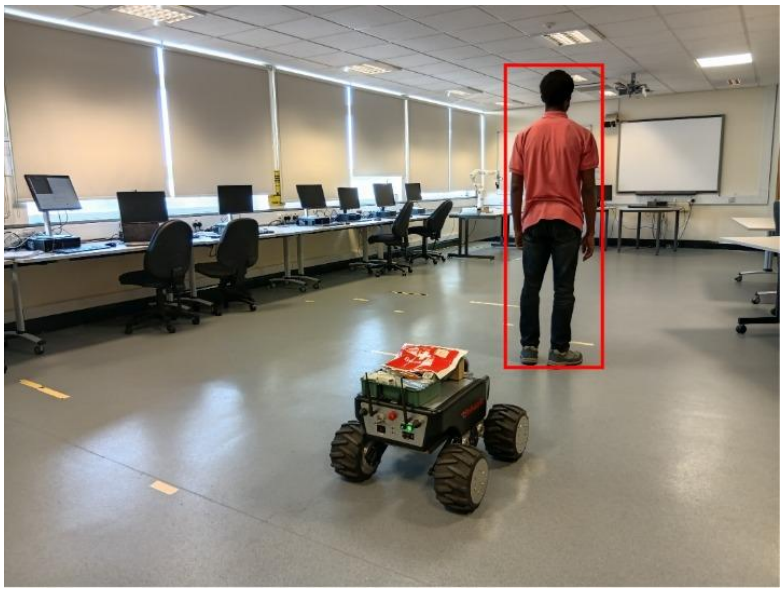


Figure 4.8a - Real-World Stop Scenario

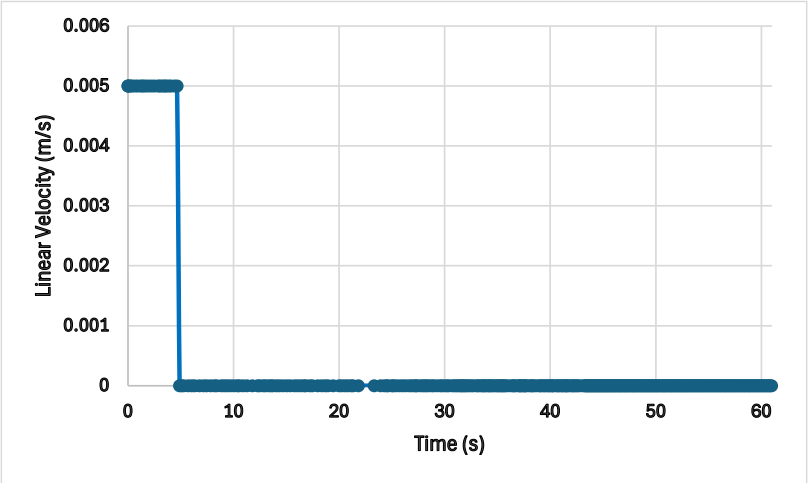


Figure 4.8b - Velocity-Time Graph for Stop Accuracy

The evaluation of the stop accuracy is very important for a follow-me robot because the robot should not continue moving when the human target has stopped. The standing experiment provided the clearest assessment of this behaviour. During the experiment, the controller generated only 22 non-zero linear commands out of 349 velocity messages at the starting, which is equivalent to 6.3%, and the remaining commands were zero-velocity commands.

This result indicates that the controller mostly maintained the robot in a stationary state while the target remained stationary. It therefore provides evidence that the motion controller was able to distinguish between sustained target presence and target movement sufficiently to avoid continuous forward or backward motion.

4.1.2.4 Response Time

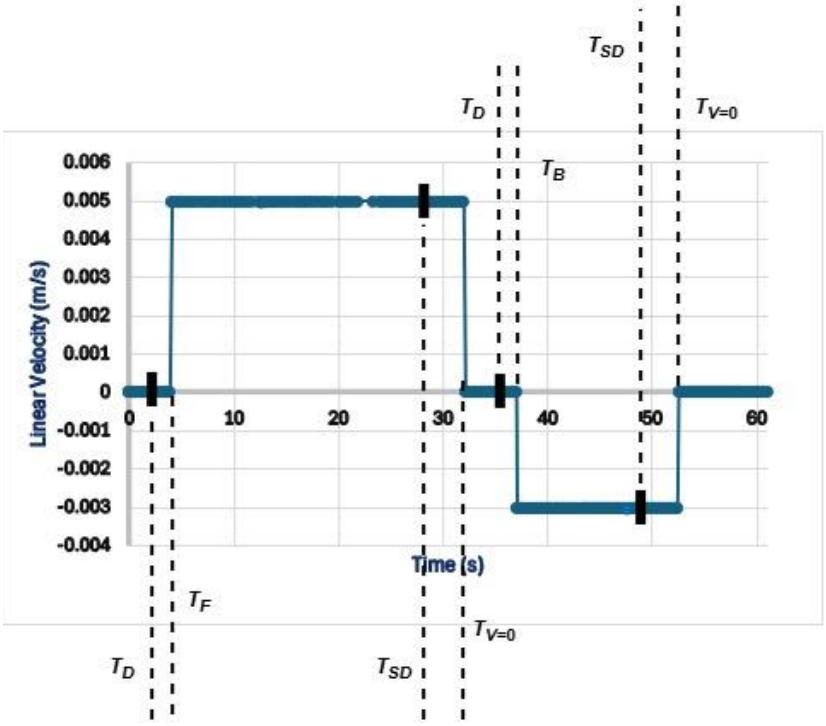


Figure 4.9 - Response-Time Graph for Forward, Backward and Stop Action

Response time represents the delay between a change in the human target's observed motion state and the generation of the corresponding robot velocity command. The extracted data from the *rosbag* analysis provided the timestamps of both */person_detection* and */summit_xl/cmd_vel*, allowing the response delay to be estimated by identifying a target-state transition and the subsequent velocity-command transition. The experiment performed and plotted on the graph above were for the forward motion and backward motion with stop action in both scenarios. These scenarios were considered to determine a satisfactory evaluation.

To derive the response time for the forward motion T_{RF}, the timestamp at which the human target motion change is detected T_D and the timestamp the robot forward velocity command appeared T_F were extracted and subtracted.

$T_{RF} = T_F - T_D$ (4.1)

Where T_F = 4.076789 s and T_D = 3.87943 s

T_{RF} = (4.076789 - 3.87943) s = 0.1974s = 197 ms

To determine the response time for the stop action after forward motion T_{RFS}, the timestamp at which the human target stop was detected T_{SD}, and the timestamp at which the robot reaches v = 0, $T_{v=0}$, were extracted and subtracted.

$T_{RFS} = T_{v=0} - T_{SD}$ (4.2)

Where $T_{v=0}$ = 32.15203 s and T_{SD} = 29.5669 s

T_{RFS} = (32.15203 - 29.5669) s = 2.585 s

To derive the response time for the backward motion T_{RB}, the timestamp at which the human target motion change is detected T_D and the timestamp the robot forward velocity command appeared T_B were extracted and subtracted.

$T_{RB} = T_B - T_D$ (4.3)
Where T_B = 37.09535 s and T_D = 36.73962 s
T_{RB} = (37.09535 - 36.73962) s = 0.3557s = 356 ms

To determine the response time for the stop action after backward motion $T_{RBS,}$ the timestamp at which the human target stop was detected T_{SD}, and the timestamp at which the robot reaches v = 0, $T_{v=0}$, were extracted and subtracted.
$T_{RBS} = T_{v=0} - T_{SD}$ (4.4)
Where $T_{v=0}$ = 52.53336 s and T_{SD} = 51.03264 s
T_{RBS} = (52.53336 - 51.03264) s = 1.501 s

4.1.2.5 Motion Smoothness

The motion smoothness was evaluated by examining the sequence and variation of consecutive linear velocity commands. Smooth motion is desirable because abrupt changes in commanded velocity can result in jerky robot behaviour, reduced tracking stability, and unnecessary mechanical stress. It can be observed that from Fig. 4.6, the recorded velocity commands were limited to a relatively small range of $-0.003 \leq v_x \leq 0.005$ m/s. Thus, indicating that the controller operated within a conservative velocity range which translates to a quantitative motion smoothness as the mobile robot moves from forward motion, 0.005 m/s to zero velocity and backward motion to zero velocity. The stop action experiment also showed that the controller could maintain a predominantly zero-velocity command when the target remained stationary.

The motion control experiments demonstrate that the follower controller could generate both forward and backward linear commands based on changes in the detected target distance. The results indicate that the implemented controller provided conservative and predominantly stable linear motion, with appropriate stopping behavior in stationary conditions. The findings support the functionality of the developed follow-me control architecture while also identifying response latency and velocity command smoothness.

4.1.3 Position Estimation Performance

The position estimation component of the system was evaluated using the bounding-box information generated by the person detector. The detector provided the human target's bounding-box centre coordinates and dimensions for each valid person detection. These measurements were used to estimate the human target's position within the camera image and to infer changes in the human target's relative distance from the robot. The evaluation was on centre estimation, distance estimation, and positional stability. These characteristics are important because the quality of the position estimate directly affects the subsequent motion-control decisions of the follow-me system.

4.1.3.1 Centre Estimation

The horizontal centre of the detected person was determined from the bounding-box coordinates and used to represent the human target's position relative to the centre of the camera image. For a bounding box with left coordinate X_{min} and width w, the horizontal center will be

$X_c = X_{min} + \frac{W}{2}$ from equation 3.8

Similarly, the vertical centre is

$Y_c = Y_{min} + \frac{H}{2}$ from equation 3.9

Where h is the bounding box height.

Also, the centre position error used to determine the direction where the human target is mostly aligned is

$$e_x = X_c - X_{image_centre}, \quad (4.5)$$

Where X_{image_centre}, the horizontal centre of the camera image = 960 px

Table 4.3 – Centre Position Estimation Analysis across all Scenarios

S/No.	Scenarios	Mean X_c (px)	Mean Y_c (px)	Center Position Error (px)
1.	Standing	881.77	616.56	-78.23
2.	Forward	893.33	608.15	-66.67
3.	Backward	91.91	522.31	-668.09
4.	Stopping	763.96	573.71	-196.04
5.	Standing >3m	829.71	618.59	-130.29
6.	Standing at 2m to 3m	924.77	375.34	-35.23
7.	Standing <3m	864.11	429.58	-95.89
8.	Light Variation	963.01	389.62	3.01
9.	No Target	0	0	-

The horizontal centre position provides an indication of whether the detected person is located to the left or right of the camera's optical centre particularly when it's used to evaluate the centre position error. The magnitude of this error provides a direct measure of the target's lateral displacement from the desired tracking position. Positive and negative centre position error indicates that the human target is aligned to left and right respectively. And the target positioned near the image centre indicates that the robot is approximately aligned with the person, whereas movement of the centre coordinate away from the image centre represents a change in the target's lateral position.

The recorded experiments demonstrated substantial variation in the horizontal position of the detected person especially as the detected centre values varied according to the target's movement within the camera's FOV. This variation was expected during walking experiments, particularly when the person changed position relative to the robot.

4.1.3.2 Distance Estimation

Since the system uses a monocular camera, direct physical distance cannot be measured from the image without additional calibration information. Instead, the apparent size of the detected person which formed the bounding box height was used as a relative distance indicator. Thus, this is determined by the inverse relationship between the bounding-box height h and target distance D. As the person approaches the camera, the bounding box becomes larger, whereas as the person moves away, the bounding box becomes smaller.

$$h = \frac{1}{D} \tag{4.6}$$

This relationship was particularly important for the forward and backward movement experiments. Distance *D* is therefore evaluated in pixels using the predefined vertical height.

Table 4.4 - Distance Estimation Analysis

S/No	Scenarios	Mean BBox Width (w)	Mean BBox Height (h)	$D = h - h_{predefined}$ (px)
1.	Standing	230.62	742.98	22.98
2.	Forward	227.63	649.48	-70.52
3.	Backward	242.81	953.66	233.66
4.	Stopping	249.58	789.42	69.42
5.	Standing >3m	198.71	652.71	-67.29
6.	Standing at 2m to 3m	319.11	742.86	22.86
7.	Standing <3m	407.18	852.02	132.02
8.	Light Variation	296.68	735.88	15.88
9.	No Target	0	0	-

The recorded data showed substantial variation in bounding-box height across the scenario 1 to 8 experiments. As it produced bounding-box heights ranging from approximately 649.48 to 953.66 px, revealing substantially large variations considering the predefined vertical height is 720 px. This was expected because walking towards or away from the robot produces significant changes in the apparent size of the target. From the table above the large distance values for the backward and <3 m scenarios of 233.66 px and 132.02 px show a large image size positioned closer to the robot while the very reduced distance values for forward and >3m scenarios of -70.52 and -67.29 respectively indicates a small image size positioned far away from the robot.

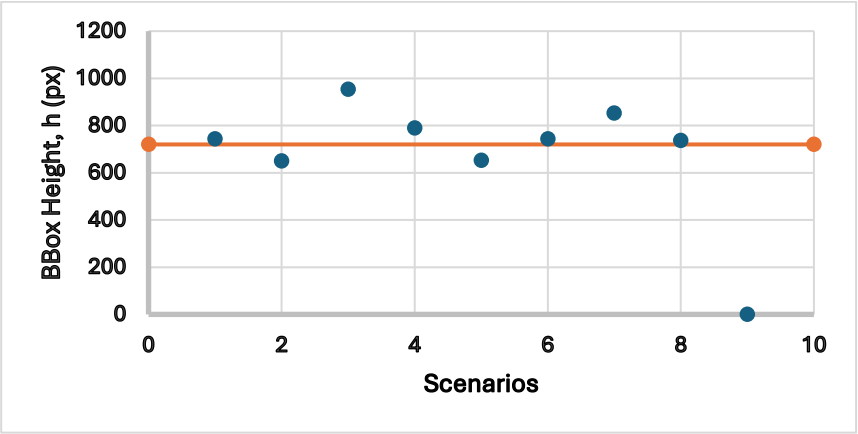


Figure 4.10 - Distance Estimation Graph

In the graph above, the dots plotted below the orange line indicate the human target to be closer to the mobile robot while the dots above the line show the human target is far from the mobile robot. The measure of distance between each dot to the line reveals how near or distant the human target is to the mobile robot.

4.1.3.3 Positional Stability

Positional stability refers to the degree to which the estimated target position remains consistent when the person is stationary. This characteristic is particularly important for standing, standing at <3 m, and standing at >3 m scenarios. This is because a stationary person should produce relatively stable bounding-box centre coordinates and bounding-box dimensions. Small variations are expected because of camera noise, changes in illumination, YOLO detection variability, and minor changes in the person's posture. The stability of the estimated position was evaluated using the standard deviation of the centre position.

$$\sigma_x = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2} \tag{4.7}$$

where x_i is an individual horizontal centre measurement and $\bar{x}$ is the mean centre position. Similarly, standard deviation for the bounding box height was calculated using

$$\sigma_H = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(H_i - \bar{H})^2} \tag{4.8}$$

where H_i is an individual bounding box height and $\bar{H}$ is the mean bounding box height.

Table 4.5 - Positional Stability Analysis

S/No	Scenarios	Mean X_c (px)	Mean BBox Height (h) (px)	σ_x (px)	σ_H (px)
1.	Standing	881.77	742.98	144.79	34.17
2.	Forward	893.33	649.48	289	125.7
3.	Backward	291.91	953.66	266.96	197.93
4.	Stopping	763.96	789.42	193.39	178.71
5.	Standing >3m	829.71	652.71	209	215.58
6.	Standing at 2m to 3m	924.77	742.86	98.04	58.30
7.	Standing <3m	864.11	852.02	150.89	64.04
8.	Light Variation	963.01	735.88	160.73	102.99
9.	No Target	0	0	-	-

These results in the table 4.5 demonstrate that the proposed detector maintained varying levels of positional consistency under the different experimental conditions. The Standing 2–3 m scenario produced the lowest horizontal position variation at 98.04 px, indicating the most stable lateral target localisation among the evaluated scenarios. The largest horizontal variations were observed during the forward and backward scenarios, with values of 289.00 px and 266.96 px, respectively. These higher values are primarily associated with intentional target movement and therefore would not be interpreted directly as detection instability. Similarly, the large bounding-box-height variations observed during forward and backward movement reflect changes in the target's relative distance from the robot.

For relative-distance stability, the standing scenario produced the lowest bounding-box-height standard deviation of 34.17 pixels, followed by the standing 2–3 m and standing <3 m scenarios with values of 58.30px and 64.04 px, respectively. This indicates that the detector produced relatively consistent target-size estimates when the person remained stationary. The

lighting variation scenario resulted in a higher value of 102.99 pixels, suggesting that changes in illumination introduced additional variation into the apparent target size.

The standing at >3 m scenario produced relatively high variation in both horizontal position and bounding-box height. This shows that target localisation became less stable at greater distances, potentially due to the smaller apparent target size and increased sensitivity to changes in detection geometry. Summarily, the results indicate that the most consistent position stability was achieved when the target was within the 2–3 m range, while target motion, greater operating distance, and lighting changes introduced greater variation in the estimated position and apparent target size because the target was deliberately moving during most of these experiments, and therefore a larger variation in horizontal position is expected.

4.2 DISCUSSION OF FINDINGS

The experimental results demonstrated the feasibility of the developed vision-based follow-me system for detecting and responding to a human target under a range of operating conditions. The results are discussed in relation to the objectives of the study and subsequently compared with findings reported in existing person-following research.

4.2.1 Relating Results with Research Objectives

The first objective of the study was to develop a vision-based person detection system capable of identifying a human target in real time. The experimental results indicate that this objective was largely achieved. Detection accuracy ranged from 94.4% to 100% across the person-present scenarios, with confidence scores generally between 0.85 and 0.90. The highest detection performance was obtained under the standing, standing at >3 m and standing at <3 m scenarios, each achieving 100% detection accuracy. The forward, backward, movement along the 2–3 m range and lighting variation scenarios also achieved high detection accuracy of 97.8%, 98.5%, 98.5% and 98.5%, respectively. The lowest result was observed in the Stopping scenario at 94.4%. These results indicate that the YOLO-based detector was generally capable of maintaining reliable person detection during both stationary and dynamic conditions. The relatively high confidence scores further indicate that detected person instances were generally classified with high confidence. This is important for a person-following application because unreliable detection can propagate errors into the subsequent position estimation and motion-control stages.

The second objective was to estimate the target's position and relative distance from visual information. The experimental results demonstrate that bounding box information provided useful measurements for this purpose. The horizontal centre of the bounding box was used to represent the target's position within the camera image, while bounding-box height was used as a relative indicator of target distance. The position-stability analysis showed that the lowest horizontal variation occurred in the 2–3 m scenario, with σ_x = 98.04 px. The standing and standing at <3 m scenarios also showed relatively low horizontal variation, at 144.79 and 150.89 px, respectively. The bounding box height variation provided additional evidence of the system's ability to distinguish changes in relative target distance. The standing scenario

produced the lowest σ_H of 34.17 px, followed by the 2–3 m and <3 m scenarios with 58.30 and 64.04 px, respectively. In contrast, the forward and backward scenarios exhibited greater variation because the human target was intentionally changing distance from the robot. Thus, the variation in bounding-box height should not necessarily be interpreted as detection error rather, it reflects the changing apparent size of the target.

In the third objective which was to enable the robot to respond appropriately to changes in the target's motion. The motion control evaluation demonstrated that the system generated linear velocity commands in response to detected changes in the target's bounding-box height. Forward and backward experiments produced corresponding changes in the commanded linear velocity, while the no-target experiment generated zero linear velocity commands. This is particularly important because the no-target experiment achieved 100% specificity, with no false-positive movement commands recorded. Consequently, the robot remained stationary when no target was present, demonstrating an important safety characteristic of the developed system.

The fourth objective was to evaluate the robustness of the system under different environmental and operational conditions. The lighting variation experiment, which was a distinct condition, achieved 98.5% detection accuracy, with an average confidence of 0.85. Although the position and bounding box height variations increased relative to some stationary scenarios, the detector continued to identify the person reliably. This demonstrates a degree of robustness to changes in illumination. However, the result also indicates that lighting remains an influencing factor, which is consistent with the established challenges of vision-based person detection on mobile robots. Previous research has identified lighting conditions, viewing angle, distance and person appearance as factors that can affect visual person detection and tracking performance.

4.2.2 Comparison with Existing Literature

The results are broadly consistent with the established challenges and approaches reported in person-following robotics research. Person following is recognised as a complex task because it combines human detection, target localisation, and robot motion control. A comprehensive review of person-following robots has identified sensor selection, environmental conditions, target dynamics and robot-control requirements as important factors affecting system performance (Islam *et al*., 2019). Existing research has demonstrated that vision can provide sufficiently rich information for person following, but that maintaining reliable detection and localisation on a moving robot remains challenging. Treptow *et al*. (2006), highlighted the requirements for person-tracking systems on mobile robots to operate in real time while dealing with moving backgrounds and changing environmental conditions. Their work also identified illumination, viewing angle and target distance as important factors affecting visual person detection.

However, this present study addresses the challenges using a monocular RGB camera and a deep-learning person detector. This differs from approaches that employ thermal cameras, stereo cameras, RGB-D sensors or multiple sensing modalities. Koide *et al*. (2020), developed a monocular person-following system that estimates person position in robot space using ground-plane information and human-height estimation, subsequently using this information

for person following. Their work demonstrated the potential of monocular vision for person-following applications while also illustrating the additional processing required when absolute target position is required. In comparison, this present study adopted a simpler image-space representation. Rather than estimating absolute three-dimensional target coordinates, the system uses the bounding box centre and height obtained from the person detector. This reduces the sensing and computational requirements of the system, although it also means that the distance measurement is regarded as relative rather than absolute. Research into monocular distance estimation similarly recognises the difficulty of obtaining accurate absolute distance from a single image and has investigated additional geometric, calibration and depth-estimation methods to overcome this limitation (Shi *et al*., 2022).

The performance of the overall average detection accuracy of 98.5%, across the evaluated scenarios can be comparable with results reported in related person-following research by Park *et al*., (2024). They had a precision of 98.25% and recall of 97.78% for human detection in a clear environment using a 3D human-leg detection approach. Although the evaluation metrics and experimental configurations differ, the comparable magnitude of the results suggests that the vision-based detector in this dissertation study provided competitive person-detection performance for a mobile robot person-following application.

Furthermore, most person-following systems have incorporated tracking and filtering techniques to improve temporal stability. Such techniques include Unscented Kalman Filter (UKF) together with ground-plane information to estimate target position in robot space, while other approaches have combined detection, tracking and Kalman prediction to maintain target localisation during challenging conditions. But this current system provides a simpler proof-of-concept approach based primarily on consecutive bounding-box measurements. The position stability results suggest that this approach can provide useful information for basic follow-me control, although the relatively large σ_x and σ_H values observed in scenarios where motion occurred indicate potential for improvement through temporal filtering.

CHAPTER FIVE

SUMMARY AND CONCLUSION

5.1 SUMMARY

This study investigated the development and experimental evaluation of a deep-learning-based vision system for autonomous person-following using a Robotnik Summit XL mobile robot. The work was guided by four research questions addressing human detection, continuous positional estimation, adaptive motion control, and the overall performance of the proposed system.

It considered how a deep-learning-based vision system could be developed for robust human detection in an autonomous person-following robot by integrating a YOLOv8-based person detector with the Summit XL's front PTZ camera to identify a human target from RGB images. The detector generated the target's bounding-box position, dimensions and confidence score, which were subsequently made available to the robot-following controller. It examined how human positional changes could be estimated and tracked continuously in real time by using the horizontal centre of the detected bounding box to estimate the target's image position and bounding-box height as an indirect indicator of relative target distance. Thus, an increased bounding-box height was interpreted as the human target approaching the robot, while a decrease indicated increasing separation.

The experiment, performing positional stability analysis showed that the lowest horizontal positional variation occurred in the 2–3 m scenario. While developing an adaptive motion-control algorithm capable of responding to dynamic human movement, the follower controller used changes in the detected target's visual characteristics to generate linear velocity commands for various human target movements and no target gave zero velocity command. Therefore, this demonstrates the feasibility of using image-based target measurements as the basis for adaptive robot motion. However, the results also showed that target movement and intermittent detection gaps can affect the consistency of the control response.

Furthermore, the overall extent to which the proposed system could improve detection accuracy, following stability and real-time performance on the Summit XL was evaluated across nine experimental scenarios, and the system demonstrated high person-detection performance. These have shown that a relatively lightweight monocular RGB vision architecture can provide sufficient information for basic autonomous person-following on a mobile robot. It also established an experimental framework for evaluating detection, position estimation and motion-control performance under different target behaviours, operating distances and lighting conditions.

5.2 CHALLENGES AND LIMITATIONS OF THE STUDY

Despite the successful implementation and experimental evaluation of the proposed system, several challenges and limitations were identified.

i. Monocular distance estimation

The system uses bounding-box height as a relative indicator of target distance rather than directly measuring physical distance in metres. Although the approach provides useful information about whether the target is approaching or moving away, it does not provide a calibrated absolute distance measurement. Consequently, variations in target posture, camera perspective and bounding-box dimensions can affect the estimated distance. This limitation is particularly relevant when the target is located at greater distances from the robot.

ii. **Limited experimental repetitions**

Due to constrained time during experimentation, few trials were conducted for each of the nine experimental scenarios. Although this provided useful evidence of system behaviour, the limited trial did not provide sufficient statistical evidence to establish repeatability or generalise the results across different users and environments. Increased repeated trials would provide stronger estimates of mean performance intervals.

iii. Limited laboratory support

Although, there was access to the hardware mobile robotic system used to perform the experiment and personal desktop computer for system development, setting up the entire system was faced with compatibility issues, outdated software versions and limited memory capacity for effective computational performance. Also, performing the trials and collecting the data for evaluation was very difficult due to the lack of a laboratory assistant.

iv. Controlled experimental environment

The experiments were conducted under well-defined scenarios rather than highly unstructured real-world conditions. Consequently, the results may not fully represent performance in environments containing multiple people, severe occlusion, highly cluttered backgrounds, abrupt lighting transitions or rapidly changing pedestrian trajectories.

v. Limited motion control evaluation

This system primarily evaluates linear following behaviour. The angular velocity component was not extensively evaluated as part of the final experiment because integrating a PID controller for a balanced angular tracking was taking too long to achieve. However, the results provided presents stronger evidence for longitudinal following behaviour.

5.3 IMPLICATIONS OF THE STUDY

The findings of this study have several implications for the development of autonomous person-following mobile robots. The results demonstrate that deep-learning-based monocular vision can provide an effective and relatively low-cost perception solution for person-following applications. The use of an RGB camera avoids the need for specialised thermal or depth sensors while still providing sufficient visual information for target detection and relative

movement estimation. The results demonstrate the usefulness of bounding-box geometry as a simple representation of target state. The horizontal centre provided information about the target's position within the camera FOV, while bounding-box height provided a practical relative measure of target distance. This approach reduces the complexity of the perception pipeline and can be implemented without requiring full three-dimensional reconstruction.

Also, the 100% specificity achieved in the No Target scenario has an important safety implication. The robot did not generate non-zero linear velocity commands when no person was detected. This behaviour is desirable in autonomous mobile robots because failure to distinguish between target-present and target-absent conditions could result in unintended robot motion. Furthermore, the experiment outcome shows that operating distance influences perception stability. The 2–3 m scenario produced particularly stable horizontal position estimation, whereas the >3 m scenario exhibited greater variation. This suggests that the practical operating range of a monocular person-following system should be considered during controller and experiment design.

These imply that high detection accuracy alone does not guarantee effective person following. The detected target must also provide sufficiently stable positional and distance information for the motion controller to generate appropriate commands. This work therefore provides a practical foundation for developing more sophisticated person-following systems on the Summit XL platform, particularly systems incorporating temporal tracking, filtering, calibrated distance estimation and more advanced motion control.

5.4 FUTURE WORK

There are several aspects that can be further investigated to improve the system such as

i. Multi-object tracking and target identification

Future work should incorporate a dedicated multi-object tracking algorithm, such as a Kalman-filter-based tracker, SORT, Deep SORT or a comparable modern tracking framework. This would allow the system to maintain the identity and predicted position of the selected person during temporary detection failures. Target re-identification could also be introduced to allow the robot to recover the correct person after temporary occlusion or detection loss.

ii. Calibrated distance estimation

Camera calibration, known human-height information, depth sensing or monocular depth-estimation techniques could be investigated to obtain an estimate of the target's physical distance in metres. This would allow the controller to operate using explicit distance errors rather than relying only on changes in bounding-box size.

iii. Improved motion control

A more advanced adaptive controller could be implemented using PID, fuzzy logic, model-predictive control or another adaptive control technique. The controller could simultaneously consider target distance error, horizontal position error, target velocity, rate of change of target

distance and detection confidence. Such an approach could improve responsiveness while reducing unnecessary robot movement.

iv. Improved angular following

In addition to the linear following system developed, future experiments should therefore include deliberate angular target movement and evaluate the robot's response.

v. **Dynamic and uncertain environments**

The system should be further developed and evaluated in a hospital styled environments containing multiple people, partial target occlusion, moving obstacles, complex backgrounds, and rapid illumination changes. Such testing would provide a more comprehensive assessment of real-world robustness.

5.5 CONCLUSION

Following the enumerated research questions, this study explored the use of a deep-learning-based vision system to develop an autonomous person-following mobile robot. The objective of developing a robust vision-based human detection system was achieved through the integration of a YOLOv8-based person detector with the Summit XL's RGB PTZ camera. The experimental results demonstrated high detection performance across the evaluated person-present scenarios, with detection accuracy ranging from 94.4% to 100%. The system also achieved high confidence scores, generally between 0.85 and 0.90, demonstrating its ability to reliably identify the target person under different movement, distance, and lighting conditions.

In addition, the developed system was able to estimate and continuously monitor changes in the target's position and relative distance. This was achieved by using the horizontal centre of the detected bounding box to represent the target's image position, while the bounding-box height was used as a relative indication of distance. The results demonstrated that these measurements provided useful information about target movement and distance variation. In particular, the 2–3 m scenario produced relatively stable horizontal position measurements, while stationary scenarios generally exhibited lower bounding-box-height variation.

Furthermore, after developing an adaptive motion-control approach capable of responding to changes in human movement, its experimental outcomes demonstrated that the detected changes in target position and bounding-box dimensions could be translated into robot linear velocity commands. The system responded differently to forward, backward, and stationary conditions, demonstrating the feasibility of using visual target measurements as inputs to autonomous motion control. Finally, the overall effectiveness of the developed system was determined on the Summit XL platform across nine experimental scenarios. Thus, the system demonstrated strong detection performance and real-time operation. During the no target experiment, 100% specificity was achieved with no false-positive movement commands, thereby demonstrating an appropriate stationary behaviour when no target was present. However, generally, the detection performance remained high during forward movement, backward movement, stopping, different operating distances, and lighting variations.

Although several positive results were achieved, some limitations were also identified. However, the results obtained after the system analysis can be interpreted as evidence of system feasibility and proof of performance. Therefore, this study demonstrates that a YOLO-based monocular vision system combined with image-based positional estimation and motion control can provide a cost-effective and practical architecture for autonomous person-following on a Robotnik Summit XL mobile robot. While considering further improvements that can achieve robust performance in more complex and less controlled real-world environments.

REFERENCES

[1] Aharon, N., Orfaig, R. and Bobrovsky, B.-Z. (2022) 'BoT-SORT: Robust Associations Multi-Pedestrian Tracking', *arXiv preprint*, arXiv:2206.14651.

[2] Ali, M. and Zhang, Z. (2024) 'The YOLO Framework: A Comprehensive Review of Evolution, Applications, and Benchmarks in Object Detection', *Computers*, 13(12), p. 336. doi: 10.3390/computers13120336.

[3] Algabri, R., Choi, M.-T., Kwon, J.-S., & Lee, S. (2020). *Deep-Learning-Based Indoor Human Following of Mobile Robot Using Color Feature*. Sensors, 20(9), 2699. https://doi.org/10.3390/s20092699.

[4] Alkandary, K., Yildiz, A. S., & Meng, H. (2025), 'A Comparative Study of YOLO Series (v3–v10) with DeepSORT and StrongSORT: A Real-Time Tracking Performance Study', *Electronics, 14*(5), 876. https://doi.org/10.3390/electronics14050876.

[5] Álvarez-Santos, V., Pardo, X.M., Iglesias, R., Canedo-Rodríguez, A. and Regueiro, C.V. (2012) 'Feature analysis for human recognition and discrimination: Application to a person-following behaviour in a mobile robot', *Robotics and Autonomous Systems*, 60(8), pp. 1021–1036. https://doi.org/10.1016/j.robot.2012.05.014.

[6] Arcos, L., Calala, C., Maldonado, D. and Cruz, P.J. (2020) 'ROS based Experimental Testbed for Multi-Robot Formation Control', *2020 IEEE ANDESCON*, Quito, Ecuador, pp. 1–6. doi: 10.1109/ANDESCON50619.2020.9272073.

[7] Arregi, M. O., and Secco, E. L. (2023) "Operative Guide for 4-wheel Summit XL Mobile Robot Set-up". *Acta Scientific Computer Sciences* 5.4: 39-47

[8] Åström, K.J. and Hägglund, T. (2006) *Advanced PID Control*. Research Triangle Park, NC: ISA.

[9] Åström, K. J., and Murray, R. M. (2021). *Feedback Systems: An Introduction for Scientists and Engineers* (2nd ed.). Princeton University Press.

[10] Bay, H., Tuytelaars, T., and Van Gool, L. (2008). SURF: Speeded Up Robust Features. *Computer Vision and Image Understanding, 110*(3), 346–359.

[11] Bertinetto, L., Valmadre, J., Henriques, J. F., Vedaldi, A., and Torr, P. H. S. (2016). *Fully-Convolutional Siamese Networks for Object Tracking*. European Conference on Computer Vision (ECCV) Workshops.

[12] Bewley, A., Ge, Z., Ott, L., Ramos, F., and Upcroft, B. (2016). *Simple Online and Realtime Tracking (SORT)*. IEEE International Conference on Image Processing (ICIP), 3464–3468.

[13] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

[14] Blessing, L.T.M. and Chakrabarti, A. (2009) *DRM: A Design Research Methodology*. London: Springer.

[15] Blueye Robotics (2026) 'Product launch 2026: The next generation is here', *Blueye Robotics*, 2026. Available at: https://www.blueyerobotics.com/ (Accessed: 11 August 2026).

[16] Bochkovskiy, A., Wang, C. Y., and Liao, H. Y. M. (2020). *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv:2004.10934.

[17] Borenstein, J., Everett, H. R., and Feng, L. (1996). *Navigating Mobile Robots: Systems and Techniques*. A K Peters.
[18] Boston Dynamics (2024) *Spot User Guide and Technical Documentation*. Waltham, MA: Boston Dynamics.
[19] Bradski, G.R. (1998) 'Computer Vision Face Tracking for Use in a Perceptual User Interface', *Intel Technology Journal*, 2(2), pp. 12–21.
[20] Bradski, G., and Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.
[21] Brooks, R.A. (1986) 'A robust layered control system for a mobile robot', *IEEE Journal of Robotics and Automation*, 2(1), pp. 14–23. doi: 10.1109/JRA.1986.1087032.
[22] Bruyninckx, H., Lefebvre, T., Mihaylova, L., Staffetti, E., De Schutter, J. and Xiao, J. (2001) 'A roadmap for autonomous robotic assembly', *Proceedings of the IEEE International Symposium on Assembly and Task Planning (ISATP 2001)*, Fukuoka, Japan, pp. 49–54. doi: 10.1109/ISATP.2001.928965.
[23] Camacho, E.F. and Bordons, C. (2007) *Model Predictive Control*. 2nd edn. London: Springer.
[24] Canonical (2020) *Ubuntu 20.04 LTS Release Notes*. Available at: https://ubuntu.com (Accessed: 7 August 2026).
[25] Cao, Z., Hidalgo, G., Simon, T., Wei, S. E., and Sheikh, Y. (2021). OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 43*(1), 172–186.
[26] Chacon, S. and Straub, B. (2014) *Pro Git*. 2nd edn. New York: Apress.
[27] Chaumette, F., and Hutchinson, S. (2006). Visual Servo Control, Part I: Basic Approaches. *IEEE Robotics & Automation Magazine, 13*(4), 82–90.
[28] Chaumette, F., and Hutchinson, S. (2007). Visual Servo Control, Part II: Advanced Approaches. *IEEE Robotics & Automation Magazine, 14*(1), 109–118.
[29] Chen, B. X., Sahdev, R., & Tsotsos, J. K. (2017), 'Integrating Stereo Vision with a CNN Tracker for a Person-Following Robot', In *International Conference on Computer Vision Systems*, pp. 313–325. https://doi.org/10.1007/978-3-319-68345-4_27
[30] Chen, X., Li, Y., Wang, H. and Zhao, J. (2024) 'Vision-Based Person Following for Mobile Robots: A Comprehensive Review', *Artificial Intelligence Review*, 57, Article 164.
[31] Chen, Z., Zhang, T., and Huang, K. (2020). Vision-based person-following robots: A survey of perception, tracking, and control techniques. *Robotics and Autonomous Systems, 124*, 103376.
[32] Ciardo, F., De Tommaso, D. and Wykowska, A. (2019) 'Humans socially attune to their "follower" robot', in *2019 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, Daegu, South Korea, 11–14 March 2019. Piscataway, NJ: IEEE, pp. 538–539. doi:10.1109/HRI.2019.8673262.
[33] Clearpath Robotics (2024) *Husky Unmanned Ground Vehicle User Manual*. Kitchener, ON: Clearpath Robotics.

[34] Clearpath Robotics (2026) *Husky A300: Unmanned Ground Vehicle*. Clearpath Robotics, by Rockwell Automation. Available at: https://clearpathrobotics.com/husky-a300-unmanned-ground-vehicle-robot/ (Accessed: 12 August 2026).

[35] Comaniciu, D., Ramesh, V. and Meer, P. (2003) 'Kernel-Based Object Tracking', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25(5), pp. 564–577.

[36] Corke, P. (2017) *Robotics, Vision and Control: Fundamental Algorithms in MATLAB*. 2nd edn. Cham: Springer. doi: 10.1007/978-3-319-54413-7.

[37] Cousins, S. (2010) 'ROS on the PR2 [ROS Topics]', *IEEE Robotics & Automation Magazine*, 17(3), pp. 23–25.

[38] Creswell, J.W. and Creswell, J.D. (2023) *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*. 6th edn. Thousand Oaks, CA: SAGE Publications.

[39] Dalal, N., and Triggs, B. (2005). Histograms of oriented gradients for human detection. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 886–893.

[40] Dam, A., Verma, A., Pangi, C.T., Raviteja, R. and Prasad, C.S. (2020) 'Person Following Mobile Robot using Pedestrian Dead-Reckoning with Inertial data of Smartphones', *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, Kharagpur, India, pp. 1–4. doi: 10.1109/ICCCNT49239.2020.9225292.

[41] EU Perspectives (2026) 'The EU has a killer robots rule. The world does not', *EU Perspectives*, 1 June. Available at: https://euperspectives.eu/2026/06/the-eu-has-a-killer-robots-rule-the-world-does-not/ (Accessed: 11 August 2026).

[42] Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J., and Zisserman, A. (2010). The Pascal Visual Object Classes (VOC) Challenge. *International Journal of Computer Vision, 88*(2), 303–338.

[43] Fetch Robotics (2021) *Fetch Mobile Manipulator Platform Documentation*. San Jose, CA: Fetch Robotics.

[44] Forsyth, D.A. and Ponce, J. (2012) *Computer Vision: A Modern Approach.* 2nd edn. Upper Saddle River, NJ: Pearson.

[45] Garcia, G.J., Corrales, J.A., Pomares, J. and Torres, F. (2009) 'Survey of visual and force/tactile control of robots for physical interaction in Spain', *Sensors*, 9(12), pp. 9689–9733. doi: 10.3390/s91209689.

[46] Girshick, R., Donahue, J., Darrell, T., and Malik, J. (2014). Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 580–587.

[47] Gonzalez, R. C., and Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.

[48] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.

[49] Goodrich, M. A., and Schultz, A. C. (2007). Human–robot interaction: A survey. *Foundations and Trends in Human–Computer Interaction, 1*(3), 203–275

[50] GP, B., P, K., Abhang, L.B., Rathinasabapathi, G., Sharma, V. and Sathish, T. (2024) 'Machine Vision Applications in Robotics and Automation', *2024 Ninth International Conference on Science Technology Engineering and Mathematics (ICONSTEM)*, Chennai, India, pp. 1–5. doi: 10.1109/ICONSTEM60960.2024.10568755.

[51] Graf, F., Odabaşi, Ç., Jacobs, T., Graf, B. and Födisch, T. (2019) 'MobiKa – Low-cost mobile robot for human-robot interaction', in *2019 28th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, New Delhi, India, 14–18 October 2019. Piscataway, NJ: IEEE, pp. 1–6. doi:10.1109/RO-MAN46459.2019.8956405.

[52] Hartley, R. and Zisserman, A. (2004) *Multiple View Geometry in Computer Vision*. 2nd edn. Cambridge: Cambridge University Press.

[53] Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson.

[54] He, K., Gkioxari, G., Dollár, P., and Girshick, R. (2017). Mask R-CNN. *Proceedings of the IEEE International Conference on Computer Vision*, 2961–2969.

[55] Honig, S.S., Oron-Gilad, T., Zaichyk, H., Sarne-Fleischmann, V., Olatunji, S. and Edan, Y. (2018) 'Toward Socially Aware Person-Following Robots', *IEEE Transactions on Cognitive and Developmental Systems*, 10(4), pp. 936–954. doi: 10.1109/TCDS.2018.2825641.

[56] Horn, B.K.P. and Schunck, B.G. (1981) 'Determining Optical Flow', *Artificial Intelligence*, 17(1–3), pp. 185–203.

[57] Humanoid Robotics Technology (2025) 'Legged Robot Basics: An Introduction', *Humanoid Robotics Technology*, 14 March. Available at: https://humanoidroboticstechnology.com/articles/legged-robot-basics-introduction/ (Accessed: 11 August 2026).

[58] Hutchinson, S., Hager, G. D., and Corke, P. I. (1996). A Tutorial on Visual Servo Control. *IEEE Transactions on Robotics and Automation, 12*(5), 651–670.

[59] Ioannou, P. A., and Sun, J. (2012). *Robust Adaptive Control*. Dover Publications.

[60] Irfan, B., Ramachandran, A., Spaulding, S., Glas, D.F., Leite, I. and Koay, K.L. (2019) 'Personalization in long-term human-robot interaction', in *2019 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, Daegu, South Korea, 11–14 March 2019. Piscataway, NJ: IEEE, pp. 685–686. doi:10.1109/HRI.2019.8673076.

[61] Iqbal, H., Rex, K., Shirley, J., Baiz, C. and Claudel, C. (2025) 'A novel autonomous microplastics surveying robot for beach environments', *arXiv*, arXiv:2508.02952. doi: 10.48550/arXiv.2508.02952.

[62] Islam, M.R., Hong, J. and Sattar, J. (2018) 'Person Following by Autonomous Robots: A Categorical Overview', *International Journal of Robotics Research*, 37(13–14), pp. 1581–1618.

[63] Islam, M.J., Hong, J. and Sattar, J. (2019) 'Person-following by autonomous robots: A categorical overview', *The International Journal of Robotics Research*, 38(14), pp. 1581–1618. doi: 10.1177/0278364919881683.

[64] Islam, M. R., Hong, K. S., and Sattar, J. (2019). Vision-Based Human Following Robots: A Survey. *Robotics and Autonomous Systems, 120*, 103261.

[65] ISO 13482. (2014). *Robots and robotic devices—Safety requirements for personal care robots*. International Organization for Standardization.

[66] Jocher, G., Chaurasia, A. and Qiu, J. (2023) *Ultralytics YOLO Documentation*. Ultralytics. Available at: https://docs.ultralytics.com/

[67] Kaehler, A. and Bradski, G. (2016) *Learning OpenCV 3: Computer Vision in C++ with the OpenCV Library*. Sebastopol, CA: O'Reilly Media.

[68] Kalal, Z., Mikolajczyk, K. and Matas, J. (2012) 'Tracking-Learning-Detection', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34(7), pp. 1409–1422.

[69] Kalman, R.E. (1960) 'A New Approach to Linear Filtering and Prediction Problems', *Transactions of the ASME—Journal of Basic Engineering*, 82(1), pp. 35–45.

[70] Kim, M., Lee, D. and Kim, K.-Y. (2015) 'System architecture for real-time face detection on analog video camera', *International Journal of Distributed Sensor Networks*, 11(5), Article 251386, pp. 1–11. doi: 10.1155/2015/251386.

[71] Kober, J., Bagnell, J. A., and Peters, J. (2013). Reinforcement Learning in Robotics: A Survey. *The International Journal of Robotics Research*, 32(11), 1238–1274.

[72] Koide, K., Miura, J. and Menegatti, E. (2020) 'Monocular person tracking and identification with on-line deep feature selection for person following robots', *Robotics and Autonomous Systems*, 124, p. 103348. doi: 10.1016/j.robot.2019.103348.

[73] Kothari, C.R. (2004) *Research Methodology: Methods and Techniques*. 2nd edn. New Delhi: New Age International Publishers.

[74] Koubaa, A. (Ed.). (2017). *Robot Operating System (ROS): The Complete Reference (Volume 1).* Springer.

[75] Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105.

[76] Laganière, R. (2017) *OpenCV 3 Computer Vision Application Programming Cookbook*. 3rd edn. Birmingham: Packt Publishing.

[77] Lasri, I., Solh, A.R. and El Belkacemi, M. (2019) 'Facial Emotion Recognition of Students using Convolutional Neural Network', *2019 Third International Conference on Intelligent Computing in Data Sciences (ICDS)*, Marrakech, Morocco, pp. 1–6. doi: 10.1109/ICDS47004.2019.8942386.

[78] LeCun, Y., Bengio, Y., and Hinton, G. (2015). Deep Learning. *Nature, 521*(7553), 436–444.

[79] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

[80] Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. (2017). *Feature Pyramid Networks for Object Detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2117–2125.

[81] Lin, T. Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal Loss for Dense Object Detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2980–2988.

[82] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. and Zitnick, C.L. (2014) 'Microsoft COCO: Common Objects in Context', *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 740–755.

[83] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y. and Berg, A.C. (2016) 'SSD: Single Shot MultiBox Detector', in Leibe, B., Matas, J., Sebe, N. and Welling, M. (eds.) *Computer Vision – ECCV 2016*. Lecture Notes in Computer Science, vol. 9905. Cham: Springer, pp. 21–37. doi: 10.1007/978-3-319-46448-0_2.

[84] Lowe, D. G. (2004). Distinctive Image Features from Scale-Invariant Keypoints. *International Journal of Computer Vision, 60*(2), 91–110.

[85] Lucas, B.D. and Kanade, T. (1981) 'An Iterative Image Registration Technique with an Application to Stereo Vision', *Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 674–679.

[86] Luo, W., Xing, J., Milan, A., Zhang, X., Liu, W. and Kim, T.-K. (2021) 'Multiple Object Tracking: A Literature Review', *Artificial Intelligence*, 293, 103448.

[87] Lutz, M. (2013) *Learning Python*. 5th edn. Sebastopol, CA: O'Reilly Media.

[88] Ma, Y. (2024) 'A visualized analysis of service assistive robots for families with disabled children based on CiteSpace data', in *2024 3rd International Conference on Service Robotics (ICoSR)*, Hangzhou, China, 22–24 November 2024. Piscataway, NJ: IEEE, pp. 66–69. doi:10.1109/ICoSR63848.2024.00024.

[89] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., and Woodall, W. (2022). *Robot Operating System 2: Design, Architecture, and Uses in the Wild*. Science Robotics, 7(66), eabm6074.

[90] Malla, D. and Bhandari, P. (2023) 'Socially assistive robot "Sister Robot" as a COVID-19 response and its future plans in health care and clinical applications', in *2023 32nd IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, Busan, Republic of Korea, 28–31 August 2023. Piscataway, NJ: IEEE, pp. 579–584. doi:10.1109/RO-MAN57019.2023.10309600.

[91] Maralit, M. (2024) 'How the Robot Operating System (ROS) is Changing the Game for Mobile Robot Developments', *EngineerZone – EZ Blogs*, 23 October. Available at: https://ez.analog.com/ez-blogs/b/engineerzone-spotlight/posts/how-the-robot-operating-system-ros-is-changing-the-game-for-mobile-robot-developments (Accessed: 5 September 2026).

[92] Marder-Eppstein, E., Berger, E., Foote, T., Gerkey, B. and Konolige, K. (2010) 'The Office Marathon: Robust Navigation in an Indoor Office Environment', *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 300–307.

[93] Marian, M., Stîngă, F., Georgescu, M.-T., Roibu, H., Popescu, D. and Manta, F. (2020) 'A ROS-based Control Application for a Robotic Platform Using the Gazebo 3D Simulator', *2020 21st International Carpathian Control Conference (ICCC)*, High Tatras, Slovakia, pp. 1–5. doi: 10.1109/ICCC49264.2020.9257256.

[94] Martinez, A. and Fernández, E. (2015) *Learning ROS for Robotics Programming.* 2nd edn. Birmingham: Packt Publishing.

[95] Matthes, E. (2019) *Python Crash Course*. 2nd edn. San Francisco, CA: No Starch Press.

[96] Microsoft (2024) *Visual Studio Code Documentation*. Available at: https://code.visualstudio.com/docs (Accessed: 7 August 2026).

[97] Milan, A., Leal-Taixé, L., Reid, I., Roth, S., and Schindler, K. (2016). MOT16: A Benchmark for Multi-Object Tracking. *arXiv preprint arXiv:1603.00831*.

[98] Mohamed, N., Al-Jaroodi, J. and Jawhar, I. (2008) 'Middleware for Robotics: A Survey', *Proceedings of the IEEE International Conference on Robotics, Automation and Mechatronics*, pp. 736–742.

[99] Montesdeoca, J., Orozco, W., Pinos, E., Toibero, M., Roberti, F. and Guevara, B.S. (2024) 'Person-Following Robots With Social Robot Motion From Control Design,

Simulation Results', *2024 IEEE ANDESCON*, Cusco, Peru, pp. 1–5. doi: 10.1109/ANDESCON61840.2024.10755713.

[100] Montesdeoca, J., Toibero, J.M., Jordan, J., Zell, A. and Carelli, R. (2022) 'Person-Following Controller with Socially Acceptable Robot Motion', *Robotics and Autonomous Systems*, 153, 104075. https://doi.org/10.1016/j.robot.2022.104075

[101] Munaro, M., Basso, F. and Menegatti, E. (2015) 'Tracking People Within Groups with RGB-D Data', *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2101–2107.

[102] Murphy, R. R. (2019). *Introduction to AI Robotics* (2nd ed.). MIT Press.

[103] Norman, D.A. (2013) *The Design of Everyday Things*. Revised and expanded edn. New York: Basic Books.

[104] Odabasi, C., Graf, F., Lindermayr, J., Patel, M., Baumgarten, S.D. and Graf, B. (2022) 'Refilling Water Bottles in Elderly Care Homes With the Help of a Safe Service Robot', *2022 17th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, Sapporo, Japan, pp. 101–110. doi: 10.1109/HRI53351.2022.9889391.

[105] Padilla, R., Passos, W. L., Dias, T. L. B., Netto, S. L., & da Silva, E. A. B. (2021). A Comparative Analysis of Object Detection Metrics with a Companion Open-Source Toolkit. *Electronics, 10*(3), 279.

[106] Park, J., Jo, J.H. and Moon, C. (2024) 'Development of a human-following scheme using Point-Voxel R-CNN-based 3D human leg detection for the robust human-following of mobile robots in cluttered environments', *International Journal of Advanced Robotic Systems*, 21(5). doi: 10.1177/17298806241287313.

[107] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. *et al.* (2019) 'PyTorch: An imperative style, high-performance deep learning library', *Advances in Neural Information Processing Systems*, 32, pp. 8024–8035.

[108] Pressman, R.S. and Maxim, B.R. (2020) *Software Engineering: A Practitioner's Approach*. 9th edn. New York: McGraw-Hill.

[109] Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., and Ng, A. Y. (2009). *ROS: An Open-Source Robot Operating System*. Proceedings of the IEEE International Conference on Robotics and Automation (ICRA) Workshop on Open Source Software.

[110] Rakhmetova, P., Bektilevov, A., Kurmangalieva, L., Bekbossynova, B. and Shingissov, B. (2025) 'Experimental Investigation of Computer Vision Architectures for Object Detection in Mobile Robotics', *2025 6th International Conference on Communications, Information, Electronic and Energy Systems (CIEES)*, Ruse, Bulgaria, pp. 1–4. doi: 10.1109/CIEES66347.2025.11300112.

[111] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You Only Look Once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.

[112] Redmon, J., and Farhadi, A. (2017). *YOLO9000: Better, Faster, Stronger*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 7263–7271.

[113] Redmon, J., and Farhadi, A. (2018). *YOLOv3: An Incremental Improvement*. arXiv:1804.02767.

[114] Reegård, K., Eitrheim, M., Kaarstad, M., Fernandes, A. and Sørensen, L. (2024) 'Beyond acceptance: Patients' perspectives on humanoid assistive robots in healthcare', in *2024 33rd IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, Pasadena, CA, USA, 26–30 August 2024. Piscataway, NJ: IEEE, pp. 851–856. doi:10.1109/RO-MAN60168.2024.10731409.

[115] Ren, S., He, K., Girshick, R., and Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems*, 91–99.

[116] Robot Operating System (ROS) (2024) *ROS Message Documentation*. Available at: https://docs.ros.org (Accessed: 7 August 2026).

[117] ROBOTIS (2024) *TurtleBot3 e-Manual*. Available at: https://emanual.robotis.com/docs/en/platform/turtlebot3/ (Accessed: 4 August 2026).

[118] Robotnik Automation (2019) *Summit XL Mobile Robot User Manual.* Valencia, Spain: Robotnik Automation S.L.

[119] Robotnik Automation (2024) *Summit XL Autonomous Mobile Robot Technical Specifications*. Valencia: Robotnik Automation.

[120] Russell, S., and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[121] Satake, J., Chiba, M., & Miura, J. (2013) 'Visual Person Identification Using a Distance-dependent Appearance Model for a Person Following Robot', *International Journal of Automation and Computing, 10*(5), 438–446. https://doi.org/10.1007/s11633-013-0740-y

[122] Shafique, S., Abid, S., Riaz, F. and Ejaz, Z. (2021) 'Computer Vision based Autonomous Navigation in Controlled Environment', *2021 International Conference on Robotics and Automation in Industry (ICRAI)*, Rawalpindi, Pakistan, pp. 1–6. doi: 10.1109/ICRAI54018.2021.9651414.

[123] Shandong Guoxing Intelligent Technology Co., Ltd. (n.d.) *All Terrain Smart Tracked Mobile Robot Tank Platform Chassis: Safari-600T*. GUOXING. Available at: https://www.gxsuprobot.com/All-Terrain-Smart-Tracked-Mobile-Robot-Tank-Platform-Chassis-pd41587899.html (Accessed: 11 August 2026).

[124] Sharanya, S., Varma, D.R. and Paul, A. (2023) 'Navigation of ground robots with reinforcement learning', *Proceedings of the Asia-Pacific Conference on Intelligent Robot Systems*. doi:10.1109/ACIRS58671.2023.10240285.

[125] Shi, Z., Xu, Z. and Wang, T. (2022) 'A method for detecting pedestrian height and distance based on monocular vision technology', *Measurement*, 199, p. 111418. doi: 10.1016/j.measurement.2022.111418.

[126] Siciliano, B. and Khatib, O. (eds.) (2016) *Springer Handbook of Robotics*. 2nd edn. Cham: Springer. doi: 10.1007/978-3-319-32552-1.

[127] Siegwart, R., Nourbakhsh, I.R. and Scaramuzza, D. (2011) *Introduction to Autonomous Mobile Robots*. 2nd edn. Cambridge, MA: MIT Press.

[128] Silberschatz, A., Galvin, P.B. and Gagne, G. (2018) *Operating System Concepts*. 10th edn. Hoboken, NJ: John Wiley & Sons.

[129] Sim, J., Hagiwara, Y. and Choi, Y. (2025) 'Small Obstacle-Aware Person-Following for Mobile Robots Using Vision-Based Segmentation', *2025 IEEE 14th Global Conference on Consumer Electronics (GCCE)*, Osaka, Japan, pp. 1209–1213. doi: 10.1109/GCCE65946.2025.11275154.

[130] Smeulders, A. W. M., Chu, D. M., Cucchiara, R., Calderara, S., Dehghan, A., and Shah, M. (2014). Visual Tracking: An Experimental Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 36(7), 1442–1468.

[131] Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson Education Limited.

[132] Song, D., Chen, M., Zhang, C., Fu, H., Lin, Y., Li, Z., Ma, A., & Liu, J. (2026), 'RVPF-YOLO: A robust visual person-following framework for mobile robots based on enhanced YOLO model', *Displays*. DOI: 10.1016/j.displa.2026.103534.

[133] Sonka, M., Hlavac, V., and Boyle, R. (2015). *Image Processing, Analysis, and Machine Vision* (4th ed.). Cengage Learning.

[134] Statzon (2023) *Autonomous Mobile Robot (AMR) Market – World Opportunity Analysis and Industry Forecast, 2024–2030*. Statzon Market Research Report. Published November 2023. Available at: https://statzon.com/

[135] Stauffer, C. and Grimson, W.E.L. (1999) 'Adaptive Background Mixture Models for Real-Time Tracking', *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, Vol. 2, pp. 246–252.

[136] Sutton, R. S., and Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[137] Szeliski, R. (2022). *Computer Vision: Algorithms and Applications* (2nd ed.). Springer.

[138] Tai, L., Paolo, G., & Liu, M. (2017). Virtual-to-Real Deep Reinforcement Learning: Continuous Control of Mobile Robots for Mapless Navigation. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 31–36.

[139] Tan, M., Pang, R. and Le, Q.V. (2020) 'EfficientDet: Scalable and Efficient Object Detection', *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10781–10790.

[140] Terven, J. and Cordova-Esparza, D. (2023) 'A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and Beyond', *Machine Learning and Knowledge Extraction*, 5(4), pp. 1680–1716.

[141] Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics*. MIT Press.

[142] Treptow, A., Cielniak, G. and Duckett, T. (2006) 'Real-time people tracking for mobile robots using thermal vision', *Robotics and Autonomous Systems*, 54(9), pp. 729–739. doi: 10.1016/j.robot.2006.04.013.

[143] Tsai, T.-H. and Yao, C.-H. (2021), 'A robust tracking algorithm for a human-following mobile robot', *IET Image Processing*, 15(3), pp. 786–796. https://doi.org/10.1049/ipr2.12062.

[144] Viola, P. and Jones, M. (2001) 'Rapid Object Detection Using a Boosted Cascade of Simple Features', *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, Vol. 1, pp. I-511–I-518.

[145] Wang, A. (2026) ‘Mobile Robot Drive Systems Comparison: 2WD, 4WD, 4WS, and Tracked Robots’, *Fdata Robot*, 8 June. Available at: https://www.fdatabot.com/mobile-robot-drive-systems-2wd-4wd-4ws-tracked-comparison/ (Accessed: 11 August 2026).

[146] Wang, A., Makino, Y., & Shinoda, H. (2021). Machine Learning-based Human-Following System: Following the Predicted Position of a Walking Human. *2021 IEEE International Conference on Robotics and Automation (ICRA)*. DOI: 10.1109/ICRA48506.2021.9561691.

[147] Wang, C.-Y., Bochkovskiy, A., and Liao, H.-Y. M. (2022). *YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors*. arXiv:2207.02696.

[148] Wilson, G., Aruliah, D.A., Brown, C.T., Chue Hong, N.P., Davis, M., Guy, R.T., Haddock, S.H.D., Huff, K.D., Mitchell, I.M., Plumbley, M.D. *et al.* (2014) ‘Best practices for scientific computing’, *PLoS Biology*, 12(1), e1001745.

[149] Wojke, N., Bewley, A., and Paulus, D. (2017). *Simple Online and Realtime Tracking with a Deep Association Metric*. IEEE International Conference on Image Processing (ICIP), 3645–3649.

[150] Xu, L. (2025) ‘Clinical applications and existing problems of medical robots’, in *2025 International Conference on Advanced Computing and Intelligent Robotics Applications (ACIRA)*, Guangzhou, China. Piscataway, NJ: IEEE, pp. 201–204. doi:10.1109/ACIRA67680.2025.11334972.

[151] Ye, H., Cai, K., Zhan, Y., Xia, B., Ajoudani, A., & Zhang, H. (2025). *RPF-Search: Field-based Search for Robot Person Following in Unknown Dynamic Environments*. arXiv:2503.02188.

[152] Yilmaz, A., Javed, O., and Shah, M. (2006). Object Tracking: A Survey. *ACM Computing Surveys*, 38(4), 13.

[153] Zadeh, L.A. (1965) 'Fuzzy Sets', *Information and Control*, 8(3), pp. 338–353.

[154] Zhang, B. (2024) ‘Algorithm of Robot Visual Perception and Environment Interaction Based on Deep Learning’, *2024 International Conference on Power, Electrical Engineering, Electronics and Control (PEEEC)*, Athens, Greece, pp. 1066–1070. doi: 10.1109/PEEEC63877.2024.00197.

[155] Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., and Wang, X. (2022). *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. European Conference on Computer Vision (ECCV), 1–21.

[156] Zhang, Z., Zou, Y., Tan, Y. and Zhou, C. (2024) ‘YOLOv8-seg-CP: a lightweight instance segmentation algorithm for chip pad based on improved YOLOv8-seg model’, *Scientific Reports*, 14, article 27716. doi: 10.1038/s41598-024-78578-x.

[157] Zhou, J. and Guowei, Z. (2025) ‘Design of a ROS-Based Vision-Guided Mobile Robot for Object Transportation’, *2025 IEEE 8th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, Guiyang, China, pp. 1289–1292. doi: 10.1109/IAEAC65194.2025.11166096.

[158] Zhou, X., Wang, D. and Krähenbühl, P. (2019) 'Objects as Points', *arXiv preprint*, arXiv:1904.07850.

[159] Zou, Z., Shi, Z., Guo, Y. and Ye, J. (2019) 'Object Detection in 20 Years: A Survey', *arXiv preprint*, arXiv:1905.05055.

APPENDIX A

A1 – Bounding-Box Height

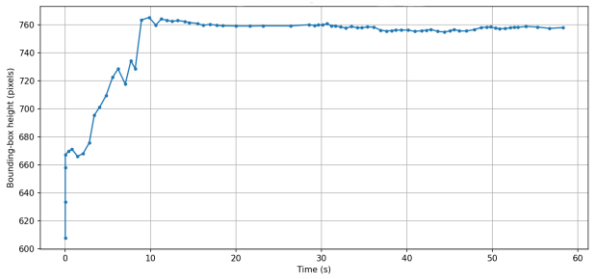


Figure 1.A1 - Scenario One Bounding-Box Height

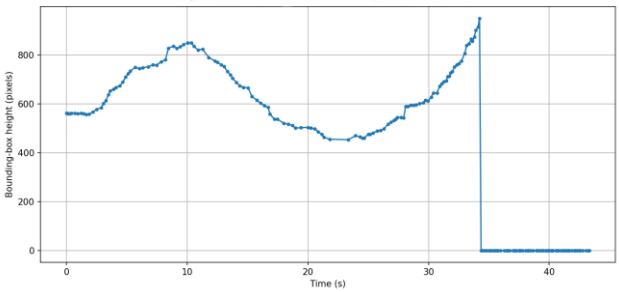


Figure 2.A1 - Scenario Two Bounding-Box Height

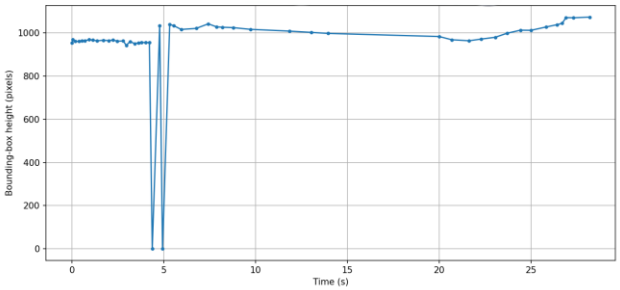


Figure 3.A - Scenario Three Bounding-Box Height

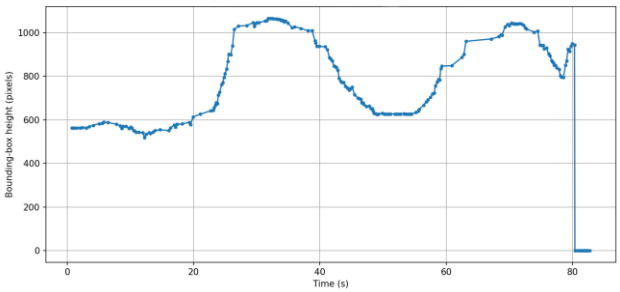


Figure 4.A - Scenario Four Bounding-Box Height

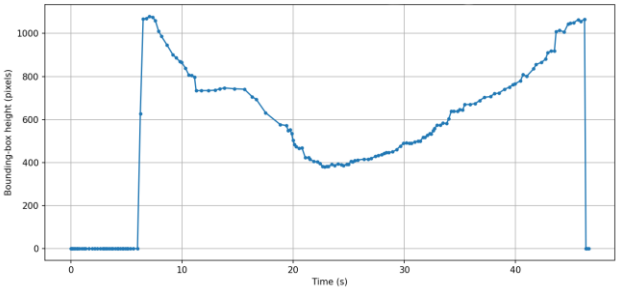


Figure 5.A - Scenario Five Bounding-Box Height

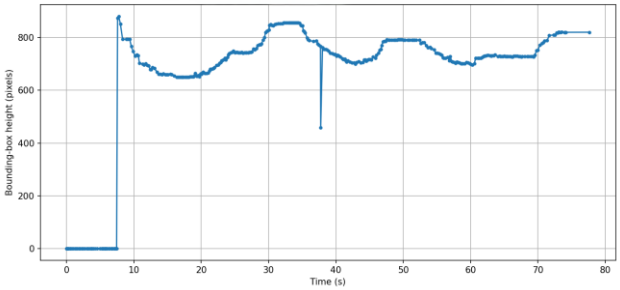


Figure 6.A - Scenario Six Bounding-Box Height

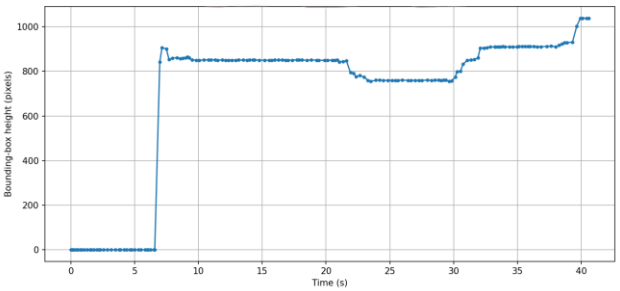


Figure 7.A - Scenario Seven Bounding-Box Height

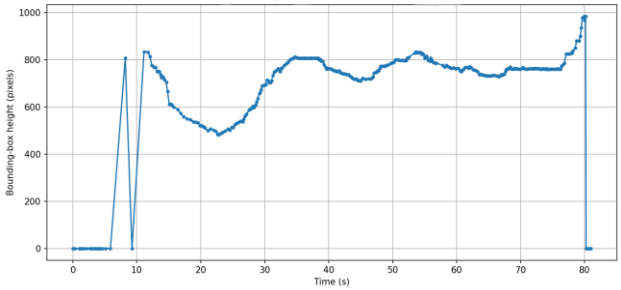


Figure 8.A - Scenario Eight Bounding-Box Height

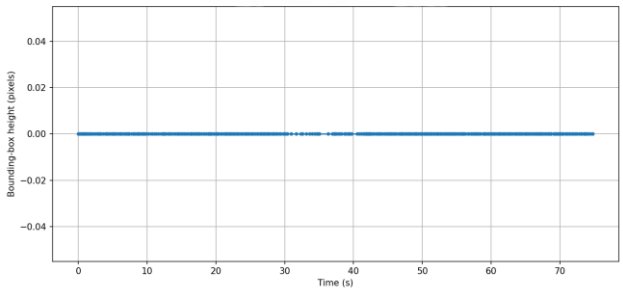


Figure 9.A - Scenario Nine Bounding-Box Height

A2 – Linear Velocity

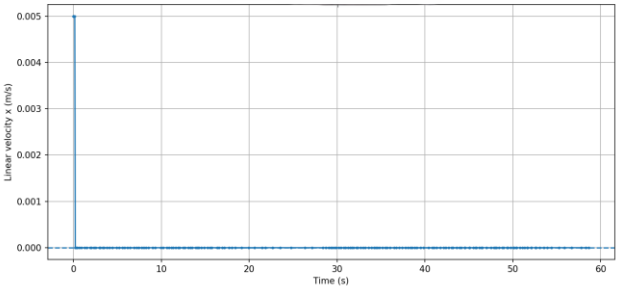


Figure 1.A2 - Scenario One Linear Velocity

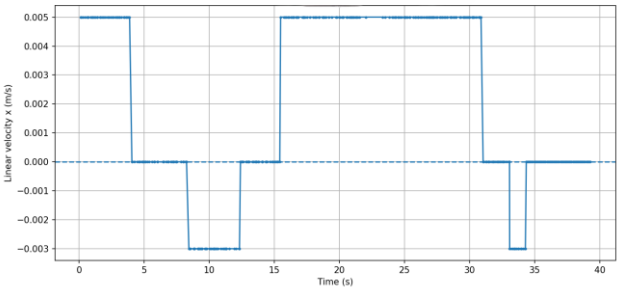


Figure 2.A2 - Scenario Two Linear Velocity

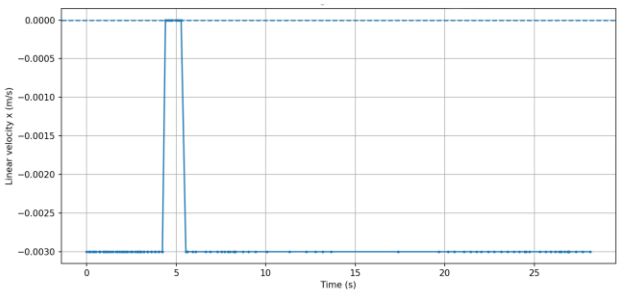


Figure 3.A2 - Scenario Three Linear Velocity

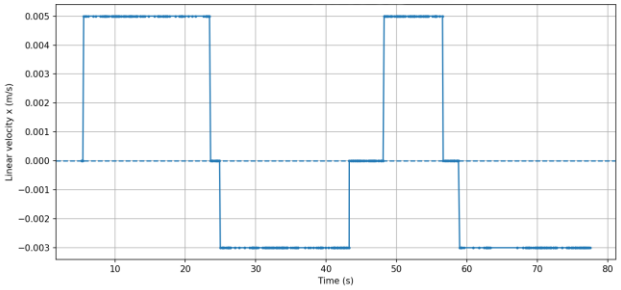


Figure 4.A2 - Scenario Four Linear Velocity

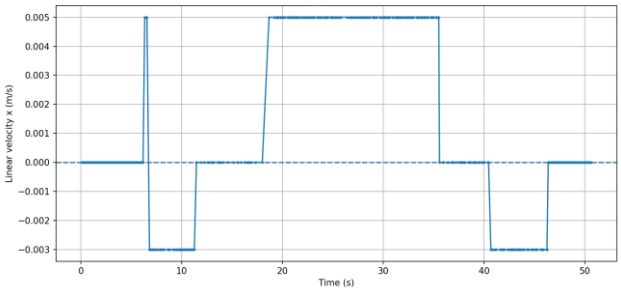


Figure 5.A2 - Scenario Five Linear Velocity

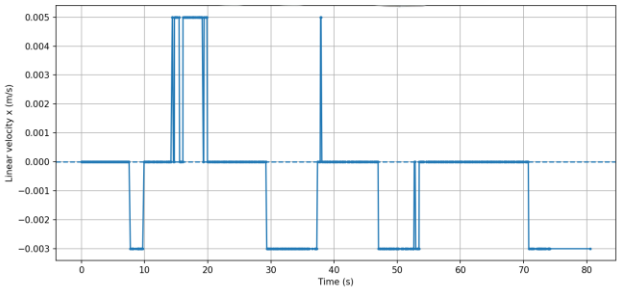


Figure 6.A2 - Scenario Six Linear Velocity

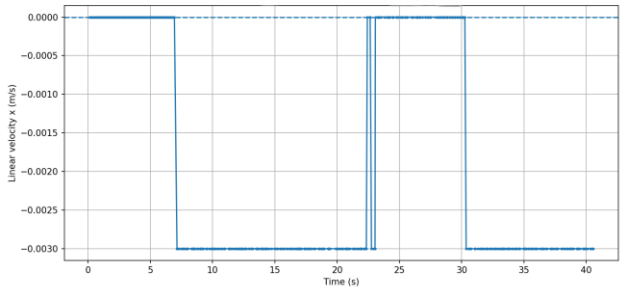


Figure 7.A2 - Scenario Seven Linear Velocity

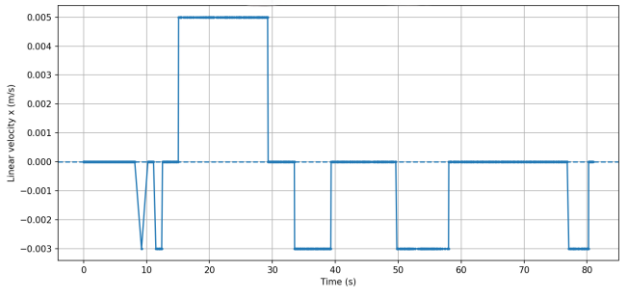


Figure 8.A2 - Scenario Eight Linear Velocity

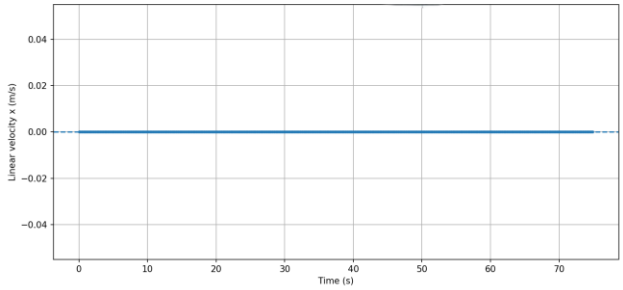


Figure 9.A2 - Scenario Nine Linear Velocity

A3 – Human Target Position

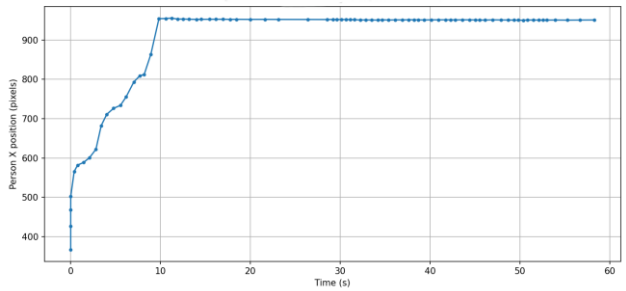


Figure 1.A3 - Scenario One Human Target Position

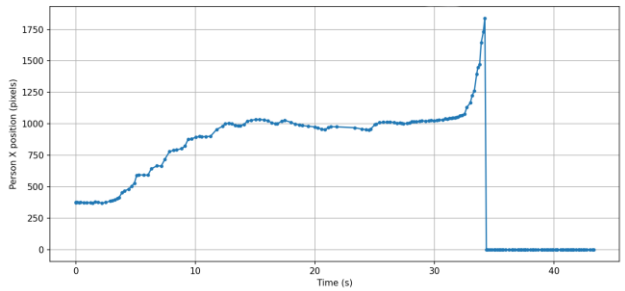


Figure 2.A3 - Scenario Two Human Target Position

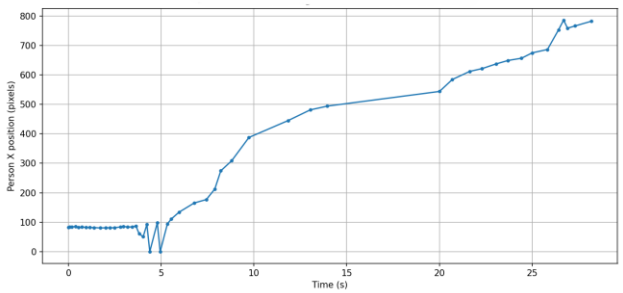


Figure 3.A3 - Scenario Three Human Target Position

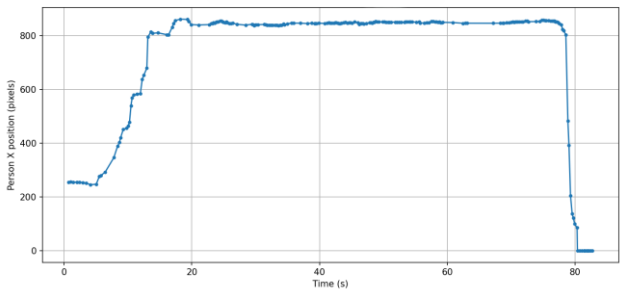


Figure 4.A3 - Scenario Four Human Target Position

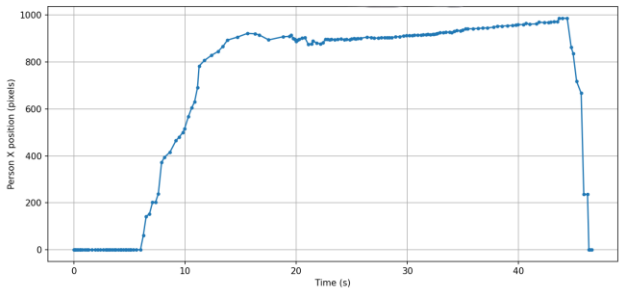


Figure 5.A3 - Scenario Five Human Target Position

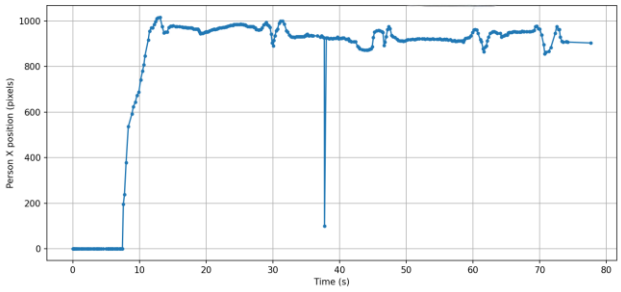


Figure 6.A3 - Scenario Six Human Target Position

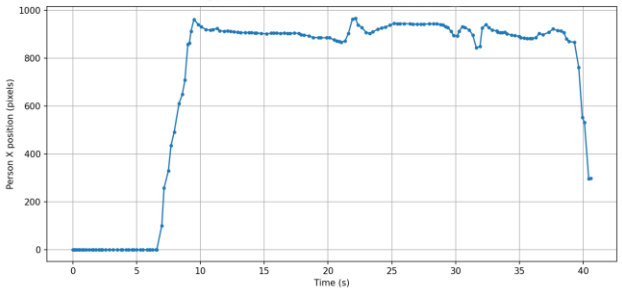


Figure 7.A3 - Scenario Seven Human Target Position

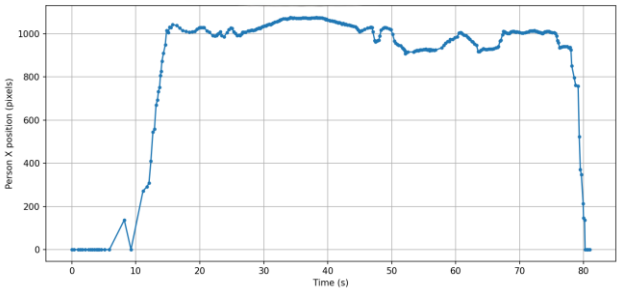


Figure 8.A3 - Scenario Eight Human Target Position

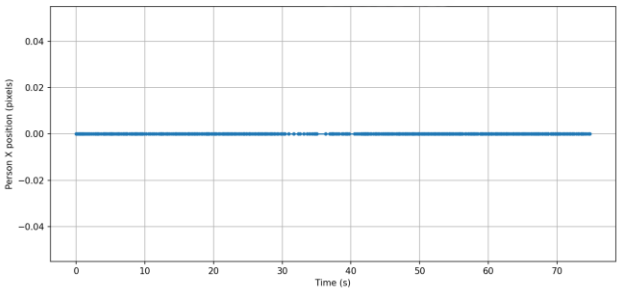


Figure 9.A3 - Scenario Nine Human Target Position

APPENDIX B

Table A.1 - Ground Truth Annotations

Scenario	Start Time	End Time	Ground Truth
1	0	58.737	1
2	0	43.359	1
3	0	28.352	1
4	0	82.867	1
5	0	50.659	1
6	0	81.732	1
7	0	40.583	1
8	0	80.961	1
9	0	74.93	0

Experimental Data Summary of all 9 Scenarios

==========

Scenario 01 - Standing

Duration: 63.737 s
Detections: 299
cmd_vel messages: 176
Detection rate: 4.338 Hz
Command rate: 2.984 Hz
Average detection gap: 0.747 s
Maximum detection gap: 3.253 s
BBox height range: 607.65 - 765.40 px
Average confidence: 0.895
Linear velocity range: 0.0000 - 0.0050 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 5
Non-zero angular commands: 0
Linear command ratio: 2.84%

Scenario 02 - Walking Forward

Duration: 63.359 s
Detections: 271
cmd_vel messages: 345
Detection rate: 4.269 Hz
Command rate: 8.790 Hz
Average detection gap: 0.234 s
Maximum detection gap: 1.523 s

BBox height range: 0.00 - 951.03 px
Average confidence: 0.591
Linear velocity range: -0.0030 - 0.0050 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 209
Non-zero angular commands: 0
Linear command ratio: 60.58%

Scenario 03 - Walking Backward

Duration: 68.352 s
Detections: 324
cmd_vel messages: 108
Detection rate: 4.737 Hz
Command rate: 3.804 Hz
Average detection gap: 0.576 s
Maximum detection gap: 6.048 s
BBox height range: 0.00 - 1073.55 px
Average confidence: 0.808
Linear velocity range: -0.0030 - 0.0000 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 99
Non-zero angular commands: 0
Linear command ratio: 91.67%

Scenario 04 - Stopping

Duration: 82.867 s
Detections: 214
cmd_vel messages: 435
Detection rate: 2.585 Hz
Command rate: 6.007 Hz
Average detection gap: 0.387 s
Maximum detection gap: 4.040 s
BBox height range: 0.00 - 1066.53 px
Average confidence: 0.851
Linear velocity range: -0.0030 - 0.0050 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 357
Non-zero angular commands: 0
Linear command ratio: 82.07%

Scenario 05 - Standing >3 m

Duration: 60.659 s
Detections: 273
cmd_vel messages: 409
Detection rate: 4.500 Hz
Command rate: 8.069 Hz
Average detection gap: 0.286 s
Maximum detection gap: 1.320 s
BBox height range: 0.00 - 1080.00 px
Average confidence: 0.683
Linear velocity range: -0.0030 - 0.0050 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 247
Non-zero angular commands: 0
Linear command ratio: 60.39%

Scenario 06 - Forward/Backward 2-3 m

Duration: 81.732 s
Detections: 369
cmd_vel messages: 724
Detection rate: 4.509 Hz
Command rate: 8.976 Hz
Average detection gap: 0.222 s
Maximum detection gap: 3.470 s
BBox height range: 0.00 - 880.72 px
Average confidence: 0.802
Linear velocity range: -0.0030 - 0.0050 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 232
Non-zero angular commands: 0
Linear command ratio: 32.04%

Scenario 07 - Standing <3 m

Duration: 60.583 s
Detections: 300
cmd_vel messages: 376
Detection rate: 4.943 Hz
Command rate: 9.275 Hz
Average detection gap: 0.254 s
Maximum detection gap: 0.471 s
BBox height range: 0.00 - 1039.11 px

Average confidence: 0.720
Linear velocity range: -0.0030 - 0.0000 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 240
Non-zero angular commands: 0
Linear command ratio: 63.83%

Scenario 08 - Lighting Variation

Duration: 80.961 s
Detections: 328
cmd_vel messages: 736
Detection rate: 4.040 Hz
Command rate: 9.078 Hz
Average detection gap: 0.248 s
Maximum detection gap: 2.319 s
BBox height range: 0.00 - 984.49 px
Average confidence: 0.846
Linear velocity range: -0.0030 - 0.0050 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 276
Non-zero angular commands: 0
Linear command ratio: 37.50%

Scenario 09 - No Target

Duration: 74.930 s
Detections: 401
cmd_vel messages: 750
Detection rate: 5.349 Hz
Command rate: 10.000 Hz
Average detection gap: 0.187 s
Maximum detection gap: 1.282 s
BBox height range: 0.00 - 0.00 px
Average confidence: 0.000
Linear velocity range: 0.0000 - 0.0000 m/s
Angular velocity range: 0.0000 - 0.0000 rad/s
Non-zero linear commands: 0
Non-zero angular commands: 0
Linear command ratio: 0.00%